Input Reference

SWAT+ Documentation

docs.swat.tamu.edu

Contents

Older projects . . . 9
Simulation control . . . 9
Basin parameters . . . 9
Spatial objects and connectivity . . . 10
Climate . . . 10
Soils and land use . . . 10
Channels . . . 10
Aquifers, reservoirs and wetlands . . . 10
Management . . . 10
Initial conditions . . . 11
Structural practices . . . 11
Pollutants . . . 11
Groundwater . . . 11
Salts and constituents . . . 11
Water allocation . . . 11
Purpose . . . 11
Format . . . 11
Categories and default file names . . . 12
Missing and null files . . . 13
Weather data folders . . . 13
Output folder . . . 14
Older file.cio files . . . 14
Example . . . 14
Related files . . . 15
Purpose . . . 16
Format . . . 16
Simulation period . . . 16
Time step . . . 17
Weather outside the observed record . . . 17
Example . . . 17
Related files . . . 17
Purpose . . . 18
Format . . . 18
Output group rows . . . 20
Print timing . . . 21
Average annual values . . . 21
Example . . . 21
Related files . . . 22
Purpose . . . 23
Format . . . 23
Object counts . . . 24
Basin areas . . . 24
gwflow . . . 24
Example . . . 24

Related files . . . 25
Purpose . . . 25
Format . . . 26
PET method . . . 27
Extra input columns . . . 28
Crack flow . . . 28
Peak runoff rate . . . 28
Channel routing . . . 28
USLE C factor . . . 29
Carbon model . . . 29
Lapse rates . . . 29
Unit hydrograph . . . 29
Runoff method . . . 30
In-stream nutrients . . . 30
gwflow module . . . 30
Tillage effect on decomposition . . . 30
Example . . . 30
Related files . . . 31
Purpose . . . 32
Format . . . 32
Zero and missing values . . . 36
Initial soil water . . . 36
Surface runoff lag . . . 36
Peak rate factor . . . 36
Pesticides in aquifers . . . 37
Pesticide decay and temperature . . . 37
Uptake distribution . . . 37
Denitrification . . . 37
Frozen soil . . . 38
Dormancy threshold . . . 38
USLE C factor . . . 38
Sub-daily erosion . . . 38
CO2 . . . 38
Random numbers . . . 38
Calibration . . . 39
Example . . . 39
Related files . . . 39
Purpose . . . 41
Activation and file names . . . 41
Format . . . 41
Initial carbon pools . . . 43
Soluble carbon losses . . . 43
Organic fertilizer and manure . . . 43
Tillage and biological mixing . . . 43
Temperature factor . . . 44
Water factor . . . 44
Residue decomposition . . . 44
Notes on individual columns . . . 45
Calibration . . . 45
Example . . . 45
Related files . . . 45
Purpose . . . 46
Activation and file name . . . 46
Format . . . 46
Use in the carbon model . . . 48
Calibration . . . 48

Example . . . 48
Related files . . . 49
Purpose . . . 49
Format . . . 51
Area, location and elevation . . . 52
Weather station lookup . . . 52
Receiving object codes . . . 52
Hydrograph codes . . . 53
Inflow to HRUs and routing units . . . 53
Flow-control rule . . . 53
Export coefficient objects . . . 54
Groundwater flow (gwflow) models . . . 54
Routing loops . . . 54
Example . . . 54
Related files . . . 54
Purpose . . . 55
Format . . . 55
Name lookup . . . 56
Tile drainage . . . 56
Example . . . 57
Related files . . . 57
Purpose . . . 58
hru-lte.hru . . . 58
Name matching . . . 60
Daily water balance . . . 60
Plant growth . . . 63
Irrigation . . . 64
Erosion . . . 64
Area units . . . 64
Calibration . . . 65
Example . . . 65
Related files . . . 65
Purpose . . . 66
rout_unit.def . . . 67
rout_unit.ele . . . 68
rout_unit.rtu . . . 70
ls_unit.def . . . 71
ls_unit.ele . . . 72
aqu_catunit.ele . . . 73
Columns in earlier documentation . . . 74
Related files . . . 74
Purpose . . . 75
chan-surf.lin . . . 75
aqu_cha.lin . . . 76
Example . . . 78
Related files . . . 78
Purpose . . . 79
hydrology.hyd . . . 79
topography.hyd . . . 83
field.fld . . . 84
shade_factor.shf . . . 84
Calibration . . . 85
Examples . . . 85
Related files . . . 85
Purpose . . . 86
recall.rec . . . 86

Recall data file . . . 87
recall_db.rec . . . 88
pest.com . . . 89
Salt and constituent recall files . . . 89
Region files . . . 89
Output . . . 90
Columns in earlier documentation . . . 90
Example . . . 90
Related files . . . 90
Purpose . . . 91
Common rules . . . 91
weather-sta.cli . . . 92
weather-wgn.cli . . . 93
Observed data lists . . . 96
atmodep.cli . . . 96
Salt and constituent deposition . . . 98
Related files . . . 98
Purpose . . . 99
Format . . . 99
Reading and storage . . . 100
Missing days . . . 100
Sub-daily runs . . . 100
Elevation adjustment . . . 101
Calibration . . . 101
Example . . . 101
Related files . . . 101
Purpose . . . 102
Format . . . 102
Missing days . . . 103
Elevation adjustment . . . 103
Calibration . . . 103
Example . . . 103
Related files . . . 104
Purpose . . . 104
Format . . . 105
Example . . . 105
Related files . . . 105
Purpose . . . 105
Format . . . 105
Snow processes . . . 106
Calibration and decision tables . . . 107
Example . . . 107
Related files . . . 108
Purpose . . . 108
Format . . . 108
Layering . . . 110
Derived layer properties . . . 111
Use of the soil properties . . . 111
Calibration and conditions . . . 112
soils_lte.sol . . . 113
Example . . . 113
Related files . . . 113
Purpose . . . 114
Format . . . 114
Name matching . . . 115
Use at the start of the run . . . 115

Land use change . . . 116
Calibration groups . . . 116
Septic systems . . . 116
Example . . . 116
Related files . . . 117
Purpose . . . 117
cntable.lum . . . 118
cons_practice.lum . . . 120
ovn_table.lum . . . 122
Related files . . . 123
Purpose . . . 124
Files on the channel line . . . 124
initial.cha . . . 124
initial.cha_cs . . . 125
temperature.cha . . . 125
Legacy channel files . . . 128
Example . . . 129
Related files . . . 129
Purpose . . . 130
Links between the files . . . 130
channel-lte.cha . . . 130
hyd-sed-lte.cha . . . 131
sed_nut.cha . . . 133
nutrients.cha . . . 134
Channel geometry and rating curve . . . 137
Flow routing . . . 137
Sediment . . . 138
Nutrients . . . 140
Pesticides . . . 140
Calibration and decision tables . . . 140
Example . . . 140
Related files . . . 141
Purpose . . . 142
aquifer.aqu . . . 143
Aquifer water balance . . . 146
Nitrate and phosphorus . . . 147
Pesticides, salts and constituents . . . 147
Aquifer flow to channels with aqu_cha.lin . . . 147
Calibration and decision tables . . . 147
initial.aqu . . . 148
initial.aqu_cs . . . 148
Example . . . 148
Related files . . . 149
Purpose . . . 150
reservoir.res . . . 150
initial.res . . . 151
hydrology.res . . . 151
Release rules . . . 153
sediment.res . . . 154
nutrients.res . . . 154
weir.res . . . 156
Calibration . . . 156
Example . . . 156
Related files . . . 157
Purpose . . . 158
wetland.wet . . . 158

hydrology.wet . . . 159
Wetland behaviour . . . 160
Management and decision tables . . . 162
Example . . . 162
Related files . . . 162
Purpose . . . 163
Format . . . 163
harv.ops . . . 164
graze.ops . . . 164
irr.ops . . . 165
chem_app.ops . . . 166
fire.ops . . . 166
sweep.ops . . . 166
Example . . . 167
Related files . . . 167
Purpose . . . 168
Format . . . 169
Condition variables . . . 170
Land use and management actions . . . 172
Reservoir release actions . . . 177
Land use scenarios . . . 178
Example . . . 178
Related files . . . 179
Purpose . . . 179
Format . . . 180
Plant types . . . 182
Heat units to maturity . . . 182
Leaf area development . . . 183
Radiation use efficiency . . . 183
Perennials and trees . . . 183
Nutrient uptake . . . 183
Residue lignin . . . 184
Values set by the model . . . 184
Calibration . . . 184
Example . . . 184
Related files . . . 184
Purpose . . . 185
fertilizer.frt . . . 186
manure_db.frt . . . 187
manure_om.frt . . . 187
Related files . . . 188
Purpose . . . 189
Format . . . 189
Tillage mixing . . . 191
Biological mixing . . . 191
Surface roughness . . . 191
Example . . . 191
Related files . . . 192
Replacement files . . . 192
Keyword migration . . . 193
Example . . . 193
Related files . . . 193
Purpose . . . 194
Format . . . 194
Example . . . 194
Related files . . . 194

Purpose . . . 194
Format . . . 194
Example . . . 195
Related files . . . 196
management.sch . . . 196
puddle.ops . . . 199
transplant.ops . . . 200
co2_yr.dat and co2.out . . . 200
object.prt . . . 201
Purpose . . . 202
Initialization files . . . 202
plant.ini . . . 202
soil_plant.ini . . . 204
nutrients.sol . . . 205
om_water.ini . . . 206
Related files . . . 207
Purpose . . . 208
File layout . . . 208
tiledrain.str . . . 208
septic.str . . . 209
septic.sep . . . 211
filterstrip.str . . . 211
grassedww.str . . . 212
bmpuser.str . . . 214
satbuffer.str . . . 214
urban.urb . . . 215
References . . . 215
Related files . . . 216
Purpose . . . 217
pesticide.pes . . . 217
pest_hru.ini . . . 220
pest_water.ini . . . 220
pest_metabolite.pes . . . 221
Related files . . . 221
Purpose . . . 222
pathogens.pth . . . 222
path_hru.ini . . . 223
path_water.ini . . . 224
Behaviour in 62.0.1 . . . 224
Related files . . . 224
Purpose . . . 225
hmet_hru.ini . . . 225
Related files . . . 225
Purpose . . . 226
Activation . . . 226
File overview . . . 226
Format conventions . . . 227
codes.gw . . . 227
zones.gw . . . 228
cells.gw . . . 229
cellcon.gw . . . 229
outputs.gw . . . 230
chancell.gw . . . 230
gwflow.con . . . 231
hrucell.gw . . . 231
lsucell.gw . . . 231

chan_depth.gw . . . 231
pumpex.gw . . . 232
hru_pump.gw . . . 232
tile.gw . . . 232
rescell.gw . . . 233
gwflow.wetland . . . 233
floodplain.gw . . . 233
gwflow_canal.con . . . 233
phreato.gw and phreato_cell.gw . . . 233
tvheads.gw . . . 233
solute.gw . . . 234
cell_sol.gw . . . 234
minerals.gw . . . 234
ponds.gw, pond_cell.gw and pond_div.gw . . . 234
sw_group.gw . . . 235
transit.gw . . . 235
Example . . . 235
Related files . . . 236
Purpose . . . 236
Activation . . . 237
File overview . . . 237
Missing files . . . 237
salt_hru.ini . . . 238
salt_aqu.ini . . . 238
salt_channel.ini . . . 238
salt_res . . . 238
salt_irrigation . . . 239
salt_fertilizer.frt . . . 239
salt_road . . . 239
salt_plants . . . 240
salt_uptake . . . 240
salt_urban . . . 241
Example . . . 241
Related files . . . 241
Purpose . . . 242
constituents.cs . . . 242
File overview . . . 243
Missing files . . . 243
cs_hru.ini . . . 243
cs_aqu.ini . . . 244
cs_channel.ini . . . 244
cs_res . . . 244
cs_irrigation . . . 245
fertilizer.frt_cs . . . 245
cs_reactions . . . 245
cs_uptake . . . 246
cs_plants_boron . . . 246
cs_urban . . . 246
cs_streamobs . . . 246
Example . . . 246
Related files . . . 247
Purpose . . . 248
Files and activation . . . 248
Daily sequence . . . 248
place_of_use.wro . . . 249
point_of_diver.wro . . . 251

water_hru_irr.wal . . . 251
wtps_wuses.wal . . . 252
water_treat.wal . . . 252
water_use.wal . . . 253
outside_src.wal . . . 253
om_treat.wal, om_use.wal and om_osrc.wal . . . 253
water_tower.wal . . . 254
water_canal.wal . . . 254
water_pipe.wal . . . 255
outside_rcv.wal . . . 255
Decision tables . . . 255
Limitations in 62.0.1 . . . 255
Example . . . 256
Outputs . . . 257
Related files . . . 257

This section describes the SWAT+ input files for revision 62.0.1. Each page gives the purpose of a file, its columns with units and defaults, how the model uses the values, an example from the reference datasets, and the files it is linked to. Where the code behaves in a way a user would not expect (a column that is read but has no effect, a default that differs from the one SWAT+ Editor writes), the page says so.

Every page was checked against the SWAT+ source at the commit recorded as `verified_at_commit` in its front matter. Where the source and older documentation disagree, the pages follow the source. The last section of each page, Sources, links to the Fortran routines and lists the variable names for developers.

`file.cio` names every other input file, so it is the place to start.

Older projects

A project set up for an earlier revision needs these changes before it runs correctly with 62.0.1:

- The `basin` line of `file.cio` now has three file names, and the `water_rights` line has two. An old `basin` line shifts every later line of `file.cio` by one without a message. See older file.cio files.
- The dynamic carbon model is now `carbon = 2` in `codes.bsn`. With `carbon = 1` no organic matter routine runs. See carbon model.
- `carb_coefs.cbn` is no longer read. Its settings moved to `carbon.bsn` and `carbon_lyr.bsn`, and its output switch to `print.prt`.

Simulation control

- `file.cio`: master index. Names the input file for every category, the weather data folders and the output folder.
- `time.sim`: simulation period and number of time steps per day.
- `print.prt`: warm-up years, daily print window and the output files written at each interval.
- `object.cnt`: basin name, areas and the number of spatial objects of each type.

Basin parameters

- `codes.bsn`: basin-wide method codes (PET method, channel routing, carbon model, runoff method and others).
- `parameters.bsn`: basin-wide coefficients for runoff, peak rate, nutrients, pesticides, routing, lapse rates, erosion and CO_2.
- `carbon.bsn`: parameters of the dynamic soil carbon model.
- `carbon_lyr.bsn`: decomposition rates and CO_2 fractions of the dynamic soil carbon model, for the surface layer and the deeper layers.

Spatial objects and connectivity

- Connect files: `hru.con`, `rout_unit.con`, `chandeg.con`, `aquifer.con`, `reservoir.con`, `recall.con` and the other connect files, which place each object, assign its weather station and route its outflow.
- `hru-data.hru`: the records each HRU uses for topography, hydrology, soil, land use, initial conditions, surface storage, snow and field geometry.
- Hydrology and topography: `hydrology.hyd`, `topography.hyd`, `field.fld` and `shade_factor.shf`.
- Recall: point sources and inlets given as time series.
- `hru-lte.hru`: the simplified HRU-lte objects.
- Routing and landscape units: `rout_unit.def`, `rout_unit.ele`, `rout_unit.rtu`, `ls_unit.def`, `ls_unit.ele` and `aqu_catunit.ele`.
- Link files: `chan-surf.lin` for overbank flow to floodplain HRUs and `aqu_cha.lin` for aquifer return flow to several channels.

Climate

- Climate files: weather stations (`weather-sta.cli`), weather generator statistics (`weather-wgn.cli`), the lists of observed data files and atmospheric deposition.
- `*.pcp`: measured daily or sub-daily precipitation for one gauge.
- `*.tmp`: measured daily maximum and minimum air temperature for one gauge.
- `*.tem`: temperature files with the `.tem` extension, read like `.tmp` files.
- `snow.sno`: snowfall, snowmelt and snow cover parameters.

Soils and land use

- `soils.sol`: soil profiles and layer properties.
- `landuse.lum`: land use and management records used by HRUs.
- Land use tables: curve number, conservation practice and overland flow Manning's n tables named in `landuse.lum`.

Channels

- Channel group: the `channel` line of `file.cio` and which of its files are used, initial channel contents, stream temperature and the legacy channel files.
- Chandeg channel files: `channel-lte.cha`, `hyd-sed-lte.cha`, `sed_nut.cha` and `nutrients.cha`, with the geometry, hydraulic, sediment and water quality parameters of the channels SWAT+ routes.

Aquifers, reservoirs and wetlands

- Aquifer files: `aquifer.aqu` and its initial conditions.
- Reservoir files: `reservoir.res` and its initial condition, hydrology, sediment, nutrient and weir databases.
- Wetland files: `wetland.wet` and `hydrology.wet`, for HRUs with ponded water including rice paddies.

Management

- Operation databases: harvest, grazing, irrigation, chemical application, fire and street sweeping.
- Decision tables: `lum.dtl`, `res_rel.dtl`, `scen_lu.dtl` and `flo_con.dtl`.
- `plants.plt`: plant parameter database.
- `fertilizer.frt`: fertilizer and manure databases.
- `tillage.til`: tillage implements and biological mixing.
- `carb_coefs.cbn`: no longer read in 62.0.1.
- `treatment.trt`: placeholder file, not read.
- Manure allocation: manure sources and the HRUs that draw from them.

- Management schedules and other inputs: `management.sch`, `puddle.ops`, `transplant.ops`, `co2_yr.dat` and `object.prt`.

Initial conditions

- Initialization files: `plant.ini`, `soil_plant.ini`, `nutrients.sol` and `om_water.ini`. Pesticide, pathogen, salt and constituent initial conditions are on the pages for those modules.

Structural practices

- Structural practice files: tile drains, septic systems, filter strips, grassed waterways, user-defined practices, the septic and urban databases, and saturated buffers.

Pollutants

- Pesticides: `pesticide.pes`, initial pesticide amounts and the metabolite chain.
- Pathogens: `pathogens.pth` and initial pathogen amounts.
- Heavy metals: heavy metal inputs.

Groundwater

- gwflow: the cell-based groundwater module, used in place of the lumped aquifers when `gwflow = 1` in `codes.bsn`.

Salts and constituents

- Salt module: inputs for the eight major salt ions.
- Constituent module: `constituents.cs` and the inputs for selenate, selenite and boron.

Water allocation

- Water allocation: places of use, points of diversion and return, and the treatment, use, tower, canal, pipe and outside-basin files.

  Sources

  - File names and their order in `file.cio`: `readcio_read.f90` and `input_file_module.f90`.
  - Each page lists the readers and the routines that use its inputs in its Sources section.

Purpose

`file.cio` is the master index of a SWAT+ project. It is the first file SWAT+ opens, and it must be in the run folder under that name. Each line names the files for one input category, so every other input file is reached through `file.cio`. The last six lines give optional folders for the weather data files and for the output files.

Format

Line 1 is a title and is skipped. Lines 2 to 26 each start with a category label and continue with file names in a fixed order. Lines 27 to 31 each give one weather data folder, and line 32 gives the output folder.

The reader takes the lines by position. The category label is read and discarded, so its text is not checked, and the file names are matched to slots by their order on the line. Values are separated by spaces.

- Write `null` for a file the project does not use. Most readers treat a name that does not exist in the run folder the same way as `null` and skip the file. The exceptions are listed under Missing and null files.
- File names can be up to 25 characters long. Longer names are cut to 25 characters.
- Each line must give at least as many names as its category has slots. If a line is short, the reader continues onto the next line to fill the missing slots, and every later line is then read one line out of step. Extra names at the end of a line are ignored.

Categories and default file names

The labels below are the ones the SWAT+ Editor writes. The default names are used only for slots that are never read, which happens when `file.cio` ends early. A project normally gives every name explicitly.

Line	Label	Files, in slot order (default names)	Documented in
2	`simulation`	`time.sim`, `print.prt`, `object.prt`, `object.cnt`, `constituents.cs`	time.sim, print.prt, object.prt, object.cnt, constituents.cs
3	`basin`	`codes.bsn`, `parameters.bsn`, `carbon.bsn`	codes.bsn, parameters.bsn, carbon.bsn
4	`climate`	`weather-sta.cli`, `weather-wgn.cli`, `pet.cli`, `pcp.cli`, `tmp.cli`, `slr.cli`, `hmd.cli`, `wnd.cli`, `atmodep.cli`	climate index files
5	`connect`	`hru.con`, `hru-lte.con`, `rout_unit.con`, `gwflow.con`, `aquifer.con`, `aquifer2d.con`, `channel.con`, `reservoir.con`, `recall.con`, `exco.con`, `delratio.con`, `outlet.con`, `chandeg.con`	connectivity files
6	`channel`	`initial.cha`, `channel.cha`, `hydrology.cha`, `sediment.cha`, `nutrients.cha`, `channel-lte.cha`, `hyd-sed-lte.cha`, `temperature.cha`	legacy channel files, channel tables
7	`reservoir`	`initial.res`, `reservoir.res`, `hydrology.res`, `sediment.res`, `nutrients.res`, `weir.res`, `wetland.wet`, `hydrology.wet`	reservoir files, wetland files
8	`routing_unit`	`rout_unit.def`, `rout_unit.ele`, `rout_unit.rtu`, `rout_unit.dr`	connectivity files
9	`hru`	`hru-data.hru`, `hru-lte.hru`	hru-data.hru
10	`exco`	`exco.exc`, `exco_om.exc`, `exco_pest.exc`, `exco_path.exc`, `exco_hmet.exc`, `exco_salt.exc`	connectivity files
11	`recall`	`recall.rec`	recall
12	`dr`	`delratio.del`, `dr_om.del`, `dr_pest.del`, `dr_path.del`, `dr_hmet.del`, `dr_salt.del`	connectivity files
13	`aquifer`	`initial.aqu`, `aquifer.aqu`	aquifer files
14	`herd`	`animal.hrd`, `herd.hrd`, `ranch.hrd`	not read
15	`water_rights`	`place_of_use.wro`, `point_of_diver.wro`	water allocation
16	`link`	`chan-surf.lin`, `aqu_cha.lin`	Link files
17	`hydrology`	`hydrology.hyd`, `topography.hyd`, `field.fld`	hydrology and topography
18	`structural`	`tiledrain.str`, `septic.str`, `filterstrip.str`, `grassedww.str`, `bmpuser.str`	structural BMPs
19	`hru_parm_db`	`plants.plt`, `fertilizer.frt`, `tillage.til`, `pesticide.pes`, `pathogens.pth`, `metals.mtl`, `salt.slt`, `urban.urb`, `septic.sep`, `snow.sno`	plants.plt, fertilizer.frt, tillage.til, pesticide.pes, pathogens.pth, metals.mtl, salt inputs, urban.urb and septic.sep, snow.sno
20	`ops`	`harv.ops`, `graze.ops`, `irr.ops`, `chem_app.ops`, `fire.ops`, `sweep.ops`	operation databases
21	`lum`	`landuse.lum`, `management.sch`, `cntable.lum`, `cons_practice.lum`, `ovn_table.lum`	landuse.lum, management.sch, lookup tables
22	`chg`	`cal_parms.cal`, `calibration.cal`, `codes.sft`, `wb_parms.sft`, `water_balance.sft`, `ch_sed_budget.sft`, `ch_sed_parms.sft`, `plant_parms.sft`, `plant_gro.sft`	parameter change files, soft calibration
23	`init`	`plant.ini`, `soil_plant.ini`, `om_water.ini`, `pest_hru.ini`, `pest_water.ini`, `path_hru.ini`, `path_water.ini`, `hmet_hru.ini`, `hmet_water.ini`, `salt_hru.ini`, `salt_water.ini`	initial conditions
24	`soils`	`soils.sol`, `nutrients.sol`, `soils_lte.sol`	soils.sol
25	`decision_table`	`lum.dtl`, `res_rel.dtl`, `scen_lu.dtl`, `flo_con.dtl`	decision tables
26	`regions`	`ls_unit.ele`, `ls_unit.def`, `ls_reg.ele`, `ls_reg.def`, `ls_cal.reg`, `ch_catunit.ele`, `ch_catunit.def`, `ch_reg.def`, `aqu_catunit.ele`, `aqu_catunit.def`, `aqu_reg.def`, `res_catunit.ele`, `res_catunit.def`, `res_reg.def`, `rec_catunit.ele`, `rec_catunit.def`, `rec_reg.def`	aquifer files, recall, soft calibration
27	`pcp_path`	folder for the precipitation data files	precipitation data

Line	Label	Files, in slot order (default names)	Documented in
28	`tmp_path`	folder for the temperature data files	temperature data
29	`slr_path`	folder for the solar radiation data files	climate index files
30	`hmd_path`	folder for the relative humidity data files	climate index files
31	`wnd_path`	folder for the wind speed data files	climate index files
32	any word	output folder	Output folder

Unused slots

These names are read from `file.cio`, but SWAT+ 62.0.1 never opens the file they name. Any name or `null` can be given.

- `connect`: `aquifer2d.con` (slot 6).
- `routing_unit`: `rout_unit.dr` (slot 4).
- `herd`: all three slots.
- `hru_parm_db`: `metals.mtl` and `salt.slt` (slots 6 and 7).
- `init`: `hmet_water.ini`, `salt_hru.ini` and `salt_water.ini` (slots 9 to 11).
- `regions`: `ls_cal.reg` and `ch_catunit.ele` (slots 5 and 6).

Two of these files are read under a fixed name instead. When salts are simulated, the HRU salt initial conditions come from a file named `salt_hru.ini` in the run folder, whatever slot 10 of the `init` line says. Channel catchment unit elements come from a file named `element.ccu`.

The `climate` line has no wind direction file. It has nine slots, and slot 3 is `pet.cli`.

Missing and null files

For most slots, a name of `null` or a name that does not exist in the run folder means the file is skipped and the related feature is off. Some files behave differently:

- `object.cnt`: if it is missing or `null`, SWAT+ prints `Cannot find object.cnt input file` and stops.
- `carbon.bsn`: read only when `carbon = 2` in `codes.bsn`. It is then required, and SWAT+ stops with an error, written to the screen and to `diagnostics.out`, if the file named in slot 3 of the `basin` line does not exist or cannot be opened. The per-layer carbon file has no slot on the line: its name is built from the carbon file name, so `carbon.bsn` gives `carbon_lyr.bsn`.
- `time.sim` and `print.prt`: there is no check for a missing file. If the name is not `null` and the file does not exist, SWAT+ creates an empty file of that name and continues with built-in values. For `time.sim` this leaves the run without a valid simulation period, so the file is required in practice.
- `place_of_use.wro` and `point_of_diver.wro`: if either is missing or `null`, SWAT+ creates an empty file of that name (a file called `null` in the second case), reads nothing from it, and water allocation is off. See water allocation.
- `chan-surf.lin`: if the name is not `null` and the file does not exist, SWAT+ creates an empty file of that name. See link files.

Weather data folders

Lines 27 to 31 let the precipitation, temperature, solar radiation, humidity and wind data files be kept outside the run folder. Each line has a label and one value of up to 80 characters. A value of `null` means the data files are in the run folder.

SWAT+ joins the folder and the file name from the climate index file (for example `pcp.cli`) without adding a separator, so the folder must end with `/` or `\`. The folder is read in the same way as a file name, with two consequences:

- A folder name that contains spaces must be in quotes.
- An unquoted `/` ends the value. `data/pcp/` is read as `data`, and an unquoted value that starts with `/` is read as empty, so the files are looked for in the run folder. Put any folder that uses forward slashes in quotes, for example `"C:/swat/weather/pcp/"`, or use backslashes on Windows.

There is no folder line for potential evapotranspiration data. Files listed in pet.cli are always read from the run folder.

Output folder

Line 32 is optional. It has a label (any word) and a folder name. SWAT+ reads the whole line, drops the first word and takes the rest of the line as the folder, so the folder name can contain spaces and forward slashes without quotes. Up to 256 characters are kept. If the line is missing or empty, or the folder is `null` or `NULL`, the output files are written to the run folder.

When a folder is given, SWAT+ converts the separators to the ones the operating system uses, creates the folder and any missing parent folders, and prints `Output path set to:` with the folder name. A relative folder is taken from the run folder. On Linux and macOS a folder that contains `:` is treated as a Windows path, and SWAT+ stops with an error. On Windows a folder that starts with `/` is placed on drive `C:`. SWAT+ also stops if it cannot create or open the folder.

The standard output files go to this folder, including `files_out.out`, `diagnostics.out`, `checker.out` and the time series outputs switched on in print.prt. In 62.0.1 the gwflow output files and the soft calibration output files (`hru-out.cal`, `hru-new.cal` and others) are still written to the run folder.

Older file.cio files

Two lines changed length in recent revisions, and a `file.cio` written for an earlier revision needs both updated before it is run with 62.0.1:

- The `basin` line now has three slots. Add carbon.bsn or `null` after parameters.bsn. With only two names on the `basin` line, SWAT+ takes the first word of line 4 (`climate`) as the carbon file name and discards the rest of that line. Every later category is then read from the line below the one it belongs to: the climate files are taken from the `connect` line, the connection files from the `channel` line, and so on. No message reports the shift. If `carbon = 2` in codes.bsn, SWAT+ stops with `ERROR: climate is required when carbon is enabled`; otherwise the run continues with wrong file names in every later slot.
- The `water_rights` line (line 15) now has two slots, the place-of-use file and the point-of-diversion file. An older line with `water_allocation.wro element.wro water_rights.wro` makes SWAT+ read `water_allocation.wro` as the place-of-use file and `element.wro` as the point-of-diversion file, and the third name is ignored. See water allocation for those files.

Example

`refdata/Osu_1hru/file.cio` in the SWAT+ repository (long lines cut at the right):

```
file.cio: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
simulation        time.sim          print.prt         null              object.cnt        null
basin             codes.bsn         parameters.bsn    null
climate           weather-sta.cli   weather-wgn.cli   null              pcp.cli           tmp.cli
connect           hru.con           null              rout_unit.con     null              aquifer.con
channel           initial.cha       null              null              null              nutrients.cha
reservoir         initial.res       null              null              sediment.res      nutrients.res
routing_unit      rout_unit.def     rout_unit.ele     rout_unit.rtu     null
hru               hru-data.hru      null
exco              null              null              null              null              null
recall            recall.rec
dr                null              null              null              null              null
aquifer           initial.aqu       aquifer.aqu
herd              null              null              null
water_rights      null              null              null
link              null              null
hydrology         hydrology.hyd     topography.hyd    field.fld
```

structural	tiledrain.str	septic.str	filterstrip.str	grassedww.str	bmpuser.str
hru_parm_db	plants.plt	fertilizer.frt	tillage.til	pesticide.pes	null
ops	harv.ops	graze.ops	irr.ops	chem_app.ops	fire.ops
lum	landuse.lum	management.sch	cntable.lum	cons_practice.lum	ovn_table.lum
chg	cal_parms.cal	calibration.cal	null	null	null
init	plant.ini	soil_plant.ini	om_water.ini	null	null
soils	soils.sol	nutrients.sol	null		
decision_table	lum.dtl	res_rel.dtl	null	null	
regions	ls_unit.ele	ls_unit.def	null	null	null
pcp_path	null				
tmp_path	null				
slr_path	null				
hmd_path	null				
wnd_path	null				

The `basin` line has three slots, with `null` for the carbon file because the project does not use the carbon model. The `climate` line uses measured precipitation, temperature, solar radiation and wind; the PET, humidity and atmospheric deposition files are not used. The `water_rights` line still has the older three slots, all `null`; the first two switch water allocation off and the third is ignored. All weather data files are in the run folder, and there is no output folder line, so the outputs are written to the run folder.

In `refdata/Ames_sub1/file.cio` the `aquifer` line reads `aquifer nbull null`. `nbull` is taken as a file name. No file of that name exists, so the initial aquifer file is skipped in the same way as for `null`.

Related files

- `time.sim`, `print.prt` and `object.cnt`: the simulation control files named on line 2.
- `codes.bsn`: its `carbon` code decides whether the carbon file on line 3 is read.
- Input reference: every input file reached from `file.cio`.

Sources

- Reader: `readcio_read.f90`, called first from `proc_bsn.f90`
- Slot names and defaults: `input_file_module.f90`
- Output folder: `output_path_module.f90`
- Weather folders: `cli_pmeas.f90`, `cli_tmeas.f90`, `cli_smeas.f90`, `cli_hmeas.f90`, `cli_wmeas.f90`, `cli_petmeas.f90`
- Carbon file check: `carbon_bsn_read.f90`
- Water allocation files: `water_allocation_read.f90`
- Fixed-name readers: `ch_read_elements.f90` (`element.ccu`), `salt_hru_read.f90` (`salt_hru.ini`)

Line	Fortran variable
2 `simulation`	`in_sim` (`time`, `prt`, `object_prt`, `object_cnt`, `cs_db`)
3 `basin`	`in_basin` (`codes_bas`, `parms_bas`, `carbon_bsn`)
4 `climate`	`in_cli`
5 `connect`	`in_con`
6 `channel`	`in_cha`
7 `reservoir`	`in_res`
8 `routing_unit`	`in_ru`
9 `hru`	`in_hru`
10 `exco`	`in_exco`
11 `recall`	`in_rec`
12 `dr`	`in_delr`
13 `aquifer`	`in_aqu`
14 `herd`	`in_herd`
15 `water_rights`	`in_wallo` (`pou`, `pod`)
16 `link`	`in_link`
17 `hydrology`	`in_hyd`
18 `structural`	`in_str`

Line	Fortran variable
19 `hru_parm_db`	`in_parmdb`
20 `ops`	`in_ops`
21 `lum`	`in_lum`
22 `chg`	`in_chg`
23 `init`	`in_init`
24 `soils`	`in_sol`
25 `decision_table`	`in_cond`
26 `regions`	`in_regs`
27 to 31	`in_path_pcp%pcp`, `in_path_tmp%tmp`, `in_path_slr%slr`, `in_path_hmd%hmd`, `in_path_wnd%wnd`
32	`out_path` in `output_path_module`

Purpose

`time.sim` sets the first and last day of the simulation and the number of time steps per day. Weather input, routing and every output file follow this period. The file is named in slot 1 of the `simulation` line of `file.cio` (default `time.sim`).

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Line 3 contains five integers in the order below. The reader does not match columns by header name.

#	Column	Units	Default	Description
1	`day_start`	day of year	1	Day of the year on which the simulation starts, 1 to 366. A value of 0 or less is set to 1 (1 January).
2	`yrc_start`	year	-	Calendar year in which the simulation starts.
3	`day_end`	day of year	0	Day of the year on which the simulation ends in `yrc_end`. A value of 0 or less means the last day of that year (31 December).
4	`yrc_end`	year	-	Calendar year in which the simulation ends.
5	`step`	time steps per day	1	Number of time steps per day for precipitation, surface runoff and routing. A value of 0 or less is set to 1 (daily).

Simulation period

The simulation runs over the calendar years `yrc_start` to `yrc_end`, which is

$$N_{yr} = \text{yrc_end} - \text{yrc_start} + 1$$

years. The first year starts on `day_start`, and every later year starts on 1 January. Every year but the last runs to 31 December. The last year ends on `day_end`, or on 31 December if `day_end` is 0. A `day_end` of 366 in a year that is not a leap year is treated as day 365. Leap years follow the Gregorian rule: years divisible by 4 are leap years, except century years that are not divisible by 400.

`day_start` and `day_end` make it possible to simulate water years or other periods that do not follow the calendar year. For example, `day_start = 274`, `yrc_start = 1999`, `day_end = 273` and `yrc_end = 2019` runs from 1 October 1999 to 30 September 2019. Neither year is a leap year, so day 274 is 1 October and day 273 is 30 September; in a leap year the same dates are days 275 and 274.

The warm-up years and the print window for daily output are set in `print.prt`. A zero `yrc_start` or `yrc_end` in `print.prt` is filled from this file.

Time step

`step` is the number of time steps in a day:

`step`	Time step
0 or 1	1 day
2	12 hours
24	1 hour
96	15 minutes
1440	1 minute

Any positive whole number works. The length of one time step in minutes is

$$\Delta t = \frac{1440}{\text{step}}$$

With `step` greater than 1, the day is split into sub-daily steps for precipitation, infiltration and surface runoff, hydrograph routing through channels and reservoirs, and sub-daily channel output. The Green and Ampt infiltration method (`gampt = 1` in `codes.bsn`) uses the sub-daily rainfall. Other processes, such as plant growth, soil water and nutrient cycling, still run once a day.

Sub-daily precipitation records (`*.pcp` files with a time step above 0) must have the same number of values per day as `step`. SWAT+ stores `step` values per day and does not check the record against it.

When `step` is greater than 1 and precipitation is generated, the weather generator also distributes each day's rainfall over the time steps. For a station with measured daily records, 62.0.1 does not fill the sub-daily values: the daily total is read, but the rainfall in each time step stays at zero. A sub-daily run therefore needs sub-daily precipitation records or generated precipitation.

Weather outside the observed record

When the simulation starts before, or ends after, the measured daily weather records listed in the climate index files, SWAT+ generates the weather for the days outside the record with the weather generator (`weather-wgn.cli`). The same applies to days that are flagged as missing in a record.

Example

`refdata/Ames_sub1/time.sim` in the SWAT+ repository:

```
time.sim:
day_start  yrc_start   day_end   yrc_end      step
        0       1975         0      2020         0
```

The simulation runs from 1 January 1975 to 31 December 2020, 46 years. `day_start` and `day_end` are 0, so the first and last days of the year are used. `step` is 0, which SWAT+ sets to 1, so the simulation is daily.

Related files

- `file.cio`: names this file.
- `print.prt`: warm-up years and print window for the outputs inside this period.
- `pcp.cli` and the precipitation data files: their time step must match `step` for sub-daily records.
- `codes.bsn`: selects Green and Ampt infiltration, which uses sub-daily rainfall.

Sources

- Reader: `time_read.f90`
- Type definition: `type time_current` in `time_module.f90`

- Time step length: proc_bsn.f90
- Year and day loop, leap years, end day: time_control.f90
- Measured and generated precipitation: cli_pmeas.f90, cli_precip_control.f90
- File name slot: `in_sim%time` in input_file_module.f90

Column	Fortran variable
day_start	time%day_start
yrc_start	time%yrc_start
day_end	time%day_end
yrc_end	time%yrc_end
step	time%step

Purpose

`print.prt` controls the output of a SWAT+ run. It sets the number of warm-up years left out of the outputs, the period and spacing of daily output, the extra output formats, a few special output files, and one row per output group with switches for daily, monthly, yearly and average annual files. The file is named in slot 2 of the `simulation` line of file.cio (default `print.prt`).

If the name in file.cio is `null`, SWAT+ uses the built-in values: no warm-up, every output group off, and crop yields written as average annual values only. A name that does not exist in the run folder has the same effect, and SWAT+ creates an empty file of that name. The files listed in Output naming convention are written in every case.

Format

Free format, whitespace-separated. The file has four short blocks, each a header line followed by a line of values, then a header line and one row per output group. Line 1 is a title. All header lines are skipped, and values are read by position within each line.

Line	Content
1	title
2	header
3	`nyskip`, `day_start`, `yrc_start`, `day_end`, `yrc_end`, `interval`
4	header
5	`aa_int_cnt`, followed by `aa_int_cnt` end years
6	header
7	`csvout`, `use_obj_lbls`, `cdfout`
8	header
9	`crop_yld`, `mgtout`, `hydcon`, `fdcout`
10	header (`objects daily monthly yearly avann`)
11 and on	output group rows

Print period (line 3)

#	Column	Units	Default	Description
1	`nyskip`	years	0	Number of years at the start of the simulation left out of all outputs (warm-up period).
2	`day_start`	day of year	1	Day of the year on which daily output starts. 0 means 1.
3	`yrc_start`	year	first simulated year	Year in which daily output starts. 0 means the first year in time.sim.
4	`day_end`	day of year	366	Day of the year on which daily output ends. 0 means 366, the end of the year.

#	Column	Units	Default	Description
5	`yrc_end`	year	after the last simulated year	Year in which daily output ends. 0 means daily output runs to the end of the simulation.
6	`interval`	days	1	Days between daily output rows. 0 or less means 1. With 2, every other day is written.

A warm-up period lets soil water, nutrient pools and plant cover settle from their initial values before results are used. It matters more for short simulations; for a simulation of five years or less, a warm-up period is recommended.

Average annual intervals (line 5)

#	Column	Units	Default	Description
1	`aa_int_cnt`	count	0	Number of average annual print intervals. When it is above 0, the end year of each interval follows on the same line, in order.

For example, `3 1995 2005 2015` would end intervals in 1995, 2005 and 2015. In 62.0.1 the end years are read but have no effect on the outputs: every average annual file covers the whole printed period. See Average annual values.

Output formats (line 7)

#	Column	Values	Default	Description
1	`csvout`	`y`, `n`	`n`	`y` also writes a comma-separated `.csv` copy of most output files.
2	`use_obj_lbls`	`y`, `n`	`n`	How the output group rows are read: `y` by name, `n` by position. See Output group rows.
3	`cdfout`	`y`, `n`	`n`	netCDF output. Read and not used.

`print.prt` files written by older versions of the SWAT+ Editor have the header `csvout dbout cdfout` on line 6. SWAT+ reads the second value as `use_obj_lbls` whatever the header says, so an old file with `n` there is read by position.

Special outputs (line 9)

#	Column	Values	Default	Description
1	`crop_yld`	`a`, `y`, `b`, `n`	`a`	Crop yield files: `a` average annual (`crop_yld_aa`), `y` yearly (`crop_yld_yr` and the basin crop yield files), `b` both, `n` none.
2	`mgtout`	`y`, `n`	`n`	`y` writes `mgt_out.txt`, the log of management operations, decision table actions and water allocation deliveries.
3	`hydcon`	`y`, `n`	`n`	`y` writes `hydcon.out`, the list of hydrograph connections between objects. It is mostly useful for finding connection errors.
4	`fdcout`	`y`, `n`	`n`	`y` computes flow duration curves and a flashiness index for SWAT+ channels and writes them to `flow_duration_curve.out`.

Older Editor versions write the header `soilout` in place of `crop_yld`; the value is read as `crop_yld`.

Output group rows

Each row from line 11 on has a group name and four switches, for the daily, monthly, yearly and average annual files of that group. A switch of `y` opens the file for that interval and any other value leaves it off. The legacy carbon rows `hru_cb` and `hru_cb_vars` also accept `l`. The files each row opens, and the pages that describe them, are listed in Output naming convention.

How the rows are read depends on `use_obj_lbls` on line 7:

- `use_obj_lbls = y`: each row is matched by its name, so rows can be in any order and rows can be left out. A row that is left out has all four switches off. A name SWAT+ does not know stops the run with `Error: The output object <name> in the input file print.prt is not a valid object`, and a name that appears twice stops it with `print object is duplicated`. Names are case-sensitive.
- `use_obj_lbls = n`: the rows are read in the fixed order of the table below, and the names in the first column are not checked. A missing row or an extra row shifts every later row, so its switches are applied to the wrong output group, with no message. Rows after the end of the file are off. The carbon and gwflow rows cannot be set in this mode.

Order	Row	Output
1	`basin_wb`	basin water balance
2	`basin_nb`	basin nutrient balance
3	`basin_ls`	basin losses from the land
4	`basin_pw`	basin plant and weather
5	`basin_aqu`	basin aquifers
6	`basin_res`	basin reservoirs
7	`basin_cha`	basin legacy channels
8	`basin_sd_cha`	basin SWAT+ channels, channel morphology and channel budget
9	`basin_psc`	basin point sources (recall objects summed)
10 to 17	`region_wb`, `region_nb`, `region_ls`, `region_pw`, `region_aqu`, `region_res`, `region_sd_cha`, `region_psc`	read, but no region files are written in 62.0.1
18	`water_allo`	water allocation, water use and treatment
19 to 22	`lsunit_wb`, `lsunit_nb`, `lsunit_ls`, `lsunit_pw`	landscape unit water balance, nutrient balance, losses, plant and weather
23 to 26	`hru_wb`, `hru_nb`, `hru_ls`, `hru_pw`	HRU water balance, nutrient balance (with the nitrogen cycle file), losses, plant and weather
27 to 30	`hru-lte_wb`, `hru-lte_nb`, `hru-lte_ls`, `hru-lte_pw`	HRU-lte water balance, losses, plant and weather; `hru-lte_nb` is read but writes nothing
31	`channel`	legacy channels
32	`channel_sd`	SWAT+ channels, channel morphology, channel budget, and the sediment budget files per channel, per stream order and for the basin
33	`aquifer`	aquifers
34	`reservoir`	reservoirs and wetlands
35	`recall`	recall objects (point sources and inlets)
36	`hyd`	inflow and outflow hydrographs of every object, and deposition
37	`ru`	routing unit hydrographs
38	`pest`	pesticides in HRUs, channels, reservoirs, aquifers and the basin
39 to 45	`basin_salt`, `hru_salt`, `ru_salt`, `aqu_salt`, `channel_salt`, `res_salt`, `wetland_salt`	salts
46 to 52	`basin_cs`, `hru_cs`, `ru_cs`, `aqu_cs`, `channel_cs`, `res_cs`, `wetland_cs`	constituents; `res_cs` also opens the wetland constituent files, and `wetland_cs` writes nothing

These rows are recognised only when `use_obj_lbls = y`:

Row	Output
`hru_cb_gl`, `hru_cb_trf`, `hru_cb_lyr`, `hru_cb_cpool`, `hru_cb_npool`, `hru_cb_plt`, `hru_cb_flux`, `hru_cb_drv`, `hru_cb_dyn`, `hru_cb_snap`	HRU carbon output, one row per file family. See Carbon outputs.
`lsu_cb_gl`, `lsu_cb_trf`, `lsu_cb_plt`	landscape unit carbon output, area-weighted from the HRUs. See Carbon outputs.
`hru_cb`, `hru_cb_vars`	legacy CSU carbon files, switched with `y` or `1`. See Legacy CSU carbon outputs.
`gwflow_wb`, `gwflow_flux`, `gwflow_heat`, `gwflow_solute`, `gwflow_obs`, `gwflow_pump`	gwflow output. A row that is present replaces the setting from the gwflow input files; a row that is absent leaves it. See gwflow output switches.

Some rows write data only when the project has the matching objects. The `channel_sd` and `basin_sd_cha` files need SWAT+ channels, and the `channel` and `basin_cha` files need legacy channels. The basin landscape files (`basin_wb`, `basin_nb`, `basin_ls`, `basin_pw`) are summed from the landscape unit element file `ls_unit.ele`, and the landscape unit files need `ls_unit.def`; both are named on the `regions` line of `file.cio`.

Print timing

Every output file leaves out the first `nyskip` years of the simulation. After those years:

- Daily rows are written from `day_start` of `yrc_start` until `day_end` of `yrc_end`, every `interval` days, starting with the first day of the window. Output starts on the first simulated day whose day of the year is at least `day_start` in a year at or after `yrc_start`. If the simulation starts in a later year than `yrc_start`, daily output therefore starts on day `day_start` of the first simulated year. Days before it in that year are not written.
- Monthly rows are written at the end of each month, yearly rows at the end of each year, and average annual rows once, at the end of the simulation.

The print window applies to daily rows only. Monthly, yearly and average annual files always cover all years after the warm-up.

Average annual values

The average annual files give the sum over the printed period divided by the number of printed years Y. SWAT+ counts the printed days D (all days after the warm-up) and divides by 365:

$$Y = \frac{D}{365}$$

Leap days are counted in D, so Y is slightly larger than the number of calendar years. When `nyskip` is above 0, 62.0.1 starts the count at the number of simulated years N_{yr} instead of 0:

$$Y = \frac{D + N_{yr}}{365}$$

For a 10-year simulation with 2 warm-up years, Y is about 8.03 instead of 8.00, and the average annual values are about 0.3% lower than the true averages over the printed years. If `nyskip` is equal to or larger than the number of simulated years, no output is written.

Example

`refdata/Ames_sub1/print.prt` in the SWAT+ repository:

```
print.prt:
nyskip      day_start  yrc_start  day_end   yrc_end   interval
0           0               1975         0      2020          1
```

```
aa_int_cnt
0
csvout        use_obj_lbls  cdfout
n             y             n
crop_yld      mgtout        hydcon        fdcout
b             y             n             n
objects                  daily       monthly        yearly         avann
hru_wb                     y             n             n             y
hru_nb                     y             n             n             y
hru_ls                     n             n             n             y
hru_pw                     y             n             n             y
hru_cb                     l             y             y             y
hru_cb_vars                l             y             y             y
```

There is no warm-up period. Daily output covers the whole simulation (1975 to 2020) at an interval of one day. There is no `.csv` output. Crop yields are written both yearly and as average annual values, and the management log `mgt_out.txt` is written.

`use_obj_lbls = y`, so the rows are read by name and every row not listed is off. The HRU water balance, nutrient balance, and plant and weather files are written daily and as average annual values; the HRU losses only as average annual values. The `hru_cb` and `hru_cb_vars` rows switch on the legacy carbon files, with `l` in the daily column for the full set of layer diagnostics. The `hru_cb_vars` files are written only when the carbon model is on (`carbon = 2` in `codes.bsn`).

`refdata/0su_1hru/print.prt`, written by an older SWAT+ Editor, has `dbout` in place of `use_obj_lbls` with the value `n`, so its rows are read by position. Its list has an `hru_cb` row after `hru_pw`, which is not part of the fixed order: the reader applies that row to `hru-lte_wb` and every later row moves down one group. All rows in that file have the same switches (`n n n y`), so the shift has no visible effect there.

Related files

- `file.cio`: names this file, the output folder, and the landscape unit files the basin and landscape unit outputs need.
- `time.sim`: the simulation period that contains the print period.
- Output naming convention: file names and the files each row opens.
- Output reference: contents of each output file.

Sources

- Reader: `basin_print_codes_read.f90`, called from `proc_bsn.f90` after `time_read`
- Type definitions: `type basin_print_codes` and `type print_interval`, and `print_prt_error`, in `basin_module.f90`
- Print window, warm-up and average annual years: `time_control.f90`
- Output calls and the `nyskip` test: `command.f90`
- File opening: `output_landscape_init.f90`, `header_write.f90`, `header_sd_channel.f90`, `header_yield.f90`, `header_mgt.f90`, `header_hyd.f90`
- gwflow overrides: `gwflow_read.f90`
- Legacy carbon flags: `carbon_legacy_module.f90`
- Region output (not called): `lsreg_output.f90`

Column or row	Fortran variable
`nyskip`, `day_start`, `yrc_start`, `day_end`, `yrc_end`, `interval`	`pco%nyskip`, `pco%day_start`, `pco%yrc_start`, `pco%day_end`, `pco%yrc_end`, `pco%int_day`
`aa_int_cnt` and end years	`pco%aa_numint`, `pco%aa_yrs`
`csvout`, `use_obj_lbls`, `cdfout`	`pco%csvout`, `pco%use_obj_labels`, `pco%cdfout`
`crop_yld`, `mgtout`, `hydcon`, `fdcout`	`pco%crop_yld`, `pco%mgtout`, `pco%hydcon`, `pco%fdcout`
`basin_wb` … `basin_psc`	`pco%wb_bsn`, `nb_bsn`, `ls_bsn`, `pw_bsn`, `aqu_bsn`, `res_bsn`, `chan_bsn`, `sd_chan_bsn`, `recall_bsn`

Column or row	Fortran variable
region_wb … region_psc, water_allo	pco%wb_reg, nb_reg, ls_reg, pw_reg, aqu_reg, res_reg, sd_chan_reg, recall_reg, water_allo
lsunit_*, hru_*, hru-lte_*	pco%{wb,nb,ls,pw}_lsu, pco%{wb,nb,ls,pw}_hru, pco%{wb,nb,ls,pw}_sd
channel, channel_sd, aquifer, reservoir, recall, hyd, ru, pest	pco%chan, sd_chan, aqu, res, recall, hyd, ru, pest
salt rows	pco%salt_basin, salt_hru, salt_ru, salt_aqu, salt_chn, salt_res, salt_wet
constituent rows	pco%cs_basin, cs_hru, cs_ru, cs_aqu, cs_chn, cs_res, cs_wet
hru_cb, hru_cb_vars, hru_cb_<family>, lsu_cb_<family>	pco%cb_hru, pco%cb_vars_hru, pco%cb_<family>_hru, pco%cb_<family>_lsu
gwflow_<part>	pco%gwflow_<part>

Purpose

object.cnt gives the basin name, the basin areas and the number of spatial objects of each type: HRUs, routing units, aquifers, channels, reservoirs, point sources and the others listed below. SWAT+ uses the counts to set up its list of objects before any connection file is read. The file is named in slot 4 of the simulation line of file.cio (default object.cnt). It is required: if it is missing or null, SWAT+ prints Cannot find object.cnt input file and stops.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Line 3 contains 21 values in the order below. The reader does not match columns by header name.

#	Column	Units	Default	Description
1	name	-	blank	Basin name, up to 25 characters. It appears in the name column of the basin output files.
2	ls_area	ha	0	Land area of the basin.
3	tot_area	ha	0	Total area of the basin, including reservoirs and other water bodies.
4	obj	count	0	Total number of spatial objects. Must equal the sum of columns 5 to 18.
5	hru	count	0	Number of HRUs (hru.con).
6	lhru	count	0	Number of HRU-lte objects (hru-lte.con).
7	rtu	count	0	Number of routing units (rout_unit.con).
8	mfl	count	0	Number of gwflow objects (gwflow.con). See gwflow.
9	aqu	count	0	Number of aquifers (aquifer.con).
10	cha	count	0	Number of legacy channels (channel.con).
11	res	count	0	Number of reservoirs (reservoir.con).
12	rec	count	0	Number of recall objects: point sources and inlets (recall.con).
13	exco	count	0	Number of export coefficient objects (exco.con).
14	dlr	count	0	Number of delivery ratio objects (delratio.con).
15	can	count	0	Number of canals. No connection file is read for canals in 62.0.1.
16	pmp	count	0	Number of pumps. No connection file is read for pumps in 62.0.1.
17	out	count	0	Number of outlets (outlet.con).
18	lcha	count	0	Number of channels routed with the SWAT+ channel method (chandeg.con).

#	Column	Units	Default	Description
19	aqu2d	count	0	Number of 2D aquifers. Read and not used.
20	hrd	count	0	Number of herds. Read and not used.
21	wro	count	0	Number of water rights objects. Read and not used.

The column headers are the ones the SWAT+ Editor writes. The old GitBook documentation calls column 8 `gwfl`. Only the order matters to SWAT+.

Object counts

Each count must match the number of objects in the corresponding connection file named on the `connect` line of `file.cio`. SWAT+ numbers all objects in one list, in this order: HRUs, HRU-lte objects, routing units, gwflow objects, aquifers, legacy channels, reservoirs, recall objects, export coefficient objects, delivery ratio objects, canals, pumps, outlets and SWAT+ channels. The object list has `obj` entries, so `obj` must equal the sum of these counts. Counts for canals and pumps reserve places in the list, but no objects are read into them.

If line 3 ends early, the reader reaches the end of the file and the missing counts stay at zero. The `refdata/Osu_1hru` example uses this and leaves out `hrd` and `wro`. If the file has more lines after line 3, SWAT+ keeps reading them in groups of title, header and values, and the last complete group is the one used. A file with a single data line avoids this.

Basin areas

`ls_area` and `tot_area` are not used to compute HRU or object areas; those come from the connection files. They are used in these places:

- `ls_area`: the HRU area when a decision table changes the fraction of the basin an HRU covers, and the HRU areas of the regions in soft calibration (`water_balance.sft`, see soft calibration).
- `tot_area`: the area weights for the basin aquifer output (`basin_aqu_*` files), and the conversion of gwflow basin water balance volumes to depths.

gwflow

When `gwflow = 1` in `codes.bsn`, SWAT+ looks for `chancell.gw` in the run folder after reading `object.cnt`:

- If `chancell.gw` exists and `mfl` is 0, SWAT+ counts the channel cell rows in `chancell.gw` and uses that count as the number of gwflow objects. It adds that count to `obj`, removes the aquifers from `obj`, sets the aquifer count to 0 and reads the gwflow connections from `gwflow.con` in place of `aquifer.con`.
- If `chancell.gw` exists and `mfl` is already above 0, SWAT+ keeps `mfl` and `obj` as given and sets the aquifer count to 0. In that case `obj` should already leave out the aquifers.
- If `chancell.gw` is missing or has no data after its header, SWAT+ turns gwflow off and the run uses the lumped aquifers.

In all three cases SWAT+ opens the `gwflow_record` log file. See gwflow for the gwflow input files.

Example

`refdata/Ames_sub1/object.cnt` in the SWAT+ repository:

```
object.cnt:
name                    ls_area      tot_area       obj       hru      lhru       rtu       mfl       aqu
demo                       1.          1            12         12         0         0         0         0
```

The basin is named `demo` and both areas are 1 ha. The project has 12 objects, all of them HRUs, so `obj` equals `hru`. There are no routing units, aquifers or channels.

`refdata/Osu_1hru/object.cnt` has 4 objects: one HRU, one routing unit, one aquifer and one SWAT+ channel (`lcha`). Its data line stops after `aqu2d`, and the `hrd` and `wro` counts are taken as zero.

Related files

- `file.cio`: names this file and the connection files.
- Connection files: one row per object; the number of rows must match the counts here.
- `codes.bsn`: its `gwflow` code starts the gwflow check above.
- gwflow: `chancell.gw` and the other gwflow inputs.

Sources

- Reader: `basin_read_objs.f90`, called from `proc_bsn.f90` after `codes.bsn`
- Type definitions: `type basin_inputs` in `basin_module.f90`, `type spatial_objects` in `hydrograph_module.f90`
- Object numbering and connection files: `hyd_connect.f90`
- Uses of the areas: `hru_fr_change.f90`, `lcu_read_softcal.f90`, `basin_aquifer_output.f90`, `gwflow_output.f90`
- File name slot: `in_sim%object_cnt` in `input_file_module.f90`

Column	Fortran variable
name	bsn%name
ls_area	bsn%area_ls_ha
tot_area	bsn%area_tot_ha
obj	sp_ob%objs
hru	sp_ob%hru
lhru	sp_ob%hru_lte
rtu	sp_ob%ru
mfl	sp_ob%gwflow
aqu	sp_ob%aqu
cha	sp_ob%chan
res	sp_ob%res
rec	sp_ob%recall
exco	sp_ob%exco
dlr	sp_ob%dr
can	sp_ob%canal
pmp	sp_ob%pump
out	sp_ob%outlet
lcha	sp_ob%chandeg
aqu2d	sp_ob%aqu2d
hrd	sp_ob%herd
wro	sp_ob%wro

Purpose

`codes.bsn` contains the method codes that apply to the whole watershed: the potential evapotranspiration (PET) method, the channel routing method, the soil carbon model, the surface runoff method, the tile drainage equations, plant stress switches and a few output options. It is named in the first position of the `basin` line of `file.cio` (default `codes.bsn`). SWAT+ reads it at the start of the run, before the inputs that depend on these codes.

If the file named in `file.cio` does not exist, every code keeps the built-in value listed below.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Line 3 contains 26 values in the order below. The reader matches values by position only, so all 26 must be present and in this order. The table gives the column headers the SWAT+ Editor writes.

#	Column	Type	Built-in value	Description
1	`pet_file`	text	`pet.cli`	Not used. Measured PET is set up through `file.cio` and `weather-sta.cli` (see PET method).
2	`wq_file`	text	blank	Not used.
3	`pet`	integer	0	PET method: `0` Priestley-Taylor, `1` Penman-Monteith, `2` Hargreaves, `3` PET of the weather station. See PET method.
4	`event`	integer	0	Extra column switch for two other inputs. `0`: normal format. Any other value: `plants.plt` is read with an extra plant class column and `soil_plant.ini` with an extra constituent column. See Extra input columns.
5	`crack`	integer	0	Crack flow: `0` off, `1` on. See Crack flow.
6	`swift_out`	integer	0	SWIFT input files: `0` not written, `1` written to a `SWIFT` folder at the end of the run. Older Editor versions label this column `rtu_wq`.
7	`sed_det`	integer	0	Peak runoff rate method for erosion: `0` NRCS method with the peak rate factor, `1` half-hour rainfall intensity method. See Peak runoff rate.
8	`rte_cha`	integer	0	Channel water routing: `0` variable storage, `1` Muskingum. See Channel routing.
9	`deg_cha`	integer	0	Not used.
10	`wq_cha`	integer	0	Not used.
11	`nostress`	integer	0	Plant stress: `0` all stresses applied, `1` all stresses turned off (water, temperature, nitrogen, phosphorus, aeration and salt), `2` nitrogen and phosphorus stress turned off.
12	`cn`	integer	0	Not used.
13	`c_fact`	integer	0	Read, then overridden: the model always uses the residue and biomass cover method for the USLE C factor. See USLE C factor.
14	`carbon`	integer	0	Soil carbon model: `0` static carbon (original SWAT humus and residue routines), `1` reserved for C-FARM (not implemented), `2` dynamic carbon model with CENTURY pools. See Carbon model.
15	`lapse`	integer	0	Elevation adjustment of HRU precipitation and temperature: `0` off, `1` on. See Lapse rates.
16	`uhyd`	integer	1	Unit hydrograph shape for sub-daily runs: `0` triangular, `1` gamma function. See Unit hydrograph.
17	`sed_cha`	integer	0	Not used.
18	`tiledrain`	integer	0	Tile flow equation: `0` drawdown-days equation, `1` DRAINMOD equations.

#	Column	Type	Built-in value	Description
19	`wtable`	integer	0	Shallow water table depth: `0` original routine (water above field capacity fills the profile from the bottom), `1` DRAINMOD water table routine.
20	`soil_p`	integer	0	Soil phosphorus model: `0` original SWAT model with a fixed P availability index, `1` dynamic P availability of Vadas and White (2010).
21	`gampt`	integer	0	Infiltration and surface runoff: `0` curve number, `1` Green-Ampt. See Runoff method.
22	`atmo_dep`	single character	`a`	Not used.
23	`stor_max`	integer	0	Not used.
24	`i_fpwet`	integer	0	In-stream nutrient method: `0` QUAL2E, `1` QUAL2E followed by simplified first-order transformations. Called `qual2e` in earlier documentation. See In-stream nutrients.
25	`gw_flow`	integer	0	Gridded groundwater module: `0` off, `1` on. Called `gwflow` in earlier documentation, where it was described as flood routing. See gwflow module.
26	`idc_till`	integer	3	Tillage effect on decomposition in the dynamic carbon model (`carbon = 2`): `1` DSSAT, `2` EPIC (placeholder), `3` Kemanian, `4` DNDC (placeholder). See Tillage effect on decomposition.

The 26 values are read in one free-format read. A line with fewer values continues onto the next line of the file, and the codes that are not reached keep their built-in values. A value that cannot be read (text in an integer column, for example) ends the read without a message, so the codes after it also keep their built-in values.

PET method

`pet` selects how daily potential evapotranspiration is calculated for each HRU:

`pet`	Method
0	Priestley-Taylor, with $\alpha = 1.28$.
1	Penman-Monteith, with a 40 cm alfalfa reference crop and a canopy resistance that depends on the year's CO2 concentration. This method also gives the maximum transpiration of the growing plants.
2	Hargreaves.
3	The daily PET value of the HRU's weather station.

In every case the result is multiplied by the HRU's `pet_co` from `hydrology.hyd`.

With `pet = 3`, the HRU uses the daily PET stored for its weather station. A station gets measured PET from the PET gage named in `weather-sta.cli`, and the gage files are listed in `pet.cli`, which is named on the `climate` line of `file.cio` (see climate files). In 62.0.1, however, the station PET is recalculated with the Hargreaves equation later in the same daily climate step, for every station, before the HRUs use it (the recalculated value feeds the 30-day precipitation to PET ratio). Measured PET is therefore replaced by Hargreaves PET with a minimum of 0.01 mm, and `pet = 3` gives Hargreaves PET times `pet_co`. Channel and pond evaporation and aquifer revap use the same station value under every `pet` code.

With `pet = 3` the model also opens a file called `pet.cli` in the run folder and reads its first three lines. The file is not used after that.

Extra input columns

`event` is not an event switch. When it is not 0:

- each row of `plants.plt` is read with one more value at the end, the plant class (for example row crop, tree, grass). Calibration conditions can select plants by this class.
- each row of `soil_plant.ini` is read with one more name at the end, a constituent initial-condition record (see initial conditions).

Files written by the SWAT+ Editor do not have these columns, so `event` should be 0 for them.

Crack flow

With `crack = 1`, SWAT+ calculates a daily crack volume for each HRU from the soil's crack volume potential (`crk` in `soils.sol`) and the layer water contents. Surface runoff is reduced by the crack volume (in sub-daily runs, the runoff of each time step in turn until the volume is filled), and water entering the soil flows through the cracks before the layer-by-layer percolation.

Peak runoff rate

The peak runoff rate is used in the MUSLE sediment yield of each HRU.

With `sed_det = 0`, the peak rate comes from the NRCS method:

$$q_{peak} = \frac{\text{prf}}{6578.6} \cdot \frac{A\, Q_{surf}}{t_{conc}} \cdot \frac{1}{35.3}$$

where q_{peak} is in m³/s, A is the HRU area (ha), Q_{surf} is the surface runoff (mm), t_{conc} is the time of concentration (h) and `prf` is the peak rate factor in `parameters.bsn`. The factor 6578.6 converts ha · mm to mi² · in, and 35.3 converts ft³/s to m³/s. Wetlands always use this equation, whatever the value of `sed_det`.

With `sed_det = 1`, the half-hour rainfall intensity method is used:

$$\alpha_{tc} = 1 - \exp[2\, t_{conc}\, \ln(1 - \alpha_{0.5})]\,, \qquad q_{peak} = \frac{\alpha_{tc}\, Q_{surf}\, A_{km}}{3.6\, t_{conc}}$$

where $\alpha_{0.5}$ is the day's fraction of rain falling in the maximum half hour (from the weather generator, scaled by `adj_pkrt` in `parameters.bsn`) and A_{km} is the HRU area in km².

Channel routing

`rte_cha` selects how water is routed through channels.

With `rte_cha = 0` (variable storage), the fraction of channel storage that leaves the reach in each routing step is

$$C_{sto} = \min\left(1,\ \text{scoef} \cdot \frac{2\,\Delta t}{2\,TT + \Delta t}\right)$$

where Δt is the routing step (h), TT is the travel time at the outflow of the previous step (h) and `scoef` is the channel storage coefficient in `parameters.bsn`. Daily runs use one routing step per day.

With `rte_cha = 1` (Muskingum), the outflow of a step is $O_2 = C_1 I_2 + C_2 I_1 + C_3 O_1$. The storage time constant is a weighted mean of the values at bankfull and at 0.1 bankfull depth, with the weights `msk_co1` and `msk_co2` (scaled

to sum to 1), and `msk_x` is the weighting factor; all three are in `parameters.bsn`. The day is divided into enough substeps to keep the scheme stable.

USLE C factor

`c_fact` is replaced by 1 each time the C factor is calculated, so the model always uses the method based on residue and growing biomass cover (from APEX). The residue part of that method uses `rsd_cover` in `parameters.bsn`. The older method based on the plants' minimum C factor is not used in 62.0.1.

Carbon model

`carbon` selects the soil organic matter model:

- `0`: static carbon. Soil organic matter is kept in active and stable humus pools and in residue, with the original SWAT residue decomposition, mineralization and annual biological mixing.
- `2`: dynamic carbon model with CENTURY litter, microbial and humus pools. The model requires `carbon.bsn` and `carbon_lyr.bsn` and stops with an error if either is missing. Tillage, fertilizer and manure applications, grazing and the daily soil processes use the carbon pool versions of their routines, and the carbon outputs selected in `print.prt` become available (see carbon outputs).
- `1`: reserved for C-FARM, which is not implemented. No organic matter routine runs under this code: the soil humus pools are not initialised, and residue decomposition and N and P mineralization are skipped for the whole run.

The numbering changed in revision 62. The dynamic carbon model used to be selected with `carbon = 1`. A `codes.bsn` from an older dataset that still has `carbon = 1` now runs with no organic matter cycling, so it must be changed to `2` (dynamic model) or `0` (static model). See soil carbon for the model description.

Lapse rates

With `lapse = 1`, each HRU's daily precipitation and temperatures are adjusted for the difference between the HRU's elevation and the elevation of the gage or weather generator station that supplies the data:

$$P' = \max\!\left(0,\; P + \text{plaps}\cdot\frac{z_{HRU} - z_{gage}}{1000}\right), \qquad T' = T + \text{tlaps}\cdot\frac{z_{gage} - z_{HRU}}{1000}$$

with elevations in m, `plaps` in mm/km and `tlaps` in °C/km from `parameters.bsn`. The precipitation adjustment is a fixed depth added on every day, wet or dry, so with a positive `plaps` an HRU above its gage also receives rain on dry days. The temperature adjustment applies to the maximum, minimum and mean temperature. Reservoirs and channels use the station weather without adjustment.

Unit hydrograph

In sub-daily runs (more than one time step per day in `time.sim`) with the curve number method (`gampt = 0`), daily surface runoff from HRUs and routing units is spread over the time steps with a unit hydrograph. The base time and time to peak are

$$t_b = \min(48,\; 0.5 + 0.6\,t_{conc} + \text{adj_uhyd}), \qquad t_p = 0.375\,t_b$$

in hours, where `adj_uhyd` is in `parameters.bsn`. With `uhyd = 0` the hydrograph is triangular, rising to the peak at t_p and falling to zero at t_b. With `uhyd = 1` it follows the gamma function

$$q(t) = \left(\frac{t}{t_p}\right)^{\alpha} \exp\left[\alpha\left(1 - \frac{t}{t_p}\right)\right]$$

with α = `uhyd_alpha` from `parameters.bsn`. The ordinates are scaled to a total of 1 and cover at most two days. `uhyd` has no effect in daily runs or with Green-Ampt.

Runoff method

With `gampt = 0`, daily surface runoff is calculated with the curve number method. With `gampt = 1`, infiltration and runoff are calculated with the Green-Ampt method from the rainfall of each time step, which is intended for sub-daily runs. In sub-daily runs with Green-Ampt the runoff hydrograph of a routing unit is shifted in time without attenuation, and `uhyd` is not used. The shift is a fixed 2 hours for every routing unit; the computed time of concentration is not used for it. For urban HRUs, Green-Ampt also uses `urb_init_abst` in `parameters.bsn`.

Any `gampt` value above 0 also switches on the sub-daily erosion calculation from raindrop splash and overland flow, which uses `splash`, `rill`, `surq_exp` and `cov_mgt` in `parameters.bsn`.

In-stream nutrients

Channel water quality is always calculated with the QUAL2E routine. With `i_fpwet = 1`, simplified first-order transformations of organic N and sediment-bound P follow, using each channel's N and P transformation rates and the travel time, and their results replace the QUAL2E results for those constituents.

gwflow module

With `gw_flow = 1`, SWAT+ looks for `chancell.gw` in the run folder. If it is present and has data, the gridded groundwater module replaces the aquifer objects: the aquifer connection file is set to `null` and the groundwater cells connected to channels are added as objects. If `chancell.gw` is missing or empty, `gw_flow` is set back to 0 and the aquifer objects are used. See gwflow and `object.cnt`.

Tillage effect on decomposition

`idc_till` is used only with `carbon = 2`. It sets the tillage factor that multiplies the decomposition rates of the carbon pools in each soil layer:

`idc_till`	Tillage factor
1	DSSAT method: 1.6 in the tilled layers for `till_eff_days` days after tillage (from `carbon.bsn`), with a linear transition in a partly tilled layer; 1.0 otherwise.
2	Placeholder for the EPIC method. No tillage effect (factor 1.0).
3	Kemanian method: 1 plus the layer's tillage factor from tillage and biological mixing, calculated with the mixing curves in `carbon.bsn`.
4	Placeholder for the DNDC method. No tillage effect (factor 1.0).

Example

`refdata/Ames_sub1/codes.bsn` in the SWAT+ repository:

`codes.bsn`:

pet_file	wq_file	pet	event	crack	swift_out	sed_det	rte_cha	deg_cha
null	null	2	0	0	0	0	0	0

This file selects Hargreaves PET (`pet = 2`), the NRCS peak rate method, variable storage channel routing, curve number runoff, the dynamic carbon model (`carbon = 2`) and the Kemanian tillage method (`idc_till = 3`). `cn = 2` and `atmo_dep = 0` are in unused columns and have no effect. `uhyd = 1` would select a gamma unit hydrograph if the run were sub-daily.

Related files

- file.cio: names codes.bsn on its basin line.
- parameters.bsn: coefficients used by the methods selected here.
- carbon.bsn and carbon_lyr.bsn: required when carbon = 2.
- hydrology.hyd: pet_co multiplies the PET of every method.
- Climate files: weather-sta.cli and pet.cli for station PET.
- gwflow: inputs of the gridded groundwater module.
- Soil carbon, nutrient cycling and channels: process descriptions.

Sources

- Reader: basin_read_cc.f90, called from proc_bsn.f90
- Type definition: type basin_control_codes in basin_module.f90
- File name slot: in_basin%codes_bas in input_file_module.f90, read in readcio_read.f90
- PET: et_pot.f90; station PET in climate_control.f90
- event: plant_parm_read.f90, soil_plant_init.f90
- Crack flow: sq_crackvol.f90, sq_crackflow.f90, swr_percmacro.f90, swr_percmain.f90
- SWIFT output: annual sums in command.f90, files written by swift_output.f90, called at the end of main.f90.in
- Peak rate and erosion: ero_pkq.f90, ero_cfactor.f90, ero_ovrsed.f90, surface.f90
- Channel routing: ch_rtmusk.f90, sd_hydsed_init.f90; in-stream nutrients in sd_channel_control3.f90
- Plant stress: pl_biomass_gro.f90
- Carbon model branches: hru_control.f90, soil_nutcarb_init.f90, carbon_bsn_read.f90, cbn_zhang2.f90
- Lapse rates: cli_lapse.f90, applied in hru_control.f90
- Unit hydrograph: unit_hyd.f90, unit_hyd_ru_hru.f90, hru_hyds.f90
- Runoff method: sq_volq.f90
- Tile drainage and water table: swr_percmain.f90
- gwflow activation: basin_read_objs.f90

Column	Fortran variable
pet_file	bsn_cc%petfile
wq_file	bsn_cc%wwqfile
pet	bsn_cc%pet
event	bsn_cc%nam1
crack	bsn_cc%crk
swift_out	bsn_cc%swift_out
sed_det	bsn_cc%sed_det
rte_cha	bsn_cc%rte
deg_cha	bsn_cc%deg
wq_cha	bsn_cc%wq
nostress	bsn_cc%nostress
cn	bsn_cc%cn
c_fact	bsn_cc%cfac
carbon	bsn_cc%cswat
lapse	bsn_cc%lapse
uhyd	bsn_cc%uhyd
sed_cha	bsn_cc%sed_ch
tiledrain	bsn_cc%tdrn
wtable	bsn_cc%wtdn
soil_p	bsn_cc%sol_p_model
gampt	bsn_cc%gampt
atmo_dep	bsn_cc%atmo
stor_max	bsn_cc%smax
i_fpwet	bsn_cc%qual2e
gw_flow	bsn_cc%gwflow
idc_till	bsn_cc%idc_till

Purpose

`parameters.bsn` contains the coefficients that apply to the whole watershed: initial soil water, surface runoff lag, peak rate factors, nutrient and pesticide transport coefficients, denitrification, channel routing coefficients, lapse rates, erosion coefficients for sub-daily runs and the starting CO2 concentration. It is named in the second position of the `basin` line of `file.cio` (default `parameters.bsn`). Several of the parameters only matter for the methods selected in `codes.bsn`.

If the file named in `file.cio` does not exist, every parameter keeps its built-in value, after the zero-value replacements described in Zero and missing values.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Line 3 contains 44 values in the order below: 42 real numbers followed by two integers. The reader matches values by position only, so all 44 must be present and in this order. The table gives the column headers the SWAT+ Editor writes; several no longer describe what the model does with the value.

#	Column	Units	Built-in value	Typical range	Description
1	`lai_noevap`	-	3.0	-	Not used. Read and can be changed by calibration, but no routine uses it.
2	`sw_init`	fraction	0.0	0 to 1	Initial soil water as a fraction of field capacity, the same for all soils. 0 means it is calculated from average annual precipitation. See Initial soil water.
3	`surq_lag`	-	4.0	1 to 24	Surface runoff lag coefficient. See Surface runoff lag.
4	`adj_pkrt`	-	1.0	0.5 to 2	Multiplier on the monthly mean fraction of daily rain that falls in the maximum half hour, calculated from the weather generator statistics (limited to 0.1 to 0.95). The daily fraction drawn from it is used by the half-hour intensity peak rate method (`sed_det = 1`), the rainfall erosivity of the MUSLE, urban washoff and the generation of sub-daily rainfall.
5	`adj_pkrt_sed`	-	484.0	-	Peak rate factor of the NRCS peak runoff equation, used for HRU erosion when `sed_det = 0` in `codes.bsn` and for wetlands. See Peak rate factor.
6	`lin_sed`	-	0.0	0 and up	Pesticide groundwater mixing factor. See Pesticides in aquifers.
7	`exp_sed`	-	0.0	0 or 1	Temperature-dependent pesticide decay: `1` on, any other value off. See Pesticide decay and temperature.
8	`orgn_min`	1/day	0.003	0.0003 to 0.003	Rate factor for mineralization of the active humus N and P pools (static carbon model, `carbon = 0`).
9	`n_uptake`	-	20.0	0 to 100	Nitrogen uptake distribution parameter. See Uptake distribution.
10	`p_uptake`	-	20.0	0 to 100	Phosphorus uptake distribution parameter.
11	`n_perc`	-	0.10	0 to 1	Nitrate percolation coefficient: ratio of the nitrate concentration in surface runoff to the concentration in water leaving the top layer.
12	`p_perc`	m^3/Mg	10.0	10 to 17.5	Phosphorus percolation coefficient, used in the leaching of soluble P from each layer.
13	`p_soil`	m^3/Mg	175.0	100 to 200	Phosphorus soil partitioning coefficient: ratio of soluble P in the top 10 mm of soil to soluble P in surface runoff.

#	Column	Units	Built-in value	Typical range	Description
14	p_avail	fraction	0.40	0.01 to 0.7	Phosphorus availability index: fraction of fertilizer P that is in solution after the fast reaction period. Sets the balance between the labile and active mineral P pools in the original P model (soil_p = 0).
15	rsd_decomp	fraction/day	0.05	0.02 to 0.1	Residue decomposition coefficient: daily fraction of residue that decomposes under ideal conditions. Used for every plant whose plnt_decomp in plants.plt is 0.
16	pest_perc	-	0.5	0 to 1	Pesticide percolation coefficient: ratio of the pesticide concentration in surface runoff and lateral flow to the concentration in percolate.
17	msk_co1	-	0.75	0 to 10	Weight of the storage time constant at bankfull depth in the Muskingum storage constant.
18	msk_co2	-	0.25	0 to 10	Weight of the storage time constant at 0.1 bankfull depth in the Muskingum storage constant.
19	msk_x	-	0.20	0 to 0.3	Muskingum weighting factor for inflow and outflow in channel storage.
20	nperco_lchtile	-	0.5	0 to 1	Ratio of the nitrate concentration in tile flow to the concentration in the mobile water of the tile layer.
21	evap_adj	-	0.60	0.5 to 1	Channel evaporation adjustment factor: multiplier on station PET for evaporation from the channel water surface.
22	scoef	-	1.0	0 to 1	Channel storage coefficient of the variable storage routing method (rte_cha = 0 in codes.bsn).
23	denit_exp	-	1.40	0 to 3	Denitrification exponential rate coefficient. See Denitrification.
24	denit_frac	-	1.30	see text	Denitrification threshold on the soil water factor. See Denitrification.
25	man_bact	fraction	0.15	-	Not used.
26	adj_uhyd	h	0.0	0 and up	Added to the base time of the sub-daily unit hydrograph. See unit hydrograph.
27	cn_froz	1/mm	0.000862	-	Frozen soil coefficient of the curve number retention parameter. See Frozen soil.

#	Column	Units	Built-in value	Typical range	Description
28	`dorm_hr`	h	-1.0	0 to 24	Day length threshold for dormancy. 0 or less means it is calculated from latitude. See Dormancy threshold.
29	`plaps`	mm/km	0.0	-	Precipitation lapse rate. Positive values increase precipitation with elevation. Used when `lapse = 1` in `codes.bsn`.
30	`tlaps`	°C/km	6.5	-	Temperature lapse rate. Positive values decrease temperature with elevation. Used when `lapse = 1` in `codes.bsn`.
31	`n_fix_max`	kg N/ha/day	20.0	1 to 20	Maximum daily nitrogen fixation of a plant.
32	`rsd_decay`	fraction/day	0.01	0 to 0.05	Minimum daily residue decay fraction. The daily decay rate of residue is never below this value.
33	`rsd_cover`	ha/t	0.75	-	Residue cover factor in the USLE C factor. See USLE C factor.
34	`urb_init_abst`	mm	1.0	0 to 10	Maximum initial abstraction on impervious urban areas. Used only with Green-Ampt (`gampt = 1`).
35	`petco_pmpt`	%	100.0	-	Not used. The PET of every method is adjusted with `pet_co` in `hydrology.hyd` instead.
36	`uhyd_alpha`	-	1.0	0.5 to 10	Shape coefficient of the gamma unit hydrograph (`uhyd = 1` in `codes.bsn`).
37	`splash`	-	0.0	0.9 to 3.1	Splash erosion coefficient. See Sub-daily erosion.
38	`rill`	-	0.0	0.5 to 2	Rill erosion coefficient, a multiplier on the USLE K factor of the top layer.
39	`surq_exp`	-	0.0	1 to 3	Exponent on bed shear stress in overland flow erosion.
40	`cov_mgt`	-	0.0	0.001 to 0.45	Scaling factor on the cover and management factor in overland flow erosion.
41	`cha_d50`	mm	0.0	-	Not used. The median particle size of each channel is read from the channel input.
42	`co2`	ppm	400.0	-	Atmospheric CO2 concentration, used for every year when there is no `co2_yr.dat`. See CO2.
43	`day_lag_max`	days	0	-	Not used as read. The model always sets it to 2.
44	`igen`	-	5	0 and up	Random number seed code. See Random numbers.

Values are read in one free-format read. A line with fewer values continues onto the next line, and the parameters that are not reached keep their built-in values. A value that cannot be read ends the read without a message.

Zero and missing values

After reading, SWAT+ replaces values that are zero or negative (strictly, below 10^{-6}) for some parameters. For most of them the replacement is the built-in value, so leaving a column at 0 has the same effect as entering the built-in value. These are: `lai_noevap`, `surq_lag`, `adj_pkrt`, `n_uptake`, `p_uptake`, `n_perc`, `p_perc`, `p_soil`, `p_avail`, `rsd_decomp`, `pest_perc`, `msk_co1`, `msk_co2`, `msk_x`, `nperco_lchtile`, `evap_adj`, `denit_exp`, `denit_frac`, `man_bact`, `cn_froz`, `n_fix_max`, `rsd_decay`, `rsd_cover` and `uhyd_alpha`.

The exceptions are:

Column	Value read	Value used
`adj_pkrt_sed`	0 or less	1.0 (the built-in value is 484.0)
`orgn_min`	0 or less	0.0003 (the built-in value is 0.003)
`urb_init_abst`	0 or less	0.0 (the built-in value is 1.0)
`co2`	below 100	400.0
`day_lag_max`	any	2

`sw_init`, `adj_uhyd`, `dorm_hr`, `splash`, `rill`, `surq_exp`, `cov_mgt` and `cha_d50` stay at 0 when 0 is read, and 0 has a meaning for some of them (see the sections below). `lin_sed`, `exp_sed`, `scoef`, `plaps`, `tlaps` and `igen` are used as read.

Initial soil water

All soils start at the same fraction of field capacity. If `sw_init` is 0 or less, the fraction is calculated from the average annual precipitation P (mm) of the HRU's weather generator station:

$$f_{fc} = \frac{P}{P + \exp(9.043 - 0.002135\,P)}$$

A warm-up period of at least one year (`nyskip` in `print.prt`) makes results largely independent of `sw_init`.

Surface runoff lag

In HRUs with a long time of concentration, only part of the surface runoff reaches the channel on the day it is generated. The fraction released each day is

$$f_{rel} = 1 - \exp\left(-\frac{\text{surq_lag}}{t_{conc}}\right)$$

with t_{conc} in hours. In sub-daily runs t_{conc} is expressed in time steps. The rest is kept in surface storage and added to the next day's runoff. Smaller values of `surq_lag` keep more water in storage and smooth the hydrograph.

Peak rate factor

The column labelled `adj_pkrt_sed` by the SWAT+ Editor is the peak rate factor of the NRCS peak runoff equation (see peak runoff rate). The built-in value, 484, is the standard factor for the NRCS dimensionless unit hydrograph. The Editor writes 1.0 in this column, and a value of 0 is also replaced by 1.0. A factor of 1 gives peak rates 484 times lower than the standard factor, which lowers MUSLE sediment yields from HRUs when `sed_det = 0`. The value only affects the peak rate used for erosion and in wetlands; it does not change runoff volumes.

Pesticides in aquifers

The columns labelled `lin_sed` and `exp_sed` by the Editor are pesticide parameters in 62.0.1. In each aquifer, the pesticide concentration in return flow and seepage is the pesticide mass divided by a mixing volume:

$$C = \frac{M_{pest}}{f_{mix}\, S_{aqu} + Q_{rtn} + Q_{revap} + Q_{seep}}$$

where f_{mix} is `lin_sed`, S_{aqu} is the aquifer storage and the Q terms are the day's return flow, revap and seepage (mm). With `lin_sed = 0` the concentration is based on the day's outflows only. Larger values dilute the pesticide in more of the stored water and slow its release.

Pesticide decay and temperature

With `exp_sed = 1` (the value 1.0; other values such as 2 do not count), pesticide decay in soil, on foliage, in aquifers, in channel water and in channel bed sediment uses half-lives adjusted for temperature with a Q10 model:

$$t_{1/2}(T) = t_{1/2,25}\, Q_{10}^{(25-T)/10}, \qquad Q_{10} = 2.58$$

where $t_{1/2,25}$ is the half-life from the pesticide database, taken as the value at 25 °C, and the daily fraction remaining is $\exp(-0.693/t_{1/2}(T))$. Soil decay uses the layer temperature, foliar decay the top soil layer temperature, and channel decay the water temperature. In aquifers the temperature is fixed at 0 °C, which makes aquifer half-lives about 10.7 times longer than the database value. Any other value of `exp_sed` uses the database half-lives without a temperature adjustment.

The Ames example file (see Example) has `exp_sed = 1.0`, so temperature-dependent decay is on in that dataset.

Uptake distribution

The maximum nitrogen uptake from the soil above depth z is

$$U(z) = U_{pot}\, \frac{1 - \exp(-\beta_n\, z / z_{root})}{1 - \exp(-\beta_n)}$$

where β_n is `n_uptake` and z_{root} is the root depth. Larger values concentrate uptake near the surface. Because the top 10 mm interacts with surface runoff, `n_uptake` affects the nitrate available for runoff; lower layers can make up a shortfall in the upper layers, so plant N stress changes little with this parameter. `p_uptake` works the same way for phosphorus.

Denitrification

Denitrification in a soil layer is

$$N_{denit} = NO_3 \left[1 - \exp\left(-\text{denit_exp}\ \gamma_{tmp}\ \frac{OC}{100}\right)\right]$$

where γ_{tmp} is the temperature factor and OC the organic carbon content (%). It only occurs on days when the soil water factor γ_{sw} is at or above `denit_frac`.

With the static carbon model (`carbon = 0`), $\gamma_{sw} = 0.1 + 0.9\sqrt{SW/FC}$, with no upper limit. With the default `denit_frac` of 1.3, denitrification needs $SW/FC \geq 1.78$, that is, a layer well above field capacity.

With the dynamic carbon model (`carbon = 2`), γ_{sw} comes from the water factor selected in `carbon.bsn` and is never above 1. Any `denit_frac` above 1, including the default 1.3, then switches denitrification off for the whole run. Datasets using `carbon = 2` need a `denit_frac` of 1 or less to simulate denitrification.

Frozen soil

When the second soil layer is at or below 0 °C, the curve number retention parameter S is reduced:

$$S_{frz} = S_{max}\,[1 - \exp(-\text{cn_froz} \cdot S)]$$

which increases runoff from frozen soil. The same adjustment is applied with Green-Ampt.

Dormancy threshold

A plant goes dormant when the day length falls below the station's minimum day length plus a threshold t_{dorm} (hours). If `dorm_hr` is above 0, t_{dorm} = `dorm_hr` everywhere. Otherwise it depends on the latitude ϕ of the weather generator station: 1 h above 40°, $(|\phi| - 20)/20$ h between 20° and 40°, and -1 h below 20°.

USLE C factor

The C factor is the product of a residue factor and a biomass factor:

$$C = \exp\left(-\text{rsd_cover} \cdot \frac{R+1}{1000}\right)\,[1 - f_{cov}\,\exp(-0.328\,h_c)]$$

where R is the residue on the surface (kg/ha), h_c is the canopy height (m), and f_{cov} is the ground cover fraction of the above-ground biomass. This is the only C factor method in 62.0.1 (see USLE C factor).

Sub-daily erosion

With Green-Ampt (`gampt` above 0 in `codes.bsn`), sediment from each time step is the sum of splash and overland flow erosion. Splash erosion decreases with runoff depth h (mm) as $\exp(-\text{splash} \cdot h/1000)$ and depends on rainfall kinetic energy and a soil texture erodibility. Overland flow erosion is

$$E_{ov} = 11.02\ \text{rill}\ K\ \text{cov_mgt}\ C\ \tau^{\,\text{surq_exp}}$$

in kg/h/m², where K is the USLE K factor of the top layer, C is a cover factor from biomass and residue, and τ is the bed shear stress (N/m²). With the built-in values of 0 for `rill` or `cov_mgt`, overland flow erosion is zero.

CO2

Without a `co2_yr.dat` file in the run folder, every year uses `co2`. With the file, the yearly values come from it (see co2_yr.dat). The radiation use efficiency of plants is adjusted for CO2 only when `co2` in `parameters.bsn` is above 350 ppm, even when the yearly values come from `co2_yr.dat`. Penman-Monteith PET uses the yearly values in all cases.

Random numbers

With `igen = 0`, the weather generator and other random processes start from fixed seeds, so every run gives the same random sequence. With any other value, each seed is advanced by a number of steps that depends on `igen`, so different `igen` values give different sequences. The same `igen` always gives the same sequence, so a new value is needed for each different realisation.

Calibration

These parameters can be changed during calibration through calibration.cal with object type bsn, using their model names: surlag (surq_lag), adj_pkr (adj_pkrt), prf (adj_pkrt_sed), evlai (lai_noevap), ffcb (sw_init), cmn (orgn_min), nperco (n_perc), pperco (p_perc), phoskd (p_soil), psp (p_avail), rsdco (rsd_decomp), percop (pest_perc), msk_co1, msk_co2, msk_x, nperco_lchtile, evrch (evap_adj), cdn (denit_exp), sdnco (denit_frac), tb_adj (adj_uhyd), n_updis (n_uptake), rsd_covco (rsd_cover), dorm_hr, plaps and tlaps. See parameter change files.

The changes in calibration.cal are applied once, after the HRUs, channels and weather stations have been set up. Several of these parameters are only used during that setup, so changing them in calibration.cal has no effect on the run:

- cmn, nperco, pperco, phoskd, psp and nperco_lchtile have already been copied to each HRU. Use the HRU-level names cmn_h, nperco_h, pperco_h, phoskd_h, psp_h and nperco_lchtile_h instead.
- ffcb: the initial soil water has already been set.
- adj_pkr and dorm_hr: the half-hour rainfall fractions and the dormancy thresholds of the weather stations have already been calculated.
- msk_co1, msk_co2 and msk_x: the Muskingum coefficients of the channels have already been calculated.
- rsdco: it has already been copied to the plants with plnt_decomp = 0.

surlag is recalculated into the runoff release fraction when it is changed, and the others are used directly during the simulation, so their changes take effect.

Example

refdata/Ames_sub1/parameters.bsn in the SWAT+ repository:

```
parameters.bsn: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
  lai_noevap       sw_init      surq_lag      adj_pkrt  adj_pkrt_sed       lin_sed       exp_sed      orgn
     3.00000       0.50000       4.00000       1.00000       1.00000       0.00000       1.00000       0.0
```

How the model reads some of these values:

- sw_init = 0.5: soils start at half of field capacity.
- adj_pkrt_sed = 1.0: the NRCS peak rate factor is 1. The standard factor is 484.
- exp_sed = 1.0: temperature-dependent pesticide decay is on.
- orgn_min = 0.0: replaced by 0.0003.
- cn_froz = 0.0: replaced by 0.000862.
- dorm_hr = 0.0: the dormancy threshold is calculated from latitude.
- plaps = 0.0 and tlaps = 0.0: no lapse rate adjustment, even if lapse = 1 in codes.bsn.
- denit_frac = 1.3: this dataset uses carbon = 2 in its codes.bsn, so no denitrification occurs (see Denitrification).
- petco_pmpt, cha_d50 and man_bact have no effect; day_lag_max is set to 2.
- igen = 0: fixed random seeds.

Related files

- file.cio: names parameters.bsn on its basin line.
- codes.bsn: method codes that decide which of these parameters are used.
- hydrology.hyd: HRU-level hydrology parameters, including pet_co.
- plants.plt: plnt_decomp overrides rsd_decomp for a plant.
- Parameter change files: calibration names for these parameters.
- Nutrient cycling, pesticides, erosion and sediment, channels and hydrology: process descriptions.

Sources

- Reader: basin_read_prm.f90, followed by the zero-value replacements in basin_prm_default.f90; both called from proc_bsn.f90

- Type definition: `type basin_parms` in basin_module.f90
- File name slot: `in_basin%parms_bas` in input_file_module.f90
- Copy to HRUs: hrudb_init.f90
- Hydrology: hydro_init.f90, time_conc_init.f90, sq_dailycn.f90, sq_greenampt.f90, unit_hyd.f90, cli_lapse.f90
- Weather generator: cli_initwgn.f90, gcycl.f90, co2_read.f90
- Peak rate and erosion: ero_pkq.f90, wetland_control.f90, ero_cfactor.f90, ero_ovrsed.f90
- Pesticides: aqu_1d_control.f90, pest_decay.f90, pest_tempdecay_module.f90, ch_rtpest.f90, pest_lch.f90
- Nutrients: nut_nminrl.f90, cbn_zhang2.f90, nut_nlch.f90, nut_solp.f90, nut_pminrl.f90, rsd_decomp.f90, cbn_surfrsd_decomp.f90, plantparm_init.f90
- Plants: pl_nup.f90, pl_pup.f90, pl_nfix.f90, pl_biomass_gro.f90
- Channels: sd_hydsed_init.f90, ch_rtmusk.f90, ch_rthr.f90, ch_rtday.f90
- Calibration: cal_parm_select.f90

Column	Fortran variable
lai_noevap	bsn_prm%evlai
sw_init	bsn_prm%ffcb
surq_lag	bsn_prm%surlag
adj_pkrt	bsn_prm%adj_pkr
adj_pkrt_sed	bsn_prm%prf
lin_sed	bsn_prm%pestgwfact
exp_sed	bsn_prm%temp_decay
orgn_min	bsn_prm%cmn
n_uptake	bsn_prm%n_updis
p_uptake	bsn_prm%p_updis
n_perc	bsn_prm%nperco
p_perc	bsn_prm%pperco
p_soil	bsn_prm%phoskd
p_avail	bsn_prm%psp
rsd_decomp	bsn_prm%rsdco
pest_perc	bsn_prm%percop
msk_co1, msk_co2, msk_x	bsn_prm%msk_co1, %msk_co2, %msk_x
nperco_lchtile	bsn_prm%nperco_lchtile
evap_adj	bsn_prm%evrch
scoef	bsn_prm%scoef
denit_exp	bsn_prm%cdn
denit_frac	bsn_prm%sdnco
man_bact	bsn_prm%bact_swf
adj_uhyd	bsn_prm%tb_adj
cn_froz	bsn_prm%cn_froz
dorm_hr	bsn_prm%dorm_hr
plaps, tlaps	bsn_prm%plaps, %tlaps
n_fix_max	bsn_prm%nfixmx
rsd_decay	bsn_prm%decr_min
rsd_cover	bsn_prm%rsd_covco
urb_init_abst	bsn_prm%urb_init_abst
petco_pmpt	bsn_prm%petco_pmpt
uhyd_alpha	bsn_prm%uhalpha
splash	bsn_prm%eros_spl
rill	bsn_prm%rill_mult
surq_exp	bsn_prm%eros_expo
cov_mgt	bsn_prm%c_factor
cha_d50	bsn_prm%ch_d50
co2	bsn_prm%co2
day_lag_max	bsn_prm%day_lag_mx
igen	bsn_prm%igen

Purpose

carbon.bsn contains the basin-wide parameters of the dynamic soil carbon model (carbon = 2 in codes.bsn): how soil organic carbon is split into pools at the start of the run, losses of soluble carbon in water, the split of organic N and P in fertilizer and manure, the duration and decay of tillage effects, the curves for tillage and biological mixing, the temperature and water factors on decomposition, surface residue photodegradation, and the C:N and C:P controls on residue decomposition.

It works together with carbon_lyr.bsn, which contains the decomposition rates and CO2 fractions for the surface layer and for the deeper layers. In revision 62 these two files replaced carb_coefs.cbn (see carb_coefs.cbn).

Activation and file names

The file is read only when carbon = 2 in codes.bsn. With any other carbon code it is ignored and need not exist.

With carbon = 2 the file is required. SWAT+ writes an error to the screen and to diagnostics.out and stops if:

- the file does not exist or cannot be opened;
- its data line is missing or one of the 28 values cannot be read;
- the companion file carbon_lyr.bsn is missing or has too few valid rows (see carbon_lyr.bsn).

There are no built-in values to fall back on.

The file is named in the third position of the basin line of file.cio, after codes.bsn and parameters.bsn. The default name is carbon.bsn. The name of the layer file is built from it by inserting _lyr before the extension: carbon.bsn gives carbon_lyr.bsn, mybasin.bsn gives mybasin_lyr.bsn, and a name without an extension gets _lyr.bsn added. The layer file is not listed in file.cio.

The basin line of file.cio must have three names. The line is read as three names in one free-format read, so a basin line from an older dataset with only two names takes the first word of the next line (normally climate) as the carbon file name, and the rest of file.cio is then read one line out of step. Datasets that do not use the carbon model can put null in the third position.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped and may be blank. Line 3 contains the 28 values in the order below. The reader matches values by position only. Columns 7, 11, 12 and 28 are integers and must be written without a decimal point, for example 2; a value such as 2.0 in these columns stops the run with a read error.

#	Column	Units	Example value	Description
1	init_seq	fraction	0.95	Fraction of each layer's soil organic carbon put into the humus and microbial pools at the start. The rest, 1− init_seq, goes to metabolic (85%) and structural (15%) litter.
2	init_microb	fraction	0.02	Fraction of the init_seq share that goes to the microbial biomass pool.
3	init_slow	fraction	0.44	Fraction of the init_seq share that goes to the slow humus pool (when mathers_method is 0).
4	init_passive	fraction	0.54	Fraction of the init_seq share that goes to the passive humus pool.
5	koc_c	-	1000	Partition coefficient of microbial carbon between soil and water, controlling its loss in runoff, lateral flow and percolation from the top layer. Typical range 500 to 1500.

#	Column	Units	Example value	Description
6	`solc_ratio`	-	0.5	Ratio of the soluble carbon concentration in surface runoff and lateral flow to the concentration in percolate. Typical range 0.1 to 1.
7	`till_eff_days`	days (integer)	100	Number of days a tillage operation affects decomposition.
8	`manure_c_frac`	fraction	0.5	Split of the organic N and P in applied fertilizer and manure between fresh residue and the slow humus pool. See Organic fertilizer and manure.
9	`bio_consol`	fraction/day	0.15	Daily recovery rate of biological mixing after tillage.
10	`till_consol`	fraction/day	0.10	Daily decline of the tillage factor as the soil settles.
11	`tmpf_eqn`	code (integer)	2	Temperature factor equation: `1`, `2` or `3`. See Temperature factor.
12	`watf_eqn`	code (integer)	1	Water factor equation: `1` or `2`. See Water factor.
13	`t_cbn_min`	°C	-0.5	Minimum temperature of temperature equation 2.
14	`t_cbn_opt`	°C	30.0	Optimum temperature of temperature equation 2.
15	`t_cbn_max`	°C	50.0	Maximum temperature of temperature equation 2.
16	`bmix_a`	-	3.0	Intercept a of the texture curve for biological mixing.
17	`bmix_b`	-	5.0	Coefficient b of the texture curve for biological mixing.
18	`bmix_c`	-	-5.5	Exponent coefficient c of the texture curve for biological mixing.
19	`tillmix_a`	-	3.0	Intercept a of the texture curve for tillage.
20	`tillmix_b`	-	15.0	Coefficient b of the texture curve for tillage.
21	`tillmix_c`	-	-3.5	Exponent coefficient c of the texture curve for tillage.
22	`sfc_rsd_photodeg`	fraction/day	0.001	Fraction of surface residue lost each day to photodegradation. The lost carbon is counted as a residue emission.
23	`n_act_frac`	fraction	0.02	Fraction of organic N in the active humus pool. Not used with `carbon = 2`. See Notes on individual columns.
24	`cnr_cap`	-	500	Upper limit on the residue C:N ratio in the C:N decomposition factor.
25	`cnr_ref`	-	25	Residue C:N ratio at which the C:N decomposition factor equals 1.
26	`cpr_cap`	-	5000	Upper limit on the residue C:P ratio in the C:P decomposition factor.
27	`cpr_ref`	-	200	Residue C:P ratio at which the C:P decomposition factor equals 1.
28	`mathers_method`	code (integer)	1	Slow humus initialisation: `1` Mathers method, any other value the `init_slow` fraction. See Initial carbon pools.

The example values are those of `refdata/Ames_sub1/carbon.bsn` (see Example). Since the file is required, there are no defaults in the usual sense.

Initial carbon pools

At the start of the run, the organic carbon C of each soil layer (from the organic carbon content in `soils.sol`) is divided into pools:

Pool	Carbon	C:N	C:P
passive humus	$C \cdot \text{init_seq} \cdot \text{init_passive}$	10	80
slow humus	$C \cdot \text{init_seq} \cdot \text{init_slow}$	10	80
microbial biomass	$C \cdot \text{init_seq} \cdot \text{init_microb}$	8	80
metabolic litter	$C \cdot (1 - \text{init_seq}) \cdot 0.85$	10	100
structural litter	$C \cdot (1 - \text{init_seq}) \cdot 0.15$	150	1500

The reader does not check the fractions. `init_microb`, `init_slow` and `init_passive` should add up to 1, or the pools will not add up to the soil's organic carbon.

With `mathers_method = 1`, the slow humus pool is set from the clay and silt content instead of `init_slow`, following "Updating carbon pool initialization with DSSAT-CENTURY". In 62.0.1 the test and equation are written for clay plus silt as a fraction, while the soil values are in percent. As a result, for every layer with more than 0.35% clay plus silt the slow humus share is

$$f_{slow} = 0.0041 + 0.0053\,(\text{clay} + \text{silt})$$

with clay and silt in percent (0.32 for a layer with 60% clay plus silt), and the slow humus carbon is $C \cdot \text{init_seq} \cdot f_{slow}$. The passive and microbial pools keep their `init_passive` and `init_microb` shares, so with the Mathers method the pools generally do not add up to the soil's organic carbon.

Soluble carbon losses

Each day, microbial carbon in the top layer is lost with the water leaving it:

$$C_{loss} = C_{mic}\left[1 - \exp\left(-\frac{Q_{surf} + Q_{perc} + Q_{lat}}{UL + 0.0001\,\text{koc_c} \cdot C_{tot}}\right)\right]$$

where UL is the water content of the layer at saturation (mm) and C_{tot} is the layer's total organic carbon. The loss is shared between percolation and surface runoff plus lateral flow, with the concentration in surface runoff and lateral flow equal to `solc_ratio` times the concentration in percolate. Microbial carbon attached to eroded sediment is lost in proportion to the sediment yield.

Organic fertilizer and manure

When fertilizer with organic N or P, or manure, is applied, the organic matter goes to the slow humus pool and to metabolic and structural litter. In the code, the slow humus pool receives the fraction 1− `manure_c_frac` of the organic matter in each layer, and the metabolic and structural litter together also receive 1− `manure_c_frac`, split by a metabolic fraction from the C:N ratio of the material. The code comments say litter should receive the fraction `manure_c_frac`. With the value 0.5 used in the reference datasets the two readings give the same result; with other values the amounts added differ from the comments.

Tillage and biological mixing

The tillage factor of a soil layer is calculated from a texture curve and the cumulative soil disturbance x of tillage operations:

$$Z = a + b\,\exp\left(c\,\frac{\text{clay}}{100}\right), \qquad F = Z\,\frac{x}{x + \exp(1 - 2x)}$$

with clay in percent. Tillage operations use `tillmix_a`, `tillmix_b` and `tillmix_c`; biological mixing uses `bmix_a`, `bmix_b` and `bmix_c`. The layer's tillage factor is the sum of the two parts. The original equations were $Z = 3 + 12\,\exp(-5.5\,\text{clay}/100)$ for tillage and $Z = 3 + 5\,\exp(-5.5\,\text{clay}/100)$ for biological mixing.

After tillage, the tillage part declines each day by the fraction `till_consol` (scaled down by SW/FC when the layer is drier than field capacity), and biological mixing efficiency recovers each day by `bio_consol` times its initial value until it is back to that value. `till_eff_days` days after a tillage operation the tillage effect ends and the tillage part is reset to 0.

How the tillage factor changes decomposition depends on `idc_till` in `codes.bsn`. With `idc_till = 3` the decomposition rates are multiplied by $1 + F$. With `idc_till = 1` they are multiplied by 1.6 in tilled layers for `till_eff_days` days.

Temperature factor

`tmpf_eqn` selects the temperature factor γ_{tmp} on decomposition, using the layer temperature T (°C). Decomposition only occurs when $T > 0$.

`tmpf_eqn`	Equation
1	$\gamma_{tmp} = T/[T + \exp(5.058 - 0.2504\,T)]$ for $T \le 35$; $1 - 0.04\,(T - 35)$ for $35 < T \le 60$; 0 above 60.
2	$\gamma_{tmp} = \dfrac{(T - T_n)^q\,(T_x - T)}{(T_o - T_n)^q\,(T_x - T_o)}$ with $q = \dfrac{T_n - T_o}{T_o - T_x}$, and 0 outside $T_n < T < T_x$. T_n, T_o and T_x are `t_cbn_min`, `t_cbn_opt` and `t_cbn_max`.
3	$\gamma_{tmp} = 0.9\,T/[T + \exp(9.93 - 0.312\,T)] + 0.1$

Biological mixing is also reduced with temperature: by equation 3 when `tmpf_eqn = 3`, and by equation 2 otherwise. So `t_cbn_min`, `t_cbn_opt` and `t_cbn_max` affect biological mixing even when `tmpf_eqn = 1`.

Water factor

`watf_eqn` selects the water factor γ_{sw}:

- 1: $\gamma_{sw} = 0.1 + 0.9\sqrt{SW/FC}$, limited to 0.05 to 1, where SW and FC are the layer's water content and field capacity above wilting point.
- 2: the product of a water term, rising from 0 at zero water content to 0.4 at wilting point and 1 at field capacity, and an aeration term that falls as the air-filled porosity drops below 0.1. Both terms are at most 1.

Decomposition rates are multiplied by the combined factor $\min\left(15,\ 0.9\,\sqrt{\gamma_{tmp}\,\gamma_{sw}}\,\gamma_{ox}\,f_{till}\right)$, where γ_{ox} is an oxygen factor that decreases with depth and f_{till} is the tillage factor.

With either equation γ_{sw} never exceeds 1. Denitrification in the dynamic carbon model occurs only when γ_{sw} reaches `denit_frac` in `parameters.bsn`, so a `denit_frac` above 1 (the default is 1.3) switches denitrification off.

Residue decomposition

The daily decay rate of surface residue for each plant is

$$k = \max\left(\text{rsd_decay},\ k_{pl}\ \min(f_{CN}, f_{CP}, 1)\ \sqrt{\gamma_{tmp}\,\gamma_{sw}}\right), \qquad f_{CN} = \exp\left(-0.693\,\frac{CN - \text{cnr_ref}}{\text{cnr_ref}}\right)$$

limited to 1, where k_{pl} is the plant's residue decomposition coefficient (`plnt_decomp` in `plants.plt`, or `rsd_decomp` in `parameters.bsn`), CN is the residue C:N ratio limited to `cnr_cap`, and f_{CP} is the same expression with the C:P ratio, `cpr_ref` and `cpr_cap`. A residue C:N ratio equal to `cnr_ref` gives $f_{CN} = 1$; each further `cnr_ref` halves it. Before decomposition, the fraction `sfc_rsd_photodeg` of the surface residue is removed by photodegradation.

Notes on individual columns

- `n_act_frac` is used only by the static carbon routines (`carbon = 0`), which do not read `carbon.bsn`. With `carbon = 2` the value read has no effect, and with `carbon = 0` the model uses 0.02.
- `till_eff_days` is the only tillage duration in the model. Under `idc_till = 3` it also limits how long the tillage part of the tillage factor lasts.
- `tmpf_eqn` and `watf_eqn` values other than those listed leave the factor at 0, which stops decomposition of the soil carbon pools and denitrification in every layer.

Calibration

These parameters can be changed during calibration through calibration.cal with object type `bsn`: `init_seq`, `init_microb`, `init_slow`, `init_passive`, `koc_c`, `solc_ratio`, `manure_c_frac`, `bio_consol`, `till_consol`, `sfc_rsd_photodeg`, `n_act_frac`, `cnr_cap`, `cnr_ref`, `cpr_cap`, `cpr_ref`, `t_cbn_min`, `t_cbn_opt`, `t_cbn_max`, `bmix_a`, `bmix_b`, `bmix_c`, `tillmix_a`, `tillmix_b` and `tillmix_c`. `till_eff_days`, `tmpf_eqn`, `watf_eqn` and `mathers_method` cannot be calibrated. A change to `n_act_frac` has no effect with `carbon = 2`. See parameter change files.

Example

`refdata/Ames_sub1/carbon.bsn` in the SWAT+ repository:

```
carbon.bsn: derived from carb_coefs.cbn (basin carbon scalars)
          init_seq       init_microb         init_slow      init_passive             koc_c        solc_rat
              0.95              0.02              0.44              0.54            1000.0               0
```

95% of soil organic carbon starts in the humus and microbial pools. Because `mathers_method = 1`, the slow humus pool is set from clay and silt, and `init_slow` is not used. Temperature equation 2 runs from -0.5 °C through an optimum of 30 °C to 50 °C, and the water factor is equation 1. Tillage affects decomposition for 100 days.

Related files

- carbon_lyr.bsn: decomposition rates and CO2 fractions for the surface and deeper layers.
- codes.bsn: `carbon = 2` turns the carbon model on; `idc_till` selects the tillage method.
- file.cio: names `carbon.bsn` on its `basin` line.
- parameters.bsn: `rsd_decomp`, `rsd_decay`, `denit_exp` and `denit_frac` are also used by the carbon model.
- soils.sol: organic carbon, clay and silt of each layer.
- print.prt and carbon outputs: carbon output files, and the optional `carbon_layers.prt` that sets how many soil layers the per-layer carbon outputs contain.
- Soil carbon: description of the carbon model.

Sources

- Reader: carbon_bsn_read.f90, called from proc_bsn.f90
- Type definitions and module variables: `organic_fractions`, `carbon_water_coef`, `manure_coef`, `organic_controls` and the residue tunables in carbon_module.f90; tillage variables in tillage_data_module.f90; `photo_degrade_factor` in plant_data_module.f90
- File name slot: `in_basin%carbon_bsn` in input_file_module.f90, read in readcio_read.f90
- Pool initialisation: soil_nutcarb_init.f90, called from main.f90.in
- Soluble carbon losses: nut_orgnc2.f90
- Organic amendments: pl_fert.f90, pl_manure.f90
- Tillage and mixing: mgt_tillfactor.f90, mgt_biomix.f90, mgt_newtillmix_cswat1.f90, tillage day counter in hru_control.f90
- Decomposition factors: cbn_zhang2.f90, fcgd.f90
- Residue decomposition: cbn_surfrsd_decomp.f90
- `n_act_frac`: nut_nminrl.f90
- Calibration: cal_parm_select.f90

Column	Fortran variable
init_seq	org_frac%frac_seq
init_microb	org_frac%frac_hum_microb
init_slow	org_frac%frac_hum_slow
init_passive	org_frac%frac_hum_passive
koc_c	cb_wtr_coef%prmt_21
solc_ratio	cb_wtr_coef%prmt_44
till_eff_days	till_eff_days (integer)
manure_c_frac	man_coef%rtof
bio_consol	bio_consf
till_consol	till_consf
tmpf_eqn	org_con%tmpf (integer)
watf_eqn	org_con%watf (integer)
t_cbn_min, t_cbn_opt, t_cbn_max	org_con%tn, org_con%top, org_con%tx
bmix_a, bmix_b, bmix_c	bmix_a, bmix_b, bmix_c
tillmix_a, tillmix_b, tillmix_c	tillmix_a, tillmix_b, tillmix_c
sfc_rsd_photodeg	photo_degrade_factor
n_act_frac, cnr_cap, cnr_ref, cpr_cap, cpr_ref	module variables of the same names
mathers_method	mathers_int, stored as org_frac%mathers_method = (mathers_int == 1)

Purpose

carbon_lyr.bsn contains the decomposition rates of the litter, microbial and humus pools of the dynamic soil carbon model (carbon = 2 in codes.bsn), and the fractions of decomposed carbon released as CO2. There is one row for each of two layer groups:

- layer group 1: the first soil layer of each HRU. SWAT+ adds a 10 mm surface layer when the first layer in soils.sol is deeper than 19.5 mm, so this is normally the 10 mm surface layer.
- layer group 2: all other soil layers.

The basin-wide parameters of the carbon model are in carbon.bsn.

Activation and file name

The file is read only when carbon = 2, right after carbon.bsn, and it is then required. It is not listed in file.cio. Its name is built from the carbon.bsn name on the basin line of file.cio by inserting _lyr before the extension, so the default carbon.bsn gives carbon_lyr.bsn, and mybasin.bsn gives mybasin_lyr.bsn.

SWAT+ writes an error to the screen and to diagnostics.out and stops if the file does not exist, cannot be opened, or has fewer than two valid data rows.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped and may be blank. Each later line is one layer group: an integer layer group number followed by 11 values, in the order below. The reader matches values by position only.

#	Column	Units	Group 1 (example)	Group 2 (example)	Description
1	`layer`	-	1	2	Layer group: `1` first soil layer, `2` all deeper layers.
2	`hp_rate`	1/day	1.2e-05	1.2e-05	Decomposition rate of passive humus under optimal conditions.
3	`hs_rate`	1/day	2.92e-04	1.81e-04	Decomposition rate of slow humus under optimal conditions.
4	`microb_rate`	1/day	0.0164	0.02	Decomposition rate of microbial biomass under optimal conditions. Used only in group 2.
5	`meta_rate`	1/day	0.0405	0.0507	Decomposition rate of metabolic litter under optimal conditions.
6	`str_rate`	1/day	0.0107	0.0134	Decomposition rate of structural litter under optimal conditions, before the lignin reduction.
7	`microb_top_rate`	1/day	0.0164	0.02	Decomposition rate of microbial biomass in the first soil layer. Used only in group 1.
8	`hs_hp`	fraction	0.05	0.05	Fraction of decomposed slow humus that goes to passive humus, before the clay adjustment.
9	`a1co2`	fraction	0.60	0.55	Fraction of decomposed metabolic litter and non-lignin structural litter released as CO2.
10	`asco2`	fraction	0.55	0.55	Fraction of decomposed slow humus released as CO2.
11	`apco2`	fraction	0.55	0.55	Fraction of decomposed passive humus released as CO2.
12	`abco2`	fraction	0.55	0.0	Fraction of decomposed microbial biomass released as CO2. Used only in group 1.

The example values are those of `refdata/Ames_sub1/carbon_lyr.bsn` (see Example).

Rows are read until the end of the file. A row whose layer group number is below 1 or above 2 is skipped with a message in `diagnostics.out`. A row that cannot be read (a missing value, or text in a number column) ends the reading at that row without a message, and the rows after it are ignored. The rows may be in any order. The reader counts accepted rows without checking that both groups are present, so a file with two rows for group 1 and none for group 2 passes the check and leaves the group 2 values at 0, which stops decomposition in all layers below the first.

Use in the carbon model

Each day, for each soil layer, the potential transformation of each pool is its carbon times its rate from the layer's group times the combined temperature, water, oxygen and tillage factor (see `carbon.bsn`):

$$\Delta C_{pool} = k_{pool}\, f_{env}\, C_{pool}$$

with these additions:

- metabolic litter: `a1co2` is released as CO2 and the rest goes to microbial biomass.
- structural litter: the rate is reduced by the lignin fraction L of the litter (at most 0.8), $k = \text{str_rate} \cdot \exp(-3L)$. Of the decomposed lignin, 30% is released as CO2 and 70% goes to slow humus. Of the non-lignin part, `a1co2` is released as CO2 and the rest goes to microbial biomass.
- microbial biomass: the first soil layer uses `microb_top_rate`. Deeper layers use `microb_rate` times a texture factor $0.25 + 0.0075\,\text{sand}$ (sand in %).
- slow humus: the fraction going to passive humus is $\max(0.001,\ \text{hs_hp} - 0.00009\,\text{clay})$, with clay in %. `asco2` is released as CO2 and the rest goes to microbial biomass.
- passive humus: `apco2` is released as CO2 and the rest goes to microbial biomass.
- microbial biomass: in the first soil layer `abco2` is released as CO2. In deeper layers the CO2 fraction is calculated from sand content, $0.17 + 0.0068\,\text{sand}$, and the `abco2` of group 2 is not used. The fraction going to passive humus is $0.003 + 0.00032\,\text{clay}$ in all layers.

The actual transformations are reduced when there is not enough mineral N to meet the N demand of the receiving pools. See soil carbon for the full pool structure.

Calibration

The parameter names `hp_rate`, `hs_rate`, `microb_rate`, `meta_rate`, `str_rate`, `microb_top_rate`, `hs_hp`, `a1co2`, `asco2`, `apco2` and `abco2` are recognised in `calibration.cal` with object type `bsn`, and the layer group is meant to be given as the layer of the change. In 62.0.1, basin parameters are passed to the change routine without a layer number, so these changes are skipped and have no effect (see parameter change files). Calibrating these rates means editing `carbon_lyr.bsn`.

Example

`refdata/Ames_sub1/carbon_lyr.bsn` in the SWAT+ repository:

```
carbon_lyr.bsn: derived from carb_coefs.cbn (per-layer carbon coefficients)
           layer         hp_rate         hs_rate     microb_rate       meta_rate        str_rate microb_to
               1         1.2e-05        2.92e-04          0.0164          0.0405          0.0107
               2         1.2e-05        1.81e-04            0.02          0.0507          0.0134
```

The first soil layer decomposes metabolic and structural litter more slowly than the deeper layers (0.0405 and 0.0107 per day, against 0.0507 and 0.0134) and slow humus faster (2.92e-04 against 1.81e-04 per day). The 0.0 for `abco2` in group 2 has no effect, because deeper layers calculate that fraction from their sand content. `microb_rate` in group 1 and `microb_top_rate` in group 2 are not used.

Related files

- carbon.bsn: basin-wide carbon parameters; its name sets the name of this file.
- codes.bsn: `carbon = 2` turns the carbon model on.
- soils.sol: layer depths, clay and sand contents.
- Carbon outputs: per-layer pools, fluxes and CO2.
- Soil carbon: description of the carbon model.

Sources

- Reader: second part of carbon_bsn_read.f90
- Type definitions: `type carbon_inputs` (array `carbdb(2)`) and `type organic_allocations` (array `org_allo(2)`) in carbon_module.f90
- Use: cbn_zhang2.f90
- Surface layer: soils_init.f90
- Calibration: cal_parm_select.f90, cal_conditions.f90

Column	Fortran variable
`layer`	`layer_id`, index into `carbdb` and `org_allo`
`hp_rate`, `hs_rate`, `microb_rate`, `meta_rate`, `str_rate`, `microb_top_rate`, `hs_hp`	`carbdb(layer)%...` of the same names
`a1co2`, `asco2`, `apco2`, `abco2`	`org_allo(layer)%...` of the same names

Purpose

The connect files describe how water and the material it moves travel from one spatial object to the next. Every spatial object (HRU, routing unit, aquifer, channel, reservoir, recall, export coefficient, delivery ratio, outlet, gwflow cell) has one row in the connect file for its object type. The row gives the object its number and name, its area, location and elevation, the record it uses in its property file, its weather station, and the list of objects that receive its outflow. SWAT+ builds the routing network and the order in which objects are run from these rows.

All connect files have the same format. The file names are given on the `connect` line of file.cio, in this order:

Position on the connect line	Default name	Object	Count column in object.cnt	Property column header	Property file
1	hru.con	HRU	hru	hru	hru-data.hru
2	hru-lte.con	HRU-lte	lhru	hlt	hru-lte.hru
3	rout_unit.con	Routing unit	rtu	rtu	rout_unit.rtu
4	gwflow.con	gwflow cell	mfl		see gwflow
5	aquifer.con	Aquifer	aqu	aqu	aquifer.aqu
6	aquifer2d.con	2-D aquifer	aqu2d		not read in 62.0.1
7	channel.con	Channel (legacy)	cha		channel.cha
8	reservoir.con	Reservoir	res	res	reservoir.res
9	recall.con	Recall (point source or inlet)	rec	rec	recall.rec
10	exco.con	Export coefficient	exco	exc	exco.exc
11	delratio.con	Delivery ratio	dlr	dlr	delratio.del
12	outlet.con	Outlet	out	out	none
13	chandeg.con	SWAT-DEG channel	lcha	lcha	channel-lte.cha

A file is read only when the matching count in object.cnt is above zero. When a count is above zero the file has to exist, because nothing checks for it: a missing file leaves those objects without connection data. The file named in position 6 is never opened in 62.0.1.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one object. SWAT+ reads as many rows as the count for that object type in object.cnt. It does not match columns by header name, so every row must give the values below in this order.

Each row has 13 leading values followed by out_tot groups of four values that describe the outflow connections, all on the same line.

#	Column	Units	Description
1	id	-	Object number within its type. Number the rows 1, 2, 3 and so on in file order: the obj_id column of other connect files refers to an object by its row position, while SWAT+ uses id as the object number during the run.
2	name	-	Object name, up to 16 characters. It appears in the output files and is not used in any calculation.
3	gis_id	-	Object number in the GIS interface. Not used in calculations. The SWAT+ Editor renumbers objects consecutively when it writes the files, so gis_id and id often differ; use id when matching a channel to a stream gauge.
4	area	ha	Area of the object. See Area, location and elevation.
5	lat	decimal degrees	Latitude.
6	lon	decimal degrees	Longitude. Not used in calculations.
7	elev	m	Elevation.
8	property column (hru, hlt, rtu, aqu, lcha, res, rec, exc, dlr, out)	-	Row number in the property file listed in the table above. Called obj in earlier documentation. In recall.con this column is not used: a recall object uses the recall.rec record whose id equals the recall object's id.
9	wst	-	Weather station name from weather-sta.cli. See Weather station lookup.
10	cst	-	Constituent pointer. Read but not used.
11	ovfl	-	Overbank flooding pointer. Read but has no effect in 62.0.1.
12	rule	-	Name of a flow-control decision table in flo_con.dtl, or null. See Flow-control rule.
13	out_tot	-	Number of outflow connections that follow on the line. May be 0.

For each of the out_tot connections, four values follow:

Offset	Column	Units	Description
+1	obj_typ	-	Type code of the receiving object. See Receiving object codes.
+2	obj_id	-	Row number of the receiving object in its connect file.
+3	hyd_typ	-	Which outflow hydrograph of this object is sent. See Hydrograph codes.
+4	frac	fraction	Fraction of that hydrograph sent to the receiving object.

When out_tot is 0 the row ends after column 13 and the object sends no outflow through this file. HRUs that belong to a routing unit are usually written this way, because the routing unit collects their outflow (see rout_unit.def and rout_unit.ele).

Area, location and elevation

The `area` value is used in calculations for some object types only:

Object	Use of `area`
HRU	Area of the HRU. Converts depths (mm) to volumes (m^3) and loads per hectare to totals.
Routing unit	Area of the routing unit. Converts inflow volumes to depths when water is routed across it.
Aquifer	Area of the aquifer. Converts recharge, storage and seepage between mm and m^3.
HRU-lte	Area of the HRU-lte. Converts runoff and sediment to totals.
SWAT-DEG channel	Used with `aqu_cha.lin` to order the channels linked to an aquifer by drainage area, and to express aquifer inflow as a depth in the channel output. See link files.
Other objects	Not used in calculations.

At the start of the run SWAT+ also adds up the drainage area of every object from the areas of the HRUs, routing units and recall objects upstream of it, weighted by `frac`, and writes the computed and the input areas to `area_calc.out`. Comparing the two columns in that file is a quick check of the routing.

`elev` is used for every object to adjust precipitation and temperature for the difference in elevation between the object and its weather data. With the lapse rates `plaps` (mm per km) and `tlaps` (°C per km) from `parameters.bsn`,

$$\Delta P = \text{plaps}\,\frac{z_{obj} - z_{pcp}}{1000}, \qquad \Delta T = \text{tlaps}\,\frac{z_{tmp} - z_{obj}}{1000}$$

where z_{obj} is `elev` and z_{pcp} and z_{tmp} are the elevations of the precipitation and temperature gauges, or of the weather generator station when the variable is simulated. For HRUs `elev` is also the elevation used to compute atmospheric pressure for potential evapotranspiration, and it can be changed in calibration with the parameter `elev`.

`lat` matters only for HRUs: an HRU with a negative latitude is treated as being in the Southern Hemisphere, and its yearly reset of base heat units and of `skip` operations happens on day 181 instead of 31 December.

Weather station lookup

The `wst` names are looked up in the list of stations read from `weather-sta.cli` with a binary search. The search assumes the stations are listed in ascending order of name (character-code order). If the stations are not in that order, or the name is not in the file, the object uses the first station in `weather-sta.cli` and no message is written.

Receiving object codes

`obj_typ` accepts these codes:

Code	Receiving object	Notes
`hru`	HRU	Inflow is routed across the HRU. See Inflow to HRUs and routing units.
`hlt`	HRU-lte	
`ru`	Routing unit	The routing unit and each HRU in it are registered as receivers.
`aqu`	Aquifer	
`cha`	Channel in `channel.con`	62.0.1 does not run these legacy channels, so flow sent to them does not continue downstream. Use `sdc` for SWAT-DEG channels.
`res`	Reservoir	

Code	Receiving object	Notes
`exc`	Export coefficient	Export coefficient objects are not run as part of the routing, so inflow to them has no effect.
`dr`	Delivery ratio	The object multiplies its inflow by the ratios in `delratio.del`.
`out`	Outlet	
`sdc`	SWAT-DEG channel in `chandeg.con`	

A recall object cannot receive flow. The routing code also has a branch for gwflow cells as receivers, but its code is longer than the three characters `obj_typ` can store, so no value in a connect file can select it. Any other code is not recognised and leaves the connection without a receiving object, which is likely to stop the run.

Hydrograph codes

`hyd_typ` selects one of the outflow hydrographs of the sending object:

Code	HRU, HRU-lte, routing unit	Aquifer	Channel, reservoir, recall, outlet, export coefficient, delivery ratio
`tot`	Surface runoff + lateral flow + tile flow	Return flow (groundwater discharge)	Total outflow
`rhg`	Percolation out of the bottom of the soil profile (recharge)	Deep seepage	Not filled, sends nothing
`sur`	Surface runoff	not used	not used
`lat`	Lateral soil flow	not used	not used
`til`	Tile flow	not used	not used

Any other code is not recognised. The usual connections are `tot` from an HRU or routing unit to a channel together with `rhg` to its aquifer, and `tot` from an aquifer to a channel.

`frac` is not checked. The fractions of one hydrograph type from one object should add up to 1, otherwise water is created or lost. Fractions of different types are independent, which is why a routing unit can send 100% of `tot` to a channel and 100% of `rhg` to an aquifer. Sending both `tot` and one of its parts (`sur`, `lat`, `til`) counts that part twice.

Inflow to HRUs and routing units

An HRU or routing unit that receives flow keeps surface, lateral and tile inflow separate, so it can route each through the right part of the landscape:

- From another HRU, HRU-lte or routing unit with `tot`, the sender's surface runoff is added to surface runon, its lateral flow to lateral inflow and its tile flow to tile inflow.
- From another HRU, HRU-lte or routing unit with `sur`, `lat` or `til`, that hydrograph goes to surface runon, lateral inflow or tile inflow. An `rhg` connection to an HRU or routing unit is accepted, but the water is not added anywhere.
- From any other object (channel, aquifer, reservoir, recall and so on) the total outflow is added to surface runon, whatever `hyd_typ` says.

For all other receiving objects the selected hydrograph, multiplied by `frac`, is added to the object's inflow.

Flow-control rule

A `rule` name other than `null` is matched against the decision tables in `flo_con.dtl` when the files are read. In 62.0.1 the match is stored but no routine uses it, so `rule` does not change the outflow fractions during the run.

Export coefficient objects

In `exco.con` the `exc` column is the row number in `exco.exc`. The `om` name in that row selects a row of `exco_om.exc`, and that row becomes the object's outflow hydrograph. It is set once when the files are read and stays constant for the whole run. Send it on with `hyd_typ tot`.

Groundwater flow (gwflow) models

When the gwflow module is active (`gwflow` = 1 in `codes.bsn`) and `object.cnt` lists gwflow cells, any row that has at least one connection with `obj_typ aqu` has its connection count reduced by one after reading, because gwflow replaces the aquifer objects. The connection that is dropped is the last one on the line, which is the aquifer connection only when that connection is written last. Put aquifer connections at the end of the line in gwflow models.

Routing loops

SWAT+ orders the objects so that every object runs after all objects that send it flow. If the connections contain a loop, the ordering cannot finish: after 5000 passes SWAT+ writes the objects that could not be ordered, with their inflow objects, to `looping.con`, prints an error and stops.

Example

`refdata/Osu_1hru/aquifer.con` in the SWAT+ repository:

```
aquifer.con: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                gis_id          area           lat           lon          elev       aqu
       1  aqu001                   1      10.00000      35.64206     127.38513     322.26395         1
```

One aquifer of 10 ha that uses row 1 of `aquifer.aqu` and sends all of its return flow (`tot`) to SWAT-DEG channel 1.

`refdata/Osu_1hru/rout_unit.con`:

```
rout_unit.con: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                gis_id          area           lat           lon          elev       rtu
       1  rtu001                   1       10.0000      35.52014     127.32787     113.51276         1
```

The routing unit sends its surface, lateral and tile flow (`tot`) to SWAT-DEG channel 1 and its recharge (`rhg`) to aquifer 1, each with fraction 1.0. The `rule` value `0` does not name a table in `flo_con.dtl`, so it has the same effect as `null`.

`refdata/Osu_1hru/hru.con`:

```
hru.con: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                gis_id          area           lat           lon          elev       hru
       1  hru0001                  1           10       35.65311     127.36763     350.96629         1
```

The HRU has no connections in `hru.con` (`out_tot` = 0). Its outflow reaches the channel and the aquifer through the routing unit it belongs to.

Related files

- `object.cnt`: number of objects of each type, and so the number of rows read from each connect file.
- `file.cio`: names the connect files on its `connect` line.
- `hru-data.hru`, `aquifer.aqu`, `reservoir.res`, `recall.rec` and the other property files named in the first table.
- `aqu_cha.lin`: read right after `aquifer.con` to link an aquifer to more than one channel.
- `weather-sta.cli`: station names used in `wst`.

- Decision tables: `flo_con.dtl`, named in `rule`.
- `parameters.bsn`: lapse rates applied with `elev`.
- Recall, export coefficient and delivery ratio objects in the model theory.

Sources

- Reader: `hyd_read_connect.f90`, called once per object type from `hyd_connect.f90`, which also resolves receiving objects, hydrograph codes, drainage areas and the run order
- Type definitions: `type object_connectivity`, `type spatial_objects`, `type object_total_hydrographs` in `hydrograph_module.f90`
- File names: `type input_con` in `input_file_module.f90`, read in `readcio_read.f90`
- Inflow summing and hydrograph use: `command.f90`, `hru_hyds.f90`, `aqu_1d_control.f90`
- Area, elevation and latitude: `hrudb_init.f90`, `topohyd_init.f90`, `cli_lapse.f90`, `et_pot.f90`, `time_control.f90`
- Weather station lookup: `search.f90`, station list from `cli_staread.f90`
- Rule lookup: `dtbl_flocon_read.f90`
- Export coefficient hydrographs: `exco_read_om.f90`
- `aqu_cha.lin` reader: `aqu2d_read.f90`
- `area_calc.out` unit opened in `proc_bsn.f90`

Column	Fortran variable
`id`	`ob(i)%num`
`name`	`ob(i)%name`
`gis_id`	`ob(i)%gis_id`
`area`	`ob(i)%area_ha`
`lat`	`ob(i)%lat`
`lon`	`ob(i)%long`
`elev`	`ob(i)%elev`
property column	`ob(i)%props`
`wst`	`ob(i)%wst_c` (resolved to `ob(i)%wst`)
`cst`	`ob(i)%constit`
`ovfl`	`ob(i)%props2`
`rule`	`ob(i)%ruleset` (resolved to `ob(i)%flo_dtbl`)
`out_tot`	`ob(i)%src_tot`
`obj_typ`	`ob(i)%obtyp_out(k)`
`obj_id`	`ob(i)%obtypno_out(k)` (resolved to `ob(i)%obj_out(k)`)
`hyd_typ`	`ob(i)%htyp_out(k)` (resolved to `ob(i)%ihtyp_out(k)`)
`frac`	`ob(i)%frac_out(k)`

Purpose

`hru-data.hru` lists the HRU property sets of the project. Each row names one record in each of the parameter files an HRU needs: topography, hydrology, soil, land use and management, soil and plant initialization, surface storage (wetland), snow and field. An HRU picks its row through the `hru` column of `hru.con`, which gives the `id` of a row in this file. The file is named on the `hru` line of `file.cio` (default `hru-data.hru`).

The parameter values themselves are in the files the names point to. This file only links them together, so several HRUs can share one hydrology or topography record while having different soils or land uses.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one property set with ten values in the order below. The reader does not match columns by header name, so all ten values must be present and in this order.

#	Column	Looked up in	Description
1	id	-	Number of the property set. The hru column of hru.con refers to this value. The numbers do not have to be consecutive; the largest id sets the size of the table.
2	name	-	Name of the property set, up to 40 characters. Not used in calculations.
3	topo	topography.hyd	Topography record: slope, slope lengths and deposition coefficient.
4	hydro	hydrology.hyd	Hydrology record: lateral flow, canopy storage, evaporation and percolation parameters.
5	soil	soils.sol	Soil profile.
6	lu_mgt	landuse.lum	Land use and management record, which sets the plant community, management schedule, curve numbers, conservation practices and structures such as tile drains.
7	soil_plant_init	soil_plant.ini (see initial conditions)	Initial soil water, soil nutrients and constituents. The names of the soil test, pesticide, pathogen, heavy metal, salt and constituent records given in that row are matched at the same time.
8	surf_stor	wetland.wet	Surface storage (wetland or paddy) record, or null for an HRU without surface storage.
9	snow	snow.sno	Snow parameter set.
10	field	field.fld	Field dimensions, or null to use the built-in values.

A name must match the record name in the target file character for character, including case. Names in topography.hyd, hydrology.hyd and field.fld are stored with at most 16 characters and soil names with at most 20, so a longer name in this file can never match a record there.

Name lookup

Each name is looked up once, when the files are read. The table gives what happens when a name is not found:

Column	null or a name that is not found
topo, hydro, soil, lu_mgt, soil_plant_init, snow	The HRU uses the first record of the target file. No message is written.
surf_stor	null means no surface storage. Any other name that is not in wetland.wet is reported in diagnostics.out as <name> not found (wetland.wet) and the HRU has no surface storage.
field	The HRU uses the built-in field values (length 500 m, width 100 m, angle 30°). A name other than null that is not found is reported in diagnostics.out as <name> not found (field.fld).

The reader has checks that would write not found messages for topo, hydro, soil, lu_mgt, soil_plant_init and snow, but they test for a pointer of zero while the pointers start at one, so in 62.0.1 these messages are never written. Check these names by hand, or in the SWAT+ Editor, before a run.

For snow this means that null selects the first row of snow.sno, the same as an unmatched name. A project that has no snow.sno file at all has no first row to fall back on.

The surf_stor name is matched when the wetlands are initialized, which happens only when wetland.wet has at least one record.

Tile drainage

The lu_mgt record decides whether an HRU has tile drains. For a tile-drained HRU, SWAT+ replaces two values from its hydro record at the start of the run: cn3_swf is set to 0.95 and perco to 0.1. Calibration changes to

`cn3_swf` and `perco` are also skipped for these HRUs. See hydrology.hyd.

Example

`refdata/Ames_sub1/hru-data.hru` in the SWAT+ repository (first four rows):

`hru-data.hru:`

id	name	topo	hydro	soil	lu_mgt	soil_
1	hru0001	topohru0001	hyd0001	soil_01-h1	cosy_lum	
2	hru0002	topohru0002	hyd0002	soil_02	mntill_corn_lum	
3	hru0003	topohru0003	hyd0003	soil_03-h3	cosy_lum	
4	hru0004	topohru0004	hyd0004	soil_04	cosy_lum	

Each HRU has a separate topography record, hydrology record and soil. HRU 2 uses the land use `mntill_corn_lum`, the others `cosy_lum`. No HRU has surface storage, all use the snow set `snow001`, and `field` is `null`, so every HRU uses the built-in field dimensions.

Related files

- hru.con: one row per HRU; its `hru` column selects a row of this file by `id`. The HRU's area, elevation, latitude and weather station come from hru.con.
- object.cnt: number of HRUs.
- topography.hyd, [hydrology.hyd](#input-reference-hyd-hydrologyhyd) and field.fld, soils.sol, landuse.lum, initial conditions, wetland.wet, snow.sno: the files the names point to.
- file.cio: names this file on its `hru` line.
- Water allocation: an HRU can be named as a point of diversion in point_of_diver.wro. That link is made in the water allocation files; there is no column for it here.

Sources

- Reader: hru_read.f90, called from proc_hru.f90
- Type definitions: `type hru_databases`, `type hru_databases_char`, `type hydrologic_response_unit_db` in hru_module.f90
- Copy to each HRU through `hru.con`: hrudb_init.f90
- Use of the topography, hydrology, snow and field pointers, and the tile-drain override: topohyd_init.f90
- Surface storage lookup: wet_initial.f90, called from wet_all_initial.f90
- Name lengths of the target records: hydrology_data_module.f90, topography_data_module.f90, soil_data_module.f90, landuse_data_module.f90
- Points of diversion: water_allocation_read.f90
- File name: `in_hru%hru_data` in input_file_module.f90

Column	Fortran variable	Resolved pointer
`id`	array index `i` of `hru_db`	
`name`	`hru_db(i)%dbsc%name`	
`topo`	`hru_db(i)%dbsc%topo`	`hru_db(i)%dbs%topo` into `topo_db`
`hydro`	`hru_db(i)%dbsc%hyd`	`hru_db(i)%dbs%hyd` into `hyd_db`
`soil`	`hru_db(i)%dbsc%soil`	`hru_db(i)%dbs%soil` into `soildb`
`lu_mgt`	`hru_db(i)%dbsc%land_use_mgt`	`hru_db(i)%dbs%land_use_mgt` into `lum`
`soil_plant_init`	`hru_db(i)%dbsc%soil_plant_init`	`hru_db(i)%dbs%soil_plant_init` into `sol_plt_ini`
`surf_stor`	`hru_db(i)%dbsc%surf_stor`	`hru(j)%dbs%surf_stor` into `wet_dat` (set in `wet_initial`)
`snow`	`hru_db(i)%dbsc%snow`	`hru_db(i)%dbs%snow` into `snodb`
`field`	`hru_db(i)%dbsc%field`	`hru_db(i)%dbs%field` into `field_db`

Purpose

`hru-lte.hru` describes the HRU-lte objects, a simplified kind of HRU. An HRU-lte has one soil layer described by a texture class, a single plant whose growing season is started and ended by decision tables, a curve number for surface runoff, separate shallow and deep groundwater stores and a fixed USLE factor for erosion. It computes no nutrients, pesticides or bacteria. HRU-lte objects can be used alone or together with ordinary HRUs, and they send water and sediment to channels, routing units and other objects like any other spatial object.

The file is named in the second position of the `hru` line of file.cio (default `hru-lte.hru`). The objects themselves are defined in `hru-lte.con`, the second file on the `connect` line, and their number is the `lhru` count in object.cnt. The `hlt` column of `hru-lte.con` gives the `id` of the `hru-lte.hru` row each object uses. The area, latitude, elevation and weather station of the object come from `hru-lte.con`; see the connection file format.

`hru-lte.hru` is read only when the file exists and is not named `null`. When `lhru` is above zero, the file must be present: without it the HRU-lte objects have no properties and the run fails. A project with HRU-lte objects also needs `soils_lte.sol` (texture classes), `plants.plt` (plant parameters) and `lum.dtl` (the growing-season decision tables).

hru-lte.hru

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one HRU-lte with 35 values in the order below. The reader does not match columns by header name, so all 35 values must be present and in this order. Text values must not contain spaces.

#	Column	Units	Typical range	Description
1	`id`	-	1 and up	Row number. The `hlt` column of `hru-lte.con` refers to the row by this number. Ids must be unique; the rows may be in any order.
2	`name`	-	-	Name of the HRU-lte, up to 16 characters. Not used in calculations; the names in the output files come from `hru-lte.con`.
3	`area`	km² (in the source)	-	Drainage area. Used only to estimate the time of concentration when `t_conc` is zero, which does not work in 62.0.1 (see Time of concentration). The SWAT+ Editor writes the HRU area in hectares here.
4	`cn2`	-	35 to 98	SCS curve number for moisture condition II. See Surface runoff.
5	`cn3_swf`	fraction	0 to 0.95	Soil water factor for the curve number: the soil water content at which the retention parameter reaches its condition III value, as a fraction of the way from field capacity (0) to saturation (1). A value of 1 makes the shape coefficients of the retention curve infinite; use a value below 1. The SWAT+ Editor writes 0.
6	`t_conc`	min	10 to 1000	Time of concentration. Sets the peak runoff rate for erosion, and the delivery ratio when the HRU-lte is an element of a routing unit with a calculated delivery ratio. Give a value above zero. The SWAT+ Editor writes 26.
7	`soil_dp`	mm	300 to 3000	Depth of the soil profile. Multiplies the available water capacity and porosity of the soil texture.

#	Column	Units	Typical range	Description
8	`perc_co`	fraction	0 to 1	Percolation threshold: percolation from the soil profile starts when soil water exceeds `perc_co` times the available water capacity. Called `perco_co` in earlier documentation, which gave a range of 0 to 6000. See Percolation.
9	`slp`	m/m	0 to 0.6	Average land surface slope. Used for lateral flow and the USLE slope factor.
10	`slp_len`	m	10 to 150	Slope length. Used for lateral flow and the USLE slope length factor.
11	`et_co`	-	0.8 to 1.2	Evapotranspiration coefficient. Multiplies potential evapotranspiration, and again actual evapotranspiration. Values below 0.4 make evapotranspiration outside the growing season negative. See Evapotranspiration.
12	`aqu_sp_yld`	-	0.01 to 0.3	Specific yield of the shallow aquifer. Read but not used.
13	`alpha_bf`	1/day	0 to 1	Baseflow recession constant of the groundwater store. See Groundwater.
14	`revap`	fraction	0 to 0.2	Revap coefficient: water removed from the shallow groundwater store each day as a fraction of the day's actual evapotranspiration.
15	`rchg_dp`	fraction	0 to 1	Fraction of the day's percolation that moves from the shallow store to the deep store.
16	`sw_init`	fraction	0 to 1	Initial soil water as a fraction of the available water capacity.
17	`aqu_init`	mm	-	Initial shallow aquifer storage. Read but not used: the store starts at zero.
18	`aqu_sh_flo`	mm	-	Initial groundwater flow. Read but not used: the flow starts at zero.
19	`aqu_dp_flo`	mm	-	Initial deep aquifer storage. Read but not used: the store starts at zero.
20	`snow_h2o`	mm H_2O	-	Initial snow water content. Read but not used: the snowpack starts at zero.
21	`lat`	degrees	-90 to 90	Latitude. Read, and used only in a day length calculation whose result is not used, so it has no effect.
22	`soil_text`	-	-	Soil texture class, matched against the `texture` column of `soils_lte.sol`. See Name matching.
23	`trop_flag`	-	`non_trop`, `trop`	Tropical flag. Read but not used.
24	`grow_start`	-	-	Name of the `lum.dtl` decision table that starts the growing season, usually `pl_grow_sum` (or `pl_grow_win` for winter crops).
25	`grow_end`	-	-	Name of the `lum.dtl` decision table that ends the growing season and computes the yield, usually `pl_end_sum` (or `pl_end_win`).
26	`plnt_typ`	-	-	Plant name, matched against the `name` column of `plants.plt`.

#	Column	Units	Typical range	Description
27	`stress`	fraction	0 to 1	Plant stress factor for stresses the HRU-lte does not simulate (pests, root restriction, soil quality, nutrients). It multiplies daily biomass growth; 1 means no such stress. With 0 no biomass grows and the yield is zero. The SWAT+ Editor writes 0.
28	`pet_flag`	-	`harg`, `p_t`	Potential evapotranspiration method: `harg` for Hargreaves; any other value selects Priestley-Taylor.
29	`irr_flag`	-	-	Irrigation switch. See Irrigation for the values that switch irrigation on in 62.0.1.
30	`irr_src`	-	`outside_bsn`, `shal_aqu`, `deep_aqu`	Source of irrigation water: the HRU-lte's shallow or deep groundwater store, or an unlimited source outside the basin (any value other than `shal_aqu` and `deep_aqu`).
31	`t_drain`	hr	0, or 24 to 72	Time to drain the soil to field capacity through tile drains. 0 means no tile drains.
32	`usle_k`	0.013 t m² h/(m³ t cm)	0 to 0.65	USLE soil erodibility factor K.
33	`usle_c`	-	0.001 to 0.5	USLE cover and management factor C. It is constant through the year. The SWAT+ Editor writes the `usle_c_min` of the plant.
34	`usle_p`	-	0 to 1	USLE support practice factor P.
35	`usle_ls`	-	-	USLE slope length and steepness factor LS. Read but not used: SWAT+ computes LS from `slp` and `slp_len`.

Name matching

The text columns are compared character by character with the names in the other files. The comparison is case-sensitive and trailing blanks are ignored.

- `soil_text` is looked up among the twelve texture classes of `soils_lte.sol` (see the soils_lte.sol format). The matching class gives the available water capacity and porosity per mm of soil depth and the saturated hydraulic conductivity. If no class matches, the HRU-lte has no soil water capacity, and the run fails or produces invalid numbers when it divides by it. The SWAT+ Editor datasets contain the classes `sand`, `loamy_sand`, `sandy_loam`, `loam`, `silt_loam`, `silt`, `silty_clay_loam`, `clay_loam`, `sandy_clay_loam`, `sandy_clay`, `silty_clay` and `clay`, and the Editor picks one for each HRU-lte from the sand, silt and clay content of the first soil layer.
- `plnt_typ` is looked up in the `name` column of `plants.plt`. An unknown name writes `<name> not found (plants.plt)` to `diagnostics.out`, and the HRU-lte grows with the built-in default plant record, whose leaf area curve is not set up.
- `grow_start` and `grow_end` are looked up by table name in `lum.dtl`. An unknown name writes `<name> entry in (hru-lte.hru) not found in (lum.dtl)` to `diagnostics.out`; the run then refers to a decision table that does not exist, so both names must be present in `lum.dtl`.
- `pet_flag` is compared with `harg` only.
- `irr_flag` and `irr_src` are compared as described under Irrigation.

Daily water balance

Each HRU-lte runs once a day, in the order of the object list. The steps below follow that order. SW is the soil water content (mm), AWC and POR are the available water capacity and the porosity of the profile (mm), and

K_{sat} is the saturated hydraulic conductivity of the texture class (mm/h):

$$AWC = awc \cdot \text{soil_dp}, \qquad POR = por \cdot \text{soil_dp}, \qquad SW_0 = \text{sw_init} \cdot AWC$$

where *awc* and *por* are the per-mm values of the texture class in `soils_lte.sol`.

Snow

When the mean air temperature $\bar{T}$ is below 0 °C, the whole day's precipitation is added to the snowpack and there is no runoff or infiltration that day. Otherwise snowmelt is

$$SNO_{mlt} = 4.57\,\bar{T}$$

limited to the snowpack, and rain and melt together form the water input P for the day. The `snow.sno` parameters are not used by HRU-lte objects.

Surface runoff

Surface runoff uses the SCS curve number method with a retention parameter that varies with soil water (see SCS curve number). At the start of the run the condition I and III curve numbers are derived from `cn2`:

$$CN_1 = CN_2 - \frac{20\,(100 - CN_2)}{100 - CN_2 + \exp\,(2.533 - 0.063\,(100 - CN_2))}, \qquad CN_1 \geq 0.4\,CN_2$$

$$CN_3 = CN_2 \exp\,(0.00673\,(100 - CN_2))$$

$$S_{max} = 254\left(\frac{100}{CN_1} - 1\right), \qquad S_3 = 254\left(\frac{100}{CN_3} - 1\right)$$

Each day the retention parameter is

$$S = S_{max}\left(1 - \frac{SW}{SW + \exp(w_1 - w_2\,SW)}\right), \qquad S \geq 3$$

The shape coefficients w_1 and w_2 are fixed at the start so that $S = S_3$ when $SW = AWC + \text{cn3_swf}\,(POR - AWC)$, and $S = 2.54$ when $SW = POR$. Runoff is

$$Q_{surf} = \frac{(P - 0.2\,S)^2}{P + 0.8\,S} \quad \text{for } P > 0.2\,S$$

and zero otherwise. The rest of P is added to the soil water. SWAT+ also stores `cn2` limited to 35 to 98, but the runoff curve is built from the value as given in the file, so values outside that range are used as they are.

Evapotranspiration

Potential evapotranspiration is computed with the method chosen in `pet_flag` and multiplied by `et_co`. With Hargreaves,

$$PET = \text{et_co} \cdot 0.0032\,\frac{H_0}{2.5 - 0.0022\,\bar{T}}\,(\bar{T} + 17.8)\,(T_{max} - T_{min})^{0.6}$$

where H_0 is the day's maximum possible solar radiation (MJ/m²). With Priestley-Taylor,

$$PET = \text{et_co} \cdot 1.28 \, \frac{\Delta}{\Delta + 0.68} \cdot \frac{23.9\, H\,(1 - 0.23)}{58.3}, \qquad \Delta = \frac{5304}{T_K^2} \exp\left(21.255 - \frac{5304}{T_K}\right)$$

where H is the day's solar radiation (MJ/m²) and T_K the mean air temperature in kelvin. Both are simpler forms than those used for ordinary HRUs (see Potential evapotranspiration).

Actual evapotranspiration is proportional to the soil water content:

$$E_a = b \cdot PET \, \frac{SW}{AWC}, \qquad b = \begin{cases} \text{et_co} & \text{during the growing season} \\ \text{et_co} - 0.4 & \text{outside it} \end{cases}$$

It is taken from the soil after lateral flow, tile flow and percolation, and is limited to the soil water left at that point. Because `et_co` enters both PET and b, a change in `et_co` changes actual evapotranspiration roughly with its square during the growing season.

Lateral flow

When soil water is above the available water capacity,

$$Q_{lat} = 0.024 \, \frac{SW - AWC}{POR - AWC} \, K_{sat} \, \frac{\text{slp}}{\text{slp_len}}$$

limited to the soil water.

Tile flow

When `t_drain` is above zero and soil water is above the available water capacity,

$$Q_{tile} = (SW - AWC)\left(1 - \exp\left(\frac{-24}{\text{t_drain}}\right)\right), \qquad Q_{tile} \le 10 \text{ mm}$$

Percolation

Water above `perc_co` times the available water capacity drains from the bottom of the profile:

$$w_{perc} = (SW - \text{perc_co} \cdot AWC)\left(1 - \exp\left(\frac{-24}{TT}\right)\right), \qquad TT = \max\left(2, \; \frac{POR - AWC}{K_{sat}}\right)$$

where TT is the travel time in hours. With `perc_co` at 1 only water above field capacity percolates; with `perc_co` at 0 (the value the SWAT+ Editor writes) the profile can drain well below field capacity.

Groundwater

The shallow and deep groundwater stores of an HRU-lte are separate from the aquifer objects. Percolation is added to the shallow store. Groundwater flow follows a recession with $\alpha =$ `alpha_bf`:

$$Q_{gw,d} = Q_{gw,d-1} \, e^{-\alpha} + w_{perc} \, (1 - e^{-\alpha})$$

limited to the shallow storage. Then revap ($\text{revap} \cdot E_a$) and deep percolation ($\text{rchg_dp} \cdot w_{perc}$) are taken from what is left in the shallow store, each limited to the storage. Deep percolation is added to the deep store, which only irrigation from `deep_aqu` draws on. Revap leaves the HRU-lte; it is not added to the soil or to the reported evapotranspiration. Both stores and the groundwater flow start at zero.

Water yield

The water the HRU-lte sends downstream each day is

$$WYLD = Q_{surf} + Q_{lat} + Q_{tile} + Q_{gw}$$

In the `hru-lte_wb` output files, `latq` is the sum of lateral flow and groundwater flow.

Plant growth

The plant named in `plnt_typ` grows between the two decision tables. On each day outside the growing season the `grow_start` table is checked; when its `grow_init` action runs, growth starts with zero biomass and leaf area. During the season the `grow_end` table is checked; its `grow_end` action computes the yield and ends the season. The `pl_grow_sum` and `pl_end_sum` tables of the SWAT+ reference data start growth when the fraction of the year's base-zero heat units at the weather station is between 0.15 and 0.2, and end it when the fraction is above 0.5 or on day 330. See decision table actions.

Heat units to maturity

At the start of the run SWAT+ estimates the heat units to maturity, PHU, from the long-term monthly mean temperatures in the weather generator of the HRU-lte's station. Daily values of $\bar{T} - T_{base}$ above zero are summed over the days between a start and an end day, and

$$PHU = \max(500,\ 0.9 \sum(\bar{T} - T_{base}))$$

limited to 2000 for annual plant types (`warm_annual`, `cold_annual` and their tuber forms). In 62.0.1 the start and end days of the sum are the positions of the `grow_start` and `grow_end` tables in `lum.dtl` (first table 1, second table 2, and so on). The days of the year of the growing season are not used. For two tables next to each other in `lum.dtl` the sum covers no days and PHU takes its minimum of 500.

Biomass and leaf area

During the growing season the fraction of heat units g increases each day by $(\bar{T} - T_{base})/PHU$. Biomass grows with intercepted radiation:

$$\Delta B = \gamma \cdot \text{stress} \cdot bm_e \cdot 0.5\, H \left(1 - e^{-0.65\,(LAI+0.05)}\right), \qquad \gamma = \min(ws,\ ts,\ as)$$

where bm_e is `bm_e` of `plants.plt`, $ws = E_a/PET$ is the water stress factor (before the adjustment b), ts the temperature stress factor and as the aeration stress factor:

$$ts = \exp\left(-0.1054 \left(\frac{T_{opt} - \bar{T}}{2T_{opt} - T_{base} - \bar{T}}\right)^2\right) \text{ for } \bar{T} > T_{base}, \qquad ts = 0 \text{ otherwise}$$

Aeration stress starts when SW exceeds AWC by more than 85% of the drainable range $POR - AWC$. Leaf area follows the leaf area curve of the plant (`frac_hu1`, `lai_max1`, `frac_hu2`, `lai_max2` and `lai_pot` in `plants.plt`), with the daily increase scaled by $\sqrt{\gamma}$. See plant growth for the background of these relations.

Yield

When the season ends, the yield is

$$Y = 0.8\, B\, HI', \qquad HI' = (HI - HI_{min}) \frac{wur}{wur + \exp(6.13 - 0.0883\, wur)} + HI_{min}$$

where HI is `harv_idx`, HI_{min} is `harv_idx_ws` of `plants.plt`, and wur is 100 times the ratio of actual to potential evapotranspiration summed over the season (100 when the potential sum is below 10 mm). Biomass and leaf area are then set to zero.

Plant growth has no effect on the water balance or on erosion: evapotranspiration does not depend on leaf area, and `usle_c` is constant. The plant affects the water balance only through the growing season itself (the factor b) and through irrigation.

Irrigation

During the growing season, on days when the water stress factor ws is below 0.8, the soil is filled to the available water capacity with at most 76.2 mm. With `irr_src` set to `shal_aqu` or `deep_aqu` the water is taken from the HRU-lte's shallow or deep store and limited to what it contains; any other value takes it from an unlimited source outside the basin.

In 62.0.1 irrigation is switched on when `irr_flag` sorts alphabetically after `no_irr`. The value `irr`, listed as the "irrigation" value in earlier documentation, sorts before `no_irr` and so leaves irrigation off. Values such as `yes` or `sprinkler` switch it on.

Erosion

Sediment yield is computed with MUSLE (see MUSLE). The slope length and steepness factor is computed at the start of the run from `slp` and `slp_len`; the `usle_ls` column is not used:

$$LS = \left(\frac{\text{slp_len}}{22.128}\right)^{m} \left(65.41\,\sin^2\theta + 4.56\,\sin\theta + 0.065\right), \qquad m = 0.6\left(1 - e^{-35.835\,\text{slp}}\right)$$

with $\theta = \arctan(\text{slp})$. Sediment from surface runoff is

$$sed = 11.8\,(Q_{surf}\;q_{peak} \cdot 1000\,A)^{0.56}\;K\,C\,P\,LS$$

with K, C and P from `usle_k`, `usle_c` and `usle_p`, and A the `area` of `hru-lte.con`. A concentration of 500 mg/L in lateral and groundwater flow is added to this. There is no snow cover or residue effect.

Time of concentration

The peak runoff rate uses the time of concentration t_c in seconds (`t_conc` times 60):

$$q_{peak} = \frac{2\,Q_{surf} \cdot 1000\,A}{1.5\,t_c} + \frac{(Q_{lat} + Q_{gw}) \cdot 1000\,A}{86400}$$

When `t_conc` is zero, SWAT+ tries to estimate it with the Kirpich formula, using `area` as the channel length. The slope in that formula is always zero in 62.0.1, so the estimate is infinite: depending on how the model was compiled the run either stops with a floating-point error or continues with no surface runoff in the peak rate. Give `t_conc` a value above zero.

Area units

The HRU-lte calculations convert a depth D in mm to a volume with $1000\,D\,A$ and report sediment yield per $100\,A$, which are the conversions for an area A in km². A is the `area` column of `hru-lte.con`, which is in hectares for every other object type and which the SWAT+ Editor writes in hectares. With hectares, the flow volumes and sediment loads an HRU-lte sends downstream are 100 times the depth over its area, and the peak rate and sediment yield in the output are affected in the same way. Only the total hydrograph is filled: an HRU-lte connection in `hru-lte.con` that asks for the surface, lateral, tile or recharge hydrograph sends nothing.

Calibration

Parameters with the suffix `_lte` in `calibration.cal` change `hru-lte.hru` values (see simplified HRU parameters). The changes are applied after the file has been read and the HRU-lte objects have been set up, so in 62.0.1 they have an effect only for values the daily calculation reads directly from the file:

- effective: `awc_lte` (available water capacity of the object), `slope_lte` and `slopelen_lte` (lateral flow only), `revapc_lte`, `percc_lte`, `abf_lte` and `tc_lte`
- no effect: `cn2_lte`, `etco_lte`, `soildep_lte`, `sw_lte`, `sy_lte`, `gw_lte`, `gwflow_lte`, `gwdeep_lte`, `snow_lte`, `uslek_lte`, `uslec_lte`, `uslep_lte` and `uslels_lte`

`abf_lte` changes the daily recession factor $e^{-\alpha}$ in place of `alpha_bf`, and `tc_lte` changes the time of concentration in seconds. Except for `awc_lte`, a change to object number n is applied to the `hru-lte.hru` row with `id` n; the two agree when row numbers follow the objects, as the SWAT+ Editor writes them.

Example

No `hru-lte.hru` is included in the SWAT+ reference data. The file below is illustrative, with the column headers the SWAT+ Editor writes.

`hru-lte.hru: illustrative example`

id	name	area	cn2	cn3_swf	t_conc	soil_dp	perc_co	slp	slp_len	et_co	aqu_sp_yld	alpha_bf	rev
1	hlt001	52.3	77.0	0.00	26.0	1500.0	1.00	0.04	60.0	1.00	0.05	0.05	0.

One HRU-lte with a 1.5 m loam profile that starts half full. Percolation starts above field capacity (`perc_co` 1). Corn grows between the `pl_grow_sum` and `pl_end_sum` tables with no extra stress (`stress` 1), there is no irrigation and no tile drainage, and potential evapotranspiration follows Hargreaves. The values of `area`, `aqu_sp_yld`, `aqu_init`, `aqu_sh_flo`, `aqu_dp_flo`, `snow_h2o`, `lat`, `trop_flag` and `usle_ls` have no effect in 62.0.1.

Related files

- Connection files: `hru-lte.con` defines the HRU-lte objects, their area, weather station and outflow connections; its `hlt` column selects a row of this file by `id`.
- Object counts: the `lhru` column of `object.cnt` gives the number of HRU-lte objects.
- Master file: `file.cio` names this file on its `hru` line, and `soils_lte.sol` on its `soils` line.
- Soil texture classes: `soils_lte.sol` supplies available water capacity, porosity and conductivity for `soil_text`.
- Plant parameters: `plants.plt` supplies the plant parameters for `plnt_typ`.
- Decision tables: the `grow_start` and `grow_end` tables in `lum.dtl`.
- Weather files: the weather station named in `hru-lte.con` supplies daily weather and, through its weather generator, the temperatures for the heat units to maturity.
- Water balance output: `hru-lte_wb` files; plant growth and stress go to the `hru-lte_pw` files.

Sources

- Reader: `hru_lte_read.f90`, called from `main.f90.in` after `dtbl_lum_read`
- Type definitions: `type swatdeg_hru_data` and `type swatdeg_hru_dynamic` in `hru_lte_module.f90`
- Daily water balance, plant growth, irrigation and erosion: `hru_lte_control.f90`, called from `command.f90`
- Retention curve shape: `ascrv.f90`
- Growing season actions `grow_init` and `grow_end`: `actions.f90`; conditions with `obj hlt`: `conditions.f90`
- Texture classes: `soil_lte_db_read.f90`, `type soil_lte_database` in `soil_data_module.f90`
- Plant parameters: `plant_data_module.f90`, `plant_parm_read.f90`
- Connection and object numbering: `hyd_connect.f90`, `hyd_read_connect.f90`
- Delivery ratio from `t_conc`: `dr_ru.f90`
- Calibration: `cal_parm_select.f90`, `cal_parmchg_read.f90`, `proc_cal.f90`
- Output: `hru_lte_output.f90`, `output_landscape_init.f90`
- File name slot: `in_hru%hru_ez` in `input_file_module.f90`, read in `readcio_read.f90`

Column	Fortran variable
id	row index of hlt_db
name	hlt_db%name
area	hlt_db%dakm2
cn2	hlt_db%cn2
cn3_swf	hlt_db%cn3_swf
t_conc	hlt_db%tc
soil_dp	hlt_db%soildep
perc_co	hlt_db%perco
slp	hlt_db%slope
slp_len	hlt_db%slopelen
et_co	hlt_db%etco
aqu_sp_yld	hlt_db%sy
alpha_bf	hlt_db%abf
revap	hlt_db%revapc
rchg_dp	hlt_db%percc
sw_init	hlt_db%sw
aqu_init	hlt_db%gw
aqu_sh_flo	hlt_db%gwflow
aqu_dp_flo	hlt_db%gwdeep
snow_h2o	hlt_db%snow
lat	hlt_db%xlat
soil_text	hlt_db%text
trop_flag	hlt_db%tropical
grow_start	hlt_db%igrow1
grow_end	hlt_db%igrow2
plnt_typ	hlt_db%plant
stress	hlt_db%stress
pet_flag	hlt_db%ipet
irr_flag	hlt_db%irr
irr_src	hlt_db%irrsrc
t_drain	hlt_db%tdrain
usle_k	hlt_db%uslek
usle_c	hlt_db%uslec
usle_p	hlt_db%uslep
usle_ls	hlt_db%uslels

Purpose

A routing unit collects the outflow of a group of objects, usually the HRUs of one landscape unit, and routes the sum as a single object. Its row in rout_unit.con sends the result on, typically the total flow to a channel and the recharge to an aquifer. Three files describe routing units: rout_unit.def lists the elements of each unit, rout_unit.ele describes each element, and rout_unit.rtu gives each unit its topography and field records.

A landscape unit is a group of HRUs used only for output. ls_unit.def lists the elements of each landscape unit and ls_unit.ele gives each element its area fractions. The same element file also weights the HRUs in the basin output files. aqu_catunit.ele lists aquifer elements; in 62.0.1 its only effect is to switch on the data rows of the basin aquifer output.

The files are named in file.cio:

File	file.cio line	Slot	Read
rout_unit.def	routing_unit (line 8)	1	when the rtu count in object.cnt is above 0
rout_unit.ele	routing_unit (line 8)	2	when the rtu count is above 0
rout_unit.rtu	routing_unit (line 8)	3	when the rtu count is above 0
ls_unit.ele	regions (line 26)	1	every run

File	file.cio line	Slot	Read
ls_unit.def	regions (line 26)	2	every run
aqu_catunit.ele	regions (line 26)	9	every run, with the calibration inputs

Slot 4 of the routing_unit line (rout_unit.dr) is read from file.cio but never opened.

All six files are free format and whitespace-separated. The readers take the values in column order and do not check the header names.

For rout_unit.rtu, a name of null or a name that does not exist in the run folder means the file is not read. The other five files are skipped when the name is null. When the name is not null and the file does not exist, SWAT+ creates an empty file of that name in the run folder, reads nothing from it, and continues as if the name were null. What a missing file means for the run is given under each file.

rout_unit.def

Format

Line 1 is a title and line 2 is the column header; both are skipped. Each later line defines one routing unit. The line ends with a list of element numbers, and only the first number of that list has a header.

#	Column	Units	Description
1	id	-	Number of the routing unit. Not used: the rows are taken in order, so row n defines routing unit n.
2	name	-	Name of the routing unit. Not used.
3	elem_tot	-	Number of values in the element list that follows. When the list uses ranges, this is smaller than the number of elements.
4 and on	elements	-	Element list: id values of rows in rout_unit.ele, written as described under Element lists.

Element lists

rout_unit.def and ls_unit.def use the same list format. Each value is an element number, and a negative value closes a range that starts at the value before it. The list 1 -4 7 therefore means elements 1, 2, 3, 4 and 7, and elem_tot is 3 because the list has three values. A list with a single value gives one element. An elem_tot of 0 gives a unit with no elements.

A negative value must follow a positive one. Two negative values in a row, or a negative first value in a list of more than one value, put negative element numbers into the list.

Behaviour

- Routing unit n is the routing unit whose id in rout_unit.con is n. It uses row n of rout_unit.def, counted from the first data line, whatever the id column says. SWAT+ reads as many rows as the rtu count in object.cnt. Extra rows are ignored, and units without a row have no elements.
- The define column of rout_unit.rtu is not used to find the row. The routing unit ids in rout_unit.con, the rows of rout_unit.def and the ids in rout_unit.rtu therefore have to run 1, 2, 3 in the same order. The SWAT+ Editor writes them that way.
- An object can be an element of one routing unit only. If two element rows of rout_unit.ele point to the same object and both are listed in rout_unit.def (in one unit or in two), SWAT+ stops at start-up with an allocation error.
- A routing unit with no elements produces no outflow, and no message is written.
- If rout_unit.def is missing or null in a project with routing units, every routing unit has no elements. The HRUs then reach the rest of the model only through their rows in hru.con, and an HRU without outflow connections there loses its water.

Example

`refdata/0su_1hru/rout_unit.def` in the SWAT+ repository:

```
rout_unit.def: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id              name  elem_tot  elements
       1             rtu001         1         1
```

Routing unit 1 has one element, element 1 of `rout_unit.ele`.

rout_unit.ele

Format

Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one element.

#	Column	Units	Description
1	`id`	-	Element number, used in the element lists of `rout_unit.def`. The rows can be in any order and the numbers do not have to be consecutive; the largest `id` sets the size of the table.
2	`name`	-	Name of the element. Not used.
3	`obj_typ`	-	Object type of the element, one of the codes below.
4	`obj_id`	-	Number of the object within its type: its `id` in the connection file of that type.
5	`frac`	fraction	Fraction of the element in the routing unit, used to scale the element's outflow. See Expansion factor.
6	`dlr`	-	Delivery ratio: `0` or `calc`, `1` or `full`. See Delivery ratio.

The object type codes SWAT+ recognises are:

`obj_typ`	Object	Connection file
`hru`	HRU	`hru.con`
`hlt`	HRU-lte	`hru-lte.con`
`ru`	routing unit	`rout_unit.con`
`cha`	legacy channel	`channel.con`
`sdc`	SWAT+ channel	`chandeg.con`
`res`	reservoir	`reservoir.con`
`exc`	export coefficient	`exco.con`
`dr`	delivery ratio	`delratio.con`
`out`	outlet	`outlet.con`

Any other code, including `aqu`, `rec` and `rtu`, is not recognised. No message is written, and the element is linked to object 1, the first object in the model (HRU 1 in a model with HRUs). Earlier documentation listed aquifers, recall objects and "another rtu" as possible elements; in 62.0.1 a routing unit inside a routing unit has the code `ru`.

Expansion factor

Each day a routing unit adds up the hydrographs of its elements. Each element's hydrographs are multiplied by an expansion factor e before they are added:

- HRU and HRU-lte elements with `frac` below 0.99999:

$$e = \text{frac}\ \frac{A_{ru}}{A_{obj}}$$

  where A_{ru} is the routing unit's `area` in `rout_unit.con` and A_{obj} the HRU's `area` in its connection file. When `frac` is the HRU's share of the routing unit area, e is 1 and the whole HRU outflow is added.

- HRU and HRU-lte elements with `frac` of 0.99999 or more: $e = 1$, so the whole HRU outflow is added whatever the areas.
- Export coefficient elements: e as for HRUs when the export coefficient object has an area above 0.01 ha, otherwise 1.
- All other elements: $e =$ frac, the fraction of the object's outflow that enters the routing unit.

The `frac` values are not checked. If they do not add up as intended, the routing unit outflow is scaled accordingly.

The factor and the delivery ratio apply to the total, surface, lateral and tile hydrographs. The recharge hydrograph is multiplied by the factor only. Pesticide, salt and constituent loads are added in full: 62.0.1 applies neither the expansion factor nor the delivery ratio to them.

When another object sends flow to a routing unit through its connection file, the inflow is shared among the unit's elements. Each element receives the sender's fraction multiplied by its `frac`. How an HRU uses that inflow is described under Inflow to HRUs and routing units.

Delivery ratio

The `dlr` column sets a ratio for each hydrograph variable of the element:

`dlr`	Ratio
`0` or `calc`	Flow 1. All other variables (sediment, nutrients, chlorophyll-a, CBOD, dissolved oxygen, sediment size classes) are multiplied by a calculated ratio, given below.
`1` or `full`	1 for every variable.
any other value	0 for every variable.

For HRU and HRU-lte elements the calculated ratio is

$$DR = \min\left(1,\ \sqrt{\frac{t_{c,\mathrm{elem}}}{t_{c,ru}}}\right)$$

where $t_{c,\mathrm{elem}}$ is the HRU's time of concentration (see Surface runoff lag), or for an HRU-lte the time of concentration of its `hru-lte.hru` record, and $t_{c,ru}$ is the routing unit's time of concentration (see Time of concentration). For `sdc` and `ru` elements the calculated ratio is 1. For the other element types no ratio is calculated, and the element takes the ratio last calculated for an earlier element (0 if there is none).

A `dlr` value that names a record in `delratio.del` does not work in 62.0.1. The names are looked up when the routing units are set up, and `delratio.del` is read only after that, so no name is ever found. The ratio then stays at 0 for every variable, and the element contributes only its recharge hydrograph. The same happens for any other value, such as `null`. Use `0`, `calc`, `1` or `full`. The SWAT+ Editor writes `0`.

The decision table action `hru_fr_update` (see decision tables) reads a replacement `rout_unit.ele` during the run. By then `delratio.del` has been read, so a `dlr` that names a `delratio.del` record does take that record's ratios from `dr_om.del`. Elements whose `dlr` is `0`, `calc`, `1`, `full` or an unknown name keep the ratios they had before.

Behaviour

- `rout_unit.ele` is required in a project with routing units. If it is missing or `null`, the routing unit tables are never set up and the run fails during start-up.
- In sub-daily runs (time step below one day) with Green and Ampt infiltration (`gampt` = 1 in `codes.bsn`), the routing unit's sub-daily flow is set from each HRU element in turn, so only the last HRU element in the list is kept, and the result is shifted by a fixed 2 hours instead of the routing unit's time of concentration.
- The `ru` row of `print.prt` writes the sum of the element hydrographs of each routing unit to the `ru_*` files (see ru files).

Example

`refdata/Osu_1hru/rout_unit.ele`:

```
rout_unit.ele: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                   obj_typ    obj_id          frac               dlr
       1  hru0001                    hru         1             1                 0
```

Element 1 is HRU 1. With `frac` = 1 the whole HRU outflow enters the routing unit. With `dlr` = 0 the water is delivered in full, and sediment and nutrients are multiplied by the calculated delivery ratio.

rout_unit.rtu

Format

Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one routing unit.

#	Column	Units	Description
1	`id`	-	Number of the routing unit. Routing unit n (the `id` in `rout_unit.con`) uses the row whose `id` is n. The ids must lie between 1 and the `rtu` count in `object.cnt`. An `id` of 0 ends the file.
2	`name`	-	Name of the routing unit. Not used.
3	`define`	-	Name of the `rout_unit.def` row. Not used: routing unit n always uses row n of `rout_unit.def`.
4	`dlr`	-	Delivery ratio of the routing unit. Not used; delivery ratios are set per element in `rout_unit.ele`.
5	`topo`	-	Name of a record in `topography.hyd`. Its slope and slope length are used for the routing unit's time of concentration.
6	`field`	-	Name of a record in `field.fld`. Its field length is used for the routing unit's time of concentration.

Time of concentration

The routing unit's time of concentration is the sum of an overland flow time and a channel flow time:

$$t_{ov} = \frac{0.0556\,(L_{slp}\,n)^{0.6}}{(slp + 0.001)^{0.3}}, \qquad t_{ch} = \frac{0.62\,L\,n^{0.75}}{A^{0.125}\;(0.5\,(slp + 0.001))^{0.375}}, \qquad t_{c,ru} = t_{ov} + t_{ch}$$

Symbol	Meaning	Units
t_{ov}, t_{ch}, $t_{c,ru}$	overland, channel and total time of concentration	hr
L_{slp}	slope length, `slp_len` of the `topo` record	m
slp	average slope, `slp` of the `topo` record	m/m
L	channel length, `len` of the `field` record divided by 1000	km
A	routing unit area, `area` in `rout_unit.con`	km²
n	Manning's n for overland flow, averaged over the HRU elements	-

The Manning's n is the area-weighted mean of the `ov_mann` values of the HRU elements' land uses (see `landuse.lum`):

$$n = \frac{\sum_i n_i\,A_i}{A}$$

where A_i is the full area of HRU element i; `frac` is not used. When the routing unit has an element that is not an HRU, the running sum is set to 0.1 at that element and the HRU elements after it are added to 0.1. The result then depends on the order of the elements.

The time of concentration is used for the calculated delivery ratio of the elements and, in sub-daily runs, for the unit hydrograph of the routing unit.

Behaviour

- A `topo` name that is `null` or not found in topography.hyd gives the first record of that file. A `field` name that is `null` or not found gives the built-in field length of 500 m. No message is written in either case.
- `rout_unit.rtu` is required in a project with routing units. If it is missing or `null`, no routing unit records are set up and the run fails during start-up.

Example

`refdata/Osu_1hru/rout_unit.rtu`:

```
rout_unit.rtu: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id              name            define               dlr              topo             field
       1            rtu01             rtu001              null       toportu011            fld011
```

`fld011` is in field.fld with a length of 500 m. `toportu011` is not in this dataset's topography.hyd, so the routing unit uses its first record, `topohru0001` (slope 0.01078, slope length 10 m), without a message. `define` and `dlr` have no effect.

ls_unit.def

Format

Line 1 is a title, line 2 gives the number of landscape units, and line 3 is the column header. Each later line defines one landscape unit, and the line ends with an element list as in `rout_unit.def`.

#	Column	Units	Description
1	`id`	-	Number of the landscape unit. Not used: row n defines landscape unit n.
2	`name`	-	Name of the landscape unit, written in the `name` column of the landscape unit output files.
3	`area`	ha	Area of the landscape unit. Not used.
4	`elem_tot`	-	Number of values in the element list. 0 gives a landscape unit with no elements, whose output is all zeros.
5 and on	`elements`	-	Element list: `id` values of rows in `ls_unit.ele`, written as described under Element lists.

SWAT+ reads as many rows as line 2 gives.

Behaviour

- Landscape units are used for output only. Each one gets a row in the `lsunit_wb`, `lsunit_nb`, `lsunit_ls` and `lsunit_pw` files and in the landscape unit carbon files (`lsu_cb_gl`, `lsu_cb_trf`, `lsu_cb_plt`), switched on in print.prt. Each value is the sum over the unit's elements of the HRU (or HRU-lte) value multiplied by the element's `sub_frac` in `ls_unit.ele`. See Aggregation.
- The element number is used as the HRU number: element n adds the output of HRU n (or HRU-lte n), and its `obj_typ_no` is not consulted. The element ids in `ls_unit.ele` therefore have to equal the HRU numbers.
- In the output files, `unit` is the landscape unit number and `gis_id` is taken from the routing unit with the same number in rout_unit.con.
- With gwflow and landscape unit connection (`conn_type` 2, see lsucell.gw), the element lists also decide which HRUs pass recharge and remaining evapotranspiration to the cells of each landscape unit. Here too element n is HRU n.
- `ls_unit.def` needs `ls_unit.ele`. If `ls_unit.def` is missing or `null`, the project has no landscape units and the landscape unit output files are not opened.

Example

`refdata/Osu_1hru/ls_unit.def`:

```
ls_unit.def: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
1
      id              name          area  elem_tot  elements
       1            rtu001       10.0000         1         1
```

One landscape unit of 10 ha, named `rtu001` after the routing unit, with element 1 of `ls_unit.ele`.

ls_unit.ele

Format

Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one element. Six values are read from each line.

#	Column	Units	Description
1	`id`	-	Element number, used in the element lists of `ls_unit.def`. The largest `id` sets the size of the table. Must equal the HRU number (see below).
2	`name`	-	Name of the element. Not used.
3	`obj_typ`	-	`hru` or `hlt`. Elements of any other type are skipped in the sums.
4	`obj_typ_no`	-	HRU or HRU-lte number.
5	`bsn_frac`	fraction	Fraction of the basin land area (`ls_area` in `object.cnt`) covered by the HRU. Weight of the HRU in the basin output files.
6	`sub_frac`	fraction	Fraction of the landscape unit area covered by the HRU. Weight of the HRU in the landscape unit output files. Called `lsu_frac` in earlier documentation; the `Ames_sub1` file labels it `RU_FRAC`.

Behaviour

- Basin output: the `basin_wb`, `basin_nb`, `basin_ls` and `basin_pw` files, including the starting soil water and snow in `basin_wb`, are the sums of the HRU values multiplied by `bsn_frac`. The fractions are used as given; if they do not add up to 1, the basin values are scaled by their sum.
- For each HRU number n, the basin sum reads row n, takes its `obj_typ_no` m, and then uses the `obj_typ` and `bsn_frac` of row m with the output of HRU m. The sum is right when row n describes HRU n, which is how the SWAT+ Editor writes the file. HRU-lte objects are summed the same way over the first rows of the file.
- Landscape unit output uses `sub_frac` (see `ls_unit.def`).
- The soft calibration of the whole basin in `water_balance.sft` takes each HRU's area as `ls_area` times its `bsn_frac` (see soft calibration).
- The decision table action `hru_fr_update` reads a replacement `ls_unit.ele` during the run and sets the area of each `hru` element to `bsn_frac` times `ls_area`.
- If `ls_unit.ele` is missing or `null`, the basin landscape files are opened and get their headers, and no data rows are written. The routine that stores the basin's starting soil water at the end of the warm-up years still reads the element table, so a model with HRUs and no `ls_unit.ele` can stop with a memory error at that point. Treat the file as required in any model with HRUs.

Example

`refdata/Ames_sub1/ls_unit.ele`:

```
lsu_unit.ele
          ID    NAME      OBJ_TYP  OBJ_TYP_NO    BSN_FRAC       RU_FRAC
           1    hru0001       hru           1     0.20           0.0
           2    hru0002       hru           2     0.20           0.0
```

3	hru0003	hru	3	0.20	0.0
4	hru0004	hru	4	0.20	0.0
5	hru0005	hru	5	0.10	0.0
6	hru0006	hru	6	0.10	0.0
7	hru0007	hru	7	0.20	0.0
8	hru0008	hru	8	0.20	0.0
9	hru0009	hru	9	0.20	0.0
10	hru0010	hru	10	0.20	0.0
11	hru0011	hru	11	0.10	0.0
12	hru0012	hru	12	0.10	0.0

This file is not in the SWAT+ Editor layout: it has six columns and upper-case headers. Each row describes the HRU of the same number, as the basin sums require. The `BSN_FRAC` values add up to 2.0, so every basin output of this dataset is twice the area-weighted HRU average. `Ames_sub1` has no `ls_unit.def`, so the `RU_FRAC` values of 0 are not used.

aqu_catunit.ele

Format

Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one element, and seven values are read from it. A line with fewer values takes the missing ones from the next line.

#	Column	Units	Description
1	`id`	-	Element number. The largest `id` sets the size of the table.
2	`name`	-	Name of the element.
3	`obj_typ`	-	Object type, `aqu` in files written by the SWAT+ Editor.
4	`obj_typ_no`	-	Aquifer number.
5	`bsn_frac`	fraction	Fraction of the basin covered by the aquifer.
6	`sub_frac`	fraction	Fraction of the aquifer region covered by the element.
7	`reg_frac`	fraction	Fraction of the calibration region covered by the element.

Behaviour

- In 62.0.1 the file works as a switch. When it has at least one element, the basin aquifer files (`basin_aqu` row in `print.prt`) get one data row per interval. Without it they get headers only.
- The basin aquifer values are weighted by each aquifer's area divided by `tot_area` in `object.cnt` (see aquifer output). The columns of `aqu_catunit.ele` are not used for this.
- The columns are meant for aquifer regions defined in `aqu_catunit.def` and `aqu_reg.def`. SWAT+ 62.0.1 reads those files, but no calculation or output uses the regions.

Example

The first rows of `refdata/Osu_1hru/aqu_catunit.ele`:

```
aqu_catunit.ele: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                    obj_typ  obj_typ_no      bsn_frac      sub_frac      reg_frac
       1  aqu011                      aqu           1       0.00241       0.00000       0.00000
       2  aqu012                      aqu           2       0.10460       0.00000       0.00000
       3  aqu021                      aqu           3       0.01290       0.00000       0.00000
```

The file has 21 elements, while the dataset has one aquifer. The rows have no effect beyond switching on the basin aquifer output. In `refdata/Ames_sub1` the file is empty, so that dataset writes no basin aquifer data rows.

Columns in earlier documentation

- `ls_unit.ele`: the SWAT+ Editor writes a seventh column, `reg_frac`. SWAT+ 62.0.1 reads six values per line and ignores it.
- `ls_unit.def`: earlier documentation did not list the line with the number of landscape units (line 2) or the `area` column. Both are read, and a file without them is read out of step.

Related files

- `rout_unit.con`: the routing unit objects, their areas and their outflow connections.
- `hru.con` and `hru-data.hru`: the HRUs that are elements of routing and landscape units.
- `object.cnt`: the `rtu` count, and `ls_area` and `tot_area` used with `bsn_frac` and the basin aquifer output.
- `file.cio`: names the files on the `routing_unit` and `regions` lines.
- `topography.hyd` and `field.fld`: the `topo` and `field` records of `rout_unit.rtu`.
- `aquifer.aqu`: the aquifers listed in `aqu_catunit.ele`.
- `print.prt`: switches for the `ru`, `lsunit_*`, `basin_*` and `basin_aqu` outputs.
- Decision tables: the `hru_fr_update` action.
- Outputs: water balance, nutrient balance and losses, hydrographs, aquifer output.
- Delivery ratios in the model theory.

Sources

- File name slots: `type input_ru` and `type input_regions` in `input_file_module.f90`, read in `readcio_read.f90`; read order in `main.f90.in` and `hyd_connect.f90`
- Routing unit readers: `ru_read.f90` (`rout_unit.rtu`), `ru_read_elements.f90` (`rout_unit.ele`, `rout_unit.def`); element lists: `define_unit_elements.f90`
- Routing unit types: `ru_parameters` in `ru_module.f90`, `routing_unit_data` and `routing_unit_elements` in `hydrograph_module.f90`
- Routing unit linkage and inflow sharing: `hyd_connect.f90`, `hyd_read_connect.f90`
- Daily routing unit sum: `ru_control.f90`, called from `command.f90`
- Delivery ratios: `dr_ru.f90`, `dr_db_read.f90`
- Time of concentration and unit hydrographs: `time_conc_init.f90`, `unit_hyd_ru_hru.f90`
- Landscape unit reader: `lsu_read_elements.f90`; types `landscape_units` and `landscape_elements` in `calibration_data_module.f90`
- Landscape unit and basin output: `lsu_output.f90`, `lsu_carbon_output.f90`, `basin_output.f90`, `basin_sw_init.f90` (called from `time_control.f90`), file opening in `output_landscape_init.f90`
- Other uses of `ls_unit.ele` and `ls_unit.def`: `lcu_read_softcal.f90`, `hru_fr_change.f90` (action `hru_fr_update` in `actions.f90`), `gwflow_rech.f90`, `gwflow_gwet.f90`
- Aquifer elements: `aqu_read_elements.f90`, called from `proc_cal.f90`; basin aquifer output in `basin_aquifer_output.f90`, files opened in `header_write.f90`

File	Column	Fortran variable
`rout_unit.def`	`id`	not stored
`rout_unit.def`	`name`	`ru_def%name`
`rout_unit.def`	`elem_tot`	`nspu`
`rout_unit.def`	`elements`	`elem_cnt`, expanded into `ru_def%num` and `ru_def%num_tot`
`rout_unit.ele`	`id`	index of `ru_elem`
`rout_unit.ele`	`name`	`ru_elem%name`
`rout_unit.ele`	`obj_typ`	`ru_elem%obtyp`
`rout_unit.ele`	`obj_id`	`ru_elem%obtypno` (object number in `ru_elem%obj`)
`rout_unit.ele`	`frac`	`ru_elem%frac`
`rout_unit.ele`	`dlr`	`ru_elem%dr_name` (ratios in `ru_elem%dr`)
`rout_unit.rtu`	`id`	index of `ru`
`rout_unit.rtu`	`name`	`ru%name`
`rout_unit.rtu`	`define`	`ru%dbsc%elem_def`
`rout_unit.rtu`	`dlr`	`ru%dbsc%elem_dr`
`rout_unit.rtu`	`topo`	`ru%dbsc%toposub_db` (index `ru%dbs%toposub_db`)
`rout_unit.rtu`	`field`	`ru%dbsc%field_db` (index `ru%dbs%field_db`, values in `ru%field`)

File	Column	Fortran variable
ls_unit.def	line 2	mlsu, stored as db_mx%lsu_out
ls_unit.def	name	lsu_out%name
ls_unit.def	area	lsu_out%area_ha
ls_unit.def	elements	lsu_out%num, lsu_out%num_tot
ls_unit.ele	obj_typ	lsu_elem%obtyp
ls_unit.ele	obj_typ_no	lsu_elem%obtypno
ls_unit.ele	bsn_frac	lsu_elem%bsn_frac
ls_unit.ele	sub_frac	lsu_elem%ru_frac
aqu_catunit.ele	obj_typ, obj_typ_no	acu_elem%obtyp, acu_elem%obtypno
aqu_catunit.ele	bsn_frac, sub_frac, reg_frac	acu_elem%bsn_frac, acu_elem%ru_frac, acu_elem%reg_frac
aqu_catunit.ele	largest id	db_mx%aqu_elem

Purpose

The **link** line of file.cio names two optional files that connect objects outside the routing set up by the connectivity files:

- **chan-surf.lin** lists, for a SWAT+ channel, the HRUs or routing units on its floodplain. On days with overbank flow, the wetlands of those HRUs are filled from the channel.
- **aqu_cha.lin** lists, for an aquifer, the channels that receive its return flow. The flow is shared among the channels by channel length and drainage area.

The line gives the two names in this order. The defaults, **chan-surf.lin** and **aqu_cha.lin**, apply only when file.cio ends before the line (see the master file). Projects that do not use the files give **null** for both, as the reference datasets in the SWAT+ repository do. Earlier documentation called the first file **chan_surf.lin**.

Neither file has column names that SWAT+ checks. Both readers skip the header line and take the values by position, so the column names on this page are descriptive.

chan-surf.lin

Free format, whitespace-separated. Line 1 is a title and is skipped. Line 2 is a single whole number, the number of channel rows; SWAT+ also counts the rows itself, and the count only sets a minimum size for the table. Line 3 is a column header and is skipped. Each later line links one channel to its floodplain objects:

#	Column	Units	Description
1	id	-	Number of the SWAT+ channel, as numbered in chandeg.con.
2	name	-	Name of the floodplain. Not used in calculations.
3	num_obj	count	Number of floodplain objects that follow. A row with 0 gives the channel no floodplain links.
4	obj_typ	-	Type of the first floodplain object: **hru** or **ru** (routing unit).
5	obj_id	-	Number of the first floodplain object, as numbered in hru.con or rout_unit.con.
6, 7, ...	obj_typ, obj_id	-	One further pair for each remaining object, **num_obj** pairs in all.

Object types other than **hru** and **ru** are read and then ignored. For an **ru** object, SWAT+ takes the element numbers that rout_unit.def lists for that routing unit and uses each one as an HRU number. This selects the routing unit's HRUs only when each element number in rout_unit.ele equals the number of the HRU it refers to.

The links apply to channels routed with the SWAT+ channel method (chandeg.con). Legacy channels (channel.con) make no use of them.

Overbank flow to floodplain wetlands

Overbank flow happens on days when the channel's peak flow rate is above the bankfull threshold described under floodplain deposition. The overbank volume V_{ob} is the floodplain volume of the channel's rating curve at the peak flow rate. After floodplain deposition, SWAT+ goes through the floodplain HRUs in the order listed (a routing unit contributes its HRUs in the order of its elements). For each HRU it takes the room left in the HRU's wetland,

$$V_{fill} = V_{emr} - V_{wet}$$

where V_{emr} is the wetland's emergency volume and V_{wet} its current storage (m^3), and subtracts it from the overbank volume still available. When the overbank volume runs out, V_{fill} is cut to what was left. The wetland then receives the fraction

$$r = \min\left(1,\ \frac{V_{fill}}{Q_{in}}\right)$$

of the channel's inflow Q_{in} for the day (m^3), with the same fraction of its sediment, nutrients and other constituents, and the channel inflow is reduced by that fraction.

Only HRUs with surface storage, a `surf_stor` entry in `hru-data.hru` that names a record in `wetland.wet`, receive water. Other HRUs in the list take nothing from the channel. The water each wetland receives is reported in the `overbank` column (mm) of the HRU water balance output. See also overbank flow and floodplain deposition in the model theory, and the wetland inputs for the emergency volume.

Missing file and repeated read

SWAT+ reads `chan-surf.lin` while it sets up the SWAT+ channels, whether or not the model has any.

- If the slot is `null`, the file is skipped and no channel has floodplain links.
- If the slot gives a name and no file of that name exists, SWAT+ opens it anyway. This creates an empty file of that name in the run folder, nothing is read from it, and no channel has floodplain links.
- When `object.cnt` counts legacy channels (`cha` above 0), SWAT+ reads the file a second time, earlier in the setup, before the SWAT+ channels exist. If the file exists, the second read tries to set up the same table again and the run stops with an error.

aqu_cha.lin

Free format, whitespace-separated. Line 1 is a title and line 2 a column header; both are skipped. Each later line links one aquifer to a set of channels:

#	Column	Units	Description
1	`id`	-	Number of the aquifer, as numbered in `aquifer.con`.
2	`name`	-	Name of the link. Not used in calculations.
3	`num_elem`	count	Number of values in the channel list that follows. A row with 0 links no channels.
4, 5, …	`elements`	-	Channel numbers, as numbered in `chandeg.con`. A negative value marks a range: `1 -4` means channels 1 to 4.

The aquifer numbers must lie between 1 and the number of aquifers. SWAT+ reads as many rows as the largest aquifer number in the first column, so each aquifer should appear on one row only. Earlier documentation described the file as a replacement for `aquifer.con`. In 62.0.1 `aquifer.con` is still required, see overlap with aquifer outflows.

Return flow shares

For each linked aquifer, SWAT+ sorts the linked channels by drainage area, smallest first. The drainage area is the `area` column of `chandeg.con` and the channel length L_i is the `len` column of `hyd-sed-lte.cha` (km). With L_{tot} the total length of the linked channels, the length left when channel i and all smaller channels have stopped receiving flow is

$$L_{left,i} = L_{tot} - \sum_{j \le i} L_j$$

Each day the contributing channel length grows with the aquifer's return flow Q (mm):

$$L_c = L_{tot} \frac{Q}{\text{bf_max}}$$

where `bf_max` is given in `aquifer.aqu` (mm; see aquifer files). The first contributing channel k is the smallest channel with $L_c \ge L_{left,k}$, and each channel from k upward receives the share

$$f_i = \frac{L_i}{L_{left,k} + L_k}, \qquad i \ge k$$

while the smaller channels receive nothing. The shares of the contributing channels add up to 1, so the whole return flow of the aquifer is passed on. When Q is at or above `bf_max`, every linked channel receives flow in proportion to its length. As Q falls, the channels with the smallest drainage areas stop receiving flow first. See baseflow in the model theory.

Each linked channel adds its share of the aquifer's return flow hydrograph, with its nitrate, phosphorus, salts and constituents, to its inflow. This happens when the channel is routed: on the same day if the aquifer comes before the channel in the routing order, otherwise on the next day. The water appears in the `geo_bf` column (m^3/s) of the channel output.

Channel order, lengths and bf_max

- The shares are computed in drainage-area order. Each channel looks up its share by its position in the `aqu_cha.lin` list. A channel receives its intended share only when the list for each aquifer is already in order of increasing drainage area. In any other order, channels receive each other's shares; the total for the aquifer does not change.
- The channel length is taken from the row of `hyd-sed-lte.cha` whose position equals the channel's record number in `channel-lte.cha`. This gives the right length when both files list the channels in the same order.
- Once the file is read, the calculation divides by `bf_max` for every aquifer, including aquifers the file does not list. With `bf_max` = 0 the division fails, and builds that stop on floating-point errors (the Intel builds, for example) stop the run. Give every aquifer a `bf_max` above zero when `aqu_cha.lin` is used.

Overlap with aquifer outflows

`aqu_cha.lin` is read only when `object.cnt` counts aquifers, right after `aquifer.con`. The aquifer still routes its return flow through the outflows listed in `aquifer.con`, and SWAT+ also passes the shares above to the linked channels. A model that lists the same channels as outflows in `aquifer.con` and in `aqu_cha.lin` sends the return flow to them twice.

When gwflow is active, SWAT+ sets the aquifer count to 0 and `aqu_cha.lin` is not read.

Missing or short file

- If the slot is `null` or the file does not exist, the file is skipped and no aquifer is linked to channels.
- If the file exists but has fewer than two lines, the run stops with an end-of-file error.

Example

No dataset in the SWAT+ repository uses either file. The two files below are illustrative.

```
chan-surf.lin: illustrative example
2
id  name    num_obj  obj_typ  obj_id  obj_typ  obj_id
1   fp001   2        hru      4       hru      5
3   fp003   1        ru       2
```

Channel 1 is linked to HRUs 4 and 5, and channel 3 to the HRUs of routing unit 2. On a day with overbank flow in channel 1, the wetland of HRU 4 is filled first, then that of HRU 5, as long as overbank volume remains. An HRU without surface storage receives nothing.

```
aqu_cha.lin: illustrative example
id  name    num_elem  elements
1   aqu01   2         1  -4
2   aqu02   3         5  7  9
```

Aquifer 1 shares its return flow among channels 1 to 4, and aquifer 2 among channels 5, 7 and 9. For the shares to reach the intended channels, channels 1 to 4 must have increasing drainage areas in that order, and so must channels 5, 7 and 9. Neither aquifer should also route its return flow to these channels in aquifer.con.

Related files

- file.cio: names both files on the link line (master file).
- chandeg.con, aquifer.con, hru.con, rout_unit.con, rout_unit.def and rout_unit.ele: number the objects the files refer to (connectivity files).
- channel-lte.cha and hyd-sed-lte.cha: channel length and bankfull flow (channel tables).
- aquifer.aqu: bf_max and the return flow (aquifer files).
- hru-data.hru, wetland.wet and hydrology.wet: the surface storage that receives overbank water (HRU data, wetland inputs).
- object.cnt: the aquifer and legacy channel counts (object counts).

Sources

- File name slots: type input_link in input_file_module.f90, read in readcio_read.f90
- Readers: overbank_read.f90 (chan-surf.lin), aqu2d_read.f90 (aqu_cha.lin), with element ranges expanded in define_unit_elements.f90
- Call sites: hyd_connect.f90 (both readers), proc_cha.f90 (overbank_read again, sd_channel_surf_link, aqu2d_init); aquifer count reset for gwflow in basin_read_objs.f90
- Type definitions: floodplain_parameters in sd_channel_module.f90; channel_aquifer_elements and geomorphic_baseflow_channel_data in hydrograph_module.f90
- Floodplain HRU lists: sd_channel_surf_link.f90; routing unit elements from ru_read_elements.f90
- Overbank filling of wetlands: sd_channel_sediment3.f90; floodplain wetland storage in ch_rtmusk.f90; overbank output in hru_control.f90
- Channel sorting and lengths: aqu2d_init.f90
- Daily shares: aqu_1d_control.f90; added to channel inflow in sd_channel_control3.f90

File	Column	Fortran variable
chan-surf.lin	line 2 count	mcha_sp
chan-surf.lin	id	index i of sd_ch
chan-surf.lin	name	sd_ch%fp%name
chan-surf.lin	num_obj	nspu, sd_ch%fp%obj_tot
chan-surf.lin	obj_typ	sd_ch%fp%obtyp
chan-surf.lin	obj_id	sd_ch%fp%obtypno
aqu_cha.lin	id	index iaq of aq_ch
aqu_cha.lin	name	aq_ch%name
aqu_cha.lin	num_elem	nspu

File	Column	Fortran variable
aqu_cha.lin	elements	elem_cnt, expanded to aq_ch%num
aquifer.aqu	bf_max	aqudb%bf_max
hyd-sed-lte.cha	len	sd_chd%chl
chandeg.con	area	ob%area_ha

Purpose

This page covers three HRU parameter files and one channel file:

- `hydrology.hyd`: lateral flow, canopy storage, evaporation, percolation and enrichment parameters.
- `topography.hyd`: slope, slope lengths and the deposition coefficient. Also used by routing units.
- `field.fld`: field length and width for routing water and sediment across the landscape. Also used by routing units.
- `shade_factor.shf`: daily shade factors for the channel water temperature routine.

An HRU picks one record from each of the first three files by name, in the `hydro`, `topo` and `field` columns of hru-data.hru. `hydrology.hyd`, `topography.hyd` and `field.fld` are named on the `hydrology` line of file.cio. `shade_factor.shf` is not listed in file.cio; SWAT+ looks for a file with that name in the project folder.

All four files are free format and whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one record. The readers do not match columns by header name, so every value must be present and in the order given. Record names are stored with at most 16 characters.

An unmatched or misspelt `hydro` or `topo` name in hru-data.hru makes the HRU use the first record of the file without a message (see name lookup). `hydrology.hyd` and `topography.hyd` must therefore exist and must contain every name used.

hydrology.hyd

#	Column	Units	Typical range	Description
1	name	-	-	Name of the hydrology record, matched against the `hydro` column of hru-data.hru.
2	lat_ttime	days	0 to 180	Lateral flow travel time, called `lat_time` in earlier documentation. 0 lets SWAT+ compute it from the soil. See Lateral flow.
3	lat_sed	mg/L	0 to 5000	Sediment concentration in lateral and tile flow.
4	can_max	mm H_2O	0 to 100	Maximum canopy storage when the canopy is fully developed.
5	esco	-	0.01 to 1	Soil evaporation compensation factor. See Soil evaporation.
6	epco	-	0.01 to 1	Plant uptake compensation factor.
7	orgn_enrich	-	0 to 5	Organic N enrichment ratio for loading with sediment. 0 lets SWAT+ compute it for each runoff event.
8	orgp_enrich	-	0 to 5	Organic and sediment-attached P enrichment ratio for loading with sediment. 0 lets SWAT+ compute it for each runoff event.
9	cn3_swf	-	0 to 1	Soil water factor for the moisture condition III curve number: 0 puts it at field capacity, 1 at saturation.
10	bio_mix	-	0 to 1	Biological mixing efficiency.
11	perco	-	0 to 1	Percolation coefficient for the bottom soil layer. See Percolation from the soil profile.

#	Column	Units	Typical range	Description
12	`lat_orgn`	mg/L	0 to 200	Organic N concentration in lateral flow.
13	`lat_orgp`	mg/L	0 to 200	Organic P concentration in lateral flow.
14	`pet_co`	-	0.8 to 1.2	Multiplier on potential evapotranspiration. Older files call this column `harg_pet`. See Potential evapotranspiration.
15	`latq_co`	-	0 to 1	Lateral flow coefficient. See Lateral flow.

For an HRU with tile drains (set in its `landuse.lum` record), SWAT+ replaces `cn3_swf` with 0.95 and `perco` with 0.1 at the start of the run, whatever the record gives, and calibration changes to these two parameters are skipped for that HRU.

Lateral flow

Lateral flow is computed for each soil layer below the first with a hillslope storage method:

$$q_{lat,ly} = 0.024 \text{ latq_co } \frac{2\,SW_{ex,ly}}{\phi_{d,ly}/h_{ly}}\; K_{sat,ly}\; \frac{slp}{L_{lat}}$$

where $SW_{ex,ly}$ is the water above field capacity in the layer (mm), $\phi_{d,ly}$ the drainable porosity (saturation minus field capacity, mm), h_{ly} the layer thickness (mm), $K_{sat,ly}$ the saturated hydraulic conductivity (mm/h), slp the `slp` value from `topography.hyd` and L_{lat} its `lat_len` (m). The result is limited to the water above field capacity. The top layer produces no lateral flow. `latq_co` scales lateral flow linearly; 0 turns it off.

Lateral flow does not leave the HRU on the day it is generated. A fraction of the stored lateral flow is released each day:

$$f_{lat} = 1 - \exp\left(-\frac{1}{TT_{lat}}\right)$$

where TT_{lat} is `lat_ttime` in days. When `lat_ttime` is 0, SWAT+ computes it from the soil and the lateral slope length:

$$TT_{lat} = \max\left(1,\; 10.4\,\frac{L_{lat}}{K_{sat,mx}}\right)$$

with $K_{sat,mx}$ the highest saturated hydraulic conductivity in the profile (mm/h). Set `lat_ttime` to a specific value only when the base flow behaviour of the watershed is well known.

Soil evaporation

The evaporative demand on the soil is distributed with depth. The cumulative demand down to depth z (mm) is

$$E_z = E_s\,\frac{z}{z + \exp(2.374 - 0.00713\,z)}$$

where E_s is the soil evaporation demand for the day. A layer whose top is at depth z_u and bottom at z_l is asked for

$$E_{ly} = E_{z_l} - (1 - \text{esco})\,E_{z_u}$$

so with `esco` = 0 each layer meets only its share of the depth distribution, and as `esco` increases more of the demand can be met from lower layers. Layers below the soil evaporation depth do not take part. The demand on a layer is further reduced when the layer is drier than field capacity.

Plant uptake compensation

`epco` is copied to every plant of the HRU's plant community when the plants are initialized. As `epco` approaches 1, more of the water uptake demand can be met by lower layers when upper layers are dry; as it approaches 0, uptake stays closer to the original depth distribution.

Enrichment ratios

When `orgn_enrich` or `orgp_enrich` is above 0.001, it is used as a fixed enrichment ratio for organic N, or for organic and sediment-attached P, for every runoff event. Otherwise SWAT+ computes the enrichment ratio for each event from the sediment concentration in surface runoff. The enrichment ratio is the ratio of the concentration of the constituent in the transported sediment to its concentration in the soil surface layer; it is above 1 because the sediment in runoff contains a larger share of the fine particles that the nutrients are attached to.

Curve number soil water factor

`cn3_swf` sets the soil water content at which the curve number for moisture condition III applies:

$$SW_{CN3} = FC + \text{cn3_swf}\,(SAT - FC)$$

where FC and SAT are the profile water contents at field capacity and saturation. Larger values delay the onset of surface runoff after dry periods. See hydrology in the model theory.

Biological mixing

Biological mixing redistributes residue and nutrients in the soil through the activity of earthworms and other soil biota, using the same method as a tillage operation with `bio_mix` as the mixing efficiency. With `carbon` = 0 in `codes.bsn`, it is applied once at the end of every calendar year, down to 50 mm or the bottom of the profile if that is shallower. With the other `carbon` codes this column is not used; `carbon` = 2 applies a daily biological mixing with an efficiency taken from the tillage data.

Percolation from the soil profile

`perco` limits percolation out of the bottom soil layer, for example to represent an impermeable layer or a high water table. Percolation between the upper layers is not affected. The potential percolation from the bottom layer is multiplied by

$$f_{perc} = \min[1,\ \exp(-\,(1.0052\ \ln(-\ln \text{perco}) + 5.6862))]$$

The relation is strongly non-linear:

`perco`	0	0.1	0.5	0.8	0.9	0.95	0.99	0.999 and above
fraction of potential percolation	0	0.002	0.005	0.015	0.033	0.067	0.35	1

So 0 stops percolation from the profile, values up to about 0.9 allow only a few percent of it, and the restriction is lifted only as `perco` approaches 1.

Potential evapotranspiration

`pet_co` multiplies the potential evapotranspiration of the HRU, whichever method is selected in `codes.bsn` (Priestley-Taylor, Penman-Monteith, Hargreaves or values read from file). A value of 0 is treated as 1. Values below 0.01 are taken as the old Hargreaves coefficient written to `harg_pet` by earlier editors and divided by 0.0023, so 0.0023 becomes 1.0.

Canopy storage

The canopy storage available on a day is

$$CAN_{day} = \text{can_max}\,\frac{LAI}{LAI_{mx}}$$

where LAI is the current leaf area index of the plant community and LAI_{mx} its maximum. Precipitation fills the canopy before it reaches the ground.

Sediment and organic nutrients in lateral flow

`lat_sed` adds sediment to the HRU sediment yield in proportion to lateral and tile flow, and `lat_orgn` and `lat_orgp` add organic N and P in proportion to lateral flow. In 62.0.1 the sediment from `lat_sed` is added twice: once as a mass computed from the HRU area and once again as a value that does not use the area. Sediment yields from HRUs with a non-zero `lat_sed` therefore include an extra term. The conversion used for `lat_orgn` and `lat_orgp` is 100 times smaller than mg/L of lateral flow would give, so the loads are 1% of what the stated units suggest.

topography.hyd

#	Column	Units	Built-in value	Description
1	`name`	-	`default`	Name of the topography record, matched against the `topo` column of `hru-data.hru` and of `rout_unit.rtu`.
2	`slp`	m/m	0.02	Average slope steepness, usually computed from the DEM. Used for lateral flow, the USLE slope-length factor, time of concentration and sediment transport across the HRU.
3	`slp_len`	m	50	Average slope length for erosion: the distance over which sheet flow is the main surface runoff process, measured to the point where flow begins to concentrate. Used in the USLE slope-length factor. The full value is used: 62.0.1 does not apply the maximum slope length of the HRU's conservation practice.
4	`lat_len`	m	50	Slope length for lateral subsurface flow, usually longer than `slp_len`. A value of 0 is replaced by 50 m.
5	`dist_cha`	m	100	Average distance to the stream. Not used.

#	Column	Units	Built-in value	Description
6	`depos`	-	1	Deposition coefficient for sediment transport capacity when surface runoff from another object is routed across the HRU. A larger value gives a higher transport capacity, so more sediment leaves the HRU.

The built-in values apply only to a record that is never read; an HRU always uses a record from the file.

When runoff from upslope objects is routed across an HRU, its sediment transport capacity is proportional to

$$\text{depos}\ C\ slp^{1.4}\ w^{1.4}$$

and to the overland flow, where C is the USLE cover factor of the HRU and w the field width `wd` from `field.fld`.

field.fld

#	Column	Units	Built-in value	Description
1	`name`	-	`default`	Name of the field record, matched against the `field` column of `hru-data.hru` and of `rout_unit.rtu`.
2	`len`	m	500	Field length. Used for the time of concentration of routing units; not used for HRUs.
3	`wd`	m	100	Field width. Used in the sediment transport capacity when surface runoff is routed across an HRU.
4	`ang`	degrees	30	Field angle. Not used.

An HRU whose `field` entry is `null`, or a name that is not found, uses the built-in values.

shade_factor.shf

#	Column	Units	Description
1	`jday`	day of year	Day the value applies to (1 to 365).
2	`cha`	-	Channel number (row in `chandeg.con`). In revisions before 62.0.1 this column was a landscape unit number.
3	`value`	fraction	Shade factor, the fraction of incoming solar radiation blocked before it reaches the water surface.

The file has no name column and is read as a list of (day, channel, value) rows. The channel water temperature routine reduces short-wave radiation at the water surface by the factor (1 − value). On day 366 of leap years the factor is 0.5.

The file is meant to be used when `sf_on` = 1 in `temperature.cha`. In 62.0.1 the temperature routine takes all its settings from the built-in record instead of from the rows of `temperature.cha`. The built-in record has `sf_on` = 0 and `ssff` = 0.5, so the shade factor is 0.5 for every channel and day, and `shade_factor.shf` is read but not used. No example dataset in `refdata/` has a `shade_factor.shf` file.

Calibration

These parameters can be changed during calibration through `calibration.cal` (see parameter change files), using these names: `lat_ttime`, `lat_sed`, `canmx` (`can_max`), `esco`, `epco`, `erorgn` (`orgn_enrich`), `erorgp` (`orgp_enrich`), `cn3_swf`, `biomix` (`bio_mix`), `perco`, `lat_orgn`, `lat_orgp`, `petco` (`pet_co`), `latq_co`, `slope` (`slp`), `slope_len` (`slp_len`), `lat_len`, `dis_stream` (`dist_cha`), `field_len` (`len`), `field_wid` (`wd`) and `field_ang` (`ang`).

A change to `lat_ttime` is applied to the daily release fraction f_{lat}, which SWAT+ has already computed from the input, and the result is then converted again as if it were a travel time in days. The effect of a `lat_ttime` change is therefore not the effect of changing the travel time in the file by the same amount. Changes to `dis_stream` and `field_ang` have no effect, because the values are not used.

Examples

`refdata/Ames_sub1/hydrology.hyd` in the SWAT+ repository (first two rows):

```
hydrology.hyd:
name                 lat_ttime       lat_sed       can_max          esco          epco   orgn_enrich   org
hyd0001                0.00000       0.00000       0.00000       0.95000       1.00000       0.00000
hyd0002                0.00000       0.00000       0.00000       0.95000       1.00000       0.00000
```

Lateral flow travel time and enrichment ratios are computed by SWAT+, there is no canopy storage, and `cn3_swf` = 0 puts the moisture condition III curve number at field capacity. `perco` = 0.5 lets only about 0.5% of the potential percolation leave the bottom of the profile (unless the HRU is tile drained, which resets `perco` to 0.1), and `latq_co` = 0.01 keeps lateral flow small.

`refdata/Osu_1hru/hydrology.hyd` uses the older column header `harg_pet` with the value 0, which SWAT+ treats as a `pet_co` of 1.

`refdata/Ames_sub1/topography.hyd` (first two rows):

```
topography.hyd:
name                       slp       slp_len       lat_len      dist_cha         depos
topohru0001            0.02500      91.46300      91.46300     100.00000       1.00000
topohru0002            0.02500      91.46300      91.46300     100.00000       1.00000
```

`refdata/Osu_1hru/field.fld` (first two rows):

```
field.fld: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                       len            wd           ang
fld011                500.00000     100.00000      30.00000
fld012                500.00000     100.00000      30.00000
```

Related files

- `hru-data.hru`: selects the records by name in its `hydro`, `topo` and `field` columns.
- `file.cio`: names the three HRU files on its `hydrology` line.
- `landuse.lum`: sets tile drainage, which overrides `cn3_swf` and `perco`.
- `codes.bsn`: PET method and `carbon` code that `pet_co` and `bio_mix` depend on.
- `temperature.cha`: channel temperature settings, including `sf_on` for `shade_factor.shf`.
- Hydrology and soil water in the model theory.

Sources

- Readers: `hydrol_read.f90`, `topo_read.f90`, `field_read.f90`, `shade_factor_read.f90` (called from `proc_read.f90`)
- Type definitions: `type hydrology_db` in `hydrology_data_module.f90`; `type topography_db` and `type fields_db` in `topography_data_module.f90`; `type shade_factor_data` in `hydrograph_module.f90`
- File names: `type input_hydrology` and `type shade_factor` in `input_file_module.f90`, `in_hyd` read in `readcio_read.f90`

- Copy to each HRU, tile override, `pet_co` and `perco` conversion: topohyd_init.f90; routing units in ru_read.f90
- Lateral flow: hydro_init.f90, swr_percmicro.f90, swr_substor.f90
- Evaporation, PET and uptake: et_act.f90, et_pot.f90, plant_init.f90
- Canopy: sq_canopyint.f90; curve number: curno.f90
- Enrichment ratios: nut_orgn.f90, nut_orgnc2.f90, nut_psed.f90
- Lateral sediment and organic nutrients: swr_latsed.f90
- Biological mixing: time_control.f90, mgt_newtillmix_cswat0.f90
- Landscape routing and time of concentration: rls_routesurf.f90, time_conc_init.f90
- Shade factor use: ch_temp.f90, ch_read_temp.f90, `type channel_temperature_data` in channel_data_module.f90
- Calibration: cal_parm_select.f90

File	Column	Fortran variable
hydrology.hyd	`lat_ttime`	`hyd_db%lat_ttime`
hydrology.hyd	`lat_sed`	`hyd_db%lat_sed`
hydrology.hyd	`can_max`	`hyd_db%canmx`
hydrology.hyd	`esco`	`hyd_db%esco`
hydrology.hyd	`epco`	`hyd_db%epco`
hydrology.hyd	`orgn_enrich`	`hyd_db%erorgn`
hydrology.hyd	`orgp_enrich`	`hyd_db%erorgp`
hydrology.hyd	`cn3_swf`	`hyd_db%cn3_swf`
hydrology.hyd	`bio_mix`	`hyd_db%biomix`
hydrology.hyd	`perco`	`hyd_db%perco` (converted to `hru%hyd%perco_lim`)
hydrology.hyd	`lat_orgn`	`hyd_db%lat_orgn`
hydrology.hyd	`lat_orgp`	`hyd_db%lat_orgp`
hydrology.hyd	`pet_co`	`hyd_db%pet_co`
hydrology.hyd	`latq_co`	`hyd_db%latq_co`
topography.hyd	`slp`	`topo_db%slope`
topography.hyd	`slp_len`	`topo_db%slope_len`
topography.hyd	`lat_len`	`topo_db%lat_len`
topography.hyd	`dist_cha`	`topo_db%dis_stream`
topography.hyd	`depos`	`topo_db%dep_co`
field.fld	`len`	`field_db%length`
field.fld	`wd`	`field_db%wid`
field.fld	`ang`	`field_db%ang`
shade_factor.shf	`jday`, `cha`, `value`	`shf_db%jday`, `shf_db%cha`, `shf_db%value`

Purpose

A recall object is a spatial object whose outflow is read from a file instead of simulated. It is used for point sources such as wastewater treatment plants, for inflow from outside the modelled area (an upstream gauge or another model), and for diversions. Each day the recall object sends the flow and loads for that day to the objects named in its row of recall.con, in the same way an HRU or channel sends its outflow downstream.

A recall object needs three pieces of input:

1. A row in recall.con, which gives its location and the receiving object (usually a channel, `sdc`).
2. A record in `recall.rec` with the same `id`, which gives the time step and the name of the data file.
3. The data file with the flows and loads.

The number of recall objects is the `rec` column of object.cnt. `recall.rec` is named on the `recall` line of file.cio (default `recall.rec`; `null` turns recall input off).

recall.rec

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one recall record with four values. The header names are not checked; the example in `refdata/Osu_1hru`

uses `NUM NAME TYPE FNAME`. Anything after the fourth value on a line is ignored.

#	Column	Units	Description
1	`id`	-	Record number. A recall object uses the record whose `id` equals its `id` in `recall.con`; the `rec` column of `recall.con` is not used. The largest `id` sets the size of the table.
2	`name`	-	Name of the record, up to 25 characters.
3	`typ`	-	Time step of the data: 0 = sub-daily, 1 = daily, 2 = monthly, 3 = yearly, 4 = constant taken from an export coefficient record.
4	`filename`	-	Name of the data file (up to 25 characters), or for `typ` 4 the name of a record in `exco.exc`.

In revision 8a1edd4, recall objects were read from `recall_db.rec`, with six data file names (organic and mineral, pesticide, pathogen, heavy metal, salt and constituent) each followed by its units and a time step word (`sub`, `day`, `mo`, `yr`), and `recall.rec` was not read. In 62.0.1 recall objects are read from `recall.rec` with one data file and an integer time step per record. `recall_db.rec` is still read, with plain file names, but only for the uses listed under recall_db.rec.

Recall data file

One data file per recall record, named in `recall.rec`.

- Line 1: title, skipped.
- Line 2: number of years in the file. Read but not used; SWAT+ works out the years from the rows.
- Line 3: column header, skipped.
- Each later line: one time step.

Each data line has six leading values followed by 18 values in this order:

#	Column	Units	Description
1	`jday`	-	Day of year (1 to 366). Selects the day for daily data.
2	`mo`	-	Month (1 to 12). Selects the month for monthly data.
3	`day_mo`	-	Day of month. Not used.
4	`yr`	-	Year.
5	`ob_typ`	-	Object type label. Not used.
6	`ob_name`	-	Object name label. Not used.
7	`flo`	m^3	Water volume for one day.
8	`sed`	t	Sediment.
9	`orgn`	kg N	Organic N.
10	`sedp`	kg P	Organic P.
11	`no3`	kg N	Nitrate N.
12	`solp`	kg P	Soluble (mineral) P.
13	`chla`	kg	Chlorophyll-a.
14	`nh3`	kg N	Ammonium N.
15	`no2`	kg N	Nitrite N.
16	`cbod`	kg	Carbonaceous biological oxygen demand.
17	`dox`	kg	Dissolved oxygen.
18	`san`	t	Detached sand.
19	`sil`	t	Detached silt.
20	`cla`	t	Detached clay.
21	`sag`	t	Detached small aggregates.
22	`lag`	t	Detached large aggregates.
23	`grv`	t	Gravel.
24	`temp`	°C	Water temperature.

The values are used as given, as the recall object's outflow for one day. For daily data this is the day's volume and loads. Monthly and yearly values are also applied on every day of their month or year, so give them as daily

amounts: the mean daily volume and load for the month or year. A monthly or yearly total would be applied once for every day of the period. Pesticides, pathogens, heavy metals, salts and other constituents cannot be given in this file; their recall loads are zero.

Time steps

`typ`	Rows	Value used on a simulation day
1, daily	One row per day	The row with that day of year (`jday`) in that year.
2, monthly	One row per month	The row with that month (`mo`) in that year.
3, yearly	One row per year	The row for that year.
0, sub-daily		Does not work, see below.
4, constant	No data file	Does not work, see below.

Years

SWAT+ reads the first data row to find the first year in the file.

- If the file starts in or before the first simulation year, the rows before that year are skipped. The file must then contain the first simulation year: if it ends before that year, the reader keeps searching and the run does not get past reading the inputs.
- If the file starts after the first simulation year, its first year is placed at the matching simulation year and earlier years get no recall input.

From there each change of year in the file moves to the next simulation year, and reading stops at the last simulation year. A year missing from the middle of the file is not skipped over: the following years shift one year earlier. Give every year from the start of the data to the end without gaps.

For daily and monthly data, the recall object sends nothing on days outside the years covered by the file. For yearly data the test of the year range always passes, but years without data have zero values, so the effect is the same.

Diversions

A negative `flo` in daily data removes water. When the recall object's first connection in `recall.con` is a channel with more than 10 m³ of water, SWAT+ also removes nitrate, soluble P, ammonium, nitrite, dissolved oxygen and organic N from that channel at the channel's concentration, limited to what is in the channel. Salt and constituent mass are handled in the same way when those modules are active.

Sub-daily and constant records

Sub-daily records (`typ` 0) do not work in 62.0.1. The reader has no step that reads a sub-daily data line, so it keeps returning to the same line and never reaches the end of the file; a run with a sub-daily recall record does not get past reading the inputs.

Constant records (`typ` 4) are meant to take a fixed daily hydrograph from the `exco_om.exc` row of the `exco.exc` record named in `filename`. In 62.0.1 `recall.rec` is read before the export coefficient files, so the name is looked up in an empty list and the constant hydrograph stays zero. The daily recall step also has no branch for `typ` 4. A recall object with a `typ` 4 record therefore sends nothing. Use an export coefficient object in `exco.con` for a constant source instead.

recall_db.rec

`recall_db.rec` is a second recall list, read when a file with this name is in the project folder. It is not named in `file.cio`. It is used by the water allocation files and by reservoir release decision tables, not by recall objects in the routing.

Line 1 is a title and line 2 the header; both are skipped. Each later line has nine values:

#	Column	Description
1	id	Record number.
2	name	Record name, up to 13 characters.
3	org_min	Organic and mineral data file name.
4	pest	Pesticide data file name.
5	path	Pathogen data file name.
6	hmet	Heavy metal data file name.
7	salt	Salt data file name.
8	constit	Other constituent data file name.
9	descrip	Description, one word or quoted.

All nine values are required; a missing value makes the reader continue on the next line. The file names are stored as text only. None of these files is opened through `recall_db.rec`.

The two uses in 62.0.1:

- A reservoir release decision table action of type `meas` names a `recall_db.rec` record. The reservoir then releases the daily, monthly or yearly flow of the `recall.rec` record with the same number, without the year-range test used for recall objects. Keep the numbering of the two files the same.
- Water use, water treatment and outside-source entries in the water allocation files can take their water quality from a recall record by matching their name against `org_min`. The link from a `recall_db.rec` record to the recall data is never set in 62.0.1, so this option does not deliver the intended data.

pest.com

If a file named `pest.com` is in the project folder, SWAT+ reads it as a list of pesticide recall files. The values it reads are stored in the flow and nutrient records of the recall objects with the same numbers, replacing the data read through `recall.rec`, and no pesticide load is set. Do not put a file with this name in the project folder.

Salt and constituent recall files

`salt_recall.rec` and `cs_recall.rec` give salt-ion and constituent loads for recall objects when the salt or constituent modules are active. Their readers are in the source, but 62.0.1 does not call them, so these files are not read. The daily recall step still looks up salt and constituent recall data whenever salts or constituents are simulated. A model that simulates salts or constituents and has recall objects is therefore likely to stop with a memory error. See salts and constituents.

For reference, the format the readers expect: a title, a header, then one row per recall object with `id`, `name`, `typ` (1 = daily, 2 = monthly, 3 = yearly, 4 = export coefficient) and `filename`. Each data file has a title (the word `Incoming` marks a source from outside the watershed), the number of years, a header, and rows of `jday`, `mo`, `day_mo`, `yr`, `ob_typ`, `ob_name` followed by one value per salt ion or constituent in the order of `constituents.cs`.

Region files

`rec_catunit.def`, `rec_reg.def` and `rec_catunit.ele` group recall objects into regions. They are named on the `regions` line of `file.cio` and read when the calibration inputs are set up.

- `rec_catunit.def` and `rec_reg.def`: a title, the number of regions, a header, then per region `id`, `name`, `area` (ha), the number of elements, and the element numbers. With 0 elements a region contains every recall object.
- `rec_catunit.ele`: a title, a header, then per element `id`, `name`, `obj_typ`, `obj_typ_no`, `bsn_frac`, `sub_frac` and `reg_frac`.

In 62.0.1 these regions are not used for any output or calibration step, and the tables they fill are not set up before reading, so a definition file with one or more regions is likely to stop the run. Set these entries to `null` in `file.cio`.

Output

Recall objects are printed to `recall_day.txt`, `recall_mon.txt`, `recall_yr.txt` and `recall_aa.txt` (and the matching `.csv` files), selected by the `recall` row of `print.prt`. The sum over all recall objects is printed to the `basin_psc` files. The columns are described in recall outputs.

Columns in earlier documentation

Earlier documentation listed `recall.rec` with the columns `id`, `name`, `om_name`, `om_unit`, `om_tstep`, `pest_name`, `pest_unit`, `pest_tstep`, `path_name`, `path_unit`, `path_tstep`, `hmet_name`, `hmet_unit`, `hmet_tstep`, `salt_name`, `salt_unit`, `salt_tstep`, `cst_name`, `cst_unit` and `cst_tstep`. That is the layout recall objects used before 62.0.1. SWAT+ 62.0.1 no longer reads these columns: `recall.rec` has the four columns described above, and `recall_db.rec` has plain file names without units or time steps.

Example

`refdata/Osu_1hru/recall.rec` in the SWAT+ repository:

```
recall.rec
NUM      NAME     TYPE      FNAME     OBJ_TYP      OBJ_NAME        flo      sed     orgn     sedp      no3     solp
   1     PSTYR     3       RECYR.         hru          hru1    12120.5        0        0        0        0        0
   1       1           1     2011         hru          hru2     9000.7        0        0        0        0        0
```

SWAT+ reads the four values `1 PSTYR 3 RECYR.`: record 1, named `PSTYR`, yearly data in a file named `RECYR.`. The rest of the line, and the second data line, are ignored, because the largest `id` is 1 and only one record is read. This dataset has `rec` = 0 in `object.cnt`, so the file is not used in the run, and `RECYR.` is not included.

A daily data file for one recall record looks like this (illustrative values, columns 8 to 24 shortened):

```
daily point source, wastewater plant 1
2
jday  mo  day_mo    yr  ob_typ  ob_name      flo     sed    orgn    sedp     no3    solp  ...  temp
   1   1       1  2010  sdc     cha05     4320.0   0.050    1.20    0.30    8.50    1.10  ...  12.0
   2   1       2  2010  sdc     cha05     4410.0   0.050    1.25    0.31    8.60    1.12  ...  12.0
```

Each row gives the volume (m^3) and loads (t, kg) discharged on that day, here about 0.05 m^3/s.

Related files

- `recall.con`: location of each recall object and the objects that receive its outflow.
- `object.cnt`: number of recall objects (`rec`).
- `file.cio`: names `recall.rec` on the `recall` line and the region files on the `regions` line.
- `print.prt`: recall output switches.
- Recall outputs: columns of the recall output files.
- Decision tables: reservoir release action `meas`.
- Water allocation: water quality from recall records.
- Recall, export coefficient and delivery ratio objects in the model theory.

Sources

- Reader for `recall.rec`, the data files and `pest.com`: `recall_read.f90`; reader for `recall_db.rec`: `recalldb_read.f90`. Both are called from `main.f90.in`, after `hyd_connect.f90` and before `exco_db_read.f90`
- Type definitions: `type recall_hydrograph_inputs` and `type hyd_output` in `hydrograph_module.f90`; `type recall_databases` in `recall_module.f90`
- Daily use: `case ("recall")` in `command.f90`; diversions in `recall_nut.f90`, `recall_salt.f90`, `recall_cs.f90`
- Salt and constituent readers (not called): `recall_read_salt.f90`, `recall_read_cs.f90`

- Region files: rec_read_elements.f90, called from proc_cal.f90; arrays in calibration_data_module.f90
- Uses of recall_db.rec: dtbl_res_read.f90, res_hydro.f90, water_use_read.f90, water_treatment_read.f90, water_osrc_read.f90, wallo_use.f90, wallo_treatment.f90, wallo_start_day.f90
- Export coefficient lookup: exco_read_om.f90
- File names: in_rec%recall_rec and in_regs%def_psc, def_psc_reg, ele_psc in input_file_module.f90; output files opened in header_write.f90

File	Column	Fortran variable
recall.rec	id	array index i of recall
recall.rec	name	recall(i)%name
recall.rec	typ	recall(i)%typ
recall.rec	filename	recall(i)%filename
data file	flo to temp	recall(i)%hd(istep, iyrs) (type hyd_output)
recall_db.rec	name, org_min, pest, path, hmet, salt, constit, descrip	recall_db(i)%name, %org_min, %pest, %path, %hmet, %salt, %constit, %descrip

Purpose

The climate files tell SWAT+ where each spatial object gets its weather. Every object names a weather station in the wst column of its connect file (for example hru.con). The weather station, a row of weather-sta.cli, names a weather generator station and, for each weather variable, either an observed data file or sim to generate the variable.

The files below are named on the climate line of file.cio, in this order:

Position	Default name	Contents
1	weather-sta.cli	Weather stations.
2	weather-wgn.cli	Weather generator statistics.
3	pet.cli	List of observed potential evapotranspiration files.
4	pcp.cli	List of precipitation files (*.pcp).
5	tmp.cli	List of temperature files (*.tmp).
6	slr.cli	List of solar radiation files.
7	hmd.cli	List of relative humidity files.
8	wnd.cli	List of wind speed files.
9	atmodep.cli	Atmospheric deposition of nitrogen.

Two more deposition files, salt_atmo.cli and cs_atmo.cli, have fixed names and are not listed in file.cio.

A position set to null, or naming a file that does not exist, is skipped. A model needs weather-sta.cli and, for any generated variable or missing value, weather-wgn.cli. SWAT+ 62.0.1 reads climate input only from these text files; it has no netCDF climate input.

Common rules

All climate files are free format, with values separated by spaces or tabs. The first line is a title and is skipped; most files also skip a column header line, as described for each file.

Names (stations, files) can be up to 50 characters and are matched with upper and lower case distinguished. A name that contains a slash, a comma or a space must be put in quotes.

SWAT+ looks names up with a binary search, so every list that is searched must be in ascending order by character code (digits before upper-case letters, upper case before lower case). This applies to the station names in weather-sta.cli and weather-wgn.cli, the file names in each observed data list, and the station names in atmodep.cli. A list out of order can make SWAT+ miss names that are present.

Path prefixes

The last lines of file.cio give an optional folder for each type of observed data file: pcp_path, tmp_path, slr_path, hmd_path and wnd_path. When the value is null, the data files are opened from the project folder. Otherwise the value is put in front of each file name with nothing added in between, so it must end with a path separator (for example C:\weather\pcp\), and it can be up to 80 characters. A path containing a forward slash must be put in quotes, because an unquoted slash ends a free-format value. There is no path line for PET files; they are always opened from the project folder.

weather-sta.cli

One line per weather station. Line 1 is a title and line 2 is the column header; both are skipped. Each later line has nine values in the order below.

#	Column	Description
1	name	Weather station name, referenced by the wst column of the connect files.
2	wgn	Name of a weather generator station in weather-wgn.cli.
3	pcp	Name of a precipitation file listed in pcp.cli, or sim to generate precipitation.
4	tmp	Name of a temperature file listed in tmp.cli, or sim to generate maximum and minimum temperature.
5	slr	Name of a solar radiation file listed in slr.cli, or sim.
6	hmd	Name of a relative humidity file listed in hmd.cli, or sim.
7	wnd	Name of a wind speed file listed in wnd.cli, or sim.
8	wnd_dir	Read as the name of a PET file listed in pet.cli, or null for none. SWAT+ Editor labels this column wnd_dir; SWAT+ 62.0.1 has no wind direction input.
9	atmo_dep	Name of a deposition station in atmodep.cli, or null.

A tenth value on a line is ignored, so a PET file name placed after atmo_dep is not read.

Unmatched names

Column	Name not found	Message in diagnostics.out
pcp, tmp, slr, hmd, wnd	The variable is generated for the whole run.	<name> file not found (pgage), (tgage), (sgage), (hgage) or (wgage), unless the value is sim
wnd_dir (PET)	No observed PET for the station.	<name> file not found (petgage), unless the value is null; sim gives the message
atmo_dep	See deposition stations.	<name> file not found (atmodep), unless the value is null
wgn	The station uses the first station in weather-wgn.cli.	none

An object whose wst name is not in weather-sta.cli uses the first weather station, also without a message.

The six-day air temperature record used for stream temperature starts each station with the same value, the mean of the January tmp_min_ave and tmp_max_sd of the first station in weather-wgn.cli. It is replaced by the daily mean temperatures of the run within the first six days.

Example

refdata/Ames_sub1/weather-sta.cli in the SWAT+ repository:

```
weather-sta.cli:
name                          wgn              pcp              tmp              slr              hm
wgn_sta1                  wgn_001         ames.pcp         ames.tem              sim              si
```

One station, wgn_sta1, with measured precipitation from ames.pcp and generated solar radiation, humidity and wind. Two names do not match. The generator station in weather-wgn.cli is called wgn001, so wgn_001 is not found and the first (and only) generator station is used without a message. tmp.cli lists ames.tmp, so ames.tem is not found and temperature is generated for the whole run (see *.tem).

weather-wgn.cli

Monthly weather statistics for the weather generator. Each weather station uses its generator station to generate the variables set to sim, to fill missing days in the observed files, and for quantities that depend on location. Line 1 is a title. Each generator station then takes 14 lines:

1. a station line with five values (no header line above it),
2. a column header line, which is skipped,
3. twelve monthly lines, January to December, with 14 values each.

The number of stations is set by the length of the file.

Station line

#	Column	Units	Description
1	name	-	Generator station name, referenced by the wgn column of weather-sta.cli.
2	lat	decimal degrees	Latitude. Sets day length, maximum possible solar radiation and the day length at which plants go dormant for every weather station that uses this generator station, also when all variables are observed. When SWAT+ estimates plant start dates, a latitude of 0 or less is treated as the Southern Hemisphere.
3	lon	decimal degrees	Longitude. Not used by SWAT+.
4	elev	m	Elevation. Used for the elevation adjustment of generated precipitation and temperature.
5	rain_yrs	years	Number of years of record used to find pcp_hhr. A value below 1 is replaced by 10. Called yrs_pcp in earlier documentation.

Monthly lines

The column headers below are those SWAT+ Editor writes; older files use the upper-case names in the second column. SWAT+ reads by position only.

#	Column	Older header	Units	Typical range	Description
1	`tmp_max_ave`	`TMPMX`	°C	-30 to 50	Mean daily maximum air temperature for the month.
2	`tmp_min_ave`	`TMPMN`	°C	-40 to 40	Mean daily minimum air temperature for the month.
3	`tmp_max_sd`	`TMPSTDMX`	°C	0.1 to 100	Standard deviation of daily maximum air temperature in the month.
4	`tmp_min_sd`	`TMPSTDMN`	°C	0.1 to 30	Standard deviation of daily minimum air temperature in the month.
5	`pcp_ave`	`PCPMM`	mm	0 to 600	Mean total precipitation for the month.
6	`pcp_sd`	`PCPSTD`	mm/day	0.1 to 50	Standard deviation of daily precipitation in the month, with dry days included.
7	`pcp_skew`	`PCPSKW`	-	0.2 to 20	Skew coefficient of daily precipitation in the month, with dry days included. Values below 0.2 are replaced by 0.2.
8	`wet_dry`	`PR_WD`	fraction	0 to 0.95	Probability of a wet day following a dry day in the month.
9	`wet_wet`	`PR_WW`	fraction	0 to 0.95	Probability of a wet day following a wet day in the month.
10	`pcp_days`	`PCPD`	days	0 to 31	Mean number of days with precipitation in the month.
11	`pcp_hhr`	`RAINHMX`	mm	0 to 125	Largest 0.5-hour rainfall in the month over the `rain_yrs` years of record.
12	`slr_ave`	`SOLARAV`	MJ/m²/day	0 to 40	Mean daily solar radiation for the month.
13	`dew_ave`	`DEWPT`	°C or fraction	-50 to 25	Mean daily dew point temperature for the month, or mean relative humidity as a fraction.
14	`wnd_ave`	`WINDAV`	m/s	0 to 100	Mean daily wind speed for the month.

The statistics count a day as wet when it has more than 0 mm of precipitation. During the run, the generator counts a day as wet, for the next day's wet-day probability, when it has at least 0.1 mm.

Checks and derived values

SWAT+ checks and completes the monthly values for each generator station when it reads the file.

Wet-day probabilities and wet days. If `wet_wet` is not greater than `wet_dry`, or `wet_dry` is 0 or less, the two probabilities are estimated from `pcp_days` (at least 0.1 day):

$$P(W/D) = 0.75\,\frac{d_{wet}}{N}, \qquad P(W/W) = 0.25 + P(W/D)$$

where d_{wet} is `pcp_days` and N the number of days in the month. Otherwise `pcp_days` is not used as given and is recalculated from the probabilities:

$$d_{wet} = \frac{N\,P(W/D)}{1 - P(W/W) + P(W/D)}$$

Mean precipitation per wet day. The generator draws daily amounts around $\mu = \text{pcp_ave}/d_{wet}$. `pcp_ave` must therefore be the mean monthly total. A mean daily value in this column makes generated precipitation far too small.

Dew point or humidity. If all twelve `dew_ave` values are between 0 and 1, they are taken as relative humidity (fraction). If any value is outside that range, all twelve are taken as dew point temperature in °C.

Half-hour rainfall. `pcp_hhr` is smoothed over three months (the month and its two neighbours, December and January wrapping round) and turned into the mean fraction of daily rain that falls in the wettest half hour:

$$\alpha_{0.5} = \text{adj_pkr}\left[1 - \exp\left(\frac{\bar{R}_{0.5}}{\mu\,\ln\left(\dfrac{0.5}{\text{rain_yrs}\;d_{wet}}\right)}\right)\right]$$

limited to 0.1 to 0.95, where $\bar{R}_{0.5}$ is the smoothed `pcp_hhr` and `adj_pkr` is the peak rate adjustment factor (`adj_pkrt` in the `parameters.bsn` header). Each day's fraction is drawn at random around this monthly value and is used for peak runoff rates, erosion and the generated distribution of rain within the day.

The generator equations themselves are described in Climate and weather.

Example

Start of `refdata/Ames_sub1/weather-wgn.cli` in the SWAT+ repository:

```
weather-wgn.cli: Ames
wgn001       37.48     -100.83    887.00     10
   TMPMX    TMPMN  TMPSTDMX TMPTDMN  PCPMM  PCPSTD  PCPSKW   PR_WD   PR_WW   PCPD  RAINHMX SOLARAV  DEWPT
    7.61    -8.13     8.06    5.99     7.4     3.3    0.19    0.07    0.25    2.65      5.1   10.39  -8.14
   11.07    -5.53     8.06    5.34    10.2     5.1    2.67    0.08    0.27    2.86      1.8   13.41  -5.58
   15.22    -1.87     7.99    5.22    28.1     8.9    1.64    0.11    0.34    4.43      7.6   17.85  -4.71
...
```

One generator station, `wgn001`, at 37.48° N and 887 m, with 10 years of half-hour rainfall record; ten more monthly lines follow. The January skew of 0.19 is raised to 0.2. The `DEWPT` values go below 0, so the column is read as dew point temperature.

Observed data lists

pcp.cli, tmp.cli, slr.cli, hmd.cli, wnd.cli and pet.cli all have the same layout: a title line, a header line (SWAT+ Editor writes filename), and one data file name per line. Each name is both the file to open and the name that the matching column of weather-sta.cli refers to. Every file listed is read, whether or not a station uses it.

List	weather-sta.cli column	Records in each data file	Units
pcp.cli	pcp	year, day, precipitation (or six values for sub-daily data)	mm (*.pcp)
tmp.cli	tmp	year, day, maximum, minimum temperature	°C (*.tmp)
slr.cli	slr	year, day, solar radiation	MJ/m²/day
hmd.cli	hmd	year, day, relative humidity	fraction, 0 to 1
wnd.cli	wnd	year, day, wind speed	m/s, at 10 m height
pet.cli	wnd_dir	year, day, potential evapotranspiration	mm

Example, refdata/Osu_1hru/pcp.cli:

```
pcp.cli: Precipitation file names - file written by SWAT+ editor 2023-03-22 04:22:29.020522
filename
Imsilpcp.pcp
```

Solar radiation, humidity, wind and PET files

These data files have the layout of a daily precipitation file: a title line, a header line (nbyr tstep lat lon elev), a gauge line with those five values, and one record per day with year, day of year (called jday in earlier documentation) and the value. Only nbyr is used from the gauge line. The records follow the same rules for order, storage, values of -97 or less, and days outside the record as daily precipitation. A missing solar radiation, humidity or wind value is generated from the weather generator station.

Relative humidity is limited to 0.01 to 0.99 every day after it is read or generated, so a file in percent gives 0.99 on every day. Solar radiation below 0 is set to 0.

Observed PET

A PET file is read when it is listed in pet.cli, and a station uses it when its wnd_dir column names it. With pet = 3 in codes.bsn, HRUs take their potential evapotranspiration from the station. In 62.0.1 the station value is replaced by a Hargreaves estimate (at least 0.01 mm) later in the same daily step, for every station, so the values in the PET files do not reach the HRUs and pet = 3 gives Hargreaves PET. A missing PET value is also replaced by a Hargreaves estimate. Channel and reservoir evaporation and aquifer revap use the same station value under every pet setting.

atmodep.cli

Atmospheric deposition of ammonium and nitrate, as average annual, monthly or yearly values for one or more stations. Earlier documentation calls this file atmo.cli. The deposition values are in this file; it does not point to data files. The atmo_dep column of codes.bsn is not used; the time step comes from this file.

Line 1 is a title and line 2 a column header; both are skipped. Line 3 is the control line:

#	Column	Description
1	num_sta	Number of deposition stations in the file.
2	timestep	aa average annual, mo monthly, yr yearly.
3	mo_init	First month of the series (1 to 12). Used with mo.
4	yr_init	First year of the series. Used with mo and yr.
5	num_aa	Number of months (mo) or years (yr) in each series. Not used with aa.

#	Column	Description

Each station then takes five lines:

Line	Contents	Units
1	Station name, referenced by the `atmo_dep` column of `weather-sta.cli`.	-
2	Ammonium concentration in rain.	mg N/L
3	Nitrate concentration in rain.	mg N/L
4	Ammonium dry deposition.	kg N/ha per year (`aa`, `yr`) or per month (`mo`)
5	Nitrate dry deposition.	kg N/ha per year (`aa`, `yr`) or per month (`mo`)

With `aa` each of lines 2 to 5 has one value. With `mo` or `yr` each has `num_aa` values, one per month or year from `mo_init`/`yr_init` on.

Daily deposition

Every day, each HRU receives in its top soil layer, for ammonium and nitrate separately,

$$N_{wet} = 0.01\, C\, P, \qquad N_{dry} = \frac{D}{365}\ \ (\texttt{aa}, \texttt{yr}), \qquad N_{dry} = \frac{D}{N_{mo}}\ \ (\texttt{mo})$$

where N_{wet} and N_{dry} are in kg N/ha, C is the concentration in rain (mg/L), P the HRU precipitation for the day (mm), D the dry deposition value and N_{mo} the number of days in the month.

Monthly and yearly series

SWAT+ starts the series at the entry for the first month or year of the simulation. If the series starts later than the simulation, deposition begins when its first month or year is reached. Before the series starts and after its last entry, HRUs receive no deposition. The entry used moves on at the last day of each month (`mo`) or year (`yr`), so the last day of a month already uses the next month's values.

When the series starts before the simulation and `mo_init` is not 1, the first entry used is `mo_init` - 1 months later than the simulation start month, and the whole series is shifted by that amount.

Deposition stations

A weather station with `atmo_dep` set to `null`, or to a name not in the file, uses a built-in station. With `aa` this station gives 1.0 mg/L ammonium and 0.2 mg/L nitrate in rain and no dry deposition. With `mo` or `yr` the built-in station has no series, and its values are undefined; give every weather station a deposition station in that case.

Example

Average annual deposition for one station:

```
atmodep.cli
num_sta  timestep  mo_init  yr_init  num_aa
      1        aa        0        0       0
station_001
1.0
0.2
0.0
0.0
```

Salt and constituent deposition

Wet and dry deposition of salt ions and of other constituents, used with the salt and constituent inputs. Both files have fixed names and are read from the project folder only when `atmodep.cli` is present and the corresponding ions or constituents are defined in `constituents.cs`. They have no control line: the number of stations, the time step and the series length come from `atmodep.cli`. Stations are matched by position, so they must be in the same order as in `atmodep.cli`.

salt_atmo.cli

The first six lines are skipped. For each station:

- a line with the station name,
- one line per salt ion, in the order of `constituents.cs`, with the ion label followed by the concentration in rain (mg/L), one value for `aa` or `num_aa` values for `mo` and `yr`,
- one line per salt ion with the ion label followed by the dry deposition, in the same units as nitrogen dry deposition.

cs_atmo.cli

The first three lines are skipped. For each station:

- a station line, which is read as a number, so it must contain a number such as the station's position (a text name stops the run with a read error),
- one line per constituent with the concentration in rain (mg/L), without a label,
- one line per constituent with the dry deposition, without a label.

For both files, the daily amounts are calculated as for nitrogen and added to the top soil layer.

Related files

- `file.cio`: names the climate files and the data path prefixes.
- `hru.con` and the other connect files: assign a weather station to each object in the `wst` column.
- `*.pcp`, `*.tmp`, `*.tem`: observed precipitation and temperature.
- `time.sim`: simulation period and number of steps per day.
- `codes.bsn`: PET method and `lapse` switch.
- `parameters.bsn`: lapse rates and `adj_pkrt`.
- Salt inputs and constituent inputs: ion and constituent lists for the deposition files.
- Climate and weather: weather generator and PET methods.

Sources

- File name slots: `type input_cli` and the `in_path_*` types in `input_file_module.f90`, read in `readcio_read.f90`
- Type definitions: `weather_station`, `weather_codes_station_char`, `weather_generator_db`, `climate_measured_data`, `atmospheric_deposition`, `atmospheric_deposition_control` in `climate_module.f90`
- `weather-sta.cli`: `cli_staread.f90`; name lookup `search.f90`; station lookup from connect files `hyd_read_connect.f90`
- `weather-wgn.cli`: `cli_wgnread.f90`, `cli_initwgn.f90`, `cli_clgen.f90`, `cli_tgen.f90`
- Observed data lists: `cli_pmeas.f90`, `cli_tmeas.f90`, `cli_smeas.f90`, `cli_hmeas.f90`, `cli_wmeas.f90`, `cli_petmeas.f90`
- Daily climate step, missing values, PET recalculation and humidity limits: `climate_control.f90`; HRU PET `et_pot.f90`
- `atmodep.cli`: `cli_read_atmodep.f90`, `cli_atmodep_time_control.f90`, `nut_nrain.f90`
- `salt_atmo.cli` and `cs_atmo.cli`: `cli_read_atmodep_salt.f90`, `cli_read_atmodep_cs.f90`, `salt_rain.f90`, `cs_rain.f90`
- `atmo_dep` code in `codes.bsn`: `bsn_cc%atmo` in `basin_module.f90`

File	Column	Fortran variable
weather-sta.cli	name	wst(i)%name
weather-sta.cli	wgn	wst(i)%wco_c%wgn
weather-sta.cli	pcp	wst(i)%wco_c%pgage
weather-sta.cli	tmp	wst(i)%wco_c%tgage
weather-sta.cli	slr	wst(i)%wco_c%sgage
weather-sta.cli	hmd	wst(i)%wco_c%hgage
weather-sta.cli	wnd	wst(i)%wco_c%wgage
weather-sta.cli	wnd_dir	wst(i)%wco_c%petgage
weather-sta.cli	atmo_dep	wst(i)%wco_c%atmodep
weather-wgn.cli	name	wgn_n(i)
weather-wgn.cli	rain_yrs	wgn(i)%rain_yrs
weather-wgn.cli	tmp_max_ave, tmp_min_ave	wgn(i)%tmpmx, wgn(i)%tmpmn
weather-wgn.cli	tmp_max_sd, tmp_min_sd	wgn(i)%tmpstdmx, wgn(i)%tmpstdmn
weather-wgn.cli	pcp_ave, pcp_sd, pcp_skew	wgn(i)%pcpmm, wgn(i)%pcpstd, wgn(i)%pcpskw
weather-wgn.cli	wet_dry, wet_wet, pcp_days	wgn(i)%pr_wd, wgn(i)%pr_ww, wgn(i)%pcpd
weather-wgn.cli	pcp_hhr	wgn(i)%rainhmx
weather-wgn.cli	slr_ave, dew_ave, wnd_ave	wgn(i)%solarav, wgn(i)%dewpt, wgn(i)%windav
observed lists	file name	pcp_n, tmp_n, slr_n, hmd_n, wnd_n, petm_n and %filename
atmodep.cli	control line	atmodep_cont%num_sta, %timestep, %mo_init, %yr_init, %num
atmodep.cli	aa lines 2 to 5	atmodep(i)%nh4_rf, %no3_rf, %nh4_dry, %no3_dry
atmodep.cli	mo lines 2 to 5	atmodep(i)%nh4_rfmo, %no3_rfmo, %nh4_drymo, %no3_drymo
atmodep.cli	yr lines 2 to 5	atmodep(i)%nh4_rfyr, %no3_rfyr, %nh4_dryyr, %no3_dryyr

Purpose

A precipitation data file contains the measured precipitation of one gauge, as daily totals or as values for each sub-daily time step. Each file is named on one line of pcp.cli, and a weather station uses it when the pcp column of weather-sta.cli gives the same name. Days without a usable value are filled by the weather generator station named in the wgn column of the same weather station.

SWAT+ does not check the file extension. .pcp is the usual one. If pcp_path in file.cio is not null, its value is put in front of every file name in pcp.cli (see path prefixes).

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is a column header; both are skipped. Line 3 describes the gauge and every later line is one record.

Gauge line

SWAT+ Editor writes the header nbyr tstep lat lon elev above this line.

#	Column	Units	Description
1	nbyr	years	Number of years of record. SWAT+ uses it to size the store for the data, so it must be at least the number of calendar years the record spans.
2	tstep	values per day	0 for daily data. For sub-daily data, the number of values per day, which must equal step in time.sim (24 for hourly data).
3	lat	decimal degrees	Latitude of the gauge. Not used by SWAT+.
4	lon	decimal degrees	Longitude of the gauge. Not used by SWAT+.
5	elev	m	Elevation of the gauge. Used only for the elevation adjustment.

Daily records

With tstep = 0, each record has three values:

#	Column	Units	Description
1	year	-	Calendar year.
2	day	-	Day of the year, 1 to 366. Called `jday` in earlier documentation.
3	pcp	mm	Precipitation for the day. A value of -97 or less marks the day as missing.

Sub-daily records

With `tstep` greater than 0, each record has six values:

#	Column	Units	Description
1	year	-	Calendar year.
2	day	-	Day of the year, 1 to 366. Called `jday` in earlier documentation.
3	month	-	Month, called `mon` in earlier documentation. Not used by SWAT+.
4	day of month	-	Not used by SWAT+.
5	step	-	Time step within the day, called `hr` in earlier documentation. Not used by SWAT+.
6	pcp	mm	Precipitation for the time step.

SWAT+ places the values of each day in the order they appear, so every day needs `step` records in time order. The month, day of month and step columns do not position the value.

Record order

Records must be continuous and in date order. SWAT+ counts years by watching for day 365 or 366 followed by a record from a new year, so a gap at the end of a year shifts every later year of the record. A file that starts or ends part way through a year is fine.

Reading and storage

Records before the start of the simulation are skipped when the file is read. Storage starts with the first record whose year is at or after `yrc_start` and whose day is at or after `day_start` in `time.sim`.

When the record starts in a year after the simulation start year and `day_start` is later than the first day in the file, the records of that first year before `day_start` are also skipped. Those days then read as 0 mm instead of being filled by the weather generator.

Missing days

In a daily run, SWAT+ fills a day from the weather generator when

- the value is -97 or less (SWAT+ Editor and most tools write -99), or
- the day is before the first record or after the last record of the file.

Generated precipitation uses the monthly statistics of the station's `weather-wgn.cli` entry and the wet or dry state of the previous day. The method is described in Climate and weather.

Sub-daily runs

A sub-daily file is read when `tstep` is greater than 0. `step` in `time.sim` sets the number of steps per day of the run, and SWAT+ stores that many values per day. If the two numbers differ, the values are assigned to the wrong steps: with `step` = 0 or 1 and an hourly file, only the last hour of each day is kept.

Missing and out-of-range values are handled differently from daily data in 62.0.1:

- When a step has a value of -97 or less, SWAT+ calls the generator, but the day's total is still the sum of the file values for that day, missing-value flags included.
- Days outside the record are not generated. Inside the stored years they read as 0 mm.
- Values are looked up by simulation year counted from the simulation start, so the record must start on or before the simulation start date.
- From the second simulation year on, the values used for 1 January come from 1 January of the previous simulation year.

Sub-daily files should therefore be complete, without missing values, and cover the whole simulation period.

Elevation adjustment

When `lapse` = 1 in `codes.bsn`, the precipitation of each HRU is changed for the difference between the HRU elevation and the gauge elevation:

$$P_{hru} = \max\left(0,\ P + \text{plaps}\,\frac{z_{hru} - z_{gauge}}{1000}\right)$$

where P is the station precipitation (mm), `plaps` is the precipitation lapse rate in `parameters.bsn` (mm per km, default 0), z_{hru} is the `elev` column of `hru.con` and z_{gauge} is `elev` from this file (m). When the station's `pcp` column is `sim`, the elevation of the weather generator station is used for z_{gauge}.

The adjustment is added every day, dry days included, so a positive `plaps` gives HRUs above the gauge a small amount of precipitation on every day. The adjustment values are calculated during channel and reservoir routing, so in a model without channels or reservoirs they remain zero.

Calibration

The calibration parameter `precip` (object type `cli`) changes the stored daily values of listed precipitation records over a window of days and years. See Climate changes.

Example

First lines of `refdata/Ames_sub1/ames.pcp` in the SWAT+ repository, and part of 2012:

```
AME.pcp
 NBYR    TSTEP      LAT     LONG     ELEV
   121        0    42.04   -93.89   316
1900 1 -99
1900 2 -99
1900 3 -99
...
2012 55 12.4
2012 56 0
2012 57 -99
2012 58 0
```

A daily record (`tstep` = 0) of 121 years, 1900 to 2020, at 316 m. The Ames run starts in 1975, so the records from 1900 to 1974 are skipped when the file is read. Of the days from 1975 on, 31 are marked missing (`-99`), such as day 57 of 2012, and the weather generator fills them during the run.

Related files

- `pcp.cli`: lists the precipitation files.
- `weather-sta.cli`: assigns a precipitation file to each weather station in its `pcp` column.
- `weather-wgn.cli`: weather generator statistics that fill missing days.

- time.sim: simulation period and number of steps per day.
- file.cio: names pcp.cli and the pcp_path prefix.
- codes.bsn and parameters.bsn: lapse switch and plaps rate.

Sources

- Reader: cli_pmeas.f90
- Type definition: type climate_measured_data in climate_module.f90
- Gauge name lookup from weather-sta.cli: cli_staread.f90, search.f90
- Daily use, missing values and range check: cli_precip_control.f90, cli_bounds_check.f90, called from time_control.f90
- Weather generator: cli_pgen.f90, cli_pgenhr.f90
- Elevation adjustment: cli_lapse.f90, called from sd_channel_control3.f90 and res_control.f90, applied in hru_control.f90
- Time step: time_read.f90
- Path prefix: in_path_pcp in input_file_module.f90, read in readcio_read.f90
- Calibration change precip: cal_conditions.f90

Column	Fortran variable
nbyr	pcp(i)%nbyr
tstep	pcp(i)%tstep
lat	pcp(i)%lat
lon	pcp(i)%long
elev	pcp(i)%elev
daily pcp	pcp(i)%ts(day, iyr)
sub-daily pcp	pcp(i)%tss(step, day, iyr)

Purpose

A temperature data file contains the measured daily maximum and minimum air temperature of one gauge. Each file is named on one line of tmp.cli, and a weather station uses it when the tmp column of weather-sta.cli gives the same name. Days without usable values are filled by the weather generator station named in the wgn column of the same weather station.

SWAT+ does not check the file extension. SWAT+ Editor writes .tmp; some older data sets use .tem. If tmp_path in file.cio is not null, its value is put in front of every file name in tmp.cli (see path prefixes).

Temperature is always read as daily data, also in sub-daily runs.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is a column header; both are skipped. Line 3 describes the gauge and every later line is one daily record.

Gauge line

SWAT+ Editor writes the header nbyr tstep lat lon elev above this line.

#	Column	Units	Description
1	nbyr	years	Number of years of record. SWAT+ uses it to size the store for the data, so it must be at least the number of calendar years the record spans.
2	tstep	-	Time step of the data. Not used by SWAT+; give 0.
3	lat	decimal degrees	Latitude of the gauge. Not used by SWAT+.
4	lon	decimal degrees	Longitude of the gauge. Not used by SWAT+.
5	elev	m	Elevation of the gauge. Used only for the elevation adjustment.

Daily records

#	Column	Units	Description
1	year	-	Calendar year.
2	day	-	Day of the year, 1 to 366. Called `jday` in earlier documentation.
3	tmax	°C	Maximum air temperature for the day, called `tmpmax` in earlier documentation. A value of -97 or less marks it as missing.
4	tmin	°C	Minimum air temperature for the day, called `tmpmin` in earlier documentation. A value of -97 or less marks it as missing.

Records must be continuous and in date order, as for precipitation: SWAT+ counts years by watching for day 365 or 366 followed by a record from a new year. Records before the simulation start are skipped when the file is read, by the rule described for precipitation files.

Missing days

SWAT+ generates both temperatures for a day when

- `tmax` or `tmin` is -97 or less (SWAT+ Editor and most tools write -99), or
- the day is before the first record or after the last record of the file.

When only one of the two values is missing, the measured value of the other is replaced by a generated one as well, so the pair always comes from one source. The generated maximum is lower on wet days than on dry days, using the day's precipitation from the same weather station. The method is described in Climate and weather.

The daily mean air temperature used by snow, plant growth and other processes is

$$T_{av} = \frac{T_{max} + T_{min}}{2}$$

Elevation adjustment

When `lapse` = 1 in `codes.bsn`, the temperatures of each HRU are changed for the difference between the gauge elevation and the HRU elevation:

$$T_{hru} = T + \text{tlaps}\,\frac{z_{gauge} - z_{hru}}{1000}$$

applied to the maximum, minimum and mean temperature. `tlaps` is the temperature lapse rate in `parameters.bsn` (°C per km, default 6.5), z_{hru} is the `elev` column of `hru.con` and z_{gauge} is `elev` from this file (m). When the station's `tmp` column is `sim`, the elevation of the weather generator station is used for z_{gauge}. As for precipitation, the adjustment values are calculated during channel and reservoir routing, so in a model without channels or reservoirs they remain zero.

Calibration

The calibration parameter `temp` (object type `cli`) is meant to change the stored values of listed temperature records, but in 62.0.1 it does not change the listed records. See Climate changes.

Example

First lines of `refdata/Osu_1hru/Imsiltmp.tmp` in the SWAT+ repository:

```
Imsiltmp.tmp: Temperature data - file written by SWAT+ editor 2023-03-22 04:22:29.141050
nbyr     tstep       lat       lon      elev
  42         0    35.610   127.290   247.900
1981    1    2.70000   -17.50000
1981    2   -3.00000    -7.50000
...
2023  364        2.3        -4.3
2023  365        0.8        -8.4
```

A daily record from 1 January 1981 to 31 December 2023 at 247.9 m. The record spans 43 calendar years while `nbyr` is 42. The Osu run starts in 2010, so only the 14 years from 2010 are stored and the smaller `nbyr` does no harm here; a run starting in 1981 would need `nbyr` of at least 43.

Related files

- `tmp.cli`: lists the temperature files.
- `weather-sta.cli`: assigns a temperature file to each weather station in its `tmp` column.
- `weather-wgn.cli`: weather generator statistics that fill missing days.
- `*.tem`: the same format under another extension.
- `file.cio`: names `tmp.cli` and the `tmp_path` prefix.
- `codes.bsn` and `parameters.bsn`: `lapse` switch and `tlaps` rate.
- `snow.sno`: snowfall and melt use the daily mean and maximum temperature.

Sources

- Reader: `cli_tmeas.f90`
- Type definition: `type climate_measured_data` in `climate_module.f90`
- Gauge name lookup from `weather-sta.cli`: `cli_staread.f90`, `search.f90`
- Daily use, missing values and range check: `climate_control.f90`, `cli_bounds_check.f90`
- Weather generator: `cli_weatgn.f90`, `cli_tgen.f90`
- Elevation adjustment: `cli_lapse.f90`, applied in `hru_control.f90`
- Path prefix: `in_path_tmp` in `input_file_module.f90`, read in `readcio_read.f90`
- Calibration change `temp`: `cal_conditions.f90`

Column	Fortran variable
`nbyr`	`tmp(i)%nbyr`
`tstep`	`tmp(i)%tstep`
`lat`	`tmp(i)%lat`
`lon`	`tmp(i)%long`
`elev`	`tmp(i)%elev`
tmax	`tmp(i)%ts(day, iyr)`
tmin	`tmp(i)%ts2(day, iyr)`

Purpose

`.tem` is an older extension for temperature data files, used by some data sets and preprocessing tools. SWAT+ opens a temperature file by the name given in `tmp.cli` and does not look at the extension, so a `.tem` file is read in the same way as a `.tmp` file. SWAT+ Editor writes `.tmp`.

The name has to be the same, extension included, in two places:

- a line of `tmp.cli`, which tells SWAT+ to read the file, and
- the `tmp` column of `weather-sta.cli`, which assigns it to a weather station.

If the `tmp` column names a file that is not listed in `tmp.cli`, SWAT+ writes `<name> file not found (tgage)` to `diagnostics.out` and generates the temperatures of that station for the whole run.

Format

The format is that of a .tmp file: a title line, a header line, a gauge line with **nbyr**, **tstep**, **lat**, **lon** and **elev**, and one record per day with year, day of year, maximum and minimum temperature in °C. Values of -97 or less mark missing data.

Example

refdata/Ames_sub1/ames.tem in the SWAT+ repository:

```
AME.Tmp
 NBYR    TSTEP       LAT      LONG      ELEV
   121        0     42.04    -93.89    316
1900    1   -99 -99
1900    2   -99 -99
...
1975    1   4.4 -7.2
1975    2   5   -7.8
...
2020    365 -1.7    -7.8
2020    366 -2.8    -8.9
```

A daily record of 121 years, 1900 to 2020, separated by tabs, which free-format reading accepts. In this data set the **tmp** column of weather-sta.cli names **ames.tem**, while tmp.cli lists **ames.tmp**, which is an empty file. SWAT+ therefore does not find **ames.tem**, never reads it, and generates the temperatures of the Ames run. Listing **ames.tem** in tmp.cli instead of **ames.tmp** makes the run use the measured record.

Related files

- *.tmp: the same format; missing values, elevation adjustment and storage are described there.
- tmp.cli: lists the temperature files to read.
- weather-sta.cli: assigns a temperature file to each weather station.

Sources

- Reader: cli_tmeas.f90
- Type definition: **type climate_measured_data** in climate_module.f90
- Name lookup and the **file not found (tgage)** message: cli_staread.f90, search.f90
- Daily use: climate_control.f90

Purpose

snow.sno is the snow parameter database. Each row is a named set of parameters for snowfall, snowpack temperature, snowmelt and snow cover. An HRU picks a row by name in the **snow** column of hru-data.hru. The file is named on the **hru_parm_db** line of file.cio (default **snow.sno**).

If hru-data.hru gives **null** for an HRU's snow entry, or a name that is not in **snow.sno**, the HRU uses the first row of **snow.sno**. No message is written in either case. A model therefore needs a **snow.sno** file with at least one row: if the file is missing or named **null** in file.cio, every HRU refers to a first row that does not exist.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one parameter set with nine values in the order below. The reader does not match columns by header name, so all nine values must be present and in this order.

#	Column	Units	Typical range	Description
1	`name`	-	-	Name of the snow parameter set, matched against the `snow` column of `hru-data.hru`.
2	`fall_tmp`	°C	-5 to 5	Snowfall temperature. On a day when the mean air temperature is at or below `fall_tmp`, all precipitation falls as snow and is added to the snowpack.
3	`melt_tmp`	°C	-5 to 5	Snowmelt base temperature. Melt occurs only on days when the maximum air temperature is above `melt_tmp`.
4	`melt_max`	mm H_2O/°C/day	0 to 10	Melt factor on 21 June (day 172).
5	`melt_min`	mm H_2O/°C/day	0 to 10	Melt factor on 21 December (day 355).
6	`tmp_lag`	-	0.01 to 1	Snowpack temperature lag factor. See Snowpack temperature for how 62.0.1 uses it.
7	`snow_h2o`	mm H_2O	0 to 500	Snow water content at and above which the HRU is fully covered by snow.
8	`cov50`	fraction	0.05 to 0.95	Fraction of `snow_h2o` at which 50% of the HRU is covered by snow.
9	`snow_init`	mm H_2O	0 and up	Snow water content at the start of the simulation.

Snow processes

All snow calculations run daily for each HRU, after canopy interception.

Snowfall

If the mean air temperature $\bar{T}$ is at or below `fall_tmp`, the day's precipitation (after canopy interception) is added to the snowpack and no rain reaches the ground. In sub-daily runs the rainfall for every time step of that day is set to zero.

Snowpack temperature

The intended snowpack temperature keeps part of the previous day's value:

$$T_{snow,d} = T_{snow,d-1}\,(1-\ell) + \bar{T}_d\,\ell$$

where ℓ is `tmp_lag`. Values near 1 give the current day's air temperature most of the weight. Values near 0 make the snowpack respond slowly.

In 62.0.1 the previous day's snowpack temperature is reset to zero at the start of each day, so the calculation reduces to

$$T_{snow,d} = \ell\,\bar{T}_d$$

and `tmp_lag` acts as a scaling factor on the day's mean air temperature. It has no lag effect.

Melt factor

The melt factor b_{mlt} varies through the year between `melt_min` and `melt_max`, which accounts for the effect of snowpack density on melt:

$$b_{mlt} = \frac{\text{melt_max} + \text{melt_min}}{2} + \frac{\text{melt_max} - \text{melt_min}}{2} \sin\left(\frac{2\pi}{365}(d - 81)\right)$$

where d is the day of the year. The factor equals `melt_max` on day 172 and `melt_min` on day 355 for every HRU. SWAT+ 62.0.1 does not swap the two for the Southern Hemisphere, so a model there should give the winter (June) factor in `melt_max` and the summer (December) factor in `melt_min`.

In rural areas the melt factor ranges from 1.4 to 6.9 mm H_2O/°C/day (Huber and Dickinson, 1988). Urban values tend to the high end because traffic compresses the snowpack: urban studies in Sweden reported 3.0 to 8.0 mm H_2O/°C/day (Bengston, 1981; Westerstrom, 1981), and snow on asphalt gave 1.7 to 6.5 mm H_2O/°C/day (Westerstrom, 1984).

Snow cover

Drifting, shading and topography mean that snow rarely covers an HRU evenly. SWAT+ describes the covered fraction with an areal depletion curve. With $r = SNO/\text{snow_h2o}$, the ratio of the current snow water content to `snow_h2o`,

$$sno_{cov} = \frac{r}{r + \exp(c_1 - c_2\, r)} \quad \text{for } r < 1, \qquad sno_{cov} = 1 \quad \text{for } r \geq 1$$

The shape coefficients c_1 and c_2 are set once at the start of the run so that the curve passes through 50% cover at $r = \text{cov50}$ and 95% cover at $r = 0.95$. `cov50` must therefore lie between 0 and 0.95.

The depletion curve affects melt only while the snow water content is between 0 and `snow_h2o`. A small `snow_h2o` makes its effect on melt minor; a large one makes it more important.

Snowmelt

When the maximum air temperature T_{max} is above `melt_tmp` and there is snow on the ground, melt is

$$SNO_{mlt} = b_{mlt}\, sno_{cov} \left(\frac{T_{snow} + T_{max}}{2} - \text{melt_tmp}\right)$$

limited to between zero and the snow water content. Melt water is added to the day's water input at the surface, together with any rain. In sub-daily runs it is spread evenly over the day's time steps.

Calibration and decision tables

Five of the parameters can be changed during calibration through `calibration.cal`, using these parameter names: `snofall_tmp` (`fall_tmp`), `snomelt_tmp` (`melt_tmp`), `snomelt_max` (`melt_max`), `snomelt_min` (`melt_min`) and `snomelt_lag` (`tmp_lag`). `snow_h2o`, `cov50` and `snow_init` have no calibration parameter. See parameter change files.

A decision table action `snow_change` switches an HRU to another `snow.sno` row during the run and logs the change to `lu_change_out.txt`. The snow cover curve is not recalculated after the switch, so it keeps the shape from the HRU's original `cov50`.

Example

`refdata/Ames_sub1/snow.sno` in the SWAT+ repository:

```
snow.sno: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
name                  fall_tmp      melt_tmp      melt_max      melt_min       tmp_lag      snow_h2o
snow001                1.00000       0.50000       4.50000       4.50000       1.00000       1.00000
```

One parameter set named `snow001`. `melt_max` and `melt_min` are both 4.5, so the melt factor is constant through the year. With `tmp_lag` at 1.0 the snowpack temperature equals the day's mean air temperature, and with `snow_h2o` at 1 mm the HRU is fully covered by any snowpack of 1 mm or more.

Related files

- `hru-data.hru`: selects a `snow.sno` row by name in its `snow` column; `null` or an unknown name selects the first row.
- `file.cio`: names the snow file.
- `tmp.cli` and `pcp.cli`: supply the air temperatures and precipitation the snow routine uses.

Sources

- Reader: `snowdb_read.f90`
- Type definition: `type snow_parameters` in `hru_module.f90`
- Name lookup from `hru-data.hru`: `hru_read.f90`
- HRU setup and depletion curve: `topohyd_init.f90`, `ascrv.f90`
- Snowfall and melt: `sq_snom.f90`, called from `hru_control.f90`
- Decision table action `snow_change`: `actions.f90`
- Calibration parameters: `cal_parm_select.f90`
- File name slot: `in_parmdb%snow` in `input_file_module.f90`

Column	Fortran variable
`name`	`snodb%name`
`fall_tmp`	`snodb%falltmp`
`melt_tmp`	`snodb%melttmp`
`melt_max`	`snodb%meltmx`
`melt_min`	`snodb%meltmn`
`tmp_lag`	`snodb%timp`
`snow_h2o`	`snodb%covmx`
`cov50`	`snodb%cov50`
`snow_init`	`snodb%init_mm`

Purpose

`soils.sol` defines every soil used in the watershed. Each soil is a block of lines: one profile line with the soil name, number of layers, hydrologic soil group and other whole-profile values, followed by one line per layer with depth, bulk density, available water capacity, saturated hydraulic conductivity, organic carbon, texture and other layer properties. An HRU picks a soil by name in the `soil` column of `hru-data.hru`. The file is named first on the `soils` line of `file.cio` (default `soils.sol`).

Every HRU should name a soil that is in the file. An unknown name is not reported: the HRU silently gets the first soil in `soils.sol`. Check the `sname` column of `checker.out`, which lists the soil each HRU uses.

The initial nutrient and carbon contents of the soil come from `nutrients.sol`, which an HRU selects through `soil_plant.ini`. The same `soils` line of `file.cio` names `nutrients.sol` second and `soils_lte.sol` third (see soils_lte.sol below).

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each soil then takes `nly + 1` lines: the profile line and `nly` layer lines. The reader counts soils by reading each profile line and skipping the next `nly` lines, so `nly` must equal the number of layer lines that follow. Columns are read by position; the header names are not checked. Spacing is not significant, so the wide indentation the SWAT+ Editor gives the layer lines has no meaning.

Profile line

#	Column	Units	Used if zero or out of range	Description
1	`name`	-	-	Soil name, up to 20 characters, matched against the `soil` column of `hru-data.hru`.
2	`nly`	-	-	Number of layer lines that follow. There is no upper limit in the model.
3	`hyd_grp`	-	-	Hydrologic soil group: `A`, `B`, `C` or `D`. It selects the curve number from `cntable.lum` and the conductivity used when `soil_k` is zero. Dual groups such as `A/D` and lower-case letters are not recognized.
4	`dp_tot`	mm	-	Maximum rooting depth. See Rooting depth for how 62.0.1 uses it.
5	`anion_excl`	fraction	0.5 if 0; 0.99 if 1 or more	Fraction of porosity from which anions are excluded.
6	`perc_crk`	fraction	-	Potential (maximum) crack volume of the soil profile as a fraction of total soil volume. Used only when crack flow is switched on (`crack` = 1 in `codes.bsn`).
7	`texture`	-	-	Texture label, up to 16 characters. Only the first whitespace-delimited word is read, so `L-L-L-L` is read whole but `L- L- L- L` is read as `L-`. The label is not used in any process; it can serve as a condition in `calibration.cal` (condition `texture`).

Layer lines

Layers are listed from the surface down. Each line has fourteen values.

#	Column	Units	Used if zero or out of range	Description
1	`dp`	mm	-	Depth from the soil surface to the bottom of the layer.
2	`bd`	Mg/m^3	1.3 if 0; capped at 2.0	Moist bulk density: oven-dry mass divided by the soil volume at or near field capacity.
3	`awc`	mm H_2O/mm soil	0.005 if 0; capped at 0.8	Available water capacity: water held between field capacity (-0.033 MPa) and permanent wilting point (-1.5 MPa).
4	`soil_k`	mm/h	by `hyd_grp` if 0 or less: A 50, B 20, C 5, D 2, other 10	Saturated hydraulic conductivity.
5	`carbon`	% of soil mass	0.001 if below 0.001	Organic carbon content of the fine earth (material passing a 2 mm sieve).
6	`clay`	%	-	Clay, particles below 0.002 mm.

#	Column	Units	Used if zero or out of range	Description
7	`silt`	%	-	Silt, particles between 0.002 and 0.05 mm.
8	`sand`	%	-	Sand, particles between 0.05 and 2.0 mm.
9	`rock`	% of total mass	capped at 98	Rock fragments, particles above 2 mm.
10	`alb`	fraction	-	Moist soil albedo. Only the top layer value is used.
11	`usle_k`	0.013 t m^2 h/(m^3 t cm)	-	USLE soil erodibility factor. Only the top layer value is used.
12	`ec`	dS/m	-	Electrical conductivity. Read and reported, but not used by any process in 62.0.1.
13	`caco3`	%	2.8 if 0	Calcium carbonate content.
14	`ph`	-	6.5 if 0	Soil pH. Read and reported, but not used by any process in 62.0.1.

`clay`, `silt` and `sand` are percentages of the mineral fraction and should add up to 100. The model does not check this, except that in sub-daily runs the sand content of the top layer is reset to 100 − clay − silt.

Layering

Thin first layer

After reading a soil, the reader checks the first layer. If it is shallower than 20 mm and the soil has only one layer, or the second layer reaches deeper than 20 mm, the bottom of the first layer is set to 20 mm.

Added 10 mm surface layer

Without a `soil_lyr_depths.sol` file in the run folder (the usual case), SWAT+ adds a 10 mm surface layer to every soil whose first layer is deeper than 19.5 mm. The new layer takes all its properties from the first input layer, so a soil with four input layers is simulated with five. The first input layer then runs from 10 mm to its listed depth. Outputs by layer count this added layer.

If the first layer is 19.5 mm or shallower (possible only when the second layer is also within 20 mm), no layer is added and the first layer is given a depth of 10 mm.

soil_lyr_depths.sol

If a file named `soil_lyr_depths.sol` is in the run folder, every soil is re-layered to the depths it lists. The file is not named in `file.cio`; SWAT+ looks for this name.

- Line 1 is a title, line 2 a header and line 3 a units label; all three are skipped.
- Each later line gives one depth in mm, from the surface down, as a whole number.
- Depths of 10 mm or less are skipped. The first layer is always 0 to 10 mm.
- A depth beyond the bottom of a soil is replaced by its total depth, and the list ends there for that soil. If the deepest listed depth is shallower than a soil, the profile is cut off at that depth.

Each new layer takes the depth-weighted average of the input layers it overlaps, computed millimetre by millimetre, for every layer property. `refdata/Ames_sub1/soil_lyr_depths.sol` lists 10, 25, 50, 100, 150, 200, 300, 450, 600, 750, 900, 1050, 1200, 1500, 1800 and 2100 mm, so a 2000 mm soil gets sixteen layers with the last from 1800 to 2000 mm.

Septic biozone layer

In an HRU with a septic system (the `sep` column of `landuse.lum`), the profile is split further so that the biozone set in `septic.str` is a separate layer.

Derived layer properties

At the start of the run SWAT+ derives the water contents of each layer from `bd`, `awc` and `clay`. As volume fractions:

$$\theta_{wp} = 0.40\,\frac{c\,\rho_b}{100}, \qquad \theta_{fc} = \theta_{wp} + AWC, \qquad \phi = 1 - \frac{\rho_b}{2.65}$$

where c is `clay` (%), ρ_b is `bd` and AWC is `awc`. If θ_{wp} is zero (no clay) it is set to 0.005. If θ_{fc} reaches the porosity ϕ, field capacity is set to $\phi - 0.05$ and the wilting point to $\theta_{fc} - AWC$; if that is not positive, field capacity becomes $0.75\,\phi$ and the wilting point $0.25\,\phi$.

For a layer of thickness h (mm), the plant-available water at field capacity is $FC = h\,(\theta_{fc} - \theta_{wp})$, the water between wilting point and saturation is $UL = h\,(\phi - \theta_{wp})$, and the percolation travel time (h) is

$$TT_{perc} = \frac{UL - FC}{K_{sat}}$$

with a minimum of 1 hour, where K_{sat} is `soil_k`. The profile sums of FC and UL are reported as `sumfc` and `sumul` in `checker.out`.

The crack volume of each layer is

$$V_{crk} = \text{perc_crk} \times 0.916\,e^{-0.0012\,z}\,h$$

where z is the depth to the bottom of the layer (mm).

The top layer's sand, silt and clay set the particle-size make-up of eroded sediment (sand, silt, clay, small and large aggregates). For the Green and Ampt infiltration method, the wetting front suction head is estimated from the porosity, clay and sand of the top layer.

Use of the soil properties

Initial soil water

Each layer starts at a fraction of its field capacity water. The fraction is `sw_init` in `parameters.bsn`. If `sw_init` is zero, it is estimated from the average annual precipitation P (mm) of the HRU's weather generator:

$$f = \frac{P}{P + \exp(9.043 - 0.002135\,P)}$$

The `sw_frac` column of `soil_plant.ini` is read but not used for this in 62.0.1.

Rooting depth

`dp_tot` limits root depth only while plants that start the run already growing are initialized. After that the model sets the HRU's maximum rooting depth to the bottom of the soil profile. Each planting operation then lowers it to the smaller of its current value and the plant's maximum root depth in `plants.plt`. The value is never raised again during the run, so after a shallow-rooted crop is planted, later crops in the same HRU are limited to that depth as well. The `zmx` column of `checker.out` shows this profile depth. It does not show `dp_tot`.

Runoff, percolation and solute leaching

`hyd_grp` picks one of the four curve numbers in `cntable.lum` for the HRU's land use. `anion_excl` reduces the pore volume available to nitrate, salts and other constituents moving down with percolating water. The nitrate that moves with a water volume w (mm) leaving a layer is

$$NO3_{mobile} = NO3_{ly}\left[1 - \exp\left(\frac{-w}{(1 - \text{anion_excl})\,UL}\right)\right]$$

See hydrology, soil water and temperature and nutrient cycling.

Erosion

The MUSLE soil factor of an HRU uses the top layer only:

$$K \cdot CFRG = \text{usle_k}_1 \; e^{-0.053\,\text{rock}_1}$$

which is multiplied by the support practice factor from `cons_practice.lum` and the slope length and steepness factor. See erosion and sediment.

Albedo

On days with less than 0.5 mm of snow, the albedo of a bare HRU is the top layer `alb`. With plant cover it is $0.23\,(1 - e) + \text{alb}_1\, e$, where $e = \exp(-5 \times 10^{-5}(CV + 0.1))$ and CV is above-ground biomass plus residue (kg/ha). There is no lower limit on `alb`, so a zero value gives zero albedo for bare soil.

Carbon and nutrients

`carbon`, `bd` and the layer thickness set the starting organic matter of each layer: the soil mass is $10\,000\,h\,\rho_b$ kg/ha, organic carbon is `carbon` percent of that, and organic matter is carbon divided by 0.58. How this is split into pools depends on the carbon model (the `carbon` column of `codes.bsn`) and on `nutrients.sol`; see nutrients.sol. In 62.0.1 the 0.001% floor on `carbon` applies to every layer; earlier revisions applied it to the top layer only.

With the Vadas and White phosphorus model (`soil_p` = 1 in `codes.bsn`), `clay` and `carbon` set the initial phosphorus sorption coefficient and `caco3` sets the rate of transfer between the active and stable mineral P pools:

$$k_{as} = 0.0023\,\text{caco3} + 0.005, \qquad 0.002 \le k_{as} \le 0.05$$

`caco3` is not used with the original phosphorus model.

Calibration and conditions

Soil columns can be changed in `calibration.cal` with these parameter names, for a range of layers: `anion_excl`, `crk` (`perc_crk`), `z` (`dp`), `bd`, `awc`, `k` (`soil_k`), `cbn` (`carbon`), `clay`, `silt`, `sand`, `rock`, `alb`, `usle_k`, `ec`, `cal` (`caco3`) and `ph`. Changes to `z`, `bd`, `awc` and `clay` recompute the layer water contents and the curve number coefficients. See parameter change files.

Calibration changes are applied before the soil organic pools are set up, and that setup works from the `carbon` column. A `cbn` change acts on the layer's organic carbon pool, so it is replaced when the pools are initialized and has no lasting effect in 62.0.1.

The `calibration.cal` conditions `hsg` and `texture` select HRUs by `hyd_grp` and `texture`. Decision tables can test `hyd_soil_group` and `k_factor` (the top layer `usle_k`).

soils_lte.sol

`soils_lte.sol` is used only by HRU-LTE objects (`hru-lte.hru`), which select a soil texture by name instead of a layered profile. Line 1 is a title and line 2 a header. The next twelve lines are read, one per texture class, each with four values:

#	Column	Units	Description
1	`texture`	-	Texture class name, matched against the soil texture column of `hru-lte.hru`.
2	`awc`	mm/mm	Available water capacity, multiplied by the soil depth given in `hru-lte.hru`.
3	`por`	mm/mm	Porosity, multiplied by the same soil depth.
4	`scon`	mm/h	Saturated hydraulic conductivity.

Twelve lines are always read. A project with HRU-LTE objects needs this file, since there is no built-in texture table.

Example

`refdata/Osu_1hru/soils.sol` in the SWAT+ repository, first soil, with the spacing reduced:

```
soils.sol: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name     nly hyd_grp     dp_tot anion_excl perc_crk texture  dp      bd     awc   soil_k  carbon   clay   s
SONGSAN    5       A 1200.00000    0.50000  0.50000 LS
                                                              180.0 1.39000 0.10871 16.10093 1.41760 17.300 25
                                                              250.0 1.64000 0.09959 19.31345 0.58820 15.700 25
                                                              460.0 1.48000 0.10069 18.05822 0.42350 16.100 20
                                                              820.0 2.00000 0.07661 37.49110 0.20000 11.300 18
                                                             1200.0 2.00000 0.06645 59.38071 0.26470  7.800 15
```

A loamy sand in hydrologic group A with five layers to 1200 mm. Because the first layer reaches 180 mm, SWAT+ adds a 10 mm surface layer and simulates six layers. `caco3` and `ph` are zero, so the model uses 2.8% and 6.5. The albedo (0.20) and K factor (0.24977) of the first layer are the only values of those two columns that affect the run.

Related files

- `hru-data.hru`: selects a soil by name in its `soil` column.
- `soil_plant.ini` and `nutrients.sol`: initial soil water, nutrients and other constituents of the soil.
- `cntable.lum`: curve numbers chosen by `hyd_grp`.
- `plants.plt`: plant maximum root depth, which limits rooting together with the soil depth.
- `codes.bsn` and `parameters.bsn`: crack flow switch, phosphorus model, carbon model and `sw_init`.
- `septic.str`: septic biozone layer.
- `checker.out`: profile summary by HRU.

Sources

- Readers: `soil_db_read.f90`, `soil_lte_db_read.f90`, called from `proc_read.f90`
- Type definitions: `type soil_profile_db`, `type soilayer_db` and `type soil_lte_database` in `soil_data_module.f90`; simulated soil in `soil_module.f90`
- Name lookup from `hru-data.hru`: `hru_read.f90`
- Layering, `soil_lyr_depths.sol` and septic split: `soils_init.f90`, `layersplit.f90`
- Value checks and derived properties: `soil_phys_init.f90`, `soil_awc_init.f90`, `soil_text_init.f90`
- Initial soil water, rooting depth and MUSLE factor: `hydro_init.f90`, `mgt_plantop.f90`, `pl_root_gro.f90`
- Curve number: `cn2_init.f90`, `curno.f90`
- Albedo, leaching and P: `albedo.f90`, `nut_nlch.f90`, `nut_pminrl2.f90`
- Carbon and nutrient start values: `soil_nutcarb_init.f90`, called from `main.f90.in`

- HRU-LTE texture lookup: hru_lte_read.f90
- Calibration and conditions: cal_parm_select.f90, cal_conditions.f90, conditions.f90
- File name slots: in_sol%soils_sol, in_sol%nut_sol, in_sol%lte_sol in input_file_module.f90

Column	Fortran variable
name	soildb%s%snam
nly	soildb%s%nly
hyd_grp	soildb%s%hydgrp
dp_tot	soildb%s%zmx
anion_excl	soildb%s%anion_excl
perc_crk	soildb%s%crk
texture	soildb%s%texture
dp	soildb%ly%z, then sol%phys%d
bd	soildb%ly%bd, then sol%phys%bd
awc	soildb%ly%awc, then sol%phys%awc
soil_k	soildb%ly%k, then sol%phys%k
carbon	soildb%ly%cbn, then sol%phys%cbn and soil1%cbn
clay, silt, sand, rock	soildb%ly%clay, %silt, %sand, %rock, then sol%phys%...
alb, usle_k, ec	soildb%ly%alb, %usle_k, %ec, then sol%ly%...
caco3	soildb%ly%cal, then sol%ly%cal
ph	soildb%ly%ph, then sol%ly%ph
soils_lte.sol columns	soil_lte%texture, %awc, %por, %scon

Purpose

landuse.lum lists the land use and management records of the project. Each row has a name and a set of names that point to records in other files: the plant community in plant.ini, the management schedule in management.sch, the curve numbers in cntable.lum, the support practice in cons_practice.lum, the urban land type in urban.urb, the Manning's n in ovn_table.lum and the structural practices in the .str files. An HRU picks a row by name in the lu_mgt column of hru-data.hru. The file is named first on the lum line of file.cio (default landuse.lum).

If an HRU names a land use that is not in the file, it silently gets the first row of landuse.lum; nothing is reported.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one land use with fourteen values, read by position. Every column must be filled; write null for an unused link. Names are up to 40 characters.

#	Column	Points to	Description
1	name	-	Land use and management name, matched against lu_mgt in hru-data.hru and named by the decision table action lu_change. Names must be unique.
2	cal_group	-	Calibration group label, called cal_grp in earlier documentation. See Calibration groups.
3	plnt_com	plant.ini	Plant community at the start of the run. null starts the HRU with no plants (bare soil or an urban surface).
4	mgt	management.sch	Management schedule. null means no scheduled operations: no planting, harvest or kill, so only plants that start the run growing (lc_status y in plant.ini) grow.
5	cn2	cntable.lum	Curve number record. The HRU's soil group picks one of its four values.
6	cons_prac	cons_practice.lum	Conservation practice, which sets the USLE P factor.
7	urban	urban.urb	Urban land type. null for land that is not urban.

#	Column	Points to	Description
8	urb_ro	-	Urban build-up and wash-off method: usgs_reg (USGS regression equations) or buildup_washoff (build-up and wash-off, as in SWMM). Any other value, including null, gives no urban sediment and nutrient loads even when urban is set.
9	ov_mann	ovn_table.lum	Overland flow Manning's n class.
10	tile	tiledrain.str	Tile drainage.
11	sep	septic.str	Septic system. See Septic systems.
12	vfs	filterstrip.str	Vegetative filter strip.
13	grww	grassedww.str	Grassed waterway.
14	bmp	bmpuser.str	User-defined BMP removal efficiencies.

Name matching

All names are matched case-sensitively when the files are read, before the HRUs are set up. For each name other than null that is not found, SWAT+ writes a line to diagnostics.out with the land use name and the missing name:

Column	Message	Result when not found
plnt_com	not found in plant.ini	No plants
mgt	not found in management.sch	No schedule
cn2	not found in cntable.lum	Built-in curve numbers 30, 55, 70, 77 (soil groups A to D)
cons_prac	not found (landuse-consprac)	P factor 1.0
tile	not found (landuse-tiledr)	No tile drain
sep	not found (landuse-septic)	No septic system
vfs	not found (landuse-fstrip)	No filter strip
grww	not found (landuse-grassww)	No grassed waterway
bmp	not found (landuse-bmpuser)	No user BMP

urban and ov_mann are matched later, when each HRU's plant community is set up, and an unknown name is not reported. An unknown urban name leaves the HRU non-urban. An unknown ov_mann name leaves the HRU's Manning's n at its built-in value of 0.05.

Use at the start of the run

For each HRU, SWAT+ takes from its land use:

- the plant community, the management schedule and the starting operation in the schedule (from rot_yr_ini in plant.ini);
- the curve number for moisture condition II from cntable.lum, for the soil's hydrologic group, from which the condition I and III values follow (see hydrology);
- the USLE P factor from cons_practice.lum;
- the urban land type and urban runoff method;
- the overland Manning's n (the ovn_mean column of ovn_table.lum), used for the time of concentration;
- the structural practice parameters from the .str files. The septic record is assigned before the soil layers are set up, so that a biozone layer can be added to the profile.

Land use change

The decision table action `lu_change` (see decision tables) switches an HRU to another `landuse.lum` row during the run and logs the change to `lu_change_out.txt`. The new plant community, schedule, curve number, P factor (and the USLE multiplier), urban type and Manning's n replace the old ones. Some values set at the start are not recalculated:

- The time of concentration keeps the old Manning's n.
- The structural practice parameters (tile, filter strip, grassed waterway, user BMP, septic) are not set again. The HRU keeps those of its first land use.
- The `cal_group` used by `calibration.cal` and decision table conditions stays that of the first land use, while soft calibration uses the group of the current land use.

Calibration groups

`cal_group` labels land uses for calibration:

- Soft calibration matches the `name` column of the land use regions in `water_balance.sft` against `cal_group`; the name `basin` matches every HRU.
- In `calibration.cal`, the condition `cal_group` limits a parameter change to HRUs with that group, and the condition `landuse` limits it to HRUs with that `landuse.lum` name.
- Decision tables can test `cal_group` as a condition.

A value of `null` is an ordinary label: all land uses with `null` form one group.

Septic systems

When `sep` names a record in `septic.str`, that record is assigned to every HRU with this land use. If its operation flag is not zero, SWAT+ adds a biozone layer to the soil profile at the depth and thickness given in `septic.str`, and from the year the system becomes operational it runs the biozone routine on each day the biozone layer is above freezing.

In 62.0.1 the daily volume of septic tank effluent is never calculated and stays at zero. No effluent water is added to the biozone layer, and the effluent concentrations in `septic.sep`, which are multiplied by that volume, add no nitrogen, phosphorus or BOD to the soil. The biozone routine still runs on the water and nutrients already in the layer. Septic systems set up through this column therefore have no effluent load on the HRU water or nutrient balance.

Example

`refdata/Ames_sub1/landuse.lum` in the SWAT+ repository, four of its rows, with the spacing reduced:

```
landuse.lum: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
name      cal_group plnt_com  mgt      cn2        cons_prac     urban urb_ro          ov_mann        tile
cosy_lum  null      cosy_comm mgt_01   rc_strow_g up_down_slope null  null            convtill_nores null
agrr_lum  null      agrr_comm AGRR_rot rc_strow_g cross_slope   null  null            convtill_res   null
frst_lum  null      frst_comm null     wood_f     up_down_slope null  null            forest_med     null
urld_lum  null      null      null     urban      up_down_slope URLD  buildup_washoff urban_asphalt  null
```

- `cosy_lum` grows the corn and soybean community `cosy_comm` under schedule `mgt_01`, with row crop curve numbers (straight row, good condition) and no support practice (P = 1 for up-and-down-slope tillage).
- `frst_lum` has no schedule, so the forest in `frst_comm` has no operations; it grows only if `frst_comm` starts it growing.
- `urld_lum` has no plants. It links to the urban type `URLD` in `urban.urb`, uses build-up and wash-off for urban loads, the `urban` curve numbers (98 for all soil groups) and the Manning's n of asphalt.
- In this project `AGRR_rot` is not in `management.sch`, so `diagnostics.out` reports `agrr_lum AGRR_rot not found in management.sch` and HRUs with `agrr_lum` run without a schedule.

Related files

- hru-data.hru: selects a land use by name in its `lu_mgt` column.
- plant.ini: plant communities.
- management.sch: management schedules.
- cntable.lum, [cons_practice.lum](#input-reference-lum-tables-cons-practicelum) and ovn_table.lum: the lookup tables.
- Structural practices and urban.urb: tiledrain.str, septic.str, filterstrip.str, grassedww.str, bmpuser.str, septic.sep and urban.urb.
- lum.dtl: decision tables, including `lu_change`.
- Plant growth and land use management.

Sources

- Reader: landuse_read.f90, called from proc_db.f90 after the plant community, schedule, lookup table and structural files
- Type definitions: `type land_use_management` and `type land_use_structures` in landuse_data_module.f90
- Name lookup from `hru-data.hru`: hru_read.f90; `cal_group` copied to the HRU in hrudb_init.f90
- HRU setup: hru_lum_init.f90, plant_init.f90, cn2_init.f90, time_conc_init.f90
- Structural practices: structure_init.f90, structure_set_parms.f90; septic setup in proc_hru.f90 and soils_init.f90; daily septic code in hru_control.f90, swr_percmain.f90, sep_biozone.f90
- Urban loads: hru_urban.f90
- Land use change: `lu_change` in actions.f90
- Calibration groups: cal_conditions.f90, conditions.f90, calsoft_hyd.f90
- File name slot: `in_lum%landuse_lum` in input_file_module.f90

Column	Fortran variable
name	lum%name
cal_group	lum%cal_group, copied to hru%cal_group and hru%lum_group_c
plnt_com	lum%plant_cov, index lum_str%plant_cov
mgt	lum%mgt_ops, index lum_str%mgt_ops
cn2	lum%cn_lu, index lum_str%cn_lu
cons_prac	lum%cons_prac, index lum_str%cons_prac
urban	lum%urb_lu, index hru%luse%urb_lu
urb_ro	lum%urb_ro, copied to hru%luse%urb_ro
ov_mann	lum%ovn, value hru%luse%ovn
tile	lum%tiledrain, index lum_str%tiledrain
sep	lum%septic, index lum_str%septic, then iseptic
vfs	lum%fstrip, index lum_str%fstrip
grww	lum%grasww, index lum_str%grasww
bmp	lum%bmpuser, index lum_str%bmpuser

Purpose

Three lookup tables supply values that landuse.lum names by record: `cntable.lum` (SCS curve numbers), `cons_practice.lum` (USLE support practice factor) and `ovn_table.lum` (Manning's n for overland flow). They are named third, fourth and fifth on the `lum` line of file.cio, after landuse.lum and management.sch. If a file is missing or named `null`, every land use gets the built-in values given below.

All three files have the same layout. Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one record, read by position. Values after the last numeric column (the descriptions the SWAT+ Editor writes) are not read. Record names are up to 40 characters and are matched case-sensitively.

cntable.lum

Curve numbers for moisture condition II, one value per hydrologic soil group, for each combination of land cover, treatment and hydrologic condition. The `cn2` column of `landuse.lum` names a record.

#	Column	Units	Built-in value	Range	Description
1	`name`	-	-	-	Curve number record name.
2	`cn_a`	-	30	30 to 100	Curve number for hydrologic soil group A.
3	`cn_b`	-	55	30 to 100	Curve number for hydrologic soil group B.
4	`cn_c`	-	70	30 to 100	Curve number for hydrologic soil group C.
5	`cn_d`	-	77	30 to 100	Curve number for hydrologic soil group D.

The SWAT+ Editor also writes `description`, `treat` and `cond_cov` columns, which are not read. The built-in values apply to land uses whose `cn2` is `null` or not found.

Curve number in the model

At the start of the run each HRU takes the value for the hydrologic soil group (`hyd_grp`) of its soil in `soils.sol`. The table values are those of SCS Engineering Division (1986), which apply to a 5% slope; SWAT+ 62.0.1 does not adjust them for the HRU slope. From CN_2 SWAT+ derives the dry and wet condition values

$$CN_1 = CN_2 - \frac{20\,(100 - CN_2)}{(100 - CN_2) + \exp[2.533 - 0.0636\,(100 - CN_2)]}, \qquad CN_1 \geq 0.4\,CN_2$$

$$CN_3 = CN_2\,\exp[0.006729\,(100 - CN_2)]$$

and the retention parameter curve used for daily runoff, as described in hydrology.

CN_2 can change during the run through the management operation `cnup` in `management.sch`, the decision table action `cn_update` (both limited to 35 to 95), a land use change (`lu_change`, which picks the new land use's record) and the calibration parameter `cn2` in `calibration.cal`.

Example

`refdata/Ames_sub1/cntable.lum` in the SWAT+ repository, first records, with the spacing reduced:

```
cntable.lum: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
name        cn_a      cn_b      cn_c      cn_d  description  treat              cond_cov
fal_bare    77.00000  86.00000  91.00000  94.00000  Fallow   Bare_soil          ----
fal_res_p   76.00000  85.00000  90.00000  93.00000  Fallow   Crop_residue_cover  Poor
fal_res_g   74.00000  83.00000  88.00000  90.00000  Fallow   Crop_residue_cover  Good
rc_strow_p  72.00000  81.00000  88.00000  91.00000  Row_crops  Straight_row     Poor
```

The names combine land cover, treatment and condition: `rc_strow_p` is row crops, straight row, poor condition. The SWAT+ Editor's table has 52 records, from fallow and row crops to forest, farmsteads and roads.

cons_practice.lum

USLE support practice factors for conservation practices. The `cons_prac` column of `landuse.lum` names a record.

#	Column	Units	Built-in value	Range	Description
1	`name`	-	-	-	Conservation practice name.
2	`usle_p`	-	1.0	0 to 1	USLE support practice factor P.
3	`slp_len_max`	m	1.0	-	Maximum slope length for the practice. Read but not used (see below).

The SWAT+ Editor also writes a `description` column, which is not read. The built-in values apply to land uses whose `cons_prac` is `null` or not found, so a P factor of 1 means no support practice.

P factor in the model

P enters the MUSLE sediment yield through a multiplier fixed at the start of the run:

$$K \cdot C_{frg} \cdot P \cdot LS \cdot 11.8$$

where K and C_{frg} come from the top soil layer (see soils.sol), and the slope length and steepness factor is

$$LS = \left(\frac{L}{22.128}\right)^{m} \left(65.41 \, \sin^2 \alpha + 4.56 \, \sin \alpha + 0.065\right), \qquad m = 0.6 \left[1 - \exp(-35.835 \, S)\right]$$

with L the slope length (`slp_len` in `topography.hyd`, m), S the slope (m/m) and $\alpha = \arctan S$. The model works out the smaller of L and `slp_len_max`, but then computes LS from L, so `slp_len_max` has no effect on LS in 62.0.1. See erosion and sediment.

P can also be set during the run by the decision table actions `p_factor`, `contour`, `stripcrop` and `terrace`, which recompute the multiplier. The calibration parameter `usle_p` in `calibration.cal` changes the HRU's P factor after the multiplier has been computed and does not recompute it, so by itself it does not change sediment yield.

Example

`refdata/Ames_sub1/cons_practice.lum`, first records:

```
cons_practice.lum: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
name                 usle_p   slp_len_max  description
up_down_slope       1.00000     121.00000  Up_and_down_slope
cross_slope         0.75000     121.00000  Cross_slope_tillage
contour_farming     0.50000     121.00000  Contour_tillage
strip_cros_slope    0.37000     121.00000  Strip_cropping_cross_slope
```

The SWAT+ Editor's table has 38 records: up-and-down and cross-slope tillage, contouring and strip cropping by slope class (1-2% to 21-25%, with pasture or row crops), and terraces by slope class with sod or underflow outlets.

ovn_table.lum

Manning's roughness coefficient for overland flow by tillage, residue and land cover. The `ov_mann` column of `landuse.lum` names a record.

#	Column	Units	Built-in value	Description
1	`name`	-	-	Manning's n class name.
2	`ovn_mean`	s/m1/3	0.5	Mean Manning's n for overland flow. This is the value the model uses.
3	`ovn_min`	s/m1/3	0.5	Minimum Manning's n. Read but not used; a guide for calibration ranges.
4	`ovn_max`	s/m1/3	0.5	Maximum Manning's n. Read but not used; a guide for calibration ranges.

The SWAT+ Editor also writes a `description` column, which is not read. A land use whose `ov_mann` is `null` or not found does not take a value from this file; its HRUs keep the model's starting value of 0.05.

Manning's n in the model

The HRU's Manning's n sets the overland part of its time of concentration (hours), which controls the peak runoff rate:

$$t_{ov} = \frac{0.0556\,(L\,n)^{0.6}}{(S + 0.0001)^{0.3}}$$

with L the slope length (m) and S the slope (m/m). The same n is used as the roughness of the HRU's concentrated flow channel in the channel part of the time of concentration, and in the area-weighted roughness of routing units.

The time of concentration is computed once, before calibration is applied. The calibration parameter `ovn` in `calibration.cal` changes the HRU's Manning's n after that, and a land use change (`lu_change`) also leaves the time of concentration unchanged, so neither alters the peak runoff rate in 62.0.1.

Example

`refdata/Ames_sub1/ovn_table.lum`, first records:

```
ovn_table.lum: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
name                 ovn_mean       ovn_min       ovn_max  description
fallow_nores          0.01000       0.00800       0.01200  Fallow_no_residue
convtill_nores        0.09000       0.06000       0.12000  Conventional_tillage_no_residue
convtill_res          0.19000       0.16000       0.22000  Conventional_tillage_residue
chisplow_nores        0.09000       0.06000       0.12000  Chisel_plow_no_residue
```

The SWAT+ Editor's table has 20 records, from fallow (0.01) and tilled cropland through range, grasses and forest (up to 0.80 for heavy forest) to urban surfaces (0.011 for asphalt).

Related files

- `landuse.lum`: names records in these tables in its `cn2`, `cons_prac` and `ov_mann` columns.
- `soils.sol`: the hydrologic soil group that picks the curve number, and the K factor and rock content of the MUSLE multiplier.
- `topography.hyd`: slope and slope length.
- `management.sch` and `lum.dtl`: the `cnup` operation and the `cn_update`, `p_factor`, `contour`, `stripcrop` and `terrace` actions.
- Parameter change files: calibration parameters `cn2`, `usle_p` and `ovn`.

Sources

- Readers: `cntbl_read.f90`, `cons_prac_read.f90`, `overland_n_read.f90`, called from `proc_db.f90`
- Type definitions: `type curvenumber_table`, `type conservation_practice_table` and `type overlandflow_n_table` in `landuse_data_module.f90`
- Name lookup: `landuse_read.f90` (`cn2`, `cons_prac`); `plant_init.f90` (`ov_mann`)
- Curve number: `cn2_init.f90`, `curno.f90`; changes in `mgt_sched.f90` (`cnup`) and `actions.f90` (`cn_update`)
- P factor and LS: `plant_init.f90`, `hydro_init.f90`, `ero_ysed.f90`, `actions.f90`
- Manning's n: `time_conc_init.f90`
- Calibration: `cal_parm_select.f90`; call order in `main.f90.in`
- File name slots: `in_lum%cntable_lum`, `in_lum%cons_prac_lum`, `in_lum%ovn_lum` in `input_file_module.f90`

Column	Fortran variable
`cntable.lum name`	`cn%name`
`cn_a`, `cn_b`, `cn_c`, `cn_d`	`cn%cn(1)` to `cn%cn(4)`, copied to `cn2(j)`
`cons_practice.lum name`	`cons_prac%name`
`usle_p`	`cons_prac%pfac`, copied to `hru%lumv%usle_p`
`slp_len_max`	`cons_prac%sl_len_mx`

Column	Fortran variable
ovn_table.lum name	overland_n%name
ovn_mean	overland_n%ovn, copied to hru%luse%ovn
ovn_min, ovn_max	overland_n%ovn_min, overland_n%ovn_max

Purpose

The channel line of file.cio names eight channel input files. The table below gives the use of each in 62.0.1. initial.cha (initial channel contents), temperature.cha (stream temperature) and the legacy files channel.cha, hydrology.cha and sediment.cha are described below; the other files are on the chandeg channel page.

SWAT+ has two channel objects. The channels listed in chandeg.con (SWAT-DEG or "lte" channels) are the ones 62.0.1 routes; their parameter files are on the chandeg channel page. The legacy channel object listed in channel.con is set up at the start of a run, but its daily routing is switched off in 62.0.1, so water sent to it does not leave it. Build channel networks with chandeg.con.

Files on the channel line

Each slot can be null. A missing file is treated like null.

Slot	Default name	Used in 62.0.1 for	Documented on
1	initial.cha	Initial water, sediment, nutrients and pesticides in chandeg channels.	initial.cha
2	channel.cha	Legacy channel object only. No effect on results.	legacy files
3	hydrology.cha	Legacy channel object only. No effect on results.	legacy files
4	sediment.cha	Legacy channel object only. No effect on results.	legacy files
5	nutrients.cha	In-stream water quality of chandeg channels.	chandeg channels
6	channel-lte.cha	One row per chandeg channel, pointing to the other files.	chandeg channels
7	hyd-sed-lte.cha	Geometry, hydraulics and sediment properties of chandeg channels.	chandeg channels
8	temperature.cha	Read, but the stream temperature calculation uses only built-in values.	temperature.cha

Three more channel files have fixed names and are read when present in the run folder: initial.cha_cs (initial pathogens, salts and constituents, below), sed_nut.cha (chandeg channels) and shade_factor.shf (below).

initial.cha

Named sets of initial conditions for chandeg channels. Each row of channel-lte.cha selects a set by name in its cha_ini column. Free format: two header lines are skipped, then one set per line. There is no id column, so the first value on each line is the name. The Editor adds a trailing description value, which the reader ignores.

#	Column	Description
1	name	Name of the set, matched against cha_ini in channel-lte.cha.
2	org_min	Name of a row in om_water.ini: initial water, sediment and nutrient concentrations.
3	pest	Name of a row in pest_water.ini: initial pesticide in the water and the bed.
4	path	Read but not used. Initial pathogens come from initial.cha_cs.
5	hmet	Read but not used.
6	salt	Read but not used. Initial salts come from initial.cha_cs.

A `cha_ini` name not found in `initial.cha`, or an `org_min` or `pest` name not found in its file, falls back to row 1 of that file without a message. Keep `initial.cha` and an `om_water.ini` in any model with chandeg channels.

Initial water and constituents

The `flo` value of the `om_water.ini` row is the initial water depth as a fraction of the bankfull depth `dp`. The channel starts with the volume the rating curve gives at that depth, and the concentrations of the row are converted to masses with that volume. With `flo` of 1 or less all of it is channel storage. Above 1 the volume is split between channel and floodplain in proportion to their volumes on the rating curve. Channels 0.001 km (1 m) long or shorter start empty.

Initial pesticides in the water are the `pest_water.ini` concentration (mg/kg) times the stored water; pesticides in the bed are the benthic concentration times the volume of the active bed layer (width times length times the pesticide's active bed depth).

initial.cha_cs

An optional file with a fixed name, read when present, with initial pathogens, salts and other constituents for chandeg channels. Two header lines are skipped, then one row per line:

#	Column	Description
1	`name`	Name of the row. Not checked.
2	`pest`	Read but not used. Pesticides come from `initial.cha`.
3	`path`	Name of a row in `path_water.ini`.
4	`hmet`	Read but not used.
5	`salt`	Name of a row in `salt_channel.ini`: initial salt ion concentrations (g/m^3).
6	`cs`	Name of a row in `cs_channel.ini`: initial constituent concentrations (g/m^3).

Rows are paired with `initial.cha` by position: row 3 of `initial.cha_cs` applies to channels that use row 3 of `initial.cha`, whatever its name. Keep the two files in the same order with the same number of rows. Salt and constituent masses are the concentrations times the initial water volume.

In 62.0.1 the pathogen pointer is resolved, but the initial pathogen values are then read from the `path_water.ini` row with the position of the channel's `pest_water.ini` row.

temperature.cha

Parameters of the stream temperature calculation for chandeg channels. Free format: two header lines are skipped, then one set per line with 15 values. The name is limited to 13 characters.

#	Column	Units	Built-in value	Typical range	Description
1	`name`	-	-	-	Name of the set (up to 13 characters).
2	`sno_mlt`	-	1.0	-	Weight of the snowmelt contribution.
3	`gw`	-	1.0	-	Weight of the groundwater contribution.
4	`sur_lat`	-	1.0	-	Weight of the surface runoff and lateral flow contributions.
5	`sno_lag`	days	2	1 to 3	Air temperature averaging period for snowmelt.
6	`gw_lag`	days	250	200 to 365	Air temperature averaging period for groundwater.
7	`surf_lag`	days	5	2 to 5	Air temperature averaging period for surface runoff.
8	`lat_lag`	days	10	5 to 10	Air temperature averaging period for lateral flow.
9	`lat_lag_coef`	-	0.75	-	Multiplier on the lagged air temperature for lateral flow.
10	`surf_lag_coef`	-	0.75	-	Multiplier on the lagged air temperature for surface runoff and snowmelt.
11	`gw_lag_coef`	-	0.75	-	Multiplier on the lagged air temperature for groundwater.
12	`hex_coef1`	-	0.75	-	Multiplier on the dew point temperature in the heat exchange.
13	`hex_coef2`	-	1.5	-	Divisor on the channel depth when the flow depth for heat exchange is calculated.
14	`sf_on`	-	0	0 or 1	Shade factor source: `1` daily values from `shade_factor.shf`, `0` the single value `ssff`. Integer.
15	`ssff`	fraction	0.5	0 to 1	Shade factor: the fraction of solar radiation kept off the water surface.

In 62.0.1 the rows of `temperature.cha` have no effect. The reader stores them, but the temperature calculation always uses the built-in values in the table, whether or not the file exists, for every channel. With the built-in `sf_on` of 0, `shade_factor.shf` is not used either. The stream temperature calculation itself runs every day for every chandeg channel.

Stream temperature

The temperature of the water leaving a channel is calculated in three steps. Values in this section are the built-in values above.

Local inflow. For the routing units that drain directly into the channel, the day's HRU snowmelt Q_{sno}, surface runoff Q_{sur}, lateral flow Q_{lat} and water yield Q_{wyld} are summed (mm), and the groundwater inflow Q_{gw} (m³/s, at most 10) is taken from the channel's hydrograph separation. Each component gets an air temperature averaged over its lag period: T_x is the mean of the daily $(T_{max} + T_{min})/2$ over the last `surf_lag`, `lat_lag`, `gw_lag` or `sno_lag` days, at least 0.01 °C, and for snowmelt at most 2.5 °C. In the first simulation year the lag is limited to the day of the year. Then

$$T_{loc} = \frac{a_{sno} k_{sur} T_{sno} Q_{sno} + a_{gw} k_{gw} T_{gw} Q_{gw} + a_{sl} k_{lat} T_{lat} Q_{lat} + a_{sl} k_{sur} T_{sur} Q_{sur}}{Q_{wyld}}$$

with a_{sno} = `sno_mlt`, a_{gw} = `gw`, a_{sl} = `sur_lat`, and k_{sur}, k_{lat} and k_{gw} the lag coefficients `surf_lag_coef`, `lat_lag_coef` and `gw_lag_coef`. With gwflow active the groundwater term is left out. If T_{loc} differs from the previous day's outflow temperature by more than 5 °C, or the weather station has no temperature gauge, $T_{loc} = 5 + 0.75\,\bar{T}$ with $\bar{T}$ the day's mean air temperature.

Mixing with upstream water. Water from upstream channels arrives at their flow-weighted outflow temperature T_{up} (or $5 + 0.75\,\bar{T}$ for a channel without upstream channels). It is weighted by q, the outflow of the channel itself for the day (m³/s):

$$T_0 = \frac{T_{up}\, q + T_{loc}\, Q_{wyld}}{q + Q_{wyld}}$$

or $T_0 = T_{loc}$ when the outflow is zero.

Heat exchange. Following Edinger et al. (1974), with dew point $T_d = \min(h_1\, T_{dew},\ \bar{T})$, h_1 = `hex_coef1`, $T_m = (T_0 + T_d)/2$ and wind speed U (m/s),

$$\beta = 0.35 + 0.015\, T_m + 0.0012\, T_m^2, \qquad K_e = 4.48 + 0.05\, T_0 + (\beta + 0.47)(9.2 + 0.46\, U^2)$$

The short-wave input is $H_s = 0.97 \times 11.57\, R_s\, (1 - SF)$ W/m², with R_s the solar radiation (MJ/m²/day) and SF the shade factor `ssff`. The long-wave input is $H_a = 0.96\, \varepsilon\, \sigma\, (\bar{T} + 273.15)^4$ with $\varepsilon = 0.74 + 0.0065\, e_a\, (1 + 0.17\, C^2)$, e_a the vapour pressure (hPa) and $C = 0.9\, R_s / R_{s,max} + 0.1$ the cloud factor. The equilibrium temperature is

$$T_{eq} = T_d + \frac{H_s + H_a - 305.5 - 4.48\, T_d}{K_e}$$

and the outflow temperature is

$$T = T_0 + \frac{K_e\, (T_{eq} - T_0)\, t}{\rho\, c_p\, d_f / 86400}, \qquad T \geq 0.001$$

where t is the travel time through the channel (days) at the day's peak flow, $\rho c_p = 4.186 \times 10^6$ J/m³/°C and d_f the flow depth (m). The flow depth comes from a triangular section with depth-to-width ratio $(D/h_2)/W$, h_2 = `hex_coef2`, that passes the day's outflow at the channel velocity; it is at least 0.1 m and at most $2D$.

The result is the outflow temperature passed downstream and reported in the outflow block of the `channel_sd` output (see channel outputs). The in-stream water quality and the `water_temp` column of `channel_sd` use $5 + 0.75\,\bar{T}$ instead.

Shade factor file

`shade_factor.shf` has a fixed name. Two header lines are skipped, then one record per line with three values: day of year, channel number (the chandeg channel's sequence number) and shade factor (fraction). It would be used with `sf_on` = 1 on day 1 to 365 (day 366 uses 0.5); in 62.0.1 `sf_on` is always the built-in 0.

Legacy channel files

`channel.cha`, `hydrology.cha` and `sediment.cha` describe the legacy channel object of `channel.con`. SWAT+ 62.0.1 reads them and sets up those channels at the start of the run, but does not route water through them, so the values have no effect on results. The formats are given for existing models. Two header lines are skipped in each file and the reader does not match columns by header name; the column names below are the names the reader uses.

channel.cha

#	Column	Description
1	`id`	Channel number.
2	`name`	Channel name.
3	`init`	Name of a row in `initial.cha`.
4	`hyd`	Name of a row in `hydrology.cha`.
5	`sed`	Name of a row in `sediment.cha`.
6	`nut`	Name of a row in `nutrients.cha`.

Names that are not found are written to `diagnostics.out` as `<name> not found (hydrology.cha)` and similar.

hydrology.cha

#	Column	Units	Built-in value	Description
1	`name`	-	-	Name of the set.
2	`w`	m	2.0	Average width of the main channel.
3	`d`	m	0.5	Average depth of the main channel.
4	`s`	m/m	0.01	Average slope. Values of 0 or less become 0.0001.
5	`l`	km	0.1	Length. Values of 0 or less become 0.001.
6	`n`	-	0.05	Manning's n, limited to 0.01 to 0.70.
7	`k`	mm/h	0.01	Effective hydraulic conductivity of the channel alluvium.
8	`wdr`	m/m	6.0	Width-to-depth ratio. Values of 0 or less become 3.5.
9	`alpha_bnk`	days	0.03	Bank storage recession factor, stored as $e^{-\alpha}$.
10	`side`	-	0.0	Side slope (horizontal per unit rise). Values of 10^{-6} or less become 2.0.

sediment.cha

#	Column	Units	Built-in value	Description
1	`name`	-	-	Name of the set.
2	`eqn`	-	0	Sediment routing method: `0` original SWAT, `1` Bagnold, `2` Kodatie, `3` Molinas and Wu, `4` Yang.

#	Column	Units	Built-in value	Description
3	`cov1`	-	0.1	Channel erodibility factor. Limited to 0 to 1 for `eqn` 0; for the other methods, values of 0 or less become 1 and values above 25 become 25.
4	`cov2`	-	0.1	Channel cover factor, with the same limits as `cov1`.
5	`bnk_bd`	g/cm³	0	Bank bulk density. Values near 0 become 1.40.
6	`bed_bd`	g/cm³	0	Bed bulk density. Values near 0 become 1.50.
7	`bnk_kd`	cm³/N/s	0	Bank erodibility from a jet test. Values near 0 become $0.2/\sqrt{\tau_c}$, or 0.2 without a critical shear stress.
8	`bed_kd`	cm³/N/s	0	Bed erodibility, as `bnk_kd`.
9	`bnk_d50`	µm	0	Bank median particle size. Values near 0 become 50; at most 10000.
10	`bed_d50`	µm	0	Bed median particle size. Values near 0 become 500; at most 10000.
11	`tc_bnk`	N/m²	0	Critical shear stress of the bank.
12	`tc_bed`	N/m²	0	Critical shear stress of the bed.
13 to 24	`erod(1)` to `erod(12)`	-	0	Monthly erodibility, 0 non-erosive to 1 no resistance. If all twelve are zero, all are set to `cov1`.

Example

`refdata/Osu_1hru` in the SWAT+ repository, file.cio channel line and `initial.cha`:

```
channel            initial.cha       null              null              null              nutrients.cha
```

```
initial.cha: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                        org_min             pest             path             hmet               salt
initcha1                    no_init             null             null             null               null
```

The model uses only chandeg channels: slots 2 to 4 and 8 are `null`. All channels use the initial set `initcha1`, whose `org_min` points to the row `no_init` of om_water.ini; that row has all values zero, so the channels start empty. The `pest`, `path`, `hmet` and `salt` pointers are `null`. No reference dataset includes `temperature.cha`, `initial.cha_cs`, `channel.cha`, `hydrology.cha` or `sediment.cha`.

Related files

- Chandeg channel files: channel-lte.cha, hyd-sed-lte.cha, sed_nut.cha and nutrients.cha.
- chandeg.con and channel.con: the two channel object types.
- file.cio: the `channel` line.
- om_water.ini, pest_water.ini, path_water.ini, salt_channel.ini, cs_channel.ini: initial contents referenced from `initial.cha` and `initial.cha_cs`.
- tmp.cli: the daily maximum and minimum temperatures used for the lagged air temperatures.
- Channel outputs and channels in the model theory.

Sources

- File name slots: `type input_cha` in input_file_module.f90, read in readcio_read.f90; `in_shf` in the same module
- Read order: proc_cha.f90; `temperature.cha` and `shade_factor.shf` from proc_read.f90
- Readers: ch_read_init.f90, ch_read_init_cs.f90, ch_read_temp.f90, shade_factor_read.f90, ch_read.f90, ch_read_hyd.f90, ch_read_sed.f90
- Type definitions: channel_data_module.f90 (`channel_init_datafiles`, `channel_init_datafiles_cs`, `water_temperature_data`, `channel_data_char_input`, `channel_hyd_data`, `channel_sed_data`); `swatdeg_init_datafiles` in sd_channel_module.f90; `shade_factor_data` in hydrograph_module.f90

- Initial pointers and contents: sd_channel_read.f90, sd_hydsed_init.f90, om_water_init.f90
- Stream temperature: ch_temp.f90, called from sd_channel_control3.f90; travel time and velocity from sd_channel_sediment3.f90
- Legacy object: set-up in ch_initial.f90; the "chan" case is commented out in command.f90

File and column	Fortran variable
initial.cha row	ch_init(i) (name, org_min, pest, path, hmet, salt)
initial.cha_cs row	ch_init_cs(i) (name, pest, path, hmet, salt, cs)
resolved pointers	sd_init(i)%org_min, %pest, %path, %salt, %cs
temperature.cha row	w_temp(i), components as the column names
shade_factor.shf row	shf_db(i)%jday, %cha, %value
channel.cha row	ch_dat_c(i) -> ch_dat(i)
hydrology.cha row	ch_hyd(i)
sediment.cha row	ch_sed(i)

Purpose

SWAT+ 62.0.1 routes water, sediment, nutrients and other constituents through the channels listed in chandeg.con. These are often called SWAT-DEG or "lte" channels, and their outputs are the channel_sd files. They use these parameter files:

- channel-lte.cha: one row per channel, pointing by name to the other files.
- hyd-sed-lte.cha: channel geometry, hydraulics, bank and bed properties.
- sed_nut.cha: optional extra sediment and nutrient coefficients. The SWAT+ Editor does not write it.
- nutrients.cha: in-stream water quality (QUAL2E) rate constants.

Initial channel contents come from initial.cha, described on the channel group page together with temperature.cha and the legacy files channel.cha, hydrology.cha and sediment.cha. The legacy channel object of channel.con is not routed in 62.0.1, so those legacy files have no effect on the results.

Links between the files

The lcha column of chandeg.con gives the id of a row in channel-lte.cha. That row names one row in each of the other files:

channel-lte.cha column	Matched against	If the name is not found
cha_ini	name in initial.cha	row 1 of initial.cha
cha_hyd	name in hyd-sed-lte.cha, and name in sed_nut.cha	row 1 of each file (or the built-in values of sed_nut.cha when that file is absent)
cha_sed	not used	
cha_nut	name in nutrients.cha	row 1 of nutrients.cha

An unmatched name writes no message to diagnostics.out; the channel takes row 1 without warning. Check the names before a run.

The file names of channel-lte.cha, hyd-sed-lte.cha, nutrients.cha and initial.cha are set on the channel line of file.cio (slots 6, 7, 5 and 1). sed_nut.cha has a fixed name and is read when a file of that name is in the run folder.

channel-lte.cha

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one channel with six values in this order.

#	Column	Units	Description
1	id	-	Channel number. chandeg.con refers to the row by this number in its lcha column.
2	name	-	Channel name.
3	cha_ini	-	Name of a row in initial.cha.
4	cha_hyd	-	Name of a row in hyd-sed-lte.cha, and of a row in sed_nut.cha when that file is used.
5	cha_sed	-	Read but not used. The Editor writes null.
6	cha_nut	-	Name of a row in nutrients.cha.

Number the rows 1, 2, 3 and so on in file order. The row is stored under its id, but the name lookups are applied to the row's position in the file, so rows whose id differs from their position get the wrong parameter sets.

hyd-sed-lte.cha

Free format. Two header lines are skipped, then one parameter set per line with 23 values in this order. The reader does not match columns by header name. The Editor adds a description header with no values under it, which the reader ignores.

Several column headers written by the Editor do not describe what SWAT+ 62.0.1 does with the value. The table gives the Editor header and the use in 62.0.1.

#	Column	Units	Typical range	Description
1	name	-	-	Name of the parameter set, matched against cha_hyd in channel-lte.cha.
2	order	-	1 to 12	Stream order (integer). Groups channels in the chanbud_order output and is tested by the decision table condition ch_order. Use 1 to 12: values outside that range fall outside the order budget table.
3	wd	m	from QSWAT+	Bankfull top width of the channel, calculated by QSWAT+ from the drainage area.
4	dp	m	from QSWAT+	Bankfull depth, from the top of the bank to the channel bottom, calculated by QSWAT+ from the drainage area.
5	slp	m/m	from QSWAT+	Average channel slope along its length, from the DEM. Values below 10^{-9} are raised to 10^{-6}.
6	len	km	from QSWAT+	Channel length. A channel of 0.001 km (1 m) or less passes its inflow through unchanged.
7	mann	-	0.025 to 0.15	Manning's n of the channel. Values below 0.01 are raised to 0.01. Chow (1959) gives about 0.03 for clean straight channels, 0.04 to 0.05 for winding channels with pools, weeds or stones, and 0.07 to 0.15 for sluggish or very weedy reaches.

#	Column	Units	Typical range	Description
8	`k`	mm/h	0 to 130	Effective hydraulic conductivity of the channel bed, used for transmission losses. Lane (1983) gives more than 127 mm/h for very clean gravel and large sand, 51 to 127 for clean sand and gravel, 25 to 76 for sand and gravel with little silt and clay, 6 to 25 for sand and gravel with much silt and clay, and 0.025 to 2.5 for consolidated beds with high silt and clay. Use 0 for perennial streams fed by groundwater.
9	`erod_fact`	-	-	Bank erosion exponent. Read but not used.
10	`cov_fact`	-	0 to 1	Channel cover factor. It adds a vegetation term to the critical velocity for bank erosion, so a larger value protects the banks more. See Bank erosion.
11	`wd_rto`	-	1.05 and up	Read as the channel sinuosity (channel length over straight-line length), although the header and the values the Editor writes are a width-to-depth ratio. Values below 1.05 are raised to 1.05. Used for the bend velocity in bank erosion.
12	`eq_slp`	-	about 1	Critical velocity coefficient: a multiplier on the calculated critical velocity for bank erosion. The header name suggests an equilibrium slope; 62.0.1 has no such parameter.
13	`d50`	mm	-	Median bed sediment size. Read, but replaced by 1.0 mm at the start of every day's sediment calculation, so the value in the file has no effect.
14	`clay`	%	0 to 100	Clay content of bank and bed. Sets the bank cohesion.
15	`carbon`	%	0 to 100	Organic carbon content of the bed. Sets the pesticide sorption coefficient in the channel.
16	`dry_bd`	t/m^3	1.1 to 1.9	Dry bulk density of bank and bed. Converts bank and bed erosion from metres to tonnes, and sets the bed porosity for pesticide mixing.
17	`side_slp`	-	-	Channel side slope. See Channel geometry and rating curve.
18	`bed_load`	-	-	Read as the bankfull flow multiplier, the ratio of the flow at which overbank flooding starts to the bankfull flow. Replaced by 1.5 at the start of every day's sediment calculation, so the value in the file has no effect.
19	`fps`	m/m	0.00001 to `slp`	Lateral slope of the floodplain away from the channel (rise over run). Values above `slp` are lowered to `slp`; values of 10^{-6} or less are set to 10^{-5}.
20	`fpn`	-	0.025 to 0.16	Manning's n of the floodplain. Chow (1959) gives 0.025 to 0.05 for pasture and cultivated land, 0.035 to 0.16 for brush and 0.08 to 0.2 for trees.
21	`n_conc`	mg/kg	-	Nitrogen content of eroded bank sediment, added to the flow as organic N.

#	Column	Units	Typical range	Description
22	`p_conc`	mg/kg	-	Phosphorus content of eroded bank sediment.
23	`p_bio`	fraction	0 to 1	Fraction of the bank phosphorus that enters the flow as soluble P. The rest enters as sediment-bound P.

There are no built-in values for this file. A channel whose `cha_hyd` name is not found uses row 1. Without `hyd-sed-lte.cha` the chandeg channels have no parameters, so the file is required whenever `chandeg.con` lists channels.

sed_nut.cha

An optional table with extra coefficients for the chandeg sediment and nutrient calculations. It is read only when a file named `sed_nut.cha` is present. Rows are matched by the same `cha_hyd` name as `hyd-sed-lte.cha`. Two header lines are skipped, then 13 values per line:

#	Column	Units	Built-in value	Description
1	`name`	-	-	Name, matched against `cha_hyd` in `channel-lte.cha`.
2	`order`	-	-	Text field, read and not used.
3	`pk_rto`	ratio	1.0	Multiplier on the peak-to-mean daily flow ratio. See Peak flow.
4	`fp_inun_days`	days	5.0	Days the floodplain stays inundated after a flood. Replaced by 1.0 at the start of every day's sediment calculation, so neither the file value nor the built-in value has an effect.
5	`n_setl`	ratio	0.5	Ratio of N settling to sediment settling. Not used.
6	`p_setl`	ratio	0.5	Ratio of P settling to sediment settling. Not used.
7	`n_sol_part`	1/h	0.01	Rate of nitrate conversion to organic N, used only with the simplified transformations. See Nutrients.
8	`p_sol_part`	1/h	0.01	Rate of soluble P conversion to sediment-bound P, used only with the simplified transformations.
9	`n_dep_enr`	ratio	0.5	Enrichment ratio for organic N deposited with sediment on the floodplain and in the channel.
10	`p_dep_enr`	ratio	0.5	Enrichment ratio for sediment-bound P deposited with sediment.
11	`arc_len_fr`	fraction	1.2	Fraction of the meander arc length where banks erode. Read but not used in the erosion mass in 62.0.1.
12	`bed_exp`	-	1.5	Exponent of the bed erosion equation. Replaced by 1.0 at the start of every day's sediment calculation.
13	`wash_bed_fr`	fraction	0.1	Fraction of bank erosion deposited in the channel bed. See Channel deposition.

Without `sed_nut.cha`, every channel uses the built-in values. With the file present, a channel whose name is not found uses row 1.

nutrients.cha

Free format. Two header lines are skipped, then one parameter set per line with 39 values in this order. The reader does not match columns by header name. Every value of zero or less in the columns that have a value in the "Replacement" column is replaced by that value.

All rates are for 20 °C and are adjusted to the water temperature in the calculation. The water temperature used here is $5 + 0.75\,\bar{T}$, where $\bar{T}$ is the day's mean air temperature at the channel's weather station, with a minimum of 0.1 °C. The stream temperature from the heat balance (see `temperature.cha`) sets only the outflow temperature.

#	Column	Units	Replacement	Typical range	Description
1	`name`	-	-	-	Name, matched against `cha_nut` in `channel-lte.cha`.
2	`plt_n`	ppm	-	0 to 100	Organic N concentration. Not used.
3	`ptl_p`	ppm	-	0 to 100	Organic P concentration, called `plt_p` in earlier documentation. Not used.
4	`alg_stl`	m/day	1.0	0.15 to 1.82	Algal settling rate.
5	`ben_disp`	mg P/m²/day	0.05	0.001 to 0.1	Benthic source rate for dissolved P.
6	`ben_nh3n`	mg N/m²/day	0.5	0 to 1	Benthic source rate for ammonium N.
7	`ptln_stl`	1/day	0.05	0.001 to 0.1	Organic N settling rate.
8	`ptlp_stl`	1/day	0.05	0.001 to 0.1	Organic P settling rate.
9	`cst_stl`	1/day	2.5	0.01 to 10	Settling rate of an arbitrary non-conservative constituent. Not used.
10	`ben_cst`	mg/m²/day	2.5	0.01 to 10	Benthic source rate of an arbitrary non-conservative constituent. Not used.
11	`cbn_bod_co`	1/day	1.71	0.02 to 3.4	CBOD deoxygenation rate.
12	`air_rt`	1/day	1.0	0 to 100	Reaeration rate (Fickian diffusion).
13	`cbn_bod_stl`	1/day	none	-0.36 to 0.36	Rate of CBOD loss by settling. A value of zero or less is kept.
14	`ben_bod`	mg O_2/m²/day	2.0	0 to 100	Sediment oxygen demand.
15	`bact_die`	1/day	2.0	0.05 to 4	Coliform die-off rate. Not used; pathogen die-off comes from the pathogen database.
16	`cst_decay`	1/day	1.71	0 to 10	Decay rate of an arbitrary non-conservative constituent. Not used.
17	`nh3n_no2n`	1/day	0.55	0.1 to 1	Rate of biological oxidation of NH_3 to NO_2.
18	`no2n_no3n`	1/day	1.1	0.2 to 2	Rate of biological oxidation of NO_2 to NO_3.
19	`ptln_nh3n`	1/day	0.21	0.2 to 0.4	Rate of hydrolysis of organic N to ammonium.
20	`ptlp_solp`	1/day	0.35	0.01 to 0.7	Rate of mineralisation of organic P to dissolved P.
21	`q2e_lt`	-	2	1 to 4	QUAL2E light averaging option. Not used.
22	`q2e_alg`	-	2	1 to 3	Algal growth limitation: `1` multiplicative ($\mu_{max} f_L f_N f_P$), `2` limiting nutrient ($\mu_{max} f_L \min(f_N, f_P)$), `3` harmonic mean of f_N and f_P. Integer.
23	`chla_alg`	µg chl-a/mg algae	50	10 to 100	Ratio of chlorophyll-a to algal biomass.
24	`alg_n`	mg N/mg algae	0.08	0.07 to 0.09	Fraction of algal biomass that is N.
25	`alg_p`	mg P/mg algae	0.015	0.01 to 0.02	Fraction of algal biomass that is P.

#	Column	Units	Replacement	Typical range	Description
26	`alg_o2_prod`	mg O_2/mg algae	1.6	1.4 to 1.8	O_2 production per unit of algal photosynthesis.
27	`alg_o2_resp`	mg O_2/mg algae	2.0	1.6 to 2.3	O_2 uptake per unit of algal respiration.
28	`o2_nh3n`	mg O_2/mg N	3.5	3 to 4	O_2 uptake per unit of NH_3 oxidised.
29	`o2_no2n`	mg O_2/mg N	1.07	1 to 1.4	O_2 uptake per unit of NO_2 oxidised.
30	`alg_grow`	1/day	2.0	1 to 3	Maximum specific algal growth rate.
31	`alg_resp`	1/day	2.5	0.05 to 5	Algal respiration rate.
32	`slr_act`	fraction	0.3	0 to 1	Fraction of solar radiation that is photosynthetically active.
33	`lt_co`	kJ/m²/min	0.75	-	Half-saturation coefficient for light. Multiplied by 0.06 on reading, which converts kJ/m²/min to MJ/m²/h.
34	`const_n`	mg N/L	0.02	0.01 to 0.3	Michaelis-Menten half-saturation constant for N.
35	`const_p`	mg P/L	0.025	0.001 to 0.05	Michaelis-Menten half-saturation constant for P.
36	`lt_nonalg`	1/m	1.0	0 to 10	Non-algal part of the light extinction coefficient.
37	`alg_shd_l`	1/(m · µg chl-a/L)	0.03	0.006 to 0.065	Linear algal self-shading coefficient.
38	`alg_shd_nl`	(1/m)(µg chl-a/L)^-2/3	0.054	0 to 1	Non-linear algal self-shading coefficient.
39	`nh3_pref`	-	0.5	0 to 1	Algal preference for ammonium over nitrate.

Units of the half-saturation coefficient for light differ between sources: the old GitBook pages give `lt_co` in MJ/m²/h with a range of 0.223 to 1.135, while the 62.0.1 reader treats the value as kJ/m²/min and multiplies it by 0.06. A file value of 0.75 is used as 0.045 MJ/m²/h.

In sub-daily runs the reader divides the rates in columns 4 to 8, 11 to 14, 17 to 20, 30 and 31 by the number of time steps per day, while the water quality calculation still runs once per day. Sub-daily runs therefore use smaller effective rates than daily runs with the same file.

When `nutrients.cha` is missing, the chandeg channels have no water quality parameters, so keep the file in any model with channels.

Channel geometry and rating curve

At the start of the run each channel gets a trapezoidal cross-section and a rating curve at four depths: 0.1 and 1.0 times the bankfull depth `dp`, and 1.2 and 2.0 times `dp` with water on the floodplain. Flow, velocity, area, volume and travel time at any other flow are interpolated between these points.

With top width W (`wd`), bankfull depth D (`dp`), side slope z (`side_slp`), length L (`len`, km), slope S (`slp`) and roughness n (`mann`), the bottom width is

$$b = W - 2\,D\,z$$

If this is zero or negative, b is set to $W/2$ and z is reset to $W/(4D)$. For a flow depth d up to bankfull,

$$P = b + 2\,d\,\sqrt{1 + 1/z^2}, \qquad A = b\,d + d/z, \qquad R = A/P$$

$$v = \frac{R^{2/3}\,S^{1/2}}{n + 0.001}, \qquad Q = A\,v, \qquad TT = \frac{L}{3.6\,v}$$

where P is the wetted perimeter (m), A the flow area (m²), v the velocity (m/s), Q the flow (m³/s) and TT the travel time (h). The channel volume is $1000\,A\,L$ (m³).

The bottom width treats `side_slp` as horizontal distance per unit rise, while the perimeter and area treat it as rise per unit horizontal distance. The side term of the area, d/z, is the area of two side triangles with slope z (rise over run) only at $d = 1$ m. These are the expressions SWAT+ 62.0.1 uses.

Above bankfull, with h the depth above the bank (0.2 D and 1.0 D) and s_{fp} the floodplain slope `fps`,

$$P = P_{bf} + 2\,h\,\sqrt{1 + 1/s_{fp}^2}, \qquad A = A_{bf} + b\,h + h^2/s_{fp}, \qquad W_{fp} = W + 2\,h/s_{fp}$$

The floodplain volume is $1000\,W_{fp}\,h\,L$, and flow and velocity use `fpn` in place of `mann`, with the channel slope `slp`.

A smaller `fps` gives a wider, flatter floodplain that stores more water at a given depth.

Flow routing

Each day the inflow from upstream objects is first increased by any water allocation transfer into the channel, aquifer flow from `aqu_cha.lin`, gwflow exchange when gwflow is active, and, on days with inflow, precipitation on the channel surface ($P\,L\,W$ m³, with P in mm, L in km and W in m). Water is then routed with one of two methods, chosen by `rte_cha` in `codes.bsn`.

Variable storage method

With `rte_cha` = 0, the outflow in each time step is a fraction of the stored water:

$$O = c\,V_{stor}, \qquad c = \min\!\left(1,\ \frac{2\,s_c\,\Delta t}{2\,TT + \Delta t}\right)$$

where s_c is `scoef` in `parameters.bsn`, Δt the time step (h) and TT the travel time (h) at the previous step's outflow rate.

Muskingum method

With `rte_cha` = 1, outflow follows the Muskingum equation

$$O_2 = C_1 I_2 + C_2 I_1 + C_3 O_1$$

The storage constant is a weighted mean of the travel times at bankfull and 0.1 bankfull depth, each computed with the kinematic wave celerity $5v/3$:

$$K = \frac{\text{msk_co1}\,K_{bf} + \text{msk_co2}\,K_{0.1}}{\text{msk_co1} + \text{msk_co2}}$$

With X = `msk_x` (all three in `parameters.bsn`), the coefficients are

$$C_1 = \frac{\Delta t - 2KX}{2K(1-X) + \Delta t}, \quad C_2 = \frac{\Delta t + 2KX}{2K(1-X) + \Delta t}, \quad C_3 = \frac{2K(1-X) - \Delta t}{2K(1-X) + \Delta t}$$

C_1 is kept at zero or above, and the three are scaled to sum to 1. When $\Delta t > 2K(1-X)$ the coefficients are computed for a shorter sub-step that meets this limit. Sub-steps are used only in sub-daily runs; a daily run routes once per day with the sub-step coefficients.

Stored water above the bankfull channel volume is counted as floodplain storage.

Water losses

After routing, transmission losses and evaporation are taken from the water left in the channel:

$$TL = 24\,k\,L\,W \quad (\text{m}^3/\text{day}), \qquad E = \text{evrch}\ PET\ L\,W$$

where k is `k` (mm/h), L is in km, W in m, PET is the potential evapotranspiration of the weather station (mm), and `evrch` is the reach evaporation factor (`evap_adj` in `parameters.bsn`). Both use the bankfull top width over the full channel length, and neither can exceed the stored volume. With gwflow active, gwflow also calculates an exchange with the channel; the transmission loss above is still taken from the channel, but the channel output reports it as zero seepage.

Sediment

Sediment calculations run once per day for channels with inflow. In 62.0.1 four inputs are replaced at the start of each day's calculation: the bankfull flow multiplier by 1.5, the floodplain inundation days by 1.0, the bed `d50` by 1.0 mm and the bed erosion exponent by 1.0. The file values of `bed_load`, `d50`, `fp_inun_days` and `bed_exp`, and any calibration changes to them, therefore have no effect.

Peak flow

The day's peak flow is

$$q_{pk} = \text{pk_rto}\,(1 + 2.66\,A_{km^2}^{-0.3})\,\frac{Q_{day}}{86400}$$

(Fuller's equation), where A_{km^2} is the `area` of the channel in `chandeg.con` converted to km² and Q_{day} is the day's inflow (m³). The peak sets the velocity v from the rating curve, which is used for erosion and the travel time.

Floodplain deposition

When q_{pk} exceeds 1.5 times the bankfull flow, the floodplain traps a fraction of the incoming sediment (Dynamic SedNet, 2017):

$$TE = \min\!\left(1,\; N_{inun}\,\frac{q_{fp}}{\bar{q}}\,\left(1 - e^{-0.0001\,A_{fp}/q_{fp}}\right)\right)$$

with $N_{inun} = 1$ day, q_{fp} the rating curve flow at the peak, $\bar{q}$ the mean daily flow (m³/s) and $A_{fp} = 5\,W\,L$ the floodplain area (m², L in m). Organic N and sediment-bound P are deposited with TE times `n_dep_enr` and `p_dep_enr`. Overbank water then fills wetlands of HRUs linked to the channel's floodplain up to their emergency volume.

Bank erosion

Banks erode when the reach velocity exceeds a critical velocity. The cohesion from the clay content C (`clay`, %) is

$$c = \max(1000,\; -87.1 + 42.82\,C - 0.261\,C^2 + 0.029\,C^3)$$

and the critical velocity is

$$v_{cr} = \text{eq_slp}\;\log_{10}(2200\,D)\,\sqrt{0.0004\,(f_{bd} + 0.021\,c + 4500\,\text{cov}\,e^{-5D})}$$

where $f_{bd} = \max(0.001,\; 39.24\,\rho_b - 1000)$ with ρ_b the bulk density `dry_bd` (t/m³). For bulk densities below 25.5 t/m³, which covers all real sediments, f_{bd} stays at 0.001, so `dry_bd` does not change v_{cr}.

The velocity at the bends uses the radius of curvature from the sinuosity σ (column `wd_rto`):

$$r_c = \frac{12\,W\,\sigma^{1.5}}{13\,\sqrt{\sigma - 1}}, \qquad v_{rch} = 0.33\,v\,(1 + 1/r_c) + 0.66\,v$$

If $v_{rch} > v_{cr}$, the bank retreat for the day (m) is

$$e_{bank} = \left(\frac{1}{v_c} + \frac{1}{0.0002\,W}\right)^{-1}, \qquad v_c = \frac{0.0001\,A_{km^2}^{-0.0858}\,W}{1 + e^{-4\,(v_{rch}/v_{cr} - 1)}}$$

and the eroded mass is $1000\,e_{bank}\,D\,(0.25\,L)\,\rho_b$ tonnes, with L in km. The bank sediment brings organic N at `n_conc` and P at `p_conc`; the fraction `p_bio` of that P is soluble.

Channel deposition

A fraction `wash_bed_fr` of the bank erosion is deposited in the channel bed and removed from the flow, with organic N and sediment-bound P scaled by `n_dep_enr` and `p_dep_enr`.

Bed erosion

For channels steeper than 10^{-6}, bed erosion occurs when $v > 0.293\sqrt{d_{50}}$, which is 0.293 m/s with $d_{50} = 1$ mm:

$$e_{bed} = 0.0001 \left(\frac{v}{0.293}\right) \text{ m}, \qquad M_{bed} = 1000\, e_{bed}\, W\, L\, \rho_b \text{ t}$$

In 62.0.1, e_{bed} and M_{bed} appear in the channel morphology and sediment budget outputs, but the eroded bed sediment is not added to the flow.

Nutrients

For channels with inflow, the day's water quality follows the QUAL2E kinetics with the rates of `nutrients.cha`, applied over the travel time (at most one day). This covers algae and chlorophyll-a, CBOD, dissolved oxygen, the nitrogen chain from organic N through ammonium and nitrite to nitrate, and organic and soluble P. See Channels in the model theory.

When column 24 of `codes.bsn` (headed `i_fpwet` by the Editor) is 1, a simplified transformation then replaces the organic N, nitrate, sediment-bound P and soluble P of the outflow:

$$\Delta N_{org} = \min\left(NO_3,\ \left(1 - e^{-k_N\, TT}\right) N_{org}\right)$$

with $k_N =$ `n_sol_part` and TT the travel time (h). ΔN_{org} is added to organic N and taken from nitrate. Phosphorus follows the same form with `p_sol_part`, moving soluble P to sediment-bound P.

Pesticides

In the channel, pesticide sorption uses $K_d = K_{oc}\,\text{carbon}/100$, with K_{oc} from `pesticide.pes` and `carbon` from `hyd-sed-lte.cha`. The mixing velocity between water and bed uses the bed porosity $1 - \rho_b/2.65$. Pesticides and pathogens are routed only on days with inflow.

Calibration and decision tables

Parameters can be changed during calibration through `calibration.cal` (see parameter change files), using these names:

- `hyd-sed-lte.cha`: `chw` (`wd`), `chd` (`dp`), `chs` (`slp`), `chl` (`len`), `chn` (`mann`), `chk` (`k`), `bank_exp` (`erod_fact`), `cov` (`cov_fact`), `vcr_coef` (`eq_slp`), `d50`, `ch_clay` (`clay`), `carbon`, `ch_bd` (`dry_bd`), `chss` (`side_slp`), `bankfull_flo` (`bed_load`), `fps`, `fpn`, `ch_n_conc`, `ch_p_conc`, `ch_p_bio`.
- `sed_nut.cha`: `pk_rto`, `fp_inun_days`, `n_setl`, `p_setl`, `n_sol_part`, `p_sol_part`, `n_dep_enr`, `p_dep_enr`, `arc_len_fr`, `bed_exp`, `wash_bed_fr`.
- `nutrients.cha`: `rs1` to `rs7`, `rk1` to `rk4`, `rk6`, `bc1` to `bc4`, `mumax`. The parameter name `rk5` changes `ben_disp` (the same column as `rs2`); no calibration name changes `bact_die`.

Calibration changes are applied after the rating curve and Muskingum coefficients are set, and these are not recalculated. Changes to `chw`, `chd`, `chs`, `chn`, `chss`, `fps` and `fpn` therefore do not change the rating curve or the routing coefficients; they change only the other calculations that use the values directly (transmission losses, evaporation, erosion). Changes to `bankfull_flo`, `fp_inun_days`, `d50` and `bed_exp` are overwritten each day (see Sediment). The `nutrients.cha` changes are applied to the row whose position equals the channel number.

The decision table action `chan_change` sets a channel's cover factor to the action's first constant and its stream order to the second. The condition `ch_order` tests a channel's stream order. See decision tables.

Example

`refdata/Osu_1hru` in the SWAT+ repository, first rows of each file:

```
channel-lte.cha: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                        cha_ini           cha_hyd           cha_sed           cha_nut
       1  cha01                      initcha1          hydcha01              null           nutcha1
       2  cha03                      initcha1          hydcha03              null           nutcha1
```

```
hyd-sed-lte.cha: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name        order       wd       dp      slp      len     mann        k  erod_fact  cov_fact   wd_rto   eq
hydcha01        4 27.17726  0.99164  0.00146  3.42760  0.05000  5.00000    0.01000   0.00500 27.40638  0.0
```

(columns narrowed for display)

Channel `cha01` is a fourth-order channel 27.2 m wide, 0.99 m deep and 3.43 km long, with Manning's n of 0.05 and bed conductivity of 5 mm/h. Some values in this file are read differently from what the Editor header suggests:

- `wd_rto` = 27.4 is the width-to-depth ratio, but SWAT+ reads it as a sinuosity of 27.4. The bends then have a large radius of curvature, and the bend velocity is close to the mean velocity.
- `eq_slp` = 0.001 multiplies the critical velocity for bank erosion by 0.001, so bank erosion occurs on almost every day with flow.
- `d50` = 12 and `bed_load` = 0.5 are replaced by 1.0 mm and 1.5 during the run.
- `fps` = 0.00001 gives a very flat, wide floodplain.

`nutrients.cha` in the same dataset has one set, `nutcha1`, with `air_rt` = 50 and `cbn_bod_stl` = 0.36. The other values are the replacement values in the table above, or zero for the two unused concentrations.

Related files

- `chandeg.con`: lists the channels and points to `channel-lte.cha` rows in its `lcha` column.
- `initial.cha`: initial water, sediment and constituents in each channel, through `om_water.ini`, `pest_water.ini`, `salt_channel.ini` and `cs_channel.ini`.
- `temperature.cha`: stream temperature parameters.
- `codes.bsn` (`rte_cha`, column 24) and `parameters.bsn` (`msk_co1`, `msk_co2`, `msk_x`, `evap_adj`, `scoef`): routing switches and coefficients.
- `file.cio`: the `channel` line names the files.
- Outputs: channel, sediment and nutrient files (`channel_sd`, `channel_sdmorph`, `sd_chanbud`, `ch_sedbud`, `chanbud_order`).
- Channels and erosion and sediment in the model theory.

Sources

- File name slots: `type input_cha` in `input_file_module.f90`, read in `readcio_read.f90`; read order in `proc_cha.f90`
- Readers: `sd_channel_read.f90` (`channel-lte.cha`), `sd_hydsed_read.f90` (`hyd-sed-lte.cha`, `sed_nut.cha`), `ch_read_nut.f90` (`nutrients.cha`)
- Type definitions: `swatdeg_datafiles`, `swatdeg_hydsed_data`, `swatdeg_sednut_data` and `swatdeg_channel_dynamic` in `sd_channel_module.f90`; `channel_nut_data` in `channel_data_module.f90`
- Initialisation, limits, rating curve and Muskingum coefficients: `sd_hydsed_init.f90`, `sd_rating_curve.f90`, `qman.f90`
- Daily channel calculation: `sd_channel_control3.f90`, called from `command.f90`; sediment in `sd_channel_sediment3.f90`; routing and losses in `ch_rtmusk.f90`; water quality in `ch_watqual4.f90`; pesticides in `ch_rtpest.f90`
- `chandeg.con` pointer column: `hyd_read_connect.f90`
- Calibration: `cal_parm_select.f90`; decision tables: `actions.f90`, `conditions.f90`

`channel-lte.cha`:

Column	Fortran variable
`id`	index of `sd_dat`
`name`	`sd_dat%name`
`cha_ini`	`sd_dat%initc` -> `sd_dat%init`
`cha_hyd`	`sd_dat%hydc` -> `sd_dat%hyd`, `sd_dat%sednut`

Column	Fortran variable
cha_sed	sd_dat%sedc (unused)
cha_nut	sd_dat%nutc -> sd_dat%nut

hyd-sed-lte.cha (copied to sd_ch in sd_hydsed_init):

Column	Fortran variable
order	sd_chd%order
wd	sd_chd%chw
dp	sd_chd%chd
slp	sd_chd%chs
len	sd_chd%chl
mann	sd_chd%chn
k	sd_chd%chk
erod_fact	sd_chd%bank_exp
cov_fact	sd_chd%cov
wd_rto	sd_chd%sinu
eq_slp	sd_chd%vcr_coef
d50	sd_chd%d50
clay	sd_chd%ch_clay
carbon	sd_chd%carbon
dry_bd	sd_chd%ch_bd
side_slp	sd_chd%chss
bed_load	sd_chd%bankfull_flo
fps, fpn, n_conc, p_conc, p_bio	sd_chd% same names

sed_nut.cha columns map to sd_chd1% components of the same names.

nutrients.cha:

Columns	Fortran variables (ch_nut%)
plt_n, ptl_p	onco, opco
alg_stl to ben_cst	rs1 to rs7
cbn_bod_co, air_rt, cbn_bod_stl, ben_bod, bact_die, cst_decay	rk1 to rk6
nh3n_no2n to ptlp_solp	bc1 to bc4
q2e_lt, q2e_alg	lao, igropt
chla_alg to o2_no2n	ai0 to ai6
alg_grow, alg_resp, slr_act	mumax, rhoq, tfact
lt_co, const_n, const_p	k_l, k_n, k_p
lt_nonalg, alg_shd_l, alg_shd_nl, nh3_pref	lambda0, lambda1, lambda2, p_n

Purpose

aquifer.aqu contains the properties of the one-dimensional aquifers: initial water table, specific yield, baseflow recession, deep seepage, revap and groundwater nitrate and phosphorus. Each aquifer object in aquifer.con selects a row by number in its aqu column. The aquifer line of file.cio names two files, initial.aqu and aquifer.aqu (in that order); either can be null.

A model has one aquifer object per row of aquifer.con. Usually there is a shallow aquifer for each land area and one or more deep aquifers that receive the seepage of the shallow ones. Models that simulate groundwater with the gwflow grid model use gwflow.con instead of these aquifers.

initial.aqu lists named sets of initial-condition files. In 62.0.1 its rows are read but not used; initial pesticides, pathogens, salts and constituents of aquifers come from initial.aqu_cs, described below.

aquifer.aqu

Free format, whitespace-separated. Line 1 is a title and line 2 the column header; both are skipped. Each later line is one aquifer property set with 18 values in this order. The reader does not match columns by header name.

#	Column	Units	Built-in value	Typical range	Description
1	`id`	-	-	-	Number of the property set. The `aqu` column of `aquifer.con` refers to the row by this number.
2	`name`	-	-	-	Name of the aquifer. Not used by the model.
3	`init`	-	-	-	Name of an initial-condition set, matched against `initial.aqu_cs`. See initial.aqu_cs.
4	`gw_flo`	mm	0.05	0 to 2	Return flow on the day before the simulation starts. It starts the recession, and sets the water volume for the initial nitrate mass.
5	`dep_bot`	m	0	0 to 10	Depth from the mid-slope surface to the bottom of the aquifer. Must be larger than `dep_wt`.
6	`dep_wt`	m	0	0 to 10	Initial depth from the mid-slope surface to the water table.
7	`no3_n`	mg N/L	0	0 to 1000	Initial nitrate-N concentration.
8	`sol_p`	mg P/L	0	0 to 1000	Mineral P concentration of the water entering the stream, constant through the run.
9	`carbon`	%	0.5	0 to 15	Organic carbon content. Reported in the aquifer output; not used in any calculation in 62.0.1.
10	`flo_dist`	m	50	0 to 1000	Average flow distance to the stream. Not used.
11	`bf_max`	mm	0	0 to 2	Return flow at which all channels linked to the aquifer receive groundwater. Used only when aquifer flow is distributed to channels with `aqu_cha.lin`.
12	`alpha_bf`	1/day	0	0 to 1	Baseflow recession constant.
13	`revap`	-	0	0 to 1	Revap coefficient: revap as a fraction of potential evapotranspiration.
14	`rchg_dp`	fraction	0	0 to 1	Fraction of the day's recharge that leaves the aquifer as deep seepage.
15	`spec_yld`	m^3/m^3	0	0 to 0.4	Specific yield: the volume of water drained by gravity per volume of aquifer. Must be above 0.
16	`hl_no3n`	days	30	0 to 200	Half-life of nitrate in the aquifer.
17	`flo_min`	m	0	0 to 10	Water table depth for return flow: the aquifer sends return flow only while the water table is at this depth below the surface or shallower.

#	Column	Units	Built-in value	Typical range	Description
18	`revap_min`	m	0	0 to 10	Water table depth for revap: revap occurs only while the water table is shallower than this depth.

The built-in values apply when aquifer.con points to a number between 1 and the largest id that has no row in the file. They are not filled in for missing columns.

Give every row an id, and keep the ids unique. The ids do not have to be consecutive.

Aquifer water balance

Each day, for each aquifer, in this order. Depths of water are in mm over the aquifer area (the area column of aquifer.con).

Storage and water table

At the start of the run the stored water is

$$S_0 = 1000\,(\text{dep_bot} - \text{dep_wt})\ \text{spec_yld}$$

Each day the recharge R, all water routed into the aquifer from HRUs, routing units and other aquifers, is added to storage, and withdrawals for water allocation are then taken. Irrigation drawn from the aquifer by decision tables is removed from storage when it is applied. The water table depth follows from the storage:

$$h = \max\left(0,\ \text{dep_bot} - \frac{S}{1000\ \text{spec_yld}}\right)$$

The aquifer calculation in 62.0.1 has no recharge delay: recharge is added on the day it arrives.

Return flow

When $h \le$ flo_min, return flow follows the baseflow recession

$$Q_d = Q_{d-1}\,e^{-\alpha} + R_d\,(1 - e^{-\alpha})$$

limited to between zero and the storage, where α is alpha_bf. When $h >$ flo_min, $Q_d = 0$, and the next day's recession starts from zero.

The recession constant is a direct index of how fast groundwater flow responds to recharge (Smedema and Rycroft, 1983). Values are about 0.1 to 0.3 for land with a slow response and 0.9 to 1.0 for land with a rapid response. It is best estimated from measured streamflow during periods without recharge. When the number of baseflow days BFD (the days for baseflow to decline through one log cycle) is known,

$$\alpha = \frac{1}{BFD}\ln 10 = \frac{2.3}{BFD}$$

Return flow leaves through the outflows of type tot in aquifer.con.

Deep seepage

$$\text{seep}_d = \text{rchg_dp}\ R_d$$

limited to the storage. Seepage leaves through the outflows of type rhg in aquifer.con, usually to a deep aquifer. Without such an outflow it leaves the model.

Revap

When $h <$ `revap_min`,

$$\text{revap}_d = \text{revap}\ PET_d$$

limited to the storage, where PET_d is the potential evapotranspiration at the aquifer's weather station (mm). Revap represents water that moves from the aquifer into the unsaturated zone above it when that zone is dry, and uptake by deep-rooted plants. It is removed from the aquifer and not added to the soil. As `revap` approaches 0, the transfer is restricted; as it approaches 1, revap approaches the potential evapotranspiration.

Nitrate and phosphorus

Nitrate recharge is added to the stored nitrate N (kg/ha). The concentration is $c = N/S$, and the nitrate leaving with return flow and seepage is $c\,Q$ and $c\,\text{seep}$. Each day the remaining nitrate decays by

$$N \leftarrow N\ \exp\left(\frac{-0.693}{\text{hl_no3n} + 0.1}\right)$$

The decay represents uptake by bacteria and chemical conversion where the aquifer is depleted of oxygen; it is the net loss from all processes.

The initial nitrate mass is $10\,\text{gw_flo}\,A\,\text{no3_n}/1000$, the concentration times the volume of one day's initial return flow (A in ha). This is a mass in kg for the whole aquifer, but the daily calculation treats the store as kg/ha, so the initial store scales with the aquifer area. It does not use the stored water S_0.

Soluble P in the return flow is sol_p times the day's recharge volume (m^3), divided by 1000, in kg. The load therefore follows the recharge. It appears on days with recharge even when there is no return flow, and is zero on days without recharge. The aquifer stores no phosphorus between days.

Pesticides, salts and constituents

Pesticides entering with recharge are added to the aquifer store and decay with the aquifer half-life of `pesticide.pes` (adjusted to temperature when the temperature decay option is on). The concentration is the stored mass divided by $f\,S + Q + \text{revap} + \text{seep}$, where f is the pesticide groundwater mixing factor of `parameters.bsn`. Return flow and seepage take pesticide out at that concentration, all of it dissolved. Revap takes none.

Salts and other constituents (see `salt` and `cs`) are mixed in the stored water and leave with return flow and seepage at the stored concentration.

Aquifer flow to channels with aqu_cha.lin

When `aqu_cha.lin` links aquifers to channels (see `file.cio`), the return flow is spread over the linked channels as each channel is routed: on the same day when the aquifer comes before the channel in the routing order, otherwise on the next day. The length of channel that receives flow is the total linked channel length times $Q/\text{bf_max}$; when $Q \geq$ `bf_max` all linked channels receive flow, and when it is lower only part of the channel length does, each channel in proportion to its length. Set `bf_max` to a value above zero whenever `aqu_cha.lin` is used.

Calibration and decision tables

These parameters can be changed during calibration through `calibration.cal` (see parameter change files): `flo_init_mm` (`gw_flo`), `dep_bot`, `dep_wt_init` (`dep_wt`), `no3_init` (`no3_n`), `minp_init` (`sol_p`), `cbn_init` (`carbon`), `flo_dist`, `bf_max`, `alpha` (`alpha_bf`), `revap_co` (`revap`), `deep_seep` (`rchg_dp`), `sp_yld` (`spec_yld`), `hlife_n` (`hl_no3n`), `flo_min` and `revap_min`.

Calibration changes are applied after the aquifers are initialised. A change to `sp_yld` recalculates the storage from the current water table, and a change to `no3_init` recalculates the nitrate mass. Changes to `flo_init_mm` and `dep_wt_init` come too late to change the initial state and have no effect. A change to `dep_bot` does not change the stored water; it moves the water table that is calculated from it.

The decision table condition `aqu_dep` tests the water table depth of an aquifer (m). See decision tables.

initial.aqu

Named sets of initial-condition files, in the same layout as `initial.cha`. Two header lines are skipped, then one set per line. The Editor adds a trailing `description` value, which the reader ignores.

#	Column	Description
1	`name`	Name of the set.
2	`org_min`	Name of an organic-mineral initial record.
3	`pest`	Name of a pesticide initial record.
4	`path`	Name of a pathogen initial record.
5	`hmet`	Name of a heavy metal initial record.
6	`salt`	Name of a salt initial record.

In 62.0.1 the rows are read and nothing is done with them: the `init` column of `aquifer.aqu` is not matched against this file, and the aquifer's initial water and nitrate come from `aquifer.aqu`.

initial.aqu_cs

An optional file with a fixed name, read when present. Two header lines are skipped, then one row per line with six names: `name`, `pest`, `path`, `hmet`, `salt` and `cs`.

- `pest` names a row in `pest_water.ini`. The initial pesticide in the aquifer (kg/ha) is the water concentration (mg/kg) times an assumed bulk density of 2.0 t/m³ times `dep_bot` times 10.
- `path` names a row in `path_water.ini`.
- `hmet` is read but not used.
- `salt` names a row in `salt_aqu.ini`: initial salt ion concentrations (g/m³) and salt mineral fractions.
- `cs` names a row in `cs_aqu.ini`: initial constituent concentrations (g/m³) and sorbed concentrations (mg/kg).

Pesticide and pathogen rows are selected by matching `name` with the `init` column of `aquifer.aqu`. Salt and constituent rows are selected by position instead: aquifer 3 (the third row of `aquifer.con`) uses row 3 of `initial.aqu_cs`, whatever its name. Salt and constituent masses are the concentrations times the initial stored water.

Example

`refdata/Osu_1hru` in the SWAT+ repository:

```
aquifer.aqu: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                          init        gw_flo       dep_bot        dep_wt         no3_n
       1  aqu011                    initaqu1       0.05000      10.00000       6.00000       0.00000
       2  aqu012                    initaqu1       0.05000      10.00000       6.00000       0.00000
      21  aqu_deep010               initaqu1       0.00000      10.00000      20.00000       0.00000
```

(three of the 21 rows)

The shallow aquifers start with the water table 6 m below a 10 m deep aquifer, so they contain $1000 \times (10-6) \times 0.05 = 200$ mm of water. Return flow and revap both need the water table within 5 m of the surface, so neither occurs until recharge raises it by 1 m (50 mm of storage). The recession constant of 0.95 1/day gives a fast response. One percent of recharge goes to deep seepage. `hl_no3n` is 0, which gives a daily nitrate decay factor of $e^{-6.93}$, so nearly all nitrate entering the aquifer is lost on the day it arrives.

The deep aquifer `aqu_deep010` has `dep_wt` = 20 m, below its `dep_bot` of 10 m, so its initial storage is negative ($1000 \times (10 - 20) \times 0.03 = -300$ mm) and the water table starts 20 m down. It is recharged by the seepage the shallow aquifers send to it. With `flo_min` = 0 it sends return flow only when the water table reaches the surface, and with `revap_min` = 0 it has no revap.

```
initial.aqu: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                      org_min             pest             path             hmet             salt
initaqu1                  no_init             null             null             null             null
```

Related files

- aquifer.con: the aquifer objects, their areas, the `aqu` pointer to `aquifer.aqu` and the `tot` and `rhg` outflows.
- file.cio: the aquifer line, and aqu_cha.lin on the link line.
- pest_water.ini, path_water.ini, salt_aqu.ini, cs_aqu.ini: initial constituents referenced from `initial.aqu_cs`.
- gwflow: the grid groundwater model used instead of these aquifers.
- Outputs: aquifer output files (`aquifer_day` and others).
- Aquifers in the model theory.

Sources

- File name slots: `type input_aqu` in input_file_module.f90, read in readcio_read.f90; read order in proc_aqu.f90
- Readers: aqu_read.f90 (`aquifer.aqu`), aqu_read_init.f90 (`initial.aqu`), aqu_read_init_cs.f90 (`initial.aqu_cs`)
- Type definitions: `aquifer_database`, `aquifer_data_parameters`, `aquifer_dynamic`, `aquifer_init_data_char` and `aquifer_init_data_char_cs` in aquifer_module.f90
- Initialisation: aqu_initial.f90
- Daily calculation: aqu_1d_control.f90, called from command.f90
- `aquifer.con` pointer and outflow types: hyd_read_connect.f90, hyd_connect.f90
- Calibration: cal_parm_select.f90; decision tables: conditions.f90, actions.f90

Column	Fortran variable
id	index of aqudb
name	aqudb%aqunm
init	aqudb%aqu_ini
gw_flo	aqudb%flo
dep_bot	aqudb%dep_bot
dep_wt	aqudb%dep_wt
no3_n	aqudb%no3
sol_p	aqudb%minp
carbon	aqudb%cbn
flo_dist	aqudb%flo_dist
bf_max	aqudb%bf_max
alpha_bf	aqudb%alpha
revap	aqudb%revap_co
rchg_dp	aqudb%seep
spec_yld	aqudb%spyld
hl_no3n	aqudb%hlife_n
flo_min	aqudb%flo_min
revap_min	aqudb%revap_min

`aqudb` is copied to `aqu_dat(iaq)` for each aquifer object in `aqu_initial`; `initial.aqu` rows go to `aqu_init_dat_c`, `initial.aqu_cs` rows to `aqu_init_dat_c_cs`.

Purpose

Reservoirs are impoundments on the channel network. They receive water, sediment and nutrients from upstream objects and release to downstream objects through a release rule. Ponds that are routed as separate objects use the same files. The reservoir group has six files, named on the `reservoir` line of `file.cio` in this order:

Slot	Default name	Content
1	`initial.res`	Named initial-condition sets for reservoirs and wetlands
2	`reservoir.res`	One row per reservoir, naming the records below
3	`hydrology.res`	Reservoir hydrology: spillway areas and volumes, seepage, evaporation, outflow smoothing
4	`sediment.res`	Sediment settling parameters, shared with wetlands
5	`nutrients.res`	Nutrient settling parameters, shared with wetlands
6	`weir.res`	Weir discharge parameters, used by `weir` release rules and by wetlands

Slots 7 and 8 of the same line name the wetland files `wetland.wet` and `hydrology.wet`. A slot set to `null`, or a file that does not exist, leaves that table empty.

`reservoir.res` refers to the other files by name. The release rule named in `reservoir.res` is a decision table in `res_rel.dtl` (see decision tables) or, for names starting with `ctbl_`, a conditions table in `res_conds.dat`.

The five databases are read in every run. `reservoir.res` is read, and the reservoirs are set up, only when `object.cnt` gives one or more reservoirs. `weir.res` is read before the decision tables, so that `weir` actions in `res_rel.dtl` can find their weir record.

All files are free format and whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. The readers do not match columns by header name, so every row needs all of its values in the order given below. Columns after the last one listed (for example the `description` column the SWAT+ Editor writes) are ignored. Names must match, including case.

reservoir.res

One row per reservoir.

#	Column	Units	Description
1	`id`	-	Reservoir number. The row with `id` n is used for reservoir n, the nth reservoir in `reservoir.con`. Numbers must run from 1 to the number of rows.
2	`name`	-	Reservoir name. Not used by the model.
3	`init`	-	Name of a row in `initial.res`.
4	`hyd`	-	Name of a row in `hydrology.res`.
5	`rel`	-	Release rule. A decision table name in `res_rel.dtl`, or a name starting with `ctbl_` for a conditions table in `res_conds.dat`.
6	`sed`	-	Name of a row in `sediment.res`.
7	`nut`	-	Name of a row in `nutrients.res`.

A name that is not found is reported in `diagnostics.out` as `<name> not found (res-hyd)`, `(res-release)`, `(res-sed)` or `(res-nut)`, and the reservoir keeps zero or built-in values for that part. An unmatched `rel` gives an empty release table, so the reservoir releases no water through its rule. An unmatched `init` is not reported; the reservoir then starts from the first row of `om_water.ini`.

hyd, **sed** and **nut** are taken from the row whose **id** equals the reservoir number. **init** and **rel** are taken from the row named by the reservoir's parameter pointer in column 8 of reservoir.con. The SWAT+ Editor writes the same number in both places, so the two agree in Editor-built projects.

initial.res

Named initial-condition sets, shared by reservoirs (**init** in **reservoir.res**) and wetlands (**init** in wetland.wet).

#	Column	Units	Description
1	name	-	Name of the initial-condition set.
2	org_min	-	Name of a row in om_water.ini: initial water volume and concentrations.
3	pest	-	Name of a row in pest_water.ini: initial pesticides in water and benthic sediment.
4	path	-	Name of a pathogen initialization row.
5	hmet	-	Heavy-metal initialization. Read; not used.
6	salt	-	Salt initialization. Read; not used. Reservoir and wetland salts come from the salt input files.

org_min, **pest** and **path** names that are not found fall back to the first row of the referenced file, without a message.

The initial water volume is the **flo** value of the om_water.ini row, a fraction of the principal spillway volume. The concentrations in that row are converted to masses with this volume. A reservoir that is not yet operational at the start of the run (see Operation start) starts empty instead.

hydrology.res

One row per hydrology record.

#	Column	Units	Default	Description
1	name	-	-	Name of the hydrology record, matched against **hyd** in **reservoir.res**.
2	yr_op	year	-	Calendar year in which the reservoir becomes operational. Whole number.
3	mon_op	month	-	Month of **yr_op** in which the reservoir becomes operational. Whole number.
4	area_ps	ha	0.08 × **vol_ps**	Surface area at the principal spillway.
5	vol_ps	ha-m	0.9 × **vol_es**, or 60000	Volume of water needed to fill the reservoir to the principal spillway.
6	area_es	ha	1.5 × **area_ps**	Surface area at the emergency spillway.
7	vol_es	ha-m	1.11 × **vol_ps**	Volume of water needed to fill the reservoir to the emergency spillway.
8	k	mm/hr	-	Hydraulic conductivity of the reservoir bottom. Use 0 for no seepage. Typical range 0 to 1.
9	evap_co	-	0.6	Evaporation coefficient, a multiplier on potential evapotranspiration. Typical range 0 to 1.
10	shp_co1	-	-	Smoothing factor for rising outflow. See Outflow smoothing.
11	shp_co2	-	-	Smoothing factor for falling outflow. See Outflow smoothing.

The Default column gives the value the reader puts in when the column is zero or negative. The rules are applied in this order: if **vol_ps** is not positive, it becomes 0.9 × **vol_es** when **vol_es** is positive and 60000 ha-m otherwise; then **vol_es**, **area_ps**, **area_es** and **evap_co** are filled in as listed. The **area_ps** default corresponds to a mean depth of 12.5 m at the principal spillway.

Principal spillway values describe the normal storage of the reservoir, and emergency spillway values its maximum storage. The volumes are converted from ha-m to m³ (× 10000) when the reservoirs are set up.

A reservoir whose `hyd` name is not found has zero spillway areas and volumes.

Operation start

A reservoir is operational from month `mon_op` of calendar year `yr_op`. Before then it passes its inflow straight through to its downstream objects and keeps no storage. `yr_op` is compared with the calendar year, so a value of 0, or any year before the start of the simulation, makes the reservoir operational from the first day. Only reservoirs that are operational on the first day get the initial storage from `initial.res`; one that starts later begins empty.

Surface area

Each day the surface area A (ha) is calculated from the storage V (m³):

$$A = \beta_1 \, V^{\beta_2}$$

The coefficients are set once at the start of the run from the spillway values, with volumes in m³. When `area_es` is larger than `area_ps` and `vol_es` is larger than `vol_ps`,

$$\beta_2 = \frac{\log_{10} \text{area_es} - \log_{10} \text{area_ps}}{L}, \qquad \beta_1 = \left(\frac{\text{area_es}}{\text{vol_es}} \right)^{\beta_2}$$

where L is the whole-number part of $\log_{10} \text{vol_es} - \log_{10} \text{vol_ps}$. When L is 0 the divisor 0.001 is used instead. β_2 is limited to 0.9, and when the limit applies $\beta_1 = (\text{area_ps}/\text{vol_ps})^{0.9}$. In all other cases $\beta_2 = 0.9$ and $\beta_1 = (\text{area_ps}/\text{vol_ps})^{0.9}$.

In 62.0.1 L is 0 unless the emergency volume is at least ten times the principal volume. For almost every reservoir, then, $\beta_2 = 0.9$ and the relation reduces to

$$A = \left(\text{area_ps} \, \frac{V}{\text{vol_ps}} \right)^{0.9}$$

At the principal spillway this curve gives area_ps$^{0.9}$: 63 ha for an `area_ps` of 100 ha. The surface area sets precipitation on the reservoir, evaporation and seepage, so `area_ps` and `area_es` act on the water balance only through this relation.

Daily water balance

On each day of operation the reservoir:

1. adds the inflow from upstream objects to storage;
2. calculates the release from its release rule, using the storage after inflow, and smooths it (decision tables only);
3. settles sediment (decision tables only);
4. adds precipitation $P = 10 \, R \, A$ (m³, with R the day's precipitation in mm);
5. subtracts the release, limited to the water in storage;
6. subtracts evaporation $E = 10$ evap_co PET A (m³, with PET the day's potential evapotranspiration in mm), limited to storage;
7. subtracts seepage $S = 240 \, k \, A$ (m³, k in mm/hr), limited to storage;
8. updates the surface area from the new storage and calculates nutrient settling and the nutrient load in the release.

Precipitation and potential evapotranspiration come from the weather station assigned to the reservoir in `reservoir.con`. When gwflow is active, seepage is exchanged with the groundwater model and `k` is not used (see gwflow).

Outflow smoothing

For release by decision table, the release Q is smoothed against the previous day's release Q_{d-1} to soften jumps when the rule switches between alternatives:

$$Q_d = \alpha\, Q + (1-\alpha)\, Q_{d-1}, \qquad \alpha = \begin{cases} e^{-\text{shp_co1}} & Q > Q_{d-1} \\ e^{-\text{shp_co2}} & \text{otherwise} \end{cases}$$

With both factors at 0 there is no smoothing. Larger values give more weight to the previous day. The columns keep the names `shp_co1` and `shp_co2` from earlier versions, where they were surface-area shape coefficients. In 62.0.1 the shape coefficients are always calculated by the model as described above and cannot be entered.

Release rules

The `rel` column of `reservoir.res` selects how the reservoir releases water.

Decision tables

A decision table in `res_rel.dtl` evaluates its conditions each day (storage relative to the principal or emergency volume, month, inflow and others; see decision tables) and applies the `release` actions of the alternatives that are met. The `option` field of each action gives the method. Releases from several actions that apply on the same day are added.

`option`	Release per day
`rate`	`const` m³/s.
`rate_pct`	`const` percent of the principal volume.
`inflo_rate`	The larger of inflow plus `const2` m³/s and `const` m³/s.
`inflo_frac`	Inflow multiplied by `const`.
`ab_emer`	All storage above the emergency volume.
`days`	(storage minus a base volume) / `const`, where `const` is the number of drawdown days. The base volume is 0 when `fp` is `null`, `const2` × principal volume when `fp` is `pvol`, and `const2` × emergency volume when `fp` is `evol`.
`dyrt`	(storage minus base volume) / `const` plus `const2` percent of the principal volume. The base volume is 0, the principal volume or the emergency volume for `fp` = `null`, `pvol` or `evol`.
`dyrt1`	As `dyrt`, with the base volume taken from condition 5, 4 or 3 of the table for `fp` = `con1`, `con2` or `con3`.
`inflo_targ`	Inflow plus storage above a target, where the target is the principal volume multiplied by the limit constant of the condition numbered `const`.
`weir`	Weir equation with the `weir.res` row named in `fp`. See weir.res.
`meas`	Measured outflow from the recall record named in `fp` (daily, monthly or annual).
`natlake`	Natural lake outflow after Döll et al. (2003): $Q = k_r\, S_e\, \left(S_e/(V_p - S_0)\right)^{1.5}$ with dead storage S_0 = `const` × principal volume, S_e the storage above S_0, V_p the principal volume and k_r = `const2` (1/day).
`nonirr-h06`, `irr-h06`	Hanazaki et al. (2006) operating rules for non-irrigation and irrigation reservoirs.
`hydrop`	Hydropower rule after Arheimer et al. (2019).

For `days`, the base volume is `const2` times the spillway volume, so an action with `fp` set to `pvol` and `const2` set to 0 draws the reservoir down toward empty. Set `const2` to 1 to draw down to the principal volume.

The `nonirr-h06`, `irr-h06` and `hydrop` options base the release on a five-year record of monthly mean inflow (and, for `irr-h06`, irrigation demand) that each reservoir keeps during the run. In 62.0.1 this record is never filled and stays at zero, so `nonirr-h06` and `hydrop` release no water and `irr-h06` gives an undefined release (zero divided by zero).

In sub-daily runs the actions are applied once for each time step of the day. `days`, `dyrt` (its percentage term) and `meas` divide their daily amount by the number of steps; the other options add their full daily amount at every step. The `weir` option replaces the release with one time step's weir flow at each step.

For a reservoir, the `weir` option takes the weir crest height from the wetland data of the HRU with the same number as the reservoir. When that HRU has no wetland the crest height is zero, so the whole water depth (storage divided by surface area) acts as head. The `height` column of `weir.res` and the `weir_height` decision table action do not change the crest height that the reservoir weir release uses.

Conditions tables

A `rel` name starting with `ctbl_` is looked up in `res_conds.dat`, a file with a fixed name that is read when it exists in the run directory. The release is the value given by the matching condition of the table. Outflow smoothing and sediment settling are not applied to these reservoirs, and their release contains no sediment.

sediment.res

One row per sediment record. Reservoirs refer to it through `sed` in `reservoir.res`, wetlands through `sed` in `wetland.wet`.

#	Column	Units	Description
1	`name`	-	Name of the sediment record.
2	`sed_amt`	mg/L	Equilibrium (normal) suspended sediment concentration. Settling occurs only above this concentration. Typical range 1 to 5000.
3	`d50`	µm	Median particle size of suspended and benthic sediment. Sets the settling rate.
4	`carbon`	%	Organic carbon in suspended and benthic sediment. Used in pesticide partitioning.
5	`bd`	t/m³	Bulk density of benthic sediment. Used in pesticide partitioning between water and benthic sediment.
6	`sed_stl`	-	Sediment settling rate. Read but not used.
7	`stl_vel`	m/day	Sediment settling velocity. Read but not used.

Sediment settling

Each day, when the sediment concentration C (mg/L) in the reservoir is above the equilibrium concentration C_{eq} (`sed_amt`), the excess settles at a rate set by the median particle size:

$$C' = (C - C_{eq})\, e^{-0.184\, d_{50}} + C_{eq}$$

With `d50` at 10 µm, 16% of the excess concentration remains after one day; at 2 µm, 69% remains. The release takes sediment at the concentration C', and all silt, clay, sand and aggregate fractions in the reservoir settle.

When the concentration is at or below `sed_amt`, nothing settles and the release takes no sediment that day.

Median particle sizes of sediment classes: clay 1.95 µm, fine silt 4.9 µm, medium silt 11.7 µm, coarse silt 47 µm, fine sand 94 µm, medium sand 375 µm, coarse sand 1500 µm. A reservoir receives sediment from its whole upstream area, so a representative value for that area suits it better than the texture of one HRU.

nutrients.res

One row per nutrient record. Reservoirs refer to it through `nut` in `reservoir.res`, wetlands through `nut` in `wetland.wet`.

#	Column	Units	Description
1	`name`	-	Name of the nutrient record.
2	`mid_start`	month	First month of the mid-year settling period. Whole number, 1 to 12.
3	`mid_end`	month	Last month of the mid-year settling period. Whole number, 1 to 12.

#	Column	Units	Description
4	mid_n_stl	m/year	Organic nitrogen settling rate in the mid-year period.
5	n_stl	m/year	Organic nitrogen settling rate in the rest of the year.
6	mid_p_stl	m/year	Particulate phosphorus settling rate in the mid-year period.
7	p_stl	m/year	Particulate phosphorus settling rate in the rest of the year.
8	chla_co	m/year	Loss rate for soluble nitrogen (nitrate, ammonium and nitrite).
9	secchi_co	m/year	Loss rate for soluble phosphorus.
10	theta_n	-	Temperature adjustment factor for nitrogen losses.
11	theta_p	-	Temperature adjustment factor for phosphorus losses.
12	n_min_stl	mg/L	Nitrogen concentration below which no nitrogen is lost.
13	p_min_stl	mg/L	Phosphorus concentration below which no phosphorus is lost.

The Editor headers `chla_co` and `secchi_co` come from older versions, where these columns were a chlorophyll-a coefficient and a water clarity coefficient. In 62.0.1 they are the soluble nitrogen and phosphorus loss rates. Chlorophyll-a and Secchi depth are not calculated from them.

The six rates are entered in m/year and divided by 365 when the file is read.

Nutrient losses

The mid-year rates apply in the months from `mid_start` to `mid_end`, and the other rates in the remaining months. The test is `mid_start` ≤ month ≤ `mid_end`, so a period that runs across the end of the year (for example November to March) never applies and the other rates are used all year. To use one rate all year, give both columns the same value.

Each day the fraction of organic nitrogen lost is

$$k_N = \frac{\text{mid_n_stl}}{365}\,\theta_N^{T-20}\,(C_N - C_{N,min})$$

limited to between 0 and 1, where C_N is the organic nitrogen concentration (mg/L), $C_{N,min}$ is `n_min_stl`, θ_N is `theta_n` and T is the day's mean air temperature (°C) at the weather station. Particulate phosphorus is treated the same way with `mid_p_stl` or `p_stl`, `theta_p` and `p_min_stl`. The organic nitrogen and particulate phosphorus masses are multiplied by $1 - k$.

Soluble nitrogen uses `chla_co` and the total soluble nitrogen concentration, and soluble phosphorus uses `secchi_co`. For these, the loss fraction is multiplied by a fixed 0.2 before it is applied to nitrate, ammonium, nitrite and soluble phosphorus.

The rate is multiplied by a concentration difference and the result is used as a fraction, so the units of the product do not match: a value of 1 m/year acts as a daily fraction of 1/365 per mg/L of excess concentration. The fraction cannot go below zero, so negative rates give no loss and the sediment never becomes a source of nutrients.

The release takes each nutrient in proportion to its share of the water: release / (storage + release).

Measured apparent settling velocities for phosphorus in natural lakes most often lie between 5 and 20 m/year, with reported values from below 1 to over 200 m/year (Chapra, 1997). Reservoirs show a wider range: Higgins and Kim (1981) reported −90 to 269 m/year for 18 reservoirs in Tennessee (median 42.2 m/year), and Walker and Kiihner (1978) reported −1 to 125 m/year for 27 Midwestern reservoirs (mean 12.7 m/year). Panuska and Robertson (1999) recommend these values:

Water body nutrient dynamics	Settling velocity (m/year)
Shallow, high net internal phosphorus flux	0 or less
Moderate net internal phosphorus flux	1 to 5
Minimal net internal phosphorus flux	5 to 16

Water body nutrient dynamics	Settling velocity (m/year)
High net internal phosphorus removal	more than 16

In 62.0.1 values of 0 or less give no phosphorus loss.

weir.res

One row per weir record. A reservoir uses a weir record through a `weir` action in its release decision table (the record named in the action's `fp` field). Wetlands use weir records as described under storage volumes and weir height on the wetland page.

#	Column	Units	Built-in value	Description
1	`name`	-	-	Name of the weir record.
2	`linear_c`	-	1.84	Weir discharge coefficient c.
3	`exp_k`	-	2.6	Weir discharge exponent k.
4	`width`	m	2.5	Weir width w.
5	`height`	m	0	Height of the weir crest above the bottom of the impoundment. Used for wetlands only.

The built-in values apply when a `weir` action names a record that is not in the file.

The weir discharge is

$$Q = 0.45\, A\, c\, w\, H^{k}$$

in m^3/s, where A is the surface area of the water body (ha) and H is the water depth above the weir crest (m). In daily runs the discharge is applied hour by hour for up to 24 hours, each hour limited to the water above the crest, and the depth is updated after each hour.

Calibration

These reservoir parameters can be changed through `calibration.cal`: `psa`, `pvol`, `esa` and `evol` (spillway areas in ha and volumes in m^3), `k_res` (`k`), `evrsv` (`evap_co`), `nsed` (`sed_amt`), `res_d50` (`d50`), `sed_stlr` (`sed_stl`), `velsetlr` (`stl_vel`), `nsetlr1`, `nsetlr2`, `psetlr1`, `psetlr2` (the four settling rates), `nsolr` (`chla_co`), `psolr` (`secchi_co`), `soln_stl_fr` and `solp_stl_fr` (the fixed 0.2 factors), the storage and constituent masses `vol`, `sed`, `orgn`, `orgp`, `no3`, `nh3`, `no2` and `solp`, and the release-table constants `drawdown_days` (`const`) and `withdraw_rate` (`const2`).

Calibration acts on the values after unit conversion, so settling rates are changed in m/day. Changing `psa`, `pvol`, `esa` or `evol` does not recalculate the surface area coefficients.

Example

`refdata/Osu_1hru` in the SWAT+ repository has no reservoirs. Its sediment, nutrient, weir and initial-condition files are used by its wetland, and show the formats.

`initial.res`:

```
initial.res: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                      org_min             pest             path             hmet             salt
initwet1                  no_init             null             null             null             null
```

`sediment.res`:

```
sediment.res: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                    sed_amt           d50        carbon            bd       sed_stl       stl_vel
sedwet1                   10.00           5.00       0.04            0.8          0.15        0.0002
```

Sediment settles above 10 mg/L. With `d50` at 5 µm, 40% of the excess remains after one day.

`nutrients.res`:

```
nutrients.res: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name               mid_start   mid_end     mid_n_stl         n_stl     mid_p_stl         p_stl       chla_c
nutwet1                    5        10       5.50000       5.50000      10.00000      10.00000       1.00000
```

The mid-year period is May to October, but the two rates for each nutrient are equal, so one rate applies all year. Both `theta` values are 1, so the losses do not depend on temperature.

`weir.res`:

```
weir.res: Reservoir weir inputs - asdf;lj
NAME   Linear_C    Exp_K     Width(m)   Depth(m)
weir1   1.83       1.50      5.00       0.0
```

The header line here differs from the Editor's. It is skipped, so only the order of the values matters.

A `hydrology.res` and `reservoir.res` pair for one reservoir, written for illustration:

```
hydrology.res: example
name        yr_op  mon_op   area_ps   vol_ps   area_es   vol_es      k   evap_co  shp_co1  shp_co2
hydres1         0       0     150.0   1500.0     200.0   2200.0   0.05      0.60     0.00     0.00
```

```
reservoir.res: example
id   name      init      hyd       rel             sed      nut
1    res001    initres1  hydres1   drawdown_days   sedres1  nutres1
```

The reservoir is operational from the start. Its principal volume is 1500 ha-m (15 million m³). Because `vol_es` is less than ten times `vol_ps`, the surface area follows $A = (150\,V/1.5 \times 10^7)^{0.9}$, which gives 91 ha at the principal spillway. Outflow is not smoothed. Seepage is $240 \times 0.05 \times A$ m³ per day.

The `drawdown_days` table in `refdata/Osu_1hru/res_rel.dtl`:

```
name                    conds      alts      acts
drawdown_days               2         2         2
var                       obj   obj_num           lim_var            lim_op     lim_const      alt1
vol                       res         0              pvol                 *       1.00000         >
vol                       res         0              evol                 *       1.00000         <
act_typ                   obj   obj_num              name            option         const        const2
release                   res         0         over_prin              days      15.00000       0.00000
release                   res         0    over_emergency              days       5.00000       0.00000
```

Between the principal and emergency volumes the reservoir releases storage / 15 per day; above the emergency volume, storage / 5 per day. Below the principal volume no alternative applies and nothing is released. Because `const2` is 0, the base volume is zero in both actions, and each releases a share of the whole storage.

Related files

- `reservoir.con`: places each reservoir in the routing network and assigns its weather station.
- `object.cnt`: number of reservoirs.
- `file.cio`: names the reservoir files.
- `res_rel.dtl`: release decision tables.
- `om_water.ini`: initial water and concentrations referenced from `initial.res`.
- `recall.rec`: measured outflows for `meas` releases.
- `wetland.wet` and `hydrology.wet`: wetlands, which share `initial.res`, `sediment.res`, `nutrients.res`, `weir.res` and `res_rel.dtl`.
- Reservoir output and reservoir and wetland theory.

Sources

- File names: `type input_res` in input_file_module.f90, read in readcio_read.f90
- Call order: main.f90.in, proc_res.f90
- Readers: res_read.f90, res_read_init.f90, res_read_hyd.f90, res_read_sed.f90, res_read_nut.f90, res_read_weir.f90, res_read_conds.f90, dtbl_res_read.f90
- Type definitions: reservoir_data_module.f90, reservoir_module.f90
- Setup: res_objects.f90, res_allo.f90, res_initial.f90, `res_convert_mass` in hydrograph_module.f90
- Daily routines: res_control.f90, res_hydro.f90, res_rel_conds.f90, res_sediment.f90, res_nutrient.f90, theta.f90, res_pest.f90
- Decision table conditions and actions: conditions.f90, actions.f90
- Calibration: cal_parm_select.f90

File	Column	Fortran variable
reservoir.res	`id`, `name`, `init`, `hyd`, `rel`, `sed`, `nut`	index into `res_dat_c`, then `res_dat_c%name`, `%init`, `%hyd`, `%release`, `%sed`, `%nut`
initial.res	`name`, `org_min`, `pest`, `path`, `hmet`, `salt`	`res_init_dat_c%init`, `%org_min`, `%pest`, `%path`, `%hmet`, `%salt`
hydrology.res	`yr_op`, `mon_op`	`res_hyddb%iyres`, `%mores`
hydrology.res	`area_ps`, `vol_ps`, `area_es`, `vol_es`	`res_hyddb%psa`, `%pvol`, `%esa`, `%evol`
hydrology.res	`k`, `evap_co`	`res_hyddb%k`, `%evrsv`
hydrology.res	`shp_co1`, `shp_co2`	`res_hyddb%br1`, `%br2`, copied to `res_ob%lag_up`, `%lag_down`
sediment.res	`sed_amt`, `d50`, `carbon`, `bd`, `sed_stl`, `stl_vel`	`res_sed%nsed`, `%d50`, `%carbon`, `%bd`, `%sed_stlr`, `%velsetlr`
nutrients.res	`mid_start`, `mid_end`	`res_nut%ires1`, `%ires2`
nutrients.res	`mid_n_stl`, `n_stl`, `mid_p_stl`, `p_stl`	`res_nut%nsetlr1`, `%nsetlr2`, `%psetlr1`, `%psetlr2`
nutrients.res	`chla_co`, `secchi_co`	`res_nut%nsolr`, `%psolr`
nutrients.res	`theta_n`, `theta_p`, `n_min_stl`, `p_min_stl`	`res_nut%theta_n`, `%theta_p`, `%conc_nmin`, `%conc_pmin`
weir.res	`linear_c`, `exp_k`, `width`, `height`	`res_weir%c`, `%k`, `%w`, `%h`

Purpose

A wetland in SWAT+ is an HRU that can store ponded water on its surface. The same mechanism simulates rice paddies. Two files are specific to wetlands, named in slots 7 and 8 of the `reservoir` line of file.cio:

- `wetland.wet`: one row per wetland, naming its initial conditions, hydrology, release rule, sediment and nutrient records.
- `hydrology.wet`: wetland hydrology records. Depths and areas are relative to the HRU.

Wetlands also use four reservoir files, described on the reservoir page: initial.res, sediment.res, nutrients.res and weir.res. Release rules are decision tables in res_rel.dtl (see decision tables).

An HRU becomes a wetland when the `surf_stor` column of hru-data.hru names a row of `wetland.wet`. With `null` there, the HRU has no ponded storage.

Both files are free format and whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. The readers do not match columns by header name, so every row needs all of its values in the order given below. Columns after the last one listed are ignored. Names must match, including case. A slot in file.cio set to `null`, or a file that does not exist, leaves the table empty.

wetland.wet

One row per wetland.

#	Column	Units	Description
1	`id`	-	Wetland number. Read; not used. Rows are used in the order they appear.
2	`name`	-	Wetland name, matched against `surf_stor` in `hru-data.hru`.
3	`init`	-	Name of a row in `initial.res`.
4	`hyd`	-	Name of a row in `hydrology.wet`. The name `paddy` switches on rice paddy behaviour; see Outflow.
5	`rel`	-	Name of a release decision table in `res_rel.dtl`. Not used for paddies.
6	`sed`	-	Name of a row in `sediment.res`.
7	`nut`	-	Name of a row in `nutrients.res`.

Several HRUs can use the same row. A name that is not found is reported in `diagnostics.out` as `<name> not found (wet-init)`, `(wet-hyd)`, `(wet-release)`, `(wet-sed)` or `(wet-nut)`, and a `surf_stor` name that is not in `wetland.wet` as `<name>not found (wetland.wet)`. A wetland whose `hyd` is not found uses the built-in hydrology values (zero depths and areas). A non-paddy wetland whose `rel` is not found releases no water through its rule.

Some files, such as the example below, have an eighth column naming a weir record. 62.0.1 does not read it; how a wetland gets its weir record is described under Storage volumes and weir height.

hydrology.wet

One row per wetland hydrology record.

#	Column	Units	Built-in value	Description
1	`name`	-	-	Name of the hydrology record, matched against `hyd` in `wetland.wet`.
2	`hru_ps`	fraction	0	Fraction of the HRU area covered at the principal spillway (when flow over the surface inlet riser starts). Used only in the surface area set up before the first day, which the daily calculation replaces.
3	`dp_ps`	mm	0	Average water depth at the principal spillway. Sets the principal volume and, without weir records, the weir height.
4	`hru_es`	fraction	0	Fraction of the HRU area covered at the emergency spillway (when water starts to spill into the ditch). Not used in the daily simulation.
5	`dp_es`	mm	0	Average water depth at the emergency spillway. Sets the emergency volume.
6	`k`	mm/hr	0.01	Hydraulic conductivity of the wetland bottom. Sets the seepage rate.
7	`evap`	-	0.7	Evaporation coefficient, a multiplier on potential evapotranspiration for ponded water.
8	`vol_area_co`	-	1	Coefficient a of the volume–area relation.
9	`vol_dp_a`	-	1	Coefficient b of the volume–depth relation.
10	`vol_dp_b`	-	1	Coefficient c of the volume–depth relation.

#	Column	Units	Built-in value	Description
11	`hru_frac`	fraction	0.5	Fraction of the HRU that drains into the wetland. Read but not used.

The built-in values apply to wetlands whose `hyd` name is not found. In rows of the file, the reader replaces zero or negative values in four columns:

- `evap` becomes 0.6.
- `k` becomes the saturated hydraulic conductivity of the top soil layer of the HRU whose number equals the row number of the record in `hydrology.wet` (HRU 1 for the first row, and so on), which is not in general an HRU that uses the record.
- `hru_ps` and `hru_es` are set to 0.08 × and 1.5 × values that are still zero when the file is read, so they stay at 0.

Wetland behaviour

Storage volumes and weir height

At the start of the run, and again when a decision table switches the wetland on, the storage limits are set from the HRU area A_{hru} (ha) on the assumption that the whole HRU is ponded:

$$V_p = 10\,A_{hru}\,\mathrm{dp_ps}, \qquad V_e = 10\,A_{hru}\,\mathrm{dp_es}$$

in m³. `hru_ps` and `hru_es` do not enter the volumes.

If `weir.res` has at least one record, every wetland starts with the first record of `weir.res` as its weir: the weir height is that record's `height` (m), the principal volume becomes $V_p = 10^4\,A_{hru}$ height, and V_e is raised to at least $1.2\,V_p$. Without weir records the weir height is `dp_ps` / 1000 m. A `weir` operation in the management schedule can later change the height and select another `weir.res` record by name (see Management and decision tables).

The initial water volume is the `flo` fraction of the `om_water.ini` row named in `initial.res`, multiplied by V_p, with concentrations from the same row.

Daily water balance

For an HRU with a wetland, the daily HRU routines change as follows:

- Surface run-on from upstream objects (with tile flow) is added to the wetland storage.
- The HRU generates no surface runoff. The day's precipitation, over the whole HRU area ($10\,R\,A_{hru}$ m³), and any irrigation water are added to the wetland storage.
- Evaporation from the ponded water is taken before soil evaporation; while water is ponded there is no soil evaporation (see Evaporation).
- Seepage from the ponded water is the infiltration into the soil.
- Water above saturation in the soil profile is returned to the wetland storage. If the weir height is below 1 mm it becomes surface runoff instead.
- The wetland outflow is the surface runoff of the HRU.

Water depth is storage divided by the HRU area: $d = V/(10\,A_{hru})$ mm.

Surface area

For wetlands that are not paddies, the water surface area A_w follows the storage V relative to the principal volume V_p:

$$x = b^2 + 4c\left(1 - \frac{V}{V_p}\right), \qquad h = \frac{-b - \sqrt{x}}{2c} + b, \qquad A_w = A_{hru}\,\min(1, \max(0.01,\ 1 + a\,h))$$

with a = `vol_area_co`, b = `vol_dp_a` and c = `vol_dp_b`. When x is below 10^{-6}, h is 0. With the built-in values ($a = b = c = 1$) the whole HRU is covered at $V = V_p$, and 63% at $V = 0.5\,V_p$. The area set up before the first day also multiplies the fraction by `hru_ps`.

For paddies the surface area is always the HRU area.

The surface area sets seepage and the weir flow. Precipitation, evaporation and the water depth are always calculated over the whole HRU.

Evaporation

When the HRU has ponded water and its leaf area index is 4 or less, the maximum evaporation is

$$E_{max} = \text{evap}\,\left(1 - \frac{LAI}{4}\right) PET$$

With a leaf area index above 4 there is no evaporation from the ponded water. Evaporation is taken from the ponded water first, and from the soil only when no ponded water remains.

In 62.0.1 the `evap` value is looked up through the record's row number in `hydrology.wet` used as an HRU number: the coefficient applied is that of the wetland on the HRU with that number, or 0.7 if that HRU has no wetland. In a model where HRU 1 uses the first `hydrology.wet` record, HRU 2 the second and so on, this gives the intended value.

Seepage

Daily seepage is

$$S = \min\left(V,\ 240\,k\,A_w\right)$$

in m³, with k in mm/hr. When the seepage is more than 0.1 mm over the HRU, it is reduced by the amount the soil profile cannot take before it becomes saturated. Nitrate, ammonium, soluble phosphorus, organic nitrogen and organic phosphorus move with the seepage into the top soil layer in proportion to the water. A puddling operation can replace `k` during the run. When gwflow is active, the exchange with groundwater is calculated by the groundwater model (see gwflow).

Outflow

A wetland whose `hyd` name in `wetland.wet` is `paddy` is a rice paddy. Its outflow is weir flow from the `weir.res` record selected for it, and `rel` is not used:

- If the storage exceeds V_e, the water above V_e is released.
- While the water depth is above the weir height, flow over the weir is $Q = 0.45\,A\,c\,w\,H^k$ (m³/s), with the weir.res coefficients, the HRU area A (ha) and the depth above the weir H (m). It is applied hour by hour for up to 24 hours in a daily run, each hour limited to the water above the weir.
- With the weir height at zero the whole water depth acts as head, so the ponded water drains over the weir.

Other wetlands release through the decision table named in `rel`. The conditions and release options are those of reservoirs (release rules), evaluated against the wetland's storage, principal and emergency volumes, water depth and weir height. The outflow is not smoothed.

Paddy HRUs also write a daily line to `paddy_test.csv` with precipitation, irrigation, seepage, evapotranspiration terms, weir height, water depth, weir outflow, soil water, sediment and nitrate.

Sediment and nutrients

Sediment settling and nutrient losses in the ponded water follow the reservoir equations with the wetland's `sediment.res` and `nutrients.res` records (see sediment settling and nutrient losses), using the HRU's weather. When the wetland was empty at the start of the day and releases water, the sediment yield of the outflow is calculated with a MUSLE-type equation from the HRU's cover and erosion factors, as for a flushed field.

Management and decision tables

These operations and decision-table entries act on wetlands and paddies:

- Decision table conditions `vol_wet` (storage against the principal or emergency volume), `wet_depth` (water depth, mm) and `weirh` (weir height, mm). See decision tables.
- Decision table action `weir_height` with option `wet`: sets the weir height to `const` mm, sets V_p to the HRU area times that height, and raises V_e to at least $1.2\,V_p$. It does not change the weir record.
- Management operation `weir`: sets the weir height to its third data value (mm), sets V_p the same way with V_e raised to at least $1.1\,V_p$, and selects the `weir.res` record named in its first data field.
- Decision table actions `impound_off` and `impound_on`: `impound_off` removes the wetland and its stored water from the HRU. `impound_on` sets the wetland up again from the `surf_stor` row of `hru-data.hru`; the name given in the action's file pointer is not matched in 62.0.1.
- Puddling (`puddle` action, with `puddle.ops`): replaces the wetland bottom conductivity and raises the sediment concentration of the ponded water.
- Fertilizer applied to an HRU with ponded water goes into the water when `carbon` in `codes.bsn` is 2. With other `carbon` values it is counted in the fertilizer output but added to neither the water nor the soil.
- Tillage when the water depth is above 1 mm mixes the soil layers with the ponded-water routine, including pesticides in the soil.

See management schedules and operations for the operation formats.

When gwflow is active and `gwflow.wetland` exists, the wetland bed thickness (m) for each wetland HRU is read from it: two header lines, then one line per wetland with a name and a thickness, where the digits in the name give the HRU number (for example `wet019` for HRU 19).

Example

`refdata/Osu_1hru` in the SWAT+ repository is a single rice paddy HRU. `hru-data.hru` names `paddy0001` in `surf_stor`.

`wetland.wet`:

```
wetland.       wet: written by SWAT+ edi#############for SWAT+ rev.60.5.4
     id                name        init         hyd         rel         sed         nut        WEIR
      1          paddy0001    initwet1       paddy        weir     sedwet1     nutwet1       weir1
```

The title line is damaged but is skipped. The hydrology record is `paddy`, so the HRU is a paddy and the `weir` decision table named in `rel` is not used. The `WEIR` column is not read.

`hydrology.wet`:

```
hydrology.wet: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                    hru_ps         dp_ps        hru_es         dp_es             k          evap    vol
paddy                     1.00        150.00          1.00        150.00           0.5          0.75
```

The emergency volume is 150 mm of water over the HRU. Seepage is at most $0.5 \times 24 = 12$ mm per day. Ponded water evaporates at $0.75 \times$ PET while the rice canopy is sparse, falling to zero as the leaf area index reaches 4.

`weir.res` has one record, `weir1`, with a height of 0 m, so the paddy starts with a weir height of zero and a principal volume of zero, and any ponded water drains over the weir. During the season `weir_height` actions in `lum.dtl` raise the weir to 60 mm and lower it to 0 again. While water stands above 60 mm it flows over the 5 m weir at $Q = 0.45\,A \times 1.83 \times 5 \times H^{1.5}$ m³/s.

Related files

- `hru-data.hru`: assigns a `wetland.wet` row to an HRU through `surf_stor`.
- `file.cio`: names the wetland files.
- `initial.res`, [`sediment.res`](#input-reference-res-sedimentres), [`nutrients.res`](#input-reference-res-nutrientsres) and `weir.res`: shared with reservoirs.

- res_rel.dtl and lum.dtl: wetland release rules and wetland actions.
- management.sch and operations: weir, irrigation, puddling, fertilizer and tillage operations on paddies.
- Salt and constituent inputs for wetlands.
- Water balance output and reservoir and wetland theory.

Sources

- File names: `type input_res` in input_file_module.f90; call order in main.f90.in
- Readers: wet_read.f90, wet_read_hyd.f90
- Type definitions: `type reservoir_data_char_input` and `type wetland_hyd_data` in reservoir_data_module.f90; `type wetland` in reservoir_module.f90
- Setup: hru_allo.f90, wet_all_initial.f90, wet_initial.f90
- Daily routines: hru_control.f90, et_act.f90, wetland_control.f90, res_weir_release.f90, res_hydro.f90, res_sediment.f90, res_nutrient.f90, swr_satexcess.f90
- Conditions, actions and operations: conditions.f90, actions.f90, mgt_sched.f90, pl_fert_wet.f90, mgt_newtillmix_wet.f90

File	Column	Fortran variable
wetland.wet	`id`	read into a dummy `k`
wetland.wet	`name`, `init`, `hyd`, `rel`, `sed`, `nut`	`wet_dat_c%name`, `%init`, `%hyd`, `%release`, `%sed`, `%nut`
hydrology.wet	`hru_ps`, `dp_ps`, `hru_es`, `dp_es`	`wet_hyddb%psa`, `%pdep`, `%esa`, `%edep`
hydrology.wet	`k`, `evap`	`wet_hyddb%k`, `%evrsv`
hydrology.wet	`vol_area_co`, `vol_dp_a`, `vol_dp_b`	`wet_hyddb%acoef`, `%bcoef`, `%ccoef`
hydrology.wet	`hru_frac`	`wet_hyddb%frac`

Purpose

The six operation databases contain named parameter sets for management operations. A scheduled operation in management.sch or an action in a lum.dtl decision table names an entry, and SWAT+ uses that entry's values when the operation runs.

File	Contents	Used by
`harv.ops`	harvest operations	`harv` and `hvkl` in management.sch; `harvest` and `harvest_kill` in lum.dtl
`graze.ops`	grazing operations	`graz` in management.sch; `graze` in lum.dtl
`irr.ops`	irrigation operations	`irrm`, `irrp` and `irpm` in management.sch; `irrigate` and `irr_demand` in lum.dtl
`chem_app.ops`	application methods for fertilizer, manure and pesticide	`fert`, `manu` and `pest` in management.sch; `fertilize`, `fert_future`, `manure`, `pest_apply` and `manure_demand` in lum.dtl
`fire.ops`	burning operations	`burn` in management.sch and lum.dtl
`sweep.ops`	street sweeping operations	`swep` in management.sch

The six file names are given on the `ops` line of file.cio, in the order harvest, graze, irrigation, chemical application, fire, sweep. The defaults are the names in the table. If a file is missing, or named `null` in file.cio, the database is empty, and any operation that names an entry from it uses an entry with all values zero (irrigation depth 25.4 mm).

Format

All six files share one layout. Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one operation. The columns are read by position, so every column in the tables below must be present and in this order. A trailing `description` column, which the SWAT+ Editor writes, is ignored.

Names are matched exactly against the names used in management.sch and lum.dtl. A name that is not found leaves the operation pointing at an empty entry with zero values, and no message is written.

harv.ops

#	Column	Units	Typical range	Description
1	`name`	-	-	Harvest operation name.
2	`harv_typ`	-	see below	Harvest type.
3	`harv_idx`	fraction	0 to 1	Harvest index override: the fraction of above-ground biomass cut. Used by biomass harvests and, as a multiplier, by `residue`. Ignored by grain and root harvests.
4	`harv_eff`	fraction	0 to 1	Harvest efficiency: the fraction of the harvested material removed from the HRU.
5	`harv_bm_min`	kg/ha	0 and up	Minimum plant biomass for the harvest to take place. For `residue` harvests, the minimum surface residue left behind.

The harvest types and what each one does:

`harv_typ`	Harvest only (`harv`, `harvest`)	Harvest and kill (`hvkl`, `harvest_kill`)
`grain`	grain harvest	grain harvest
`picker`	grain harvest	grain harvest
`stripper`	biomass harvest	grain harvest
`biomass`	biomass harvest	biomass harvest
`tree`	biomass harvest	no harvest; the plant is only killed
`tuber`	root harvest	root harvest
`peanuts`	root harvest	root harvest
`residue`	residue harvest	residue harvest

A harvest of a growing plant runs only if the plant's total biomass is above `harv_bm_min`. A `residue` harvest scheduled after the plant has died runs without this check.

Grain harvest. The yield is $Y = \text{harv_eff} \cdot M_{seed}$, where M_{seed} is the seed (yield) pool built up during growth from the harvest index in `plants.plt`. The seed mass that is not removed goes to the top soil layer as organic matter. `harv_idx` has no effect.

Biomass harvest. A fraction $h = \text{harv_idx} \cdot \text{harv_eff}$ of the seed, leaf and stem mass is cut. The cut takes place only if the above-ground biomass left afterwards is above `harv_bm_min`. The cut mass is reported as yield, and a further $(1 - \text{harv_eff})$ times the cut mass is added to the surface residue. Leaf area and accumulated heat units are multiplied by $1 - 0.5\,h$.

Root harvest (`tuber`, `peanuts`). The yield is `harv_eff` times the root mass. The remaining roots stay in the soil as residue.

Residue harvest. Each plant's surface residue is reduced by the fraction $e \cdot \text{harv_idx}$, but not below `harv_bm_min`. The efficiency e is the `op_data3` value of the `management.sch` row (or `const` of the decision table action) when it is greater than zero, and `harv_eff` otherwise.

For every type, the reported yield is reduced by the plant's pest stress.

graze.ops

#	Column	Units	Typical range	Description
1	`name`	-	-	Grazing operation name.

#	Column	Units	Typical range	Description
2	fert	-	-	Name of the manure in fertilizer.frt that the animals deposit. Called fertname in earlier documentation.
3	bm_eat	kg/ha/day	0 to 500	Dry biomass removed by grazing each day.
4	bm_tramp	kg/ha/day	0 to 500	Dry biomass removed by trampling each day. Read but not applied.
5	man_amt	kg/ha/day	0 to 500	Dry manure deposited each day.
6	grz_bm_min	kg/ha	0 to 5000	Minimum above-ground biomass for grazing to take place on a given day.

A graz row in management.sch starts a grazing period, and its op_data3 value is the length of the period in days. On each day of the period, if the HRU's above-ground biomass is at least grz_bm_min, the model removes bm_eat from the plants (spread over the plants in proportion to their above-ground biomass, and never more than is there) and adds man_amt of the named manure to the top soil layer. While a plant's biomass is below 2500 kg/ha, grazing also lowers its leaf area index by bm_eat/1000 and multiplies its accumulated heat units by 1 − bm_eat/1000. A graze action in a decision table applies one day of grazing each time it fires.

The parameter values in the SWAT+ Editor's default graze.ops are rough estimates. If grazing matters in a watershed, derive values from the stocking rate and type of animal. Trampling is often set equal to bm_eat (the animals trample as much as they eat), but in 62.0.1 bm_tramp has no effect.

irr.ops

#	Column	Units	Typical range	Description
1	name	-	-	Irrigation operation name.
2	amt_mm	mm	0 and up	Irrigation depth per application. Called irr_amt in earlier documentation.
3	eff_frac	fraction	0 to 1	In-field efficiency: the fraction of applied water available to the crop. Called irr_eff in earlier documentation.
4	sumq_frac	fraction	0 to 1	Fraction of the applied water that leaves as surface runoff. Called surq_rto in earlier documentation.
5	dep_sub	mm	0 and up	Application depth for subsurface irrigation. Read but not used. Called irr_dep in earlier documentation.
6	salt_ppm	ppm	0 and up	Salt concentration in the irrigation water. Used by the paddy irrigation operation irrp. Called irr_salt in earlier documentation.
7	no3_ppm	ppm	0 and up	Nitrate concentration in the irrigation water. Used by irrp. Called irr_no3n in earlier documentation.
8	po4_ppm	ppm	0 and up	Phosphate concentration in the irrigation water. Read but not used. Called irr_po4 in earlier documentation.

A scheduled irrigation (irrm or irpm in management.sch) applies

$$I_{applied} = \text{amt_mm} \cdot \text{eff_frac} \cdot (1 - \text{sumq_frac}), \qquad I_{runoff} = \text{amt_mm} \cdot \text{sumq_frac}$$

The `irrigate` action in `lum.dtl` uses the action's `const` (mm) in place of `amt_mm` and takes `eff_frac` and `sumq_frac` from the entry named in its `option` column. The `irr_demand` action with file pointer `unlim` uses `amt_mm`, but computes runoff as amt_mm · eff_frac · sumq_frac. See `*.dtl` for both actions.

chem_app.ops

#	Column	Units	Typical range	Description
1	`name`	-	-	Application method name, for example `broadcast`, `band`, `foliar`, `inject`, `aerial_liquid`, `aerial_solid`, `drill`, `side_dress`, `fertigate`, `basal`, `rope_wick`, `tree_inject`.
2	`chem_form`	-	`solid`, `liquid`	Form of the chemical. Read but not used.
3	`app_typ`	-	`spread`, `spray`, `inject`, `direct`	Application type. Read but not used.
4	`app_eff`	fraction	0 to 1	Application efficiency. Multiplies the amount of a `pest` operation or `pest_apply` action. Not applied to fertilizer or manure.
5	`foliar_eff`	fraction	0 to 1	Foliar efficiency. Read but not used.
6	`inject_dp`	mm	0 and up	Injection depth. Read but not used.
7	`surf_frac`	fraction	0 to 1	Fraction of the applied fertilizer, manure or pesticide placed in the top 10 mm soil layer. The rest goes to the second soil layer.
8	`drift_pot`	fraction	0 to 1	Drift potential. Read but not used.
9	`aerial_unif`	fraction	0 to 1	Aerial uniformity. Read but not used.

A decision table `fertilize` action on an HRU with standing water applies the fertilizer to the water instead of the soil when the method's `surf_frac` is 1.

fire.ops

#	Column	Units	Typical range	Description
1	`name`	-	-	Fire operation name, for example `grass`, `tree_intense`, `tree_low`.
2	`chg_cn2`	-	0 to 20	Amount added to the HRU's curve number for moisture condition II. The result is capped at 98.
3	`frac_burn`	fraction	0 to 1	Fraction of above-ground plant biomass, surface residue and top-layer soil organic matter burned.

A burn removes `frac_burn` of the above-ground biomass of every plant, of the surface residue, and of the slow and passive humus in the top soil layer. The phosphorus in the burned material is returned to the passive humus pool, and the burned plant carbon is reported as emitted carbon.

sweep.ops

#	Column	Units	Typical range	Description
1	name	-	-	Street sweeping operation name.
2	swp_eff	fraction	0 to 1	Removal efficiency: the fraction of built-up solids removed from the swept curb length.
3	frac_curb	fraction	0 to 1	Fraction of the curb length that can be swept. Parked cars and other obstacles often make part of the curb unavailable.

Street sweeping is meant for urban HRUs that use the build-up and wash-off method. The removal efficiency depends on the type of sweeper, whether flushing is part of the cleaning, the amount of solids and the frequency of rain, and is usually higher for particulate constituents than for nitrogen and phosphorus (Pitt, 1979). SWAT+ assumes fixed nutrient concentrations in the solids, so one efficiency applies to every constituent.

In 62.0.1 a `swep` operation reads the entry and logs it, but the sweeping routine uses a curb fraction of zero and is never called during the run, so `sweep.ops` has no effect on results.

Example

`refdata/Osu_1hru` in the SWAT+ repository, first rows of each file:

```
harv.ops: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                      harv_typ      harv_idx      harv_eff   harv_bm_min  description
forest_cut                    tree       0.95000       0.99000       0.00000
grain                        grain       0.00000       0.95000       0.00000
grass_bag                  biomass       0.50000       1.00000    2000.00000
grass_mulch                biomass       0.50000       0.00000    2000.00000
```

`grass_bag` and `grass_mulch` both use a cut fraction of 0.5. With `harv_eff` at 1, `grass_bag` removes everything it cuts; with `harv_eff` at 0, `grass_mulch` cuts nothing ($h = 0.5 \times 0$). Both run only on plants above 2000 kg/ha. `grain` removes 95% of the seed pool.

```
graze.ops: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                           fert        bm_eat      bm_tramp       man_amt    grz_bm_min  description
dairy_high                 dairy_fr      25.00000      10.00000       0.95000    3000.00000  High_Prod_Dair
```

```
irr.ops: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                       amt_mm      eff_frac     sumq_frac       dep_sub      salt_ppm       no3_ppm
sprinkler_med          25.00000       0.85000       0.00000       0.00000     100.00000      20.00000
drip                   25.00000       0.90000       0.00000     100.00000     100.00000      20.00000
```

A scheduled `sprinkler_med` irrigation puts $25 \times 0.85 = 21.25$ mm of water into the soil.

```
chem_app.ops: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                       chem_form           app_typ       app_eff    foliar_eff     inject_dp     surf_fr
broadcast                      solid            spread       0.90000       0.00000       0.00000       1.000
inject                        liquid            inject       0.95000       0.00000     150.00000       0.200
```

```
fire.ops: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                     chg_cn2     frac_burn  description
grass                    8.00000       1.00000
tree_low                 6.00000       0.70000
```

Related files

- `management.sch`: scheduled operations that name entries in these files.
- `*.dtl`: decision table actions that name entries in these files.
- `fertilizer.frt`: the manure named in `graze.ops`, and the fertilizers applied with a `chem_app.ops` method.

- plants.plt: the plant harvest index that builds the seed pool used by grain harvests.
- file.cio: names the six files on its ops line.

Sources

- Readers: mgt_read_harvops.f90, mgt_read_grazeops.f90, mgt_read_irrops.f90, mgt_read_chemapp.f90, mgt_read_fireops.f90, mgt_read_sweepops.f90, called from proc_db.f90
- Type definitions: mgt_operations_module.f90
- File name slots: type input_ops in input_file_module.f90
- Scheduled operations: mgt_sched.f90; decision table actions: actions.f90
- Harvest: mgt_harvgrain.f90, mgt_harvbiomass.f90, mgt_harvtuber.f90, mgt_harvresidue.f90
- Grazing: pl_graze.f90, daily loop in hru_control.f90
- Burning: pl_burnop.f90
- Street sweeping: hru_sweep.f90, hru_urbanhr.f90
- Application methods: pl_fert.f90, pest_apply.f90

File	Column	Fortran variable
harv.ops	harv_typ	harvop_db%typ
harv.ops	harv_idx	harvop_db%hi_ovr
harv.ops	harv_eff	harvop_db%eff
harv.ops	harv_bm_min	harvop_db%bm_min
graze.ops	fert	grazeop_db%fertnm (index stored in grazeop_db%manure_id)
graze.ops	bm_eat	grazeop_db%eat
graze.ops	bm_tramp	grazeop_db%tramp
graze.ops	man_amt	grazeop_db%manure
graze.ops	grz_bm_min	grazeop_db%biomin
irr.ops	eff_frac	irrop_db%eff
irr.ops	sumq_frac	irrop_db%surq
irr.ops	dep_sub	irrop_db%dep_mm
irr.ops	salt_ppm, no3_ppm, po4_ppm	irrop_db%salt, %no3, %po4
chem_app.ops	chem_form	chemapp_db%form
chem_app.ops	app_typ	chemapp_db%op_typ
chem_app.ops	inject_dp	chemapp_db%inject_dep
fire.ops	chg_cn2	fire_db%cn2_upd
fire.ops	frac_burn	fire_db%fr_burn
sweep.ops	swp_eff	sweepop_db%eff
sweep.ops	frac_curb	sweepop_db%fr_curb

Purpose

A decision table is a set of rules. Each day SWAT+ checks the table's conditions for an object (an HRU, a reservoir, a water allocation object), finds which of the table's alternatives are satisfied, and performs the actions that those alternatives switch on. Decision tables drive automatic management (planting, harvest, fertilizer, irrigation), reservoir and paddy releases, land use change scenarios, and water allocation demands. See decision tables for the concept.

There are four decision table files. Their names are given on the decision_table line of file.cio, in this order:

File	Default name	Used by
land use and management	lum.dtl	HRUs, through the automatic operations listed in management.sch; HRU-lte growing seasons; manure demands in manure_allo.mnu; irrigation demands, outside-source concentrations and treatment in water allocation
reservoir release	res_rel.dtl	reservoirs and HRU wetlands or paddies that name a release table (see *.res and *.wet)

File	Default name	Used by
land use scenarios	`scen_lu.dtl`	tables listed in `scen_dtl.upd`, checked for every HRU every day
flow control	`flo_con.dtl`	saturated buffers, and water allocation demand and source or receiver fractions

A file that is missing, or named `null` in `file.cio`, gives an empty set of tables.

Format

All four files have the same layout. Free format, whitespace-separated.

- Line 1: title (skipped).
- Line 2: the number of decision tables in the file.
- Line 3: skipped (normally blank).
- Then each table in turn:
  - a header line (skipped);
  - the table line: `name`, `conds`, `alts`, `acts` (table name, number of conditions, number of alternatives, number of actions);
  - a header line (skipped);
  - `conds` condition rows;
  - a header line (skipped);
  - `acts` action rows;
  - in `res_rel.dtl`, `scen_lu.dtl` and `flo_con.dtl` only: one more line, which is skipped. Leave a blank line here. Without it the next table's header is skipped in its place and the following lines are read out of step.

The counts on the table line must match the rows that follow. Extra or missing rows shift every later table. Names are matched exactly, and a name that is not found is not reported.

Condition rows

#	Column	Description
1	`var`	Condition variable (see Condition variables).
2	`obj`	Object type the variable belongs to, for example `hru`, `res`, `null`. For `prob_unif`, this column contains the application probability instead (see below).
3	`obj_num`	Object number. 0 means the object the table is being checked for.
4	`lim_var`	Limit variable. A name (land use, plant, soil group) or a reference volume, depending on the condition. `null` when not used.
5	`lim_op`	Limit operator: `*`, `+`, `-` or `/`. Combines `lim_var` with `lim_const` for `soil_water`, `vol` and `vol_wet`. `-` when not used.
6	`lim_const`	Limit constant.
7 and on	`alt1`, `alt2`, …	One alternative operator per alternative.

For most conditions the current value of the variable is compared with `lim_const`. For `soil_water`, `vol` and `vol_wet` it is compared with a target built from `lim_var`, `lim_op` and `lim_const`, for example "emergency volume × 0.8".

Alternatives

Each alternative is a column of operators, one per condition. An alternative is satisfied when every one of its conditions is true. For numeric conditions the operators are:

Operator	Condition is true when
`<`	value < limit
`>`	value > limit
`<=`	value limit
`>=`	value limit
`=`	value = limit
`/=`	value limit
`-`	always (the condition does not apply to this alternative)

Any other text in a numeric condition's alternative column is treated like `-`. In particular `/` does not mean "not equal" for numeric conditions.

The yes/no conditions (`plant_gro`, `plant_name_gro`, `hyd_soil_group`, `land_use`, `cal_group`, `tillage`, `prob_unif`, `prob_unif1`, `prob_unif_lu`, `ch_order`) use only `=`, meaning the condition must be true. `tile_drained` and `plant_com` also accept `/`, meaning the condition must be false. Every other operator is treated like `-`.

Action rows

#	Column	Description
1	`act_typ`	Action type (see Land use and management actions and Reservoir release actions).
2	`obj`	Object type. For `irrigate`, the type of the water source.
3	`obj_num`	Object number. 0 means the object the table is being checked for.
4	`name`	Action name. Used to link `days_act` conditions to this action, and by `irrigate`, `irr_demand` and `puddle` as described below. Otherwise free text.
5	`option`	Action option, specific to the action type.
6	`const`	First action constant.
7	`const2`	Second action constant. In `lum.dtl`, values below 1 are raised to 1.
8	`fp`	File pointer: a name in another input file, specific to the action type.
9 and on	`outcome1`, `outcome2`, …	`y` or `n` for each alternative.

An action runs when at least one alternative is satisfied and the action's outcome for that alternative is `y`. Actions run in the order of the rows.

In `lum.dtl`, many actions use `const2` as the maximum number of times the action may run in a calendar year for each HRU. The count is reset on 31 December. These actions are marked "per year: `const2`" in the table below.

Condition variables

Units are those of the compared value. "Plants" means the plants of the HRU's plant community.

`var`	Value compared	Notes
`w_stress`	Average water stress factor of the growing plants (1 = no stress)	1 when no plant is growing.
`n_stress`	Average nitrogen stress factor of the growing plants	0 when no plant is growing.
`p_stress`	Average phosphorus stress factor of the growing plants	0 when no plant is growing.
`phu_plant`	Fraction of potential heat units accumulated by the first plant that has started accumulating (plant 1 if none)	
`phu_base0`	Fraction of the annual base-zero heat units accumulated at the HRU's weather station	`obj` must be `hru` or `hlt`.
`precip_cur`	Precipitation today, mm	`obj` must be `hru` or `hlt`.
`precip_next`	Precipitation tomorrow, mm	`obj` must be `hru` or `hlt`.
`plant_gro`	Plant number `lim_const` (1 if 0) is growing	Yes/no.
`plant_name_gro`	The plant named in `lim_var` is in the community and growing	Yes/no.
`days_plant`	Days since the last planting	
`days_harv`	Days since the last harvest	
`days_irr`	Days since the last irrigation	

var	Value compared	Notes
days_act	Days since the action named in lim_var (an action name in the same table) last ran	Compared with lim_const + 2.
day_start	Day of year on which the simulation started	
irr_year	Irrigation applied to the HRU, mm	In 62.0.1 the total is not reset each year, so it is the total since the start of the run.
slope	HRU slope, m/m	
soil_water	Soil water content above wilting point, mm	Target: lim_var (wp = 0, fc = water at field capacity, ul = water at saturation), combined with lim_const by lim_op.
jday	Day of year	
month	Month	
year_rot	Year of the crop rotation	Starts at the plant community's initial rotation year and increases by 1 each 31 December.
year_gro, cur_yrs_mat	Years of growth of plant 1	Same value for both names.
year_cal	Calendar year	
year_seq	Year of the simulation (1 for the first year)	
year_start	First calendar year of the simulation	
biomass	Above-ground plant biomass, kg/ha	
leaf_area	Leaf area index of all plants	
ground_cov	Above-ground plant biomass, kg/ha	Residue is not included in 62.0.1, so it gives the same value as biomass.
p_factor	USLE P factor	
k_factor	USLE K factor of the top soil layer	
hyd_soil_group	Soil hydrologic group equals lim_var (A, B, C or D)	Yes/no.
p_pet	Ratio of average annual precipitation to PET from the weather generator	
p_lab_150	Labile phosphorus in the upper soil, ppm	Uses the first two soil layers.
soil_temp2	Temperature of soil layer 2, °C	
soil_carbon	Organic carbon of soil layer 1, %	
tile_drained	The HRU has tile drains	= true, / false.
prob	A uniform random number between 0 and 1	Compared with lim_const; use < to act with probability lim_const.
prob_unif	Today is the drawn application day	See Probability conditions.
prob_unif1	Random draw within a window that starts on 1 January	See Probability conditions.
prob_unif_lu	Random draw that spreads an operation over the HRUs of a land use	See Probability conditions.
channel_flo	Flow in the current hydrograph, m^3/s	
tile_flo	Tile flow from the HRU today, mm	
irr_demand	Irrigation demand of the HRU, m^3	
irr_pou_dmd	Demand of water allocation place of use obj_num, m^3	
aqu_dep	Depth to the water table of aquifer obj_num, m	
land_use	HRU land use equals lim_var (a name in landuse.lum)	Yes/no.
cal_group	HRU calibration group in landuse.lum equals lim_var	Yes/no.
tillage	The HRU's management schedule lists the decision table named in lim_var	Yes/no.
plant_com	The plant named in lim_var is in the plant community	= true, / false.
ch_order	Stream order of channel obj_num equals lim_const	Yes/no.
wtow_vol	Water in water tower obj_num, m^3	Target: lim_const × tower capacity.
vol	Reservoir volume, m^3	Target: lim_var pvol (principal volume) or evol (emergency volume) combined with lim_const by lim_op. With + and -, lim_const is in ha · m. With lim_var e-pv, the target is $V_p + \text{lim_const}\,(V_e - V_p)$.

var	Value compared	Notes
res_inflo	Inflow to reservoir obj_num, m^3/s	
wet_depth	Water depth of the HRU wetland or paddy	lim_const in mm.
weirh	Weir height of the HRU wetland or paddy	lim_const in mm.
vol_wet	Water volume of the HRU wetland or paddy, m^3	Target: lim_var pvol or evol combined with lim_const by lim_op.

Probability conditions

These three conditions let an operation happen on a random day within a window instead of on a fixed date.

prob_unif uses the columns differently from other conditions: obj is the probability f that the operation happens at all this year, obj_num is the first day of the window and lim_const is the last day (day of year). On the first day of the window SWAT+ decides, with probability f, whether the operation will happen this year, and if so draws one day in the window at random. The condition is true only on that day. A window that ends before it starts (for example 330 to 30) runs over the turn of the year.

prob_unif1 uses obj_num as a calendar year and lim_const as a number of days. The window starts on 1 January of that year. On each day of the window the condition is true with probability $1/n$, where n is the number of days left in the window.

prob_unif_lu uses obj_num as the first day and lim_const as the last day of the window. It is meant to be combined with a land_use condition in the same table: it spreads the operation over the HRUs of that land use so that the share of HRUs treated by day d follows a uniform distribution over the window.

Land use and management actions

Actions in lum.dtl. Unless stated otherwise, obj is hru and obj_num 0 means the HRU being checked.

act_typ	option	const	const2	fp	Effect
plant	plant name in the community	-	-	name in transplant.ops	Starts growth of the plant if it is not already growing. With a transplant entry, the plant starts from the transplant state (see management.sch).
harvest	plant name, or all	residue harvest efficiency	per year	name in harv.ops	Harvests the plant.
harvest_kill	plant name, or all	residue harvest efficiency	per year	name in harv.ops	Harvests and kills the plant.
kill	plant name, or all	-	per year	-	Kills the plant; biomass becomes residue.
till, tillage	name in tillage.til	-	per year	-	Tillage.
fertilize	name in fertilizer.frt	amount, kg/ha	per year	name in chem_app.ops	Applies fertilizer. On an HRU with standing water and a method with surf_frac 1, the fertilizer goes into the water.
fert_future	name in fertilizer.frt	amount, kg/ha	day of year	name in chem_app.ops	Schedules a fertilizer application on the given day. obj_num is the sequence number of the fert_future action in the table.
manure	-	amount, kg/ha	per year	-	In 62.0.1 the manure and method names are not looked up, so this action applies an empty fertilizer entry. Use fertilize with a manure from fertilizer.frt, or manure_demand.
manure_demand	name in chem_app.ops	amount, t/ha	per year	-	Sets a manure request for manure allocation. obj_num is the manure source number.
pest_apply	pesticide name listed in constituents.cs	amount, kg/ha	per year	name in chem_app.ops	Applies the amount times the method's app_eff.
graze	name in graze.ops	-	-	-	One day of grazing.
burn	name in fire.ops	-	per year	-	Burns plants and residue.

act_typ	option	const	const2	fp	Effect
irrigate	name in irr.ops	depth, mm	per year	-	Irrigates with const · eff_frac · (1 − sumq_frac). obj and obj_num name the source: aqu, cha or sdc, or res takes the water from that object and irrigates nothing if it has too little; any other obj (for example hru) is an unlimited source. With name ponding, the HRU is a paddy: const is the target ponding depth (mm), const2 the depth below which water is added, and the depth applied is the shortfall from the target.
irr_demand	name in irr.ops	depth, mm	per year	unlim or other	Sets the HRU's irrigation demand for water allocation (const× area). With fp unlim, the amt_mm of the irr.ops entry is applied directly. With name ponding, const and const2 are the target and threshold ponding depths (mm).
res_irr_dmd	-	depth, mm	-	-	Records an irrigation demand on reservoir obj_num.
rot_reset	-	rotation year	-	-	Sets the rotation year (minimum 1).
plant_reset, harv_reset, kill_reset, irr_reset	-	days	-	-	Sets the days since the last planting, harvest, kill or irrigation (minimum 1).
pheno_reset	-	-	per year	-	Restarts the phenology of plants with a moisture_gro trigger in plants.plt.
cn_update	absval, abschg, pctchg or relchg	value	per year	-	Changes the curve number for moisture condition II: set to the value, add the value, change by the value in percent, or multiply by (1 + value). The result is kept between 35 and 95.

act_typ	option	const	const2	fp	Effect
tiledep_control	-	drain depth, mm	per year	-	Sets the tile drain depth for drainage water management.
impound_on	-	-	-	name in wetland.wet	Turns on impounded water (paddy or wetland) on the HRU.
impound_off	-	-	-	-	Turns off impounded water on the HRU.
weir_height	wet, or other	height, mm	-	-	With wet, sets the HRU paddy weir height and the principal and emergency volumes from it. Otherwise sets the weir height of reservoir obj_num.
puddle	name in puddle.ops	-	per year	-	Puddles a paddy (see management.sch). The action name must be the tillage operation in tillage.til used for the soil mixing.
lu_change	-	-	-	name in landuse.lum	Changes the HRU's land use and management and reinitialises plants and curve number. Logged in lu_change_out.txt.
snow_change	-	-	-	name in snow.sno	Switches the HRU to another snow parameter set. Logged in lu_change_out.txt.
hru_fr_update	new ls_unit.ele file	-	-	new rout_unit.ele file	Updates HRU area fractions. Logged in lu_change_out.txt.
p_factor	-	new USLE P	-	-	Sets the USLE P factor.
contour	-	-	-	-	Sets $P = 2.7\,S + 0.26$, with S the slope (m/m).
stripcrop	-	-	-	pasture or other	Sets $P = 2.32\,S + 0.36$, halved when fp is pasture.
terrace	-	-	-	sod_outlet or other	Sets $P = 1.4\,S + 0.13$, doubled when fp is sod_outlet.
tile_install	-	-	-	name in tiledrain.str	Installs tile drains (see *.str).
septic_install	-	-	-	name in septic.str	Installs a septic system.
fstrip_install	-	-	-	name in filterstrip.str	Installs a filter strip.

act_typ	option	const	const2	fp	Effect
grassww_install, grassww_uninstall	-	-	-	name in grassedww.str	Installs or removes a grassed waterway.
user_def_bmp	-	-	-	name in bmpuser.str	Applies user-defined BMP removal fractions.
grow_init, grow_end	-	-	-	-	Start and end of the growing season of an HRU-lte (obj hlt). grow_end computes the yield.
chan_change	-	cover	stream order	-	Sets the cover and stream order of the channel being checked.
herd	-	-	-	-	No effect in 62.0.1.

The `plant`, `harvest` and `harvest_kill` actions of the generic tables `pl_hv_summer1`, `pl_hv_winter1` and `pl_hv_summer2` take the plant name from `management.sch` instead of `option` (see `management.sch`).

Flow control and water allocation actions, used in `flo_con.dtl`:

act_typ	option	const	const2	Effect
`tileflo_control`	`flo_mm`, `max_mm` (at most `const` mm), `min_mm` (at least `const` mm), `all_flo`, `zero_flo`, `frac` (`const` × tile flow)	mm or fraction	-	Sets the tile flow sent from a saturated buffer source HRU to its receiving HRU.
`wallo_dmd`	`set_dmd`	demand, m^3	-	Sets the demand of a water allocation place of use.
`wallo_dmd`	`irr_dmd`	-	-	Adds the irrigation demand of place of use `obj_num`.
`wallo_dmd`	`fill_res`, `fill_wtow`, `fill_can`	target fraction of capacity	fraction of the shortfall per day	Demand to fill reservoir, water tower or canal `obj_num` to `const` times its capacity (for `fill_res`, `fp` `pvol` or `evol`).
`duty_fr`	`fr1` to `fr6`	fraction	-	Fraction of the demand taken from point of diversion 1 to 6 (or sent to return 1 to 6).
`duty_reset`	-	point of diversion number	new maximum withdrawal	Resets the withdrawal limit of a point of diversion of place of use `obj_num`.

See water allocation for how these demands and fractions are used.

Reservoir release actions

In `res_rel.dtl` the action type is `release` and `option` selects how the release is computed. The releases of all actions that run on a day are added together. A table with zero alternatives runs all its actions every day. V is the current volume, V_p the principal and V_e the emergency volume.

option	const	const2	fp	Release
`rate`	rate, m^3/s	-	-	constant rate
`rate_pct`	percent	-	-	`const` % of V_p per day
`inflo_rate`	maximum rate, m^3/s	change to inflow, m^3/s	-	the larger of inflow + `const2` and `const`
`inflo_frac`	fraction	-	-	`const` × inflow
`ab_emer`	-	-	-	all volume above V_e
`days`	days	multiplier	`null`, `pvol` or `evol`	$(V - V_t)$/const, with target $V_t = 0$, `const2` × V_p or `const2` × V_e
`dyrt`	days	percent	`null`, `pvol` or `evol`	$(V - V_t)$/const plus `const2` % of V_p, with V_t = 0, V_p or V_e
`dyrt1`	days	percent	`con1`, `con2` or `con3`	as `dyrt`, with the target taken from condition 5, 4 or 3 of the table
`inflo_targ`	condition number	-	-	inflow plus all volume above V_p × that condition's `lim_const`

option	const	const2	fp	Release
weir	-	-	name in weir.res	weir equation
meas	-	-	name of a recall object	measured outflow
natlake	inactive storage, fraction of V_p	release coefficient, 1/day	-	natural lake outflow (Döll et al., 2003)
nonirr-h06	α	-	-	Hanazaki et al. (2006) scheme for non-irrigation reservoirs
irr-h06	α	β	-	Hanazaki et al. (2006) scheme for irrigation reservoirs
hydrop	amplitude	phase	-	HYPE hydropower scheme (Arheimer et al., 2019)

See *.res for the reservoir side of these options.

Land use scenarios

Tables in scen_lu.dtl run only if they are listed in scen_dtl.upd. That file has a title line, the number of entries, a header line, and one row per entry with three columns: max_hits, typ and the name of a table in scen_lu.dtl. Each listed table is checked for every HRU every day, before the HRUs are simulated. The usual actions are lu_change and snow_change.

Example

refdata/Ames_sub1/lum.dtl in the SWAT+ repository contains two tables:

```
lum.dtl: yearly irrigation limit example for Ames dataset
2

name                      conds      alts      acts       !limit yearly irrigation to 500 mm using irrigate
irr_year_irr                4         1         1
var                       obj    obj_num          lim_var            lim_op  lim_const      alt1
irr_year                  hru          0             null                 -   500.00000         <
w_stress                  hru          0             null                 -   0.80000           <
phu_plant                 hru          0           phu_mat                -     0.20000         >
leaf_area                 hru          0           null                   -     0.50000         >
act_typ                   obj    obj_num             name            option    const        const2
irrigate                  hru          0         sprinkler     sprinkler_ilm    25.00000   20.00000  unli
```

Table irr_year_irr has four conditions, one alternative and one action. The alternative is satisfied on a day when the irrigation applied so far is below 500 mm, the average water stress factor is below 0.8, the crop has accumulated more than 20% of its heat units, and the leaf area index is above 0.5. On such a day the HRU is irrigated with 25 mm using the sprinkler_ilm entry of irr.ops, at most 20 times a year. obj is hru, so the water comes from an unlimited source. The table is used by naming irr_year_irr as an automatic operation in management.sch. In 62.0.1 the irrigation total behind irr_year is not reset each year, so once 500 mm has been applied over the whole run, the table stops irrigating for the rest of the simulation.

A reservoir table from refdata/Osu_1hru/res_rel.dtl, followed by the blank line that res_rel.dtl needs:

```
name                      conds      alts             acts
paddy_auto_rel              2         2                2
var                       obj    obj_num          lim_var            lim_op     lim_const      alt1    alt
wet_depth                 hru          0         weir_hgt                 -       0.00000         >
wet_depth                 hru          0             null                 -       2.00000         >
```

act_typ	obj	obj_num	name	option	const	const2
release	hru	0	above_emer	days	1.00000	0.00000
release	hru	0	below_emer	weir	10.00000	0.00000

Both alternatives need a water depth above 2 mm, and the first also above 0 mm. When the first alternative is satisfied, the paddy releases its whole volume within one day (`days` with `const` 1 and target 0) and also releases over the weir `weir1`.

Related files

- `management.sch`: lists the `lum.dtl` tables each HRU uses.
- `*.ops`, `tillage.til`, `fertilizer.frt`, `landuse.lum`, `snow.sno`, `*.str`: databases named by actions.
- `*.res` and `*.wet`: name the release tables in `res_rel.dtl`.
- Water allocation and manure allocation: name tables in `lum.dtl` and `flo_con.dtl`.
- `file.cio`: names the four files on its `decision_table` line.

Sources

- Readers: `dtbl_lum_read.f90`, `dtbl_res_read.f90`, `dtbl_scen_read.f90`, `dtbl_flocon_read.f90`, `cal_cond_read.f90` (`scen_dtl.upd`)
- Type definitions: `conditional_module.f90`
- Conditions: `conditions.f90`, `cond_real.f90`, `cond_integer.f90`
- Actions: `actions.f90`; reservoir releases: `res_hydro.f90`
- Linking tables to HRUs: `proc_cond.f90`, `hru_dtbl_actions_init.f90`
- Callers: `hru_control.f90`, `time_control.f90`, `res_control.f90`, `wallo_start_day.f90`, `mallo_control.f90`
- File name slots: `type input_condition` in `input_file_module.f90`

Column	Fortran variable
condition `var`, `obj`, `obj_num`, `lim_var`, `lim_op`, `lim_const`	`d_tbl%cond(ic)%var`, `%ob`, `%ob_num`, `%lim_var`, `%lim_op`, `%lim_const`
`alt1` …	`d_tbl%alt(ic, ialt)`
`prob_unif` probability (`obj` column)	`d_tbl%frac_app`
action `act_typ`	`d_tbl%act(iac)%typ`
action `obj`, `obj_num`, `name`, `option`, `const`, `const2`	`d_tbl%act(iac)%ob`, `%ob_num`, `%name`, `%option`, `%const`, `%const2`
action `fp`	`d_tbl%act(iac)%file_pointer`
`outcome1` …	`d_tbl%act_outcomes(iac, ialt)`

Purpose

`plants.plt` is the plant growth database. Each row is one plant species, crop variety or land cover, with the parameters SWAT+ needs to grow it: plant type, heat units to maturity, leaf area development, radiation use efficiency, canopy height, rooting depth, temperature limits, nutrient content, stomatal conductance and residue properties.

Plant communities in `plant.ini`, plant operations in `management.sch` and decision tables, and land uses in `landuse.lum` all refer to rows of this file by name. The names are also used by the GIS interfaces to link land cover maps to SWAT+ land uses.

The file is named on the `hru_parm_db` line of `file.cio` (default `plants.plt`). If the file is missing, the database is empty.

The database distributed with SWAT+ has parameters for most common crops, grasses, trees and land covers. If a land use or plant is not in it, start from a row for a plant with similar growth habits.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one plant with 56 values in the order below. The reader does not match columns by header name, so every value must be present and in this order. It ignores anything after the 56th value, so the `description` column written by the SWAT+ Editor has no effect.

If the `event` column of `codes.bsn` is not 0, one more word is read after the 56th value: a plant class name of up to 25 characters (for example `row_crop`). In that case the plant class takes the place of the description.

#	Column	Units	Default	Description
1	`name`	-	-	Name of the plant or land cover, up to 40 characters. Must be unique.
2	`plnt_typ`	-	-	Plant type: `warm_annual`, `cold_annual`, `warm_annual_tuber`, `cold_annual_tuber` or `perennial`. See Plant types.
3	`gro_trig`	-	-	Growth trigger: `temp_gro` (growth driven by temperature and day length) or `moisture_gro` (tropical plants whose growth starts with the monsoon).
4	`nfix_co`	-	0.0	Nitrogen fixation coefficient: 0.5 for legumes, 0 for non-legumes. Any value above 0 lets the plant fix nitrogen when soil nitrate cannot meet its demand.
5	`days_mat`	days	110	Days to maturity, an integer. Used to compute the heat units to maturity. 0 means the plant uses the heat units of the whole year. See Heat units to maturity.
6	`bm_e`	(kg/ha)/(MJ/m²)	15.0	Biomass-energy ratio (radiation use efficiency): dry biomass produced, including roots, per unit of intercepted photosynthetically active radiation, with no stress, at ambient CO2 and a vapor pressure deficit of 1 kPa.
7	`harv_idx`	(kg/ha)/(kg/ha)	0.76	Harvest index for optimal growing conditions: fraction of above-ground biomass removed as yield. Values above 0.7 are replaced by 0.02 (see Values set by the model).
8	`lai_pot`	m²/m²	5.0	Maximum potential leaf area index. Values are limited to 0 to 10.
9	`frac_hu1`	fraction	0.05	Fraction of heat units to maturity at the 1st point of the optimal leaf area development curve.
10	`lai_max1`	fraction	0.05	Fraction of `lai_pot` at the 1st point of the optimal leaf area development curve.
11	`frac_hu2`	fraction	0.40	Fraction of heat units to maturity at the 2nd point of the optimal leaf area development curve.
12	`lai_max2`	fraction	0.95	Fraction of `lai_pot` at the 2nd point of the optimal leaf area development curve.
13	`hu_lai_decl`	fraction	0.99	Fraction of heat units to maturity at which leaf area starts to decline.
14	`dlai_rate`	-	1.0	Exponent of the leaf area decline of annuals after `hu_lai_decl`. 1 gives a linear decline.
15	`can_ht_max`	m	6.0	Maximum canopy height.
16	`rt_dp_max`	m	3.5	Maximum rooting depth. The soil profile depth also limits roots.
17	`tmp_opt`	°C	30.0	Optimal temperature for plant growth.
18	`tmp_base`	°C	10.0	Base (minimum) temperature for plant growth. Heat units accumulate only above it.
19	`frac_n_yld`	kg N/kg yield	0.0015	Fraction of nitrogen in the yield, dry weight.

#	Column	Units	Default	Description
20	`frac_p_yld`	kg P/kg yield	0.0003	Fraction of phosphorus in the yield, dry weight.
21	`frac_n_em`	kg N/kg biomass	0.006	Normal fraction of nitrogen in plant biomass at emergence.
22	`frac_n_50`	kg N/kg biomass	0.002	Normal fraction of nitrogen in plant biomass at 50% maturity.
23	`frac_n_mat`	kg N/kg biomass	0.0015	Normal fraction of nitrogen in plant biomass at maturity.
24	`frac_p_em`	kg P/kg biomass	0.0007	Normal fraction of phosphorus in plant biomass at emergence.
25	`frac_p_50`	kg P/kg biomass	0.0004	Normal fraction of phosphorus in plant biomass at 50% maturity.
26	`frac_p_mat`	kg P/kg biomass	0.0003	Normal fraction of phosphorus in plant biomass at maturity.
27	`harv_idx_ws`	(kg/ha)/(kg/ha)	0.01	Harvest index under severe water stress: the lowest harvest index expected, between 0 and `harv_idx`.
28	`usle_c_min`	-	0.001	Minimum USLE C factor for water erosion. Values are limited to 0.001 to 1.
29	`stcon_max`	m/s	0.002	Maximum stomatal conductance at high solar radiation and low vapor pressure deficit. Used with the Penman-Monteith ET method.
30	`vpd`	kPa	4.0	Vapor pressure deficit at the 2nd point of the stomatal conductance curve.
31	`frac_stcon`	fraction	0.75	Fraction of `stcon_max` at the 2nd point of the stomatal conductance curve.
32	`ru_vpd`	(kg/ha)/(MJ/m²)/kPa	8.0	Rate of decline in radiation use efficiency per unit increase in vapor pressure deficit above 1 kPa. 0 turns the adjustment off.
33	`co2_hi`	µL CO2/L air	660.0	Elevated CO2 concentration at the 2nd point of the radiation use efficiency curve.
34	`bm_e_hi`	(kg/ha)/(MJ/m²)	16.0	Biomass-energy ratio at the CO2 concentration `co2_hi`.
35	`plnt_decomp`	-	0.05	Plant residue decomposition coefficient: fraction of residue that decomposes in a day under optimal moisture, temperature, C:N and C:P. 0 takes `rsd_decomp` from `parameters.bsn`.
36	`lai_min`	m²/m²	0.75	Minimum leaf area index of perennials during the dormant period.
37	`bm_tree_acc`	-	0.3	Exponent that limits the maximum leaf area index of young perennials. See Perennials and trees.
38	`yrs_mat`	years	10	Years for a perennial to reach full development, an integer. Values below 1 are set to 1.
39	`bm_tree_max`	t/ha	1000.0	Maximum above-ground biomass. Checked once a year.
40	`ext_co`	-	0.65	Light extinction coefficient for intercepted photosynthetically active radiation. 0 is replaced by 0.65.
41	`leaf_tov_mn`	months	12.0	Leaf turnover time of `moisture_gro` perennials with minimum stress. Called `leaf_tov_min` in earlier documentation.
42	`leaf_tov_mx`	months	3.0	Leaf turnover time of `moisture_gro` perennials with maximum stress. Called `leaf_tov_max` in earlier documentation.
43	`bm_dieoff`	fraction	0.0	Factor on the excess biomass removed when above-ground biomass exceeds `bm_tree_max`. 0 is replaced by 1.
44	`rt_st_beg`	fraction	0.0	Root to shoot ratio at the start of the growing season. 0 is replaced by 0.4.

#	Column	Units	Default	Description
45	rt_st_end	fraction	0.0	Root to shoot ratio at the end of the growing season. 0 is replaced by 0.2. Tuber types always use 0.7.
46	plnt_pop1	plants/m²	0.0	Plant population at the 1st point of the population and leaf area curve.
47	frac_lai1	fraction	0.0	Fraction of lai_pot at the 1st point of the population and leaf area curve.
48	plnt_pop2	plants/m²	0.0	Plant population at the 2nd point of the population and leaf area curve.
49	frac_lai2	fraction	0.0	Fraction of lai_pot at the 2nd point of the population and leaf area curve.
50	frac_sw_gro	-	0.5	Intended as the 30-day precipitation to PET ratio that starts growth of moisture_gro plants. Read but not used, which uses a fixed 0.5.
51	aeration	fraction	0.2	Aeration stress threshold: the fraction of pore space above field capacity that can fill with water before the plant is stressed by lack of air. 0 is replaced by 0.2.
52	rsd_pctcov	-	0.0	Residue factor for the percent cover equation. 0 is replaced by 0.4.
53	rsd_covfac	-	0.0	Residue factor for the surface cover (C factor) equation. 0 is replaced by 0.04.
54	avg_lig_frac	fraction	0.85	Average lignin fraction of residue. Read but not used.
55	ab_lig_frac	fraction	0.15	Lignin fraction of above-ground residue. Used by the dynamic carbon model only.
56	bg_lig_frac	fraction	0.12	Lignin fraction of below-ground residue (roots). Used by the dynamic carbon model only.

The defaults are the values SWAT+ uses for a component that is not set. They are not filled in for missing columns, since every row must have all 56 values.

Plant types

plnt_typ decides how the plant goes through the year.

- warm_annual and cold_annual are annual crops. A cold_annual is planted in the fall and goes dormant over winter when day length falls below the dormancy threshold. Leaf area declines after hu_lai_decl, following dlai_rate.
- warm_annual_tuber and cold_annual_tuber are annuals whose yield is below ground. Their root to shoot ratio at the end of the season is fixed at 0.7.
- perennial covers grasses, shrubs and trees. Perennials go dormant when day length falls below the threshold, keep at least lai_min of leaf area, and develop over yrs_mat years.

gro_trig = moisture_gro is for tropical plants. If the plant was planted with the monsoon option, it starts growing when the 30-day ratio of precipitation to potential ET reaches 0.5. Leaf turnover of moisture_gro perennials varies between leaf_tov_mx and leaf_tov_mn months with moisture stress.

Heat units to maturity

SWAT+ computes the heat units a plant needs to reach maturity at the start of the run, from the monthly mean temperatures of the HRU's weather generator station.

- days_mat = 0: the plant grows through the whole year. Heat units to maturity are 95% of the year's sum of mean daily temperature above tmp_base.
- days_mat > 0 for an annual: the planting date is taken as the day when the sum of mean daily temperatures above 0 °C reaches 15% of the annual sum. Heat units above tmp_base are then summed for days_mat days from that date.

The calculation works in both hemispheres. Giving the days of a variety (for example 90, 100, 110 or 120 days for corn) lets one database support several varieties. If `days_mat` is longer than the growing season at a site, the heat units of the whole season are used.

Leaf area development

The optimal leaf area index follows a curve through two points, (`frac_hu1`, `lai_max1`) and (`frac_hu2`, `lai_max2`), where the first value is the fraction of heat units to maturity and the second the fraction of `lai_pot`:

$$f_{LAI} = \frac{f_{PHU}}{f_{PHU} + \exp(l_1 - l_2\, f_{PHU})}$$

l_1 and l_2 are set at the start of the run so that the curve passes through both points. The values of `lai_pot` in the standard database are for typical plant densities in rainfed agriculture. Drought-prone regions with low planting densities, or irrigated land with high densities, may need different values. Canopy height grows with the square root of f_{LAI} up to `can_ht_max`.

When `plnt_pop1` and `plnt_pop2` are given, a second curve through (`plnt_pop1`, `frac_lai1`) and (`plnt_pop2`, `frac_lai2`) limits `lai_pot` by plant population.

Radiation use efficiency

When the `co2` column of `parameters.bsn` is above 350 µL/L, radiation use efficiency is adjusted for CO2 along a curve through (330 µL/L, `bm_e`) and (`co2_hi`, `bm_e_hi`). Otherwise `bm_e` is used as given. Values for `co2_hi` and `bm_e_hi` are needed even when climate change is not simulated, and `bm_e_hi` should be larger than `bm_e`. A `co2_hi` of 330 is replaced by 660.

When the vapor pressure deficit is above 1 kPa, radiation use efficiency drops by `ru_vpd` for each kPa above 1, down to no less than 27% of `bm_e`. A value of 6 to 8 is a reasonable approximation for most plants, and 0 turns the adjustment off.

Perennials and trees

The maximum leaf area index of a perennial grows with its age:

$$LAI_{max} = \text{lai_pot} \left(\frac{y}{\text{yrs_mat}} \right)^{\text{bm_tree_acc}}$$

where y is the number of years since the plant was established (at least 1), and LAI_{max} is at most `lai_pot`. With `bm_tree_acc` = 0 the plant reaches `lai_pot` in its first year.

At the end of each year, if above-ground biomass B exceeds `bm_tree_max` (converted to kg/ha), the amount

$$D = (1 + \text{bm_dieoff})\,(B - 1000\,\text{bm_tree_max})$$

dies and goes to the residue pools, with roots in the same proportion. Nothing dies if D is not smaller than B. A mature forest stand typically has 30 to 50 t/ha of biomass. `bm_dieoff` is not used at the start of dormancy in 62.0.1.

Nutrient uptake

The six fractions `frac_n_em` to `frac_p_mat` describe the nitrogen and phosphorus content of the whole plant, roots included, on a dry weight basis, at emergence, at 50% maturity and at maturity. They set the plant's nutrient demand through the season. Each fraction must be smaller than the one before it: if `frac_n_50` is not at least 0.0001 below `frac_n_em`, it is set to `frac_n_em` − 0.0001, and if `frac_n_mat` is not at least 0.0001 below `frac_n_50`, it is multiplied by 0.75. The phosphorus fractions are treated the same way.

Residue lignin

With `carbon` = 2 in codes.bsn, residue entering the soil is split into metabolic, structural and lignin pools. Below ground, the lignin fraction is `bg_lig_frac`, the structural fraction is `bg_lig_frac` / 0.8, and the rest is metabolic. Above ground the same split uses `ab_lig_frac`. With `carbon` = 0 these columns are not used.

Values set by the model

After the file is read, SWAT+ changes some values:

Column	Change
`harv_idx`	Values above 0.7 are set to 0.02.
`lai_pot`	Limited to 0 to 10.
`usle_c_min`	Limited to 0.001 to 1.
`plnt_decomp`	0 is replaced by `rsd_decomp` of parameters.bsn.
`yrs_mat`	Values below 1 are set to 1.
`ext_co`	0 is replaced by 0.65.
`bm_dieoff`	0 is replaced by 1.
`rt_st_beg`, `rt_st_end`	0 is replaced by 0.4 and 0.2. Tuber types get `rt_st_end` = 0.7.
`aeration`, `rsd_pctcov`, `rsd_covfac`	0 is replaced by 0.2, 0.4 and 0.04.
`co2_hi`	330 is replaced by 660.
`frac_n_50`, `frac_n_mat`, `frac_p_50`, `frac_p_mat`	Adjusted to decrease, as described above.

The `harv_idx` change affects crops whose harvest index is above 0.7, such as alfalfa (0.9 in the standard database) and tuber crops with a harvest index above 1. Their harvest index becomes 0.02.

Calibration

These parameters can be changed during calibration through calibration.cal: `bio_e` (`bm_e`), `hvsti` (`harv_idx`), `rdmx` (`rt_dp_max`), `t_opt` (`tmp_opt`), `t_base` (`tmp_base`) and `usle_c` (`usle_c_min`).

Example

The first rows of `refdata/Ames_sub1/plants.plt` in the SWAT+ repository:

```
plants.plt
name  plnt_typ  gro_trig  nfix_co  days_mat  bm_e  harv_idx  lai_pot  frac_hu1  lai_max1  frac_hu2  lai_ma
alfa  perennial  temp_gro  0.5  0  20  0.9  4  0.15  0.01  0.5  0.95  0.9  0.5  0.9  3  20  4  0.02  0.003
cana  warm_annual  temp_gro  0  100  34  0.3  4.5  0.15  0.02  0.45  0.95  0.5  0.3  1.3  1.4  21  5  0.03
```

- `alfa` (alfalfa) is a perennial legume (`nfix_co` = 0.5) with `days_mat` = 0, so it uses the heat units of the whole year. Its `harv_idx` of 0.9 is replaced by 0.02.
- `alfa` has `frac_n_50` = `frac_n_mat` = 0.02, so `frac_n_mat` is multiplied by 0.75 to 0.015.
- `bm_tree_max` is 0 for both rows. With `bm_dieoff` = 0.1, $D = 1.1\,B$ at the end of the year, which is not smaller than B, so no biomass dies off.
- Ames_sub1 has `event` = 0 and `carbon` = 2 in codes.bsn. No plant class is read, so the `description` column is ignored, and the lignin columns are used: for `alfa`, 9% of decomposing surface residue and 20% of root residue entering the soil goes to the lignin pool. `avg_lig_frac` is read but has no effect.

Related files

- plant.ini: plant communities for HRUs, by plant name.
- management.sch and decision tables: plant, harvest and kill operations name plants from this file.
- landuse.lum: land uses and their plant communities.
- codes.bsn: `event` switches the plant class column on; `carbon` selects whether the lignin fractions are used.

- parameters.bsn: `rsd_decomp` replaces a `plnt_decomp` of 0.
- Weather generator: monthly temperatures used for heat units to maturity.

Sources

- Reader: plant_parm_read.f90
- Type definition: `type plant_db` and `type residue_partition_fracs` in plant_data_module.f90
- Derived values and overrides: plantparm_init.f90
- Heat units to maturity: plant_init.f90
- Growth: pl_leaf_gro.f90, pl_leaf_senes.f90, pl_root_gro.f90, pl_biomass_gro.f90, pl_dormant.f90, pl_mortality.f90, pl_nfix.f90, pl_waterup.f90
- Monsoon trigger: hru_control.f90
- Residue lignin: cbn_rsd_transfer.f90, cbn_surfrsd_decomp.f90
- Calibration parameters: cal_parm_select.f90

Column	Fortran variable
name	pldb%plantnm
plnt_typ	pldb%typ
gro_trig	pldb%trig
days_mat	pldb%days_mat
bm_e	pldb%bio_e
harv_idx	pldb%hvsti
lai_pot	pldb%blai
frac_hu1, lai_max1	pldb%frgrw1, pldb%laimx1
frac_hu2, lai_max2	pldb%frgrw2, pldb%laimx2
hu_lai_decl	pldb%dlai
can_ht_max	pldb%chtmx
rt_dp_max	pldb%rdmx
tmp_opt, tmp_base	pldb%t_opt, pldb%t_base
frac_n_yld, frac_p_yld	pldb%cnyld, pldb%cpyld
frac_n_em, frac_n_50, frac_n_mat	pldb%pltnfr1, pltnfr2, pltnfr3
frac_p_em, frac_p_50, frac_p_mat	pldb%pltpfr1, pltpfr2, pltpfr3
harv_idx_ws	pldb%wsyf
usle_c_min	pldb%usle_c
stcon_max	pldb%gsi
vpd	pldb%vpdfr
frac_stcon	pldb%gmaxfr
ru_vpd	pldb%wavp
co2_hi, bm_e_hi	pldb%co2hi, pldb%bioehi
plnt_decomp	pldb%rsdco_pl
lai_min	pldb%alai_min
bm_tree_acc	pldb%laixco_tree
yrs_mat	pldb%mat_yrs
bm_tree_max	pldb%bmx_peren
ext_co	pldb%ext_coef
leaf_tov_mn, leaf_tov_mx	pldb%leaf_tov_min, pldb%leaf_tov_max
rt_st_beg, rt_st_end	pldb%rsr1, pldb%rsr2
plnt_pop1, frac_lai1	pldb%pop1, pldb%frlai1
plnt_pop2, frac_lai2	pldb%pop2, pldb%frlai2
frac_sw_gro	pldb%frsw_gro
avg_lig_frac	pldb%res_part_fracs%meta_frac
ab_lig_frac	pldb%res_part_fracs%str_frac
bg_lig_frac	pldb%res_part_fracs%lig_frac
plant class (event /= 0)	pl_class
event in codes.bsn	bsn_cc%nam1

Purpose

Fertilizer and manure applications use three databases.

- `fertilizer.frt` lists the fertilizer products: the fractions of mineral and organic nitrogen and phosphorus in each, and how much of the mineral nitrogen is ammonium. `fert` operations in `management.sch` and `fertilize` actions in decision tables name a row of this file.
- `manure_db.frt` lists the manure types. Each one points to a row of `manure_om.frt` and names optional pesticide, pathogen, metal, salt and constituent profiles. `manu` operations in `management.sch`, `manure` actions in decision tables and the manure allocation file name a row of this file.
- `manure_om.frt` gives the water, carbon and nutrient make-up of each manure.

`fertilizer.frt` is named on the `hru_parm_db` line of `file.cio` (default `fertilizer.frt`). The two manure files are not listed in `file.cio`. SWAT+ looks for them under the fixed names `manure_db.frt` and `manure_om.frt`, and leaves the manure databases empty when they are missing.

fertilizer.frt

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one product with the values below in this order. The reader does not match columns by header name. It reads the first six values and ignores the rest of the line, so the `pathogens` and `description` columns written by the SWAT+ Editor have no effect.

#	Column	Units	Default	Description
1	`name`	-	-	Name of the fertilizer, up to 16 characters. Must be unique.
2	`min_n`	kg mineral N/kg fertilizer	0.0	Fraction of the fertilizer that is mineral nitrogen (nitrate plus ammonium).
3	`min_p`	kg mineral P/kg fertilizer	0.0	Fraction of the fertilizer that is mineral phosphorus.
4	`org_n`	kg organic N/kg fertilizer	0.0	Fraction of the fertilizer that is organic nitrogen.
5	`org_p`	kg organic P/kg fertilizer	0.0	Fraction of the fertilizer that is organic phosphorus.
6	`nh3_n`	kg NH3-N/kg mineral N	0.0	Fraction of the mineral nitrogen that is ammonium. The rest is nitrate.

The fractions are by weight of the product. `min_n` + `min_p` + `org_n` + `org_p` is the nutrient content and should not exceed 1; the remainder is carrier.

For a commercial fertilizer the grade gives the nitrogen and phosphate content. The phosphate value in a grade is P2O5, so it has to be converted to P (multiply by 0.44) before it goes into `min_p`.

Fertilizer application

A fertilizer operation applies an amount in kg/ha of product. The application method row in `chem_app.ops` gives the fraction placed in the top 10 mm of soil (`surf_frac`); the rest goes into the second soil layer. In each of the two layers:

- `min_n` × (1 − `nh3_n`) of the amount is added as nitrate and `min_n` × `nh3_n` as ammonium.
- `min_p` of the amount is added as labile (soluble) phosphorus.
- With `carbon` = 0 in `codes.bsn`, half of the organic N and P goes to the fresh residue pool and half to the active humus pool.
- With `carbon` = 2, organic N and P are added to the soil organic matter pools with a carbon content of 10 times the organic N. The share that goes to the metabolic and structural litter pools depends on the C:N ratio. The split between the fresh and humus parts is `manure_c_frac` in `carbon.bsn`.

fertilizer.frt example

The first rows of `refdata/Ames_sub1/fertilizer.frt` in the SWAT+ repository:

```
fertilizer.frt: written by SWAT+ editor v2.2.2 on 2023-10-17 14:43 for SWAT+ rev.60.5.4
name                      min_n         min_p         org_n         org_p         nh3_n         pathogens  
```

```
elem_n          1.00000     0.00000     0.00000     0.00000     0.00000          null  l
elem_p          0.00000     1.00000     0.00000     0.00000     0.00000          null  l
anh_nh3         0.82000     0.00000     0.00000     0.00000     1.00000          null  A
urea            0.46000     0.00000     0.00000     0.00000     1.00000          null  U
18_46_00        0.18000     0.20200     0.00000     0.00000     0.00000          null
dairy_fr        0.00700     0.00500     0.03100     0.00300     0.99000          null  l
```

- `urea` is 46% mineral N, all of it applied as ammonium.
- `18_46_00` is an 18-46-0 grade: 18% N and 46% P2O5, which is 20.2% P.
- `dairy_fr` is fresh dairy manure, mostly organic N.
- `elem_n` has `nh3_n` = 0, so all its nitrogen is applied as nitrate.

manure_db.frt

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one manure type with eight values.

#	Column	Description
1	`name`	Name of the manure type, up to 25 characters. Manure operations and the manure allocation file refer to it by this name.
2	`org_min`	Name of a row in `manure_om.frt` that gives the water, carbon and nutrient make-up.
3	`pests`	Name of a pesticide profile, or `null`.
4	`paths`	Name of a pathogen profile, or `null`.
5	`hmets`	Name of a heavy metal profile, or `null`.
6	`salts`	Name of a salt profile, or `null`.
7	`constit`	Name of a profile for other constituents, or `null`.
8	`description`	Free text, one word.

In 62.0.1 the `pests`, `paths`, `hmets`, `salts` and `constit` names are read but not looked up. Salt and constituent loads in manure come from the fertilizer files of the salt and constituent modules, taken from the row in the same position as the manure type. They are added only when the manure type's `org_min` names the second or a later row of `manure_om.frt`.

A manure application takes its make-up from the row of `manure_om.frt` in the same position as the manure type in `manure_db.frt` (first with first, second with second). The `org_min` name is matched against `manure_om.frt` when the file is read, but the application does not use the match. Keep the two files in the same order, with `org_min` naming the row in the same position.

manure_db.frt example

`refdata/Osu_1hru/manure_db.frt`:

```
manure_db.frt: manure database
name          org_min            pests  paths  hmets  salts  constit  description
beef_salt     mw_beef_liq        null   null   null   null   null     midwest_beef_liquid_salt
```

One manure type, `beef_salt`, using the `mw_beef_liq` make-up. `mw_beef_liq` is also the first row of `manure_om.frt` below, so the position used by the application and the `org_min` name agree. Because `org_min` names the first row, no salt or constituent loads are added when `beef_salt` is applied, whatever the salt and constituent fertilizer files contain.

manure_om.frt

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one manure make-up with nine values.

#	Column	Units	Description
1	`name`	-	Name of the make-up, up to 64 characters.
2	`frac_water`	kg water/kg manure as applied	Water fraction of the manure as applied. Must be below 1.
3	`fcbn`	kg C/kg dry manure	Carbon fraction.
4	`fminn`	kg mineral N/kg dry manure	Mineral nitrogen fraction (nitrate plus ammonium).
5	`fminp`	kg mineral P/kg dry manure	Mineral phosphorus fraction.
6	`forgn`	kg organic N/kg dry manure	Organic nitrogen fraction.
7	`forgp`	kg organic P/kg dry manure	Organic phosphorus fraction.
8	`fnh3n`	kg NH3-N/kg mineral N	Fraction of the mineral nitrogen that is ammonium.
9	`description`	-	Free text, one word.

Manure application

The amount in a manure operation is the dry mass of manure in kg/ha. The nutrient fractions apply to that dry mass. The water that comes with it, amount / (1 − `frac_water`) − amount, is added to the soil water of the two top layers. As for fertilizer, `surf_frac` of the application method in `chem_app.ops` goes to the top 10 mm and the rest to the second layer.

Mineral N and P are added the same way as for fertilizer. With `carbon` = 0, the organic N and P are split between the fresh and humus pools with a fixed fraction of 0.5. With `carbon` = 2, carbon from `fcbn` and organic N and P go to the manure and litter pools, and the split uses `manure_c_frac` in `carbon.bsn`.

manure_om.frt example

`refdata/Osu_1hru/manure_om.frt`:

```
manure_om.frt: manure organic matter database
name            frac_water   fcbn          fminn         fminp         forgn         forgp         fnh3n  c
mw_beef_liq     0.988        0.000780576   0.000311751   0.00081295    0.002110312   0.000365707   0.99   r
mw_beef_slur    0.926        0.016786571   0.002697842   0.00081295    0.002110312   0.000365707   0.99   r
```

Two make-ups of Midwest beef manure. The liquid form is 98.8% water and the slurry 92.6%. Applying 1000 kg/ha of `mw_beef_liq` adds about 82,300 kg/ha of water, or 8.2 mm.

Related files

- `management.sch`: `fert` and `manu` operations.
- Decision tables: `fertilize` and `manure` actions.
- `chem_app.ops`: application methods and the surface fraction.
- `manure_allo.mnu`: moves manure from sources to the land.
- `codes.bsn`: the `carbon` column selects how organic nutrients enter the soil.
- `carbon.bsn`: `manure_c_frac` for the dynamic carbon model.

Sources

- Readers: `fert_parm_read.f90`, `manure_db_read.f90`, `manure_orgmin_read.f90`, called from `proc_db.f90`
- Type definitions: `type fertilizer_db`, `type manure_database` and `type manure_attributes` in `fertilizer_data_module.f90`
- Name lookup: `read_mgtops.f90`
- Operations: `mgt_sched.f90`, `actions.f90`
- Application: `pl_fert.f90`, `pl_manure.f90`, `cs_fert.f90`, `salt_fert.f90`
- Fresh and humus split: `man_coef%rtof` in `carbon_module.f90`, read from `carbon.bsn` in `carbon_bsn_read.f90`

File and column	Fortran variable
fertilizer.frt `name`	`fertdb%fertnm`

File and column	Fortran variable
fertilizer.frt `min_n`, `min_p`	`fertdb%fminn`, `fertdb%fminp`
fertilizer.frt `org_n`, `org_p`	`fertdb%forgn`, `fertdb%forgp`
fertilizer.frt `nh3_n`	`fertdb%fnh3n`
manure_db.frt `org_min`	`manure_db%org_min`, index in `manure_db%iorg_min`
manure_db.frt `description`	`manure_db%descrip`
manure_om.frt columns	`manure_om%` with the same names

Purpose

`tillage.til` is the tillage database. Each row describes one tillage implement or operation: how much of the soil it mixes and how deep. A `till` operation in `management.sch`, a `till` or `tillage` action in a decision table, and a puddling operation all name a row of this file.

One row has a special role. A row named `biomix` sets the biological mixing efficiency and depth used by the dynamic carbon model (see Biological mixing).

The file is named on the `hru_parm_db` line of `file.cio` (default `tillage.til`). If the file is missing or named `null`, the database is empty and tillage operations find no row.

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one operation with the values below in this order. The reader does not match columns by header name. It reads the first six values and ignores the rest of the line, so the `description` column written by the SWAT+ Editor has no effect.

#	Column	Units	Default	Typical range	Description
1	`name`	-	-	-	Name of the tillage operation, up to 16 characters. Management schedules and decision tables refer to the operation by this name, so names must be unique. The name `biomix` is reserved for biological mixing.
2	`mix_eff`	fraction	0.0	0.0 to 1.0	Mixing efficiency. The fraction of the soil, residue, nutrients and other constituents in each layer within the mixing depth that is mixed uniformly over that depth. The rest stays in its original layer.
3	`mix_dp`	mm	0.0	0 to 750	Depth of mixing caused by the operation.
4	`rough`	mm	0.0	-	Random roughness. Read but not used.
5	`ridge_ht`	mm	0.0	-	Ridge height. Read but not used.
6	`ridge_sp`	mm	0.0	-	Ridge interval or row spacing. Read but not used.

Tillage mixing

On the day of a tillage operation SWAT+ takes `mix_eff` and `mix_dp` from the named row. For each soil layer whose bottom is within `mix_dp`, the fraction `mix_eff` of the layer's mass is mixed. A layer that straddles `mix_dp` contributes in proportion to the part of it above the mixing depth. The mixed material is then spread over the mixing depth in proportion to layer thickness. Soil water, texture, mineral and organic nutrient pools, residue and constituents are all mixed this way.

A mixing depth greater than 0 and less than 10 mm is raised to 11 mm. A row with `mix_dp` of 0 mixes nothing.

The `carbon` column of `codes.bsn` decides what else happens. With `carbon` = 2 (dynamic carbon), the operation also raises the tillage factor of each mixed layer, which speeds the decomposition of soil organic matter for `till_eff_days` days. The tillage factor curve uses `tillmix_a`, `tillmix_b` and `tillmix_c` from `carbon.bsn`. With `carbon` = 0, only the mixing is done.

A puddling operation (`pudl` in `management.sch`) names a row of `puddle.ops` and a row of this file. When the HRU has more than 1 mm of ponded water, the soil is mixed with a routine for flooded soils. Otherwise the mixing is the same as for an ordinary tillage operation.

Biological mixing

Biological mixing is the slow mixing of the soil by roots, worms and other organisms. How it is set depends on the carbon model.

With `carbon` = 2 in `codes.bsn`, the `biomix` row of `tillage.til` sets it. Its `mix_eff` is the biological mixing efficiency and its `mix_dp` the maximum biological mixing depth. Biological mixing runs every day in layers above 0 °C. After a tillage operation the efficiency drops to zero and builds back up over the following days at a rate set by `bio_consol` in `carbon.bsn`. If the file has no `biomix` row, the efficiency is 0.2 and the depth 50 mm. A `biomix` row with `mix_eff` of 0 turns daily biological mixing off.

With `carbon` = 0, the `biomix` row is not used for mixing. Biological mixing then happens once a year, at the end of the year, with the efficiency in the `bio_mix` column of `hydrology.hyd` and a fixed depth of 50 mm (or the soil depth, if smaller).

Surface roughness

`rough`, `ridge_ht` and `ridge_sp` are not used. The random roughness used for surface depression storage is 20 mm for every HRU in 62.0.1, whatever tillage is done.

Example

`refdata/Osu_1hru/tillage.til` in the SWAT+ repository:

```
tillage.til: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                    mix_eff        mix_dp         rough      ridge_ht      ridge_sp  description
puddle                  0.980       150.000        00.000         0.000         0.000    rice_paddy_puddlin
```

One operation named `puddle`, used for rice paddy puddling. It mixes 98% of the soil in the top 150 mm. The file has no `biomix` row; Osu_1hru runs with `carbon` = 0 in `codes.bsn`, so biological mixing comes from `hydrology.hyd` and the missing row has no effect.

The first lines of `refdata/Ames_sub1/tillage.til`, as shipped:

```
name           mix_eff         mix_dp        rough    ridge_ht    ridge_sp           description
bedderd           0.55            150           35           0           0           bedder(disk)
bedders           0.55            150           30           0           0           beddershaper
beddhipr          0.65            150           35           0           0           bedderdisk-hipper
beddkrow          0.85            100           40           0           0           bedderdisk-row
bedrollr          0.25             50            5           0           0           bedroller
```

beetcult	0.25	25	15	0	0	beetcultivator
biomix	0.01	50	0	0	0	biological_mixing

This file has no title line. SWAT+ skips the first two lines whatever they contain, so it takes the header as the title and the `bedderd` row as the header, and `bedderd` is not in the database. A schedule that names `bedderd` would not find it. The other 98 rows are read normally; for example `mldboard` (moldboard plow) mixes 95% of the top 250 mm. Ames_sub1 runs with `carbon` = 2, so its `biomix` row sets a daily biological mixing efficiency of 0.01 over the top 50 mm.

Related files

- `management.sch`: `till` and `pudl` operations name a row of this file.
- Decision tables: `till` and `tillage` actions name a row of this file.
- `puddle.ops`: puddling parameters that go with the tillage row.
- `codes.bsn`: the `carbon` column selects the mixing routine and how biological mixing is set.
- `carbon.bsn`: tillage factor coefficients, `till_eff_days` and `bio_consol` for the dynamic carbon model.
- `hydrology.hyd`: the `bio_mix` column sets biological mixing when `carbon` = 0.

Sources

- Reader: `till_parm_read.f90`
- Type definition and biological mixing defaults: `tillage_data_module.f90`
- Name lookup for schedules and decision tables: `read_mgtops.f90`, `dtbl_lum_read.f90`
- Operations: `mgt_sched.f90`, `actions.f90`
- Mixing: `mgt_newtillmix_cswat0.f90`, `mgt_newtillmix_cswat1.f90`, `mgt_newtillmix_wet.f90`, `mgt_tillfactor.f90`
- Biological mixing: `mgt_biomix.f90`, `soil_nutcarb_init.f90`, `hru_control.f90`, `time_control.f90`
- Random roughness: `zero0.f90`, `swr_depstor.f90`

Column	Fortran variable
name	tilldb%tillnm
mix_eff	tilldb%effmix
mix_dp	tilldb%deptil
rough	tilldb%ranrns
ridge_ht	tilldb%ridge_ht
ridge_sp	tilldb%ridge_sp

`carb_coefs.cbn` is no longer read.

Removed

SWAT+ 62.0.1 ignores a copy of `carb_coefs.cbn` left in the run directory. Its settings now come from `carbon.bsn` and `carbon_lyr.bsn`.

Replacement files

The dynamic carbon model is switched on with `carbon` = 2 in `codes.bsn`. In earlier revisions the value was 1; in 62.0.1 the value 1 is reserved for the C-FARM model, which is not implemented. With `carbon` = 0 soil carbon is static and neither replacement file is read.

With `carbon` = 2 SWAT+ needs two files, both with a title line, a header line and data rows like other SWAT+ inputs:

- `carbon.bsn`: one row of 28 basin-wide values. It is named on the basin line of `file.cio` (default `carbon.bsn`).
- `carbon_lyr.bsn`: decomposition rates and CO2 fractions, one row for the top soil layer (`layer` 1) and one for the layers below (`layer` 2). Its name is built from the `carbon.bsn` name by adding `_lyr` before the extension, so it is not listed in `file.cio`.

If either file is missing, or carbon.bsn does not have 28 values, SWAT+ stops with an error. There is no fallback to built-in values.

An optional file carbon_layers.prt sets how many soil layers the per-layer carbon outputs include. Without it, the outputs cover the largest number of layers of any HRU.

Keyword migration

Keyword in carb_coefs.cbn	Replacement
cbn_diagnostics	Removed. Carbon outputs are switched on per file in print.prt with the hru_cb_* and lsu_cb_* rows.
hp_rate, hs_rate, microb_rate, meta_rate, str_rate, microb_top_rate, hs_hp	Columns of the same names in carbon_lyr.bsn.
a1co2, asco2, apco2, abco2	Columns of the same names in carbon_lyr.bsn.
org_frac (4 values)	init_seq, init_microb, init_slow, init_passive in carbon.bsn.
prmt_21	koc_c in carbon.bsn.
prmt_44	solc_ratio in carbon.bsn.
till_eff_days	till_eff_days in carbon.bsn.
rtof	manure_c_frac in carbon.bsn.
cbn_consolidation_factors (2 values)	bio_consol, till_consol in carbon.bsn.
cbn_factor_approaches (2 values)	tmpf_eqn, watf_eqn in carbon.bsn.
tn, top, tx	t_cbn_min, t_cbn_opt, t_cbn_max in carbon.bsn.
zz_bmix_coefs (3 values)	bmix_a, bmix_b, bmix_c in carbon.bsn.
zz_emix_coefs (3 values)	tillmix_a, tillmix_b, tillmix_c in carbon.bsn.
photo_degrade_factor	sfc_rsd_photodeg in carbon.bsn.
nmbr_soil_test_layers, soil_test	Removed, no replacement.

Six values that were fixed in the code are now columns of carbon.bsn: n_act_frac (fraction of humus nitrogen kept in the active pool), cnr_cap and cnr_ref (cap and reference C:N ratio of surface residue, formerly 500 and 25), cpr_cap and cpr_ref (cap and reference C:P ratio, formerly 5000 and 200), and mathers_method (method for the initial slow humus pool).

Example

refdata/Ames_sub1 in the SWAT+ repository has both replacement files, converted from its former carb_coefs.cbn:

```
carbon.bsn: derived from carb_coefs.cbn (basin carbon scalars)
          init_seq       init_microb         init_slow      init_passive             koc_c        solc_rat
              0.95              0.02              0.44              0.54            1000.0               0

carbon_lyr.bsn: derived from carb_coefs.cbn (per-layer carbon coefficients)
           layer         hp_rate         hs_rate     microb_rate       meta_rate        str_rate microb_to
               1         1.2e-05        2.92e-04          0.0164          0.0405          0.0107
               2         1.2e-05        1.81e-04            0.02          0.0507          0.0134
```

Related files

- carbon.bsn and carbon_lyr.bsn: the replacement inputs.
- codes.bsn: the carbon switch.
- print.prt: carbon output switches.
- Carbon outputs: the files written by the dynamic carbon model.

Sources

- Readers: carbon_bsn_read.f90 (reads carbon.bsn and carbon_lyr.bsn), carbon_layers_read.f90, both called from proc_bsn.f90

- Variables: carbon_module.f90
- Carbon switch: bsn_cc%cswat in basin_module.f90
- Output switches: basin_print_codes_read.f90
- File name slot: in_basin%carbon_bsn in input_file_module.f90

Purpose

treatment.trt is a file name kept from the SWAT2012 treatment table. Project setup tools can still write it into a project folder, but SWAT+ 62.0.1 does not open it, and file.cio has no slot for it. Its contents have no effect on a run, and the file can be left empty or deleted.

Water treatment in SWAT+ is part of water allocation. Treatment plants are defined in water_treat.wal, and the composition of treated water in om_treat.wal.

Format

None. The file is not read.

Example

Both reference datasets in the SWAT+ repository, refdata/Ames_sub1 and refdata/Osu_1hru, contain an empty treatment.trt.

Related files

- Water allocation: water_treat.wal and om_treat.wal, the treatment inputs SWAT+ reads.
- file.cio: lists the input files SWAT+ reads.

  Sources

  - File name slots read from file.cio: input_file_module.f90, readcio_read.f90
  - Treatment inputs: water_treatment_read.f90 (water_treat.wal), om_treat_read.f90 (om_treat.wal)

Purpose

manure_allo.mnu sets up manure allocation: manure sources with a monthly production, and HRUs that receive manure from them when a decision table in lum.dtl asks for it. The file name is fixed. If the file is missing, manure allocation is off.

SWAT+ 62.0.1 reads this file and checks the decision tables every day, but applies no manure to any HRU. To apply manure, use a fertilize action in lum.dtl or a manu operation in management.sch.

Format

Free format, whitespace-separated.

- Line 1: title (skipped).
- Line 2: number of allocation objects.
- Then, for each allocation object:
  1. header line (skipped);
  2. object line: name, rule_typ, src_obs, trn_obs;
  3. header line (skipped);
  4. src_obs source lines;
  5. header line (skipped);
  6. trn_obs demand lines.

Object line

#	Column	Description
1	`name`	Allocation object name.
2	`rule_typ`	Allocation rule. Read but not used.
3	`src_obs`	Number of source lines.
4	`trn_obs`	Number of demand lines.

Source lines

#	Column	Units	Description
1	`num`	-	Source number, 1 to `src_obs`. The line is stored at this position.
2	`mois_typ`	-	Moisture class, `wet` or `dry`.
3	`manure_typ`	-	Manure name in `fertilizer.frt`.
4	`lat`	degrees	Latitude of the source.
5	`long`	degrees	Longitude of the source.
6	`stor_init`	t	Manure in storage at the start of the simulation.
7	`stor_max`	t	Storage capacity.
8 to 19	`prod_jan` … `prod_dec`	t/month	Manure produced in each month, January to December.

Demand lines

#	Column	Description
1	`num`	Demand number, 1 to `trn_obs`. The line is stored at this position.
2	`ob_typ`	Receiving object type: `hru`, `muni` (treatment) or `divert` (diversion out of the basin). Only `hru` demands are linked to a decision table.
3	`ob_num`	Number of the receiving object.
4	`dtbl`	Name of the decision table in `lum.dtl` that requests manure for this HRU, or `null`.
5	`right`	Manure right, `sr` (senior) or `jr` (junior).

The decision table requests manure with a `manure_demand` action: `obj_num` is the source number, `option` the application method in `chem_app.ops`, `const` the amount in t/ha and `const2` the maximum number of requests per year (see `*.dtl`).

Example

One allocation object with one source and one HRU demand:

```
manure_allo.mnu: example
1
name            rule_typ    src_obs  trn_obs
farm_a          fixed             1        1
num  mois_typ  manure_typ  lat       long      stor_init  stor_max   prod_jan  feb  mar  apr  may  jun  ju
  1  wet       beef_fr     41.99   -93.62          0.0       500.0    5.0  5.0  6.0  6.0  7.0  7.0  7.0  7
num  ob_typ    ob_num  dtbl                right
  1  hru            1  apply_manure         sr
```

With a table `apply_manure` in `lum.dtl` whose first action is `manure_demand` with source 1, method `broadcast` and 2 t/ha, the intent is to apply 2 t/ha of `beef_fr` from the source whenever the table's conditions are met. In 62.0.1 the request is recorded but nothing is applied.

Related files

- fertilizer.frt: the manure named in manure_typ.
- *.dtl: the decision tables named on demand lines and the manure_demand action.
- chem_app.ops: application methods.

Sources

- Reader: manure_allocation_read.f90, called from main.f90.in after dtbl_lum_read
- Types: manure_allocation_module.f90 (manure_allocation, manure_source_objects, manure_demand_objects)
- Daily calculation: mallo_control.f90, called from time_control.f90
- manure_demand action: actions.f90, cross-walk in dtbl_lum_read.f90
- HRU decision table slot: hru_dtbl_actions_init.f90
- Output: manure_source_output.f90, manure_demand_output.f90, called from command.f90

Column	Fortran variable
object line	mallo%name, %rule_typ, %src_obs, %trn_obs
source manure_typ	mallo%src%manure_typ (index in mallo%src%fertdb)
source prod_jan … prod_dec	mallo%src%prod_mon(1:12)
demand dtbl	mallo%trn%dtbl (index in mallo%trn%dtbl_num and hru%man_trn_dtbl)
manure_demand obj_num, const, option	manure_amt%src_obj, %app_t_ha, %app_method

The management schedules are in management.sch. Four small inputs referred to from other pages follow it: puddle.ops, transplant.ops, co2_yr.dat and object.prt.

management.sch

management.sch contains the management schedules. Each schedule has a list of operations on fixed dates or heat-unit fractions, a list of automatic operations (decision tables in lum.dtl), or both. An HRU uses the schedule named in the mgt column of its land use in landuse.lum. The file name is the second entry on the lum line of file.cio (default management.sch).

Format

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Then, for each schedule:

1. The schedule line: name, numb_ops, numb_auto.
2. numb_auto lines, each naming one decision table in lum.dtl.
3. numb_ops operation lines.

Column	Description
name	Schedule name, referenced by mgt in landuse.lum.
numb_ops	Number of scheduled operation lines.
numb_auto	Number of automatic operation lines.

Automatic operations

Each automatic operation line names a decision table in lum.dtl. Every day, SWAT+ checks each listed table for every HRU that uses the schedule. A name that is not in lum.dtl is not reported.

Three generic tables can be shared by several schedules with different crops. For these, the crop name follows the table name on the same line, and the plant, harvest and harvest_kill actions of the table use it instead of their option column:

Table name	Line content
pl_hv_summer1	table name and one crop
pl_hv_winter1	table name and one crop
pl_hv_summer2	table name and two crops; the crop used is the one for the current rotation year

Operation lines

#	Column	Units	Description
1	op_typ	-	Operation code (see below).
2	mon	-	Month of the operation, or 0.
3	day	-	Day of the month, or 0.
4	hu_sch	fraction	Heat-unit fraction that triggers the operation, or 0.
5	op_data1	-	First name (see below).
6	op_data2	-	Second name (see below).
7	op_data3	varies	Number: amount, days or override (see below).

op_typ	Operation	op_data1	op_data2	op_data3
plnt	plant	plant name in the HRU's plant community	name in transplant.ops	> 0 to start from the transplant entry
harv	harvest	plant name, or all	name in harv.ops	residue harvest efficiency (0 uses harv_eff)
hvkl	harvest and kill	plant name, or all	name in harv.ops	residue harvest efficiency
kill	kill	plant name, or all	-	-
till	tillage	name in tillage.til	-	-
fert	fertilizer	name in fertilizer.frt	name in chem_app.ops	amount, kg/ha
manu	manure	name in manure_db.frt	name in chem_app.ops	amount, kg/ha
pest	pesticide	pesticide name listed in constituents.cs	name in chem_app.ops	amount, kg/ha (multiplied by app_eff)
graz	grazing	name in graze.ops	-	number of grazing days
burn	burning	name in fire.ops	-	-
swep	street sweeping	name in sweep.ops	-	-
irrm	irrigation	name in irr.ops	-	-
irpm	irrigation of a rice paddy	name in irr.ops	-	-
irrp	start, adjust or stop paddy irrigation	name in irr.ops	water source: cha, res, aqu or unlim	target ponding depth, mm (see below)
pudl	puddling	name in tillage.til	name in puddle.ops	-
weir	set paddy weir height	weir name in weir.res	-	weir height, mm
dwm	drainage water management	-	-	tile drain depth, mm
cnup	change curve number	absval, abschg, pctchg or relchg	-	value
mons	monsoon season	-	-	0 or 1
skip	end of a rotation year	-	-	-

Notes on individual operations:

- plnt, harv, hvkl and kill act on the plant in the HRU's plant community (plant.ini) whose name equals op_data1. A plant that is already growing is not planted again; the event is logged as PLANT_ALREADY_GROWING.
- fert on an HRU with standing water puts the fertilizer into the water.
- pest looks the name up in the pesticide list of constituents.cs when the operation runs, so the pesticide must be simulated as a constituent.
- irrm and irpm apply the depth in irr.ops (see irr.ops).
- irrp turns on automatic irrigation of a paddy: water is added whenever the ponding depth falls below 90% of the target. A positive op_data3 is the target depth; a negative value uses amt_mm of the irr.ops entry; zero

turns paddy irrigation off. The irr.ops entry also gives the efficiency, runoff fraction and salt and nitrate concentrations of the water.

- cnup changes the curve number for moisture condition II: set it to the value (absval), add the value (abschg), change it by the value in percent (pctchg), or multiply it by 1 + value (relchg). The result is kept between 35 and 95.
- mons applies to plants with a moisture_gro trigger in plants.plt. With op_data3 1 it marks the start of the monsoon season. With 0 it restarts the phenology of such plants that are not growing.
- Codes not in this table (for example pcom or prtp) are read but do nothing.

Timing

Operations run in the order of the lines, one rotation year after another. A skip line ends a rotation year. After the last line the schedule starts again at the first line, so a schedule with n skip lines is an n-year rotation.

An operation runs on the day its mon and day match the calendar date, or, when hu_sch is greater than 0, on the first day the heat-unit fraction exceeds hu_sch. The heat-unit fraction is that of the plant named in op_data1 while it grows, and the base-zero fraction of the year otherwise. Give each operation either a date or a heat-unit fraction and set the other to 0. Within a rotation year, lines must be in chronological order: SWAT+ waits for the date of the current line before it moves on.

When the schedule reaches a skip line it waits there until the end of the year: 31 December for HRUs in the northern hemisphere, day 181 for HRUs in the southern hemisphere. At the start of the run, SWAT+ begins at the rotation year given by the plant community in plant.ini and skips the operations dated before the first simulated day.

The operations performed are written to mgt_out.txt when mgtout is y in print.prt.

Example

refdata/Ames_sub1/management.sch in the SWAT+ repository, first lines:

```
management.sch file AMES
                          NAME NUMB_OPS NUMB_AUTO OP_TYP        MON        DAY HU_SCH   OP_DATA1    OP_DAT
mgt_01                             305         1
                                               irr_year_irr
                                                   till          4          1    0.0    chisplow        n
                                                   plnt          4         28    0.0       corn         n
                                                   irrm          5         15    0.0       sprinkler_ilm
                                                   fert          6          1    0.0       null       ele
                                                   hvkl         10          7    0.0       corn        gr
                                                   skip          0          0      0       null         n
```

Schedule mgt_01 has 305 operation lines and one automatic operation, the irr_year_irr table of lum.dtl (see the *.dtl example). In the first rotation year it tills with chisplow on 1 April, plants corn on 28 April, irrigates once on 15 May, and harvests and kills the corn with the grain harvest operation on 7 October.

For a fert line, 62.0.1 reads the fertilizer name from op_data1 and the application method from op_data2. The fert line above has null in op_data1 and the fertilizer name elem_n in op_data2. No fertilizer is named null, so the line matches no entry in fertilizer.frt and applies nothing; no warning is written. Every fert line in this file has the same layout, so as written the Ames schedules apply no fertilizer in 62.0.1. A line that applies 181.4 kg/ha of elem_n in 62.0.1 reads fert 6 1 0.0 elem_n broadcast 181.4, with broadcast an entry in chem_app.ops.

Two schedules from refdata/Osu_1hru/management.sch, one using a scheduled manu operation together with decision tables and one using only decision tables:

```
rice140_rot                      1          7
     plow
     weir60r
     weir0
     pad_irr_r
     puddle
```

```
     pl_hv_rice
     fert_paddy
manu      5          1      0     beef_salt   broadcast  12.9927
agrl_rot                        0          2
    pl_hv_agro
    fert_stress
```

As shipped, the `manu` line finds `beef_salt` in manure_db.frt and `broadcast` in chem_app.ops, so 12.99 kg/ha of manure is applied on 1 May. The schedule has one operation line and no `skip`, so the line runs every year.

Related files

- landuse.lum: selects a schedule by name in its `mgt` column.
- *.ops, tillage.til, fertilizer.frt: databases the operation lines name.
- *.dtl: the decision tables listed as automatic operations.
- plant.ini: plant community and initial rotation year.

puddle.ops

Puddling parameters for rice paddies. A puddling operation (`pudl` in `management.sch`, or the `puddle` action in lum.dtl) mixes the soil with a tillage operation, lowers the hydraulic conductivity of the paddy's upper soil, and sets the sediment concentration of the ponded water. The file name is fixed.

Free format. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one entry, read by position.

#	Column	Units	Description
1	`name`	-	Puddling operation name.
2	`hydcon_mm/h`	mm/h	Hydraulic conductivity of the upper soil after puddling. 0 leaves it unchanged.
3	`sed_ppm`	ppm	Sediment concentration of the ponded water after puddling.
4	`orgn_ppm`	ppm	Organic N concentration after puddling. Read but not used.
5	`sedp_ppm`	ppm	Organic P concentration after puddling. Read but not used.
6	`no3_ppm`	ppm	NO3-N concentration after puddling. Read but not used.
7	`solp_ppm`	ppm	Soluble P concentration after puddling. Read but not used.
8	`nh3_ppm`	ppm	NH3 concentration after puddling. Read but not used.
9	`no2_ppm`	ppm	NO2 concentration after puddling. Read but not used.

The tillage operation is named in `op_data1` of the `pudl` line (or the `name` column of the `puddle` action). If the paddy has more than 1 mm of water, the mixing is done in the ponded water; otherwise it is ordinary tillage.

`refdata/Osu_1hru/puddle.ops` as shipped:

```
name                hydcon_mm/h   sed_ppm      orgn_ppm      sedp_ppm     no3_ppm       solp_ppm       nh
high_eff            0.00000         10000       0.00000     800.00000     10.00000       20.00000        1
med_eff             0.00000         10000       0.00000     500.00000     10.00000       20.00000        1
low_eff             0.0           10000       0.00000     100.00000     10.00000       20.00000        1.000
```

The file has no title line. SWAT+ skips the column header as the title and the `high_eff` line as the header, so 62.0.1 reads only `med_eff` and `low_eff`. An operation that names `high_eff` finds no entry and uses an entry with all values zero: conductivity unchanged and no sediment added. The Osu dataset's `puddle` decision table names `med_eff`, so its puddling runs with a sediment concentration of 10000 ppm and unchanged conductivity (`hydcon_mm/h` is 0).

transplant.ops

Transplant database. A planting can start a plant from a transplanted state instead of from seed: a `plnt` line in `management.sch` with a `transplant.ops` name in `op_data2` and `op_data3` greater than 0, or a `plant` action in `lum.dtl` with the name in its `fp` column. The file name is fixed. Up to revision 61 it was `transplant.plt`; 62.0.1 reads only `transplant.ops`, so a project with a `transplant.plt` file needs it renamed.

Free format. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one entry, read by position.

#	Column	Units	Description
1	`name`	-	Transplant name.
2	`lai_ini`	m^2/m^2	Leaf area index at transplanting. Read but not used.
3	`bm_ini`	kg/ha	Plant biomass at transplanting.
4	`phu_acc_ini`	fraction	Fraction of potential heat units already accumulated.
5	`fr_yrmat`	fraction	Fraction of the years to maturity already reached (perennials).
6	`pop`	plants/m^2	Plant population. Sets the maximum leaf area index through the population curve in `plants.plt`. 0 uses the maximum LAI in `plants.plt`.

`refdata/Osu_1hru` contains the file under its old name:

```
transplant.plt
      NAME       LAI_INI     BM_INI    PHU_ACC_INI    FR_YRMAT      POP
   tr_rice140         0.1        0.2      0.2             1            100
```

62.0.1 does not open `transplant.plt`, so in this dataset the transplant database is empty. The `plant` actions in its `lum.dtl` that name `tr_rice140` find no entry, no message is written, and the rice is planted from seed with the initial values of an ordinary planting.

co2_yr.dat and co2.out

Atmospheric CO2 concentration by year. `co2_yr.dat` is optional; its name is fixed. SWAT+ builds one CO2 value per simulation year and writes the series to `co2.out`.

`co2_yr.dat` layout:

Line	Content
1	title (skipped)
2	number of year records n
3	header (skipped)
4 to $n + 3$	calendar year and CO2 concentration (ppm), one year per line, consecutive years in increasing order

How the series is built:

- If `co2_yr.dat` is missing or empty, every year uses `co2` from `parameters.bsn` (default 400 ppm).
- If the last record's year is at or before the first simulation year, every year uses the last record's value.
- Otherwise the records are aligned to the simulation by the first record's year. Years before the first record use the first value, and years after the last record repeat the last value.

The alignment counts records from the first one, so the years must be consecutive.

`co2.out` has the header `YR  CO2(ppm)` and one line per simulation year with the calendar year and the CO2 value used.

object.prt

Hydrograph and state output for individual objects. Each line asks for one variable of one object, written daily to a separate file. The file name is the third entry on the `simulation` line of `file.cio` (default `object.prt`; `null` for none).

Free format. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one request.

#	Column	Description
1	`id`	Sequence number, 1, 2, 3, … in order.
2	`obj_typ`	Object type: `hru`, `hlt` (HRU-lte), `ru` (routing unit), `res`, `cha` (channel), `sdc` (SWAT-DEG channel), `exc` (export coefficient), `dr` (delivery ratio) or `out` (outlet).
3	`obj_typ_no`	Object number within its type. For `hru` with `sol_water` or `solnut_ly`, 0 writes every HRU.
4	`hyd_typ`	Variable (see below).
5	`filename`	Output file name, up to 26 characters.

`hyd_typ`	Output
`tot`	total outflow hydrograph
`rhg`	recharge (percolation) hydrograph
`sur`	surface runoff hydrograph
`lat`	lateral flow hydrograph
`til`	tile flow hydrograph
`sol_water`	soil water of each layer
`solnut_ly`	soil N and P pools of each layer (no header line)
`solnut_pr`	soil N and P pools of the profile (no header line)
`plant`	plant status
`cha_fp`	channel and floodplain water balance

Each requested file gets a header line (except `solnut_ly` and `solnut_pr`) and one line per day with day of year, month, day, year, object type and name, followed by the values.

Sources

- `management.sch` readers: `mgt_read_mgtops.f90` (schedule and automatic operation lines), `read_mgtops.f90` (operation lines and name lookups)
- Schedule types: `mgt_operations_module.f90`
- Running scheduled operations: `mgt_operatn.f90`, `mgt_sched.f90`, called from `hru_control.f90`
- Starting operation and year-end `skip` handling: `plant_init.f90`, `time_control.f90`
- Automatic operation lookup: `proc_cond.f90`; generic crop tables: `actions.f90`
- Curve number change: `chg_par.f90`
- `puddle.ops`: `mgt_read_puddle.f90`
- `transplant.ops`: `plant_transplant_read.f90`, `mgt_transplant.f90`, type `plant_transplant_db` in `plant_data_module.f90`
- CO2: `co2_read.f90`, `bsn_prm%co2` in `basin_module.f90`
- `object.prt`: `object_read_output.f90`, `obj_output.f90`, type `object_output` in `hydrograph_module.f90`
- File name slots: `in_lum%management_sch` and `in_sim%object_prt` in `input_file_module.f90`

File	Column	Fortran variable
`management.sch`	`numb_ops`, `numb_auto`	`sched%num_ops`, `sched%num_autos`
`management.sch`	automatic operation line	`sched%auto_name`, `sched%auto_crop`
`management.sch`	`op_typ`, `mon`, `day`	`mgt_ops%op`, `%mon`, `%day`
`management.sch`	`hu_sch`	`mgt_ops%husc`
`management.sch`	`op_data1`, `op_data2`, `op_data3`	`mgt_ops%op_char`, `%op_plant`, `%op3`
`puddle.ops`	`hydcon_mm/h`	`pudl_db%wet_hc`
`transplant.ops`	`lai_ini`, `bm_ini`, `phu_acc_ini`	`transpl%lai`, `%bioms`, `%phuacc`

File	Column	Fortran variable
co2_yr.dat	year, CO2	co2_inc%co2_yr%iyr, %co2
object.prt	obj_typ, obj_typ_no, hyd_typ	ob_out%obtyp, %obtypno, %hydtyp

Purpose

The initialization files set the state of plants, soils and water bodies on the first day of the run. The general ones are:

- plant.ini: plant communities, with the starting state of each plant. A land use in landuse.lum names a community in its plnt_com column.
- soil_plant.ini: for each HRU, links to the records that initialize soil nutrients, pesticides, pathogens, salts, heavy metals and constituents. An HRU names a record in the soil_plant_init column of hru-data.hru.
- nutrients.sol: initial soil nitrate, labile phosphorus and humus properties, named in the nutrients column of soil_plant.ini.
- om_water.ini: initial water, sediment and nutrient concentrations of channels, reservoirs and wetlands, named in the org_min column of their initial files.

Initialization files

The init line of file.cio has eleven file names. nutrients.sol is the second name on the soils line. Several salt and constituent files are opened under fixed names, whatever file.cio says.

File	How the name is set	Status in 62.0.1	Documented in
plant.ini	init line, name 1	Read	plant.ini
soil_plant.ini	init line, name 2	Read	soil_plant.ini
om_water.ini	init line, name 3	Read	om_water.ini
pest_hru.ini	init line, name 4	Read	pesticides
pest_water.ini	init line, name 5	Read	pesticides
path_hru.ini	init line, name 6	Read	pathogens
path_water.ini	init line, name 7	Read	pathogens
hmet_hru.ini	init line, name 8	Read, values not used	heavy metals
hmet_water.ini	init line, name 9	Not read	heavy metals
salt_hru.ini	init line, name 10	The file.cio name is not used. SWAT+ opens a file called salt_hru.ini.	salts
salt_water.ini	init line, name 11	Not read. Water bodies take their initial salts from salt_aqu.ini and salt_channel.ini (fixed names) and from the reservoir salt file.	salts
salt_aqu.ini, salt_channel.ini	Fixed names, not in file.cio	Read	salts
cs_hru.ini, cs_aqu.ini, cs_channel.ini	Fixed names, not in file.cio	Read	constituents
nutrients.sol	soils line, name 2	Read	nutrients.sol

All of these files are free format and whitespace-separated, with a title on line 1 and a column header on line 2, both skipped. Columns are read by position. Names are matched case-sensitively.

plant.ini

Each plant community is a block: one community line, then one line for each plant in the community. A community can be plants growing together (a mixed stand) or the crops of a rotation, of which only some are growing at the start.

If `plant.ini` is missing or named `null`, no community exists, and every land use whose `plnt_com` is not `null` is reported in `diagnostics.out` as `<landuse> <community> not found in plant.ini`.

Community line

#	Column	Units	Description
1	`pcom_name`	-	Community name, up to 40 characters, matched against `plnt_com` in `landuse.lum`. Names must be unique.
2	`plt_cnt`	-	Number of plant lines that follow, called `plnt_cnt` in earlier documentation. The reader counts communities with this number, so it must equal the number of plant lines.
3	`rot_yr_ini`	-	Year of the rotation in which the run starts. SWAT+ steps through the management schedule, counting a new year at each `skip` operation or each drop in day of year, and starts at the first operation of this rotation year on or after the start day of the run. Decision tables see it as the current rotation year.

Plant lines

#	Column	Units	Range	Description
1	`plt_name`	-	-	Plant name in `plants.plt`, called `plnt_name` in earlier documentation.
2	`lc_status`	-	`y`, `n`	Land cover status at the start: `y` growing, `n` not growing (it starts when a planting operation names it).
3	`lai_init`	m^2/m^2	0 to 8	Initial leaf area index. Used only when `lc_status` is `y`.
4	`bm_init`	kg/ha	0 and up	Initial total plant biomass. It is set whatever `lc_status` is; its N and P contents follow from the plant's nutrient fractions at the starting stage of growth.
5	`phu_init`	fraction	0 to 1	Fraction of the heat units to maturity already accumulated at the start of the current growing season.
6	`plnt_pop`	plants/m^2	-	Plant population. Read but not used: the maximum LAI is always the `lai_pot` of `plants.plt`, with no population adjustment.
7	`yrs_init`	fraction	0 to 1	For trees and other perennials, the current age as a fraction of the years to maturity (`mat_yrs` in `plants.plt`). 0.5 for a tree with `mat_yrs` = 30 means a 15-year-old stand. Not used for annuals.
8	`rsd_init`	kg/ha	0 to 10 000	Initial surface residue from this plant.

Starting state

For each plant in the community, SWAT+ sets at the start of the run:

- Heat units to maturity, from `days_mat` in `plants.plt` and the weather generator temperatures (see plants.plt).
- Maturity stage: `phu_init` for annuals; `yrs_init` for perennials, whose year of growth is $\max(1, \lfloor \text{yrs_init} \times \text{mat_yrs} \rfloor)$ and whose lifetime heat unit fraction is $\text{yrs_init} + \text{phu_init}/\text{mat_yrs}$.
- Canopy height: $\text{yrs_init} \times h_{max}$ for perennials; for annuals $h_{max}\sqrt{f}$, where f is the fraction of maximum LAI that the LAI curve of `plants.plt` gives for `phu_init`.
- LAI from `lai_init` and biomass from `bm_init`. If the plant is growing, its root depth, root mass and seed mass are also set from the maturity stage.

- Surface residue from `rsd_init`, with carbon 42% of the residue mass, a C:N ratio of 10 and a C:P ratio of 100. The residue of all plants in the community is added together, so a two-crop rotation with 10 000 kg/ha each starts with 20 000 kg/ha.

In 62.0.1 a perennial that starts the run growing gets its lifetime heat unit fraction from `yrs_init` and `phu_init` before its roots are set. Earlier revisions left it at zero, which kept the starting root depth of perennials at the 25.4 mm minimum whatever their age.

A plant name that is not in `plants.plt` is not reported. The reader's check for a missing name can never trigger, and the plant silently takes the parameters of the first plant in `plants.plt`. Check plant names in `plant.ini` against `plants.plt` by hand.

Example

`refdata/Ames_sub1/plant.ini` in the SWAT+ repository, first communities, with the spacing reduced:

```
plant.ini:
pcom_name  plt_cnt  rot_yr_ini  plt_name  lc_status  lai_init  bm_init      phu_init  plnt_pop  yrs_init
cosy_comm        2           1
                                corn      n          0.00000   0.00000      0.00000   0.00000   0.00000
                                soyb      n          0.00000   0.00000      0.00000   0.00000   0.00000
corn_comm        1           1
                                corn      n          0.00000   0.00000      0.00000   0.00000   0.00000
wetw_comm        1           1
                                wetw      y          2.00000   50000.00000  0.00000   0.00000   1.00000
```

`cosy_comm` is a corn and soybean rotation in which neither crop is growing at the start; the schedule plants them. Its two plant lines give 20 000 kg/ha of starting surface residue. `wetw_comm` is a wetland forest that starts growing, mature (`yrs_init` = 1), with LAI 2 and 50 000 kg/ha of biomass.

soil_plant.ini

Each line links an HRU to its soil initialization records. `hru-data.hru` names a record in its `soil_plant_init` column.

#	Column	Units	Description
1	`name`	-	Record name, up to 40 characters.
2	`sw_frac`	fraction	Initial soil water as a fraction of field capacity. Read but not used; initial soil water comes from `sw_init` in `parameters.bsn` (see soils.sol).
3	`nutrients`	-	Record in nutrients.sol.
4	`pest`	-	Record in `pest_hru.ini` (pesticides).
5	`path`	-	Record in `path_hru.ini` (pathogens).
6	`salt`	-	Record in `salt_hru.ini` (salts).
7	`hmet`	-	Record in `hmet_hru.ini` (heavy metals).
8	`cs`	-	Record in `cs_hru.ini` (constituents). Read only when the `event` column of `codes.bsn` is not 0.

The model reads column 6 as the salt record and column 7 as the heavy metal record. Some SWAT+ Editor versions write the header as `... path hmet salt`, the other way round. A salt record name placed in the column the Editor labels `salt` is then read as a heavy metal name, and the HRU uses the first `salt_hru.ini` record instead. Put the salt name in column 6 when salts are simulated.

Without a constituent column (the usual case), constituent runs use the first `cs_hru.ini` record for every HRU.

Name matching

When `hru-data.hru` is read, each HRU's `soil_plant_init` name is matched, and then the names in that record are matched against their files. When a name is not found:

- `nutrients`: the HRU uses the built-in values in the nutrients.sol table.
- `pest`, `path`, `salt`, `hmet`, `cs`: the HRU uses the first record of that file.
- `soil_plant_init` itself: nothing is reported (the check for a missing name can never trigger), and the HRU uses the first record of `soil_plant.ini`.

A project needs a `soil_plant.ini` file. If the file named in `file.cio` is missing, SWAT+ creates an empty file of that name and the soil initialization has no record to use.

Example

`refdata/0su_1hru/soil_plant.ini`:

```
soil_plant.ini: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                    sw_frac         nutrients              pest              path              hmet
soilplant1              0.00000          soilnut1              null              null              null
```

One record, `soilplant1`, with soil nutrients from `soilnut1` in `nutrients.sol` and no pesticide, pathogen, heavy metal or salt records. This file has the Editor's `hmet salt` header order.

nutrients.sol

Initial soil nitrogen and phosphorus and the humus properties of the static carbon model. Each record is one line. `soil_plant.ini` names a record in its `nutrients` column.

#	Column	Units	Built-in value	Description
1	`name`	-	`default`	Record name, up to 16 characters. A longer name in `soil_plant.ini` does not match.
2	`exp_co`	1/mm	0.001	Depth coefficient for the decrease of nitrate and labile P with depth. Values above 0.005 are replaced by 0.001.
3	`lab_p`	mg/kg	5.0	Labile (soluble) P concentration at the soil surface. 5.0 is used if 0.
4	`nitrate`	mg/kg	7.0	Nitrate N concentration at the soil surface. 7.0 is used if 0.
5	`fr_hum_act`	fraction	0.02	Fraction of humus in the active pool. 0.02 is used if 0. Static carbon model only.
6	`hum_c_n`	ratio	10.0	Humus C:N ratio (8 to 12). Static carbon model only; must not be 0.
7	`hum_c_p`	ratio	80.0	Humus C:P ratio (70 to 90). Static carbon model only; must not be 0.
8	`inorgp`	mg/kg	0.5	Inorganic P. Read but not used.
9	`watersol_p`	mg/kg	0.15	Water-soluble P. Read but not used.
10	`h3a_p`	mg/kg	0.25	H3A-extractable P. Read but not used.
11	`mehlich_p`	mg/kg	1.2	Mehlich-extractable P. Read but not used.
12	`bray_strong_p`	mg/kg	0.85	Strong Bray-extractable P. Read but not used.

The SWAT+ Editor also writes a `description` column, which is not read. The built-in values apply to HRUs whose `nutrients` name is `null` or not found, and to all HRUs when `nutrients.sol` is missing or named `null` in `file.cio`.

Initial mineral nitrogen and phosphorus

The concentrations at the surface decrease exponentially with the depth z (mm) to the bottom of each soil layer, and are converted to mass with the layer's bulk density ρ_b (Mg/m^3) and thickness h (mm):

$$NO3_{ly} = \text{nitrate}\; e^{-\text{exp_co}\, z}\, \frac{\rho_b\, h}{100}, \qquad P_{lab,ly} = \text{lab_p}\; e^{-\text{exp_co}\, z}\, \frac{\rho_b\, h}{100}$$

in kg/ha. The active mineral P pool is $P_{lab}\,(1 - PSP)/PSP$, with the phosphorus availability index PSP from `psp` in `parameters.bsn`, and the stable pool is four times the active pool. With the Vadas and White phosphorus model (`soil_p` = 1 in `codes.bsn`), PSP (0.1 to 0.7) and the stable to active ratio (1 to 7) are instead estimated from each layer's clay, organic carbon and labile P.

Initial organic pools

The organic carbon of each layer comes from the `carbon` column of `soils.sol`: $C_{ly} = 10\,000\, h\, \rho_b \times$ carbon/100 kg/ha, with organic N and P at C:N 10 and C:P 100 before the split into pools. The split depends on the `carbon` column of `codes.bsn`:

- Static carbon model (0): the active humus pool gets `fr_hum_act` of the organic matter and the stable pool the rest. The N and P of both pools are their carbon divided by `hum_c_n` and `hum_c_p`.
- CENTURY model (2): the share `init_seq` of `carbon.bsn` goes to the passive humus, slow humus and microbial pools, and the rest to metabolic and structural litter. Fixed C:N and C:P ratios are used; `fr_hum_act`, `hum_c_n` and `hum_c_p` are not. The slow humus pool follows the `mathers_method` switch of `carbon.bsn`.

In 62.0.1 the CENTURY pools are set up for carbon model 2 (earlier revisions tested for 1), the litter share is 1 − init_seq from `carbon.bsn` instead of a fixed 5%, and a non-lignin structural pool is also initialized.

Example

`refdata/Osu_1hru/nutrients.sol`:

```
nutrients.sol: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                    exp_co         lab_p       nitrate    fr_hum_act       hum_c_n       hum_c_p
soilnut1               0.00050       5.00000       7.00000       0.02000      10.00000      80.00000
```

Surface nitrate 7 mg/kg and labile P 5 mg/kg. With `exp_co` = 0.0005, the concentrations fall to $e^{-0.5} = 61\%$ of the surface value at 1000 mm.

om_water.ini

Initial water and water quality of channels, reservoirs and wetlands. Each line is one record; the `org_min` column of `initial.cha` (channels) or `initial.res` (reservoirs and wetlands) names a record.

#	Column	Units	Description
1	`name`	-	Record name, up to 16 characters.
2	`vol`	fraction	Initial water. Channels: fraction of bankfull depth; values above 1 put water on the flood plain. Reservoirs and wetlands: fraction of the principal spillway volume.
3	`sed`	mg/L	Suspended sediment.
4	`part_n`	mg/L	Organic N.
5	`part_p`	mg/L	Organic P.
6	`no3`	mg/L	Nitrate N.
7	`solp`	mg/L	Soluble mineral P.
8	`chl_a`	mg/L	Chlorophyll a.
9	`nh3`	mg/L	Ammonia N.
10	`no2`	mg/L	Nitrite N.
11	`cbn_bod`	mg/L	Carbonaceous biochemical oxygen demand.
12	`dis_ox`	mg/L	Dissolved oxygen.
13	`sand`	mg/L	Sand.
14	`silt`	mg/L	Silt.
15	`clay`	mg/L	Clay.
16	`sm_ag`	mg/L	Small aggregates.

#	Column	Units	Description
17	l_ag	mg/L	Large aggregates.
18	gvl	mg/L	Gravel.
19	tmp	°C	Water temperature.

The header names are those of the SWAT+ Editor; the reader does not use them. All values except vol and tmp are concentrations: SWAT+ converts them to masses (kg, or tonnes for sediment) with the initial water volume. A reservoir is filled only if it is already built at the start of the run (by its year and month operational); otherwise it starts empty.

If an org_min name is not found, the object uses the first record of om_water.ini. Projects with channels, reservoirs or wetlands therefore need the file. Aquifers do not use it in 62.0.1: the org_min column of initial.aqu is read but no aquifer value is set from it.

Example

refdata/Osu_1hru/om_water.ini, with the spacing reduced:

```
om_water.ini: added to match initial.cha org_min 'no_init' (channel water init)
name     flo  sed  orgn sedp no3  solp chl_a nh3  no2  cbn_bod dis_ox san  sil  cla  sag  lag  grv  tmp
no_init  0.0  0.0  0.0  0.0  0.0  0.0  0.0   0.0  0.0  0.0     0.0    0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0
```

One record, no_init, named by both initial.cha and initial.res in that project: channels and wetlands start empty. The header here uses the model's short names (flo, orgn, sedp, …) instead of the Editor's; only the order matters.

Related files

- landuse.lum: names a plant.ini community in its plnt_com column.
- hru-data.hru: names a soil_plant.ini record in its soil_plant_init column.
- plants.plt: plant parameters for every plt_name.
- management.sch: the schedule in which rot_yr_ini picks the starting year.
- soils.sol: soil layers, organic carbon and bulk density used with nutrients.sol.
- codes.bsn, parameters.bsn and carbon.bsn: carbon model, phosphorus model, psp and initial carbon pool fractions.
- initial.cha, initial.res and initial.aqu: name om_water.ini records.
- Pesticides, pathogens, heavy metals, salts and constituents: the other initialization files.
- Nutrient cycling and carbon.

Sources

- File names: type input_init and type input_soils in input_file_module.f90, read in readcio_read.f90
- plant.ini: reader readpcom.f90 (called from proc_db.f90); type plant_community_db and type plant_init_db in plant_data_module.f90; use in plant_init.f90 and pl_root_gro.f90
- soil_plant.ini: reader soil_plant_init.f90 (called from proc_read.f90); type soil_plant_initialize in hru_module.f90; name matching in hru_read.f90
- nutrients.sol: reader solt_db_read.f90; type soiltest_db in soil_data_module.f90; use in soil_nutcarb_init.f90 (called from main.f90.in) with carbon_bsn_read.f90
- om_water.ini: reader om_water_init.f90; type hyd_output, res_convert_mass and hyd_convert_conc_to_mass in hydrograph_module.f90; name matching in sd_channel_read.f90, res_read.f90, wet_initial.f90, aqu_read_init.f90; use in sd_hydsed_init.f90, res_initial.f90, wet_initial.f90
- Fixed-name salt and constituent files: salt_hru_read.f90, salt_aqu_read.f90, salt_cha_read.f90, cs_hru_read.f90, cs_aqu_read.f90, cs_cha_read.f90

File and column	Fortran variable
plant.ini pcom_name, plt_cnt, rot_yr_ini	pcomdb%name, %plants_com, %rot_yr_ini
plt_name, lc_status	pcomdb%pl%cpnm (index %db_num), %igro
lai_init, bm_init, phu_init	pcomdb%pl%lai, %bioms, %phuacc
plnt_pop, yrs_init, rsd_init	pcomdb%pl%pop, %fr_yrmat, %rsdin
soil_plant.ini name, sw_frac	sol_plt_ini%name, %sw_frac
nutrients, pest, path	sol_plt_ini%nutc, %pestc, %pathc (indices %nut, %pest, %path)
columns 6, 7, 8	sol_plt_ini%saltc, %hmetc, %csc (indices %salt, %hmet, %cs)
nutrients.sol columns 1 to 12	solt_db%name, %exp_co, %lab_p, %nitrate, %fr_hum_act, %hum_c_n, %hum_c_p, %inorgp, %watersol_p, %h3a_p, %mehlich_p, %bray_strong_p
om_water.ini name	om_init_name
om_water.ini columns 2 to 19	om_init_water%flo, %sed, %orgn, %sedp, %no3, %solp, %chla, %nh3, %no2, %cbod, %dox, %san, %sil, %cla, %sag, %lag, %grv, %temp

Purpose

Structural practices are field-scale conservation structures attached to a land use. Five files contain their parameter sets: tiledrain.str, septic.str, filterstrip.str, grassedww.str and bmpuser.str. They are named on the structural line of file.cio. Two databases on the hru_parm_db line belong with them: septic.sep (septic tank effluent quality) and urban.urb (urban land types). The saturated buffer file satbuffer.str is read from the run folder under that fixed name; it is not listed in file.cio.

Each record in a structural file has a name. A land use in landuse.lum points to records by name in its tile, sep, vfs, grww and bmp columns, and an urban land use points to urban.urb in its urban column. Every HRU that uses that land use gets the practice. Write null in a landuse.lum column for no practice. A name that is not found in the structural file is reported in diagnostics.out as <landuse> <name> not found (landuse-tiledr) (or landuse-septic, landuse-fstrip, landuse-grassww, landuse-bmpuser), and the HRU then gets no practice of that kind.

Practices can also be added during a run with the decision table actions tile_install, septic_install, fstrip_install, grassww_install, grassww_uninstall and user_def_bmp, which name a record in the matching file. See decision tables.

All files are optional. If a file is missing, or named null in file.cio, it is treated as empty.

File layout

All of these files share one layout. Line 1 is a title and line 2 is a column header; both are skipped. Each later line is one record, read as free-format values separated by spaces. The reader takes the values by position and ignores the header text, so every column must be present and in the order shown. Text after the last numeric column (the description column written by SWAT+ Editor) is ignored.

tiledrain.str

Subsurface (tile) drain parameter sets, referenced from the tile column of landuse.lum.

#	Column	Units	Typical range	Description
1	name	-	-	Name of the tile drain record.
2	dp	mm	0 to 6000	Depth of the drain tube below the soil surface. 0 means no drain: the HRU produces no tile flow.
3	t_fc	hours	0 to 100	Time to drain the soil to field capacity. Used only by the drawdown method (tiledrain = 0 in codes.bsn).

#	Column	Units	Typical range	Description
4	lag	hours	0 to 100	Drain tile lag time. Tile flow is held in a lag store that releases a fraction $1 - \exp(-24/\text{lag})$ each day.
5	rad	mm	3 to 40	Effective radius of the drains. DRAINMOD method only.
6	dist	mm	5000 to 100000	Distance between two drain tubes or tiles. DRAINMOD method only.
7	drain	mm/day	10 to 51	Drainage coefficient: the largest daily tile flow the drains can take. Used by both methods.
8	pump	mm/hr	0 to 10	Pump capacity for subirrigation. DRAINMOD method only.
9	lat_ksat	-	0.01 to 4	Multiplier on the saturated hydraulic conductivity of each soil layer (k in the soil file) to give its lateral conductivity. DRAINMOD method only.

Tile flow methods

Tile flow is computed only when the HRU's land use names a tile drain record with dp greater than 0, the soil temperature of the second layer is above 0 °C and the shallow water table is above the drains. The tiledrain column of codes.bsn selects the method, and its wtable column selects how the water table depth is found.

With tiledrain = 0 (drawdown method), the water above field capacity drains at a rate set by t_fc:

$$q_{tile} = \frac{h_{wt} - h_d}{h_{wt}} \, (SW - FC) \, \left(1 - \exp\left(\frac{-24}{t_{fc}}\right)\right)$$

where h_{wt} is the height of the water table above the bottom of the soil profile, h_d is the height of the drains above the bottom of the profile (profile depth minus dp), SW is the soil water content and FC the water content at field capacity, all in mm. If t_{fc} is below 1 hour the exponential factor is left out. The result is limited to drain.

With tiledrain = 1 (DRAINMOD method), the drainage flux comes from the Hooghoudt and Kirkham equations used in DRAINMOD, which use rad, dist, lat_ksat, drain and pump. When the water table is below the drains the same equations give a subirrigation flux, limited by pump.

In both methods the day's tile flow then passes through the lag store described under lag. If lag is 0.01 hours or less, the release fraction is zero and tile water stays in the lag store for the rest of the run.

dist, rad and dp enter the DRAINMOD equations together with the soil depth, all in millimetres, so dist must be given in millimetres. The SWAT+ Editor default record (example below) has dist = 30, which the model reads as 30 mm.

```
tiledrain.str: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                        dp          t_fc           lag           rad          dist         drain
mw24_1000           1000.00000      24.00000      96.00000     100.00000      30.00000      10.00000
```

A decision table can change the drain depth during a run with tiledep_control (drainage water management).

septic.str

On-site septic system parameter sets, adapted from Siegrist et al. (2005): system type, biozone geometry, biomass properties and reaction coefficients in the biozone. A record is meant to be referenced from the sep column of landuse.lum.

#	Column	Units	Description
1	name	-	Name of the septic system record.

#	Column	Units	Description
2	typ	-	Septic system type: the record number (row position, starting at 1) of the effluent type in septic.sep, given as an integer.
3	yr	year	Year the system starts operating. 0 means it operates from the start of the simulation.
4	operation	-	Operation flag: 0 not operating, 1 active, 2 failing. An active system becomes failing when the biozone clogs; a failing system returns to active after t_fail days.
5	residents	-	Number of permanent residents per house. A typical US value is 2.5.
6	area	m^2	Average drainfield area of one septic system. About 40 to 70 m^2 per person is usual; 100 m^2 for a household of 2.5.
7	t_fail	days	Time until a failing system is fixed (0 to 150).
8	dp_bioz	mm	Depth to the top of the biozone layer from the ground surface.
9	thk_bioz	mm	Thickness of the biozone layer, the thin soil layer under the effluent distribution chamber where pollutants are degraded by live bacteria.
10	cha_dist	km	Distance from the septic system to the stream.
11	sep_dens	per km^2	Number of septic systems per square kilometre.
12	bm_dens	kg/m^3	Density of biomass.
13	bod_decay	m^3/day	BOD decay rate coefficient in the biozone.
14	bod_conv	-	Ratio of bacterial growth to BOD degraded in the septic tank effluent.
15	fc_lin	-	Linear coefficient for field capacity in the biozone.
16	fc_exp	-	Exponential coefficient for field capacity in the biozone.
17	fecal_decay	m^3/day	Fecal coliform decay rate coefficient.
18	tds_conv	-	Conversion factor for plaque from total dissolved solids.
19	mort	-	Mortality rate coefficient.
20	resp	-	Respiration rate coefficient.
21	slough1	-	Linear coefficient for the rate of biomass sloughing.
22	slough2	-	Exponential coefficient for the rate of biomass sloughing.
23	nit	-	Nitrification rate coefficient.
24	denit	-	Denitrification rate coefficient.
25	p_sorp	L/kg	Linear P sorption distribution coefficient.
26	p_sorp_max	mg P/kg soil	Maximum P sorption capacity.
27	solp_slp	-	Slope of the linear effluent soluble P equation.
28	solp_int	-	Intercept of the linear effluent soluble P equation.

Septic simulation in 62.0.1

In revision 62.0.1 septic systems run, but receive no effluent:

- The sep column of landuse.lum assigns the named septic.str record to every HRU with that land use before the soil layers are set up. If operation is not 0, a biozone layer is added to the soil profile at dp_bioz with thickness thk_bioz, and from year yr the biozone routine runs on each day the biozone layer is above freezing. Unknown names are reported in diagnostics.out.
- The daily septic tank effluent volume is never computed and stays at zero. No effluent water reaches the biozone, and the effluent concentrations in septic.sep, which are multiplied by that volume, add no nitrogen, phosphorus or BOD to the soil. The biozone routine works only on the water and nutrients already in the layer. residents, area, cha_dist, sep_dens and the q_rate column of septic.sep are read but not used.
- The decision table action septic_install assigns a septic.str record to an HRU during the run. The biozone layer is placed only when the soils are set up at the start of the run, so an HRU that gets a septic system this way has no biozone layer.

septic.str: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4

name	typ	yr	operation	residents	area	t_fail	dp_bioz	tl
standard	1	0	0	2.50000	100.00000	70	500.00000	50

septic.sep

Septic tank effluent quality database. Each record describes the effluent of one type of on-site wastewater system (from Siegrist et al., 2005; McCray et al., 2005). A `septic.str` record selects a row by its position through `typ`.

#	Column	Units	Description
1	`name`	-	Name of the effluent type (for example `gcon`, generic conventional system).
2	`q_rate`	m^3/capita/day	Septic tank effluent flow rate per person. McCray et al. (2005) give 0.227 as the US median. Not used.
3	`bod`	mg/L	7-day biological oxygen demand of the effluent. Typically 170 for a conventional system.
4	`tss`	mg/L	Total suspended solids in the effluent. Typically 75 for a conventional system.
5	`nh4_n`	mg N/L	Ammonium nitrogen in the effluent. Typically 60 (17 to 78) for a conventional system.
6	`no3_n`	mg N/L	Nitrate nitrogen in the effluent. 0 to 1.94 for a conventional system.
7	`no2_n`	mg N/L	Nitrite nitrogen in the effluent. Usually very low.
8	`org_n`	mg N/L	Organic nitrogen in the effluent. 9.4 to 15 for a conventional system.
9	`min_p`	mg P/L	Mineral phosphorus in the effluent.
10	`org_p`	mg P/L	Organic phosphorus in the effluent. Typically 1 for a conventional system.
11	`fcoli`	cfu/100 mL	Fecal coliform in the effluent. Typically 1E7 for a conventional system.

Values vary widely between system types; Table A-1 of Siegrist et al. (2005) lists them. The database distributed with SWAT+ Editor has 28 types, from generic conventional (`gcon`) and advanced (`gadv`) systems to sand filters, constructed wetlands, biofilters and textile filters.

`septic.sep: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4`

name	q_rate	bod	tss	nh4_n	no3_n	no2_n
gcon	0.22700	170.00000	75.00000	42.40000	0.00000	0.00000
gadv	0.22700	22.00000	14.00000	18.90000	9.60000	0.00000

filterstrip.str

Vegetative filter strip parameter sets, referenced from the `vfs` column of `landuse.lum`. A filter strip is a strip of dense vegetation at the field edge that intercepts runoff from upslope. It slows overland flow so particles settle, and it infiltrates part of the runoff. The model follows White and Arnold (2009). It reduces sediment, organic N, nitrate, particulate and soluble P, and pesticides in surface runoff leaving the HRU. The runoff volume itself is not changed.

#	Column	Units	Range used	Description
1	`name`	-	-	Name of the filter strip record.
2	`flag`	-	0 or 1	On/off switch. The filter strip is simulated only when `flag` is greater than 0.
3	`fld_vfs`	ratio	above 0, up to 300	Ratio of field area to filter strip area. Values of 30 to 60 are most common. Values above 300 are set to 300.

#	Column	Units	Range used	Description
4	con_vfs	fraction	0.1 to 0.95	Fraction of the field runoff that enters the most concentrated 10% of the strip. Runoff is rarely spread evenly: 10% of the strip often receives 25 to 75% of the runoff. A value of 0 or less is replaced by 0.5; other values are kept between 0.1 and 0.95.
5	cha_q	fraction	0 to 0.95	Fraction of the flow in the most concentrated 10% of the strip that is fully channelized. Channelized flow passes the strip without filtering or infiltration. Values above 0.95 are set to 0.95.

fld_vfs divides the field area to give the strip area, so it must be greater than 0 when the strip is switched on.

```
filterstrip.str: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                  flag       fld_vfs       con_vfs         cha_q  description
field_border             0       0.10000       0.00300       0.20000  Field_border
high_engineered          0       0.10000       0.00100       0.05000  Highly_engineered_low_channelized
```

Both records in this example have flag = 0, so a land use pointing to either one gets no filter strip.

grassedww.str

Grassed waterway parameter sets, referenced from the grww column of landuse.lum. A grassed waterway is a broad, shallow vegetated channel that takes runoff away from a field. The vegetation slows the flow, which protects the channel from scour and lets sediment and attached pollutants settle. SWAT+ simulates it as a trapezoidal channel with 8:1 side slopes.

#	Column	Units	Typical range	Default used when 0	Description
1	`name`	-	-	-	Name of the grassed waterway record.
2	`flag`	-	-	-	Read but not used. The waterway is simulated whenever the land use names a record.
3	`mann`	-	0.001 to 0.5	0.35	Manning's n of the waterway.
4	`sed_co`	-	0 to 1	0.005	Sediment transport coefficient, used to compute the transport capacity of the waterway.
5	`dp`	m	0 to 10	3/64 of `wd`	Depth of the waterway.
6	`wd`	m	0 to 1000	-	Top width of the waterway.
7	`len`	km	0 to 10000	square root of the HRU area	Length of the waterway.
8	`slp`	m/m	0 to 1	0.75 × HRU slope	Slope of the waterway.

Values of 0 or less in `mann`, `sed_co`, `dp`, `len` and `slp` are replaced by the defaults shown. If `dp` and `wd` together cannot form a trapezoid with 8:1 side slopes (the bottom width would be zero or negative), `dp` is reset to 3/64 of `wd`.

```
grassedww.str: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                  flag          mann        sed_co            dp            wd           len
grwway_high              0       0.05000       0.02000       1.00000       4.00000       0.50000       0.1
grwway_med               0       0.05000       0.02000       0.75000       3.00000       0.75000       0.0
grwway_low               0       0.05000       0.02000       0.50000       2.00000       1.00000       0.0
```

bmpuser.str

User-defined BMP removal efficiencies, referenced from the `bmp` column of `landuse.lum`. Many conservation practices are not modelled explicitly in SWAT+ but have published removal efficiencies. This record applies fixed reductions to the loads leaving the HRU. Surface runoff volume is not reduced, which keeps the water balance closed.

#	Column	Units	Range	Description
1	`name`	-	-	Name of the user BMP record.
2	`flag`	-	-	Read but not used. The BMP is applied whenever the land use names a record.
3	`sed_eff`	fraction	0 to 1	Sediment removal.
4	`ptlp_eff`	fraction	0 to 1	Particulate P removal (organic P and sorbed mineral P).
5	`solp_eff`	fraction	0 to 1	Soluble P removal in surface runoff.
6	`ptln_eff`	fraction	0 to 1	Particulate (organic) N removal.
7	`soln_eff`	fraction	0 to 1	Soluble N removal: nitrate in surface runoff and in lateral flow.
8	`bact_eff`	fraction	0 to 1	Bacteria removal. Read but not used.

Each load is multiplied by $(1 - \text{eff})$, so the efficiencies are fractions: 0.2 removes 20%. A value of 20 would give a negative load.

```
bmpuser.str: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                  flag        sed_eff       ptlp_eff       solp_eff       ptln_eff       soln_eff       bact
bmpusr1                  1        0.20000        0.20000        0.20000        0.20000        0.20000        0.2
```

satbuffer.str

Saturated buffer links. A saturated buffer diverts part of the tile flow from a source HRU into the soil of a receiving (buffer) HRU. Each record links one source to one receiver. The file is read only if a file named `satbuffer.str` is present in the run folder.

#	Column	Units	Description
1	`name`	-	Name of the buffer record.
2	`hru_src`	-	Number of the source HRU whose tile flow feeds the buffer.
3	`frac_src`	fraction	Fraction of the source HRU contributing tile flow to the buffer.
4	`flocon_dtbl`	-	Name of the flow control decision table (in the flow control decision table file) that sets how much tile flow goes into the buffer.
5	`hru_rcv`	-	Number of the receiving (buffer) HRU.
6	`lyr`	-	Soil layer of the receiving HRU that takes the diverted tile flow (0 = surface).

urban.urb

Urban land type database, referenced from the `urban` column of `landuse.lum`. The `urb_ro` column of `landuse.lum` selects the urban load method: `usgs_reg` (USGS regression equations) or `buildup_washoff` (build-up and wash-off of solids from impervious surfaces).

#	Column	Units	Built-in value	Description
1	`name`	-	-	Name of the urban land type.
2	`frac_imp`	fraction	0.05	Fraction of the HRU area that is impervious.
3	`frac_dc_imp`	fraction	0.05	Fraction of the HRU area that is directly connected impervious area.
4	`curb_den`	km/ha	0	Curb length density.
5	`urb_wash`	1/mm	0	Wash-off coefficient for removal of constituents from impervious surfaces.
6	`dirt_max`	kg/curb km	1000	Maximum amount of solids that can build up on impervious surfaces.
7	`t_halfmax`	days	1	Time for solids on impervious surfaces to build up to half of `dirt_max`.
8	`conc_totn`	mg N/kg sed	0	Total nitrogen concentration in suspended solids from impervious areas.
9	`conc_totp`	mg P/kg sed	0	Total phosphorus concentration in suspended solids from impervious areas.
10	`conc_no3n`	mg N/kg sed	0	Nitrate nitrogen concentration in suspended solids from impervious areas.
11	`urb_cn`	-	98	Curve number for impervious areas (moisture condition II). Read but not used: 62.0.1 uses a fixed curve number of 98 for impervious areas.

With the curve number method, surface runoff from an urban HRU is a weighted mix of runoff from the pervious part (the HRU's daily curve number) and from the directly connected impervious part (curve number 98), with `frac_dc_imp` as the weight. `frac_imp` is the weight for urban sediment and nutrient loads in both urban load methods.

```
urban.urb: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                  frac_imp   frac_dc_imp      curb_den      urb_wash      dirt_max     t_halfmax     c
urhd                   0.60000       0.44000       0.24000       0.18000     225.00000       0.75000     5
urmd                   0.38000       0.30000       0.24000       0.18000     225.00000       0.75000     5
```

References

- McCray, J.E., Kirkland, S.L., Siegrist, R.L. and Thyne, G.D. (2005). Model parameters for simulating fate and transport of on-site wastewater nutrients. Ground Water 43(4), 628–639.
- Siegrist, R.L., McCray, J., Weintraub, L., Chen, C., Bagdol, J., Lemonds, P., Van Cuyk, S., Lowe, K., Goldstein, R. and Rada, J. (2005). Quantifying site-scale processes and watershed-scale cumulative effects of decentralized wastewater systems. Project No. WU-HT-00-27, National Decentralized Water Resources Capacity Development Project, Washington University, St. Louis, MO.
- White, M.J. and Arnold, J.G. (2009). Development of a simplistic vegetative filter strip model for sediment and nutrient retention at the field scale. Hydrological Processes 23, 1602–1616. doi:10.1002/hyp.7291

Related files

- file.cio: the `structural` line names the five `.str` files; the `hru_parm_db` line names `urban.urb` and `septic.sep`.
- landuse.lum: points to records in all of these files.
- codes.bsn: `tiledrain` and `wtable` select the tile flow and water table methods.
- Decision tables: actions that install practices or change drain depth during a run.

Sources

- Readers, called from proc_db.f90:
  - sdr_read.f90 (`tiledrain.str`)
  - sep_read.f90 (`septic.str`)
  - septic_parm_read.f90 (`septic.sep`)
  - scen_read_filtstrip.f90 (`filterstrip.str`)
  - scen_read_grwway.f90 (`grassedww.str`)
  - scen_read_bmpuser.f90 (`bmpuser.str`)
  - sat_buff_read.f90 (`satbuffer.str`)
  - urban_parm_read.f90 (`urban.urb`)
- File name slots: `type input_structural` and `type input_parameter_databases` in input_file_module.f90
- Type definitions: `subsurface_drainage_parameters` and `saturated_buffer_parameters` in hru_module.f90; `septic_system` and `septic_db` in septic_data_module.f90; `filtstrip_operation`, `grwaterway_operation` and `bmpuser_operation` in mgt_operations_module.f90; `urban_db` in urban_data_module.f90
- Name lookup and assignment to HRUs: landuse_read.f90, hru_lum_init.f90, structure_init.f90, structure_set_parms.f90, plant_init.f90 (urban)
- Tile flow: swr_percmain.f90, swr_origtile.f90, swr_drains.f90, swr_substor.f90
- Septic: proc_hru.f90, soils_init.f90, sep_biozone.f90, hru_control.f90
- Filter strip, waterway, user BMP: smp_filter.f90, smp_grass_wway.f90, smp_bmpfixed.f90
- Urban: sq_daycn.f90, hru_urban.f90
- Decision table actions: actions.f90

File	Column	Fortran variable
tiledrain.str	dp, t_fc, lag, rad, dist, drain, pump, lat_ksat	sdr%depth, %time, %lag, %radius, %dist, %drain_co, %pumpcap, %latksat
septic.str	operation, residents, t_fail, dp_bioz, thk_bioz, cha_dist, sep_dens, bm_dens	sep%opt, %cap, %tfail, %z, %thk, %strm_dist, %density, %bd
septic.str	bod_decay, fc_lin, fc_exp, fecal_decay, tds_conv, mort, resp, slough1, slough2	sep%bod_dc, %fc1, %fc2, %fecal, %plq, %mrt, %rsp, %slg1, %slg2
septic.str	nit, denit, p_sorp, p_sorp_max, solp_slp, solp_int	sep%nitr, %denitr, %pdistrb, %psorpmax, %solpslp, %solpintc
septic.sep	name, q_rate, bod, tss, nh4_n, no3_n, no2_n, org_n, min_p, org_p, fcoli	sepdb%sepnm, %qs, %bodconcs, %tssconcs, %nh4concs, %no3concs, %no2concs, %orgnconcs, %minps, %orgps, %fcolis
filterstrip.str	flag, fld_vfs, con_vfs, cha_q	filtstrip_db%vfsi, %vfsratio, %vfscon, %vfsch
grassedww.str	flag, mann, sed_co, dp, wd, len, slp	grwaterway_db%grwat_i, %grwat_n, %grwat_spcon, %grwat_d, %grwat_w, %grwat_l, %grwat_s
bmpuser.str	flag, sed_eff, ptlp_eff, solp_eff, ptln_eff, soln_eff, bact_eff	bmpuser_db%bmp_flag, %bmp_sed, %bmp_pp, %bmp_sp, %bmp_pn, %bmp_sn, %bmp_bac
urban.urb	name, frac_imp, frac_dc_imp, curb_den, urb_wash, dirt_max, t_halfmax	urbdb%urbnm, %fimp, %fcimp, %curbden, %urbcoef, %dirtmx, %thalf
urban.urb	conc_totn, conc_totp, conc_no3n, urb_cn	urbdb%tnconc, %tpconc, %tno3conc, %urbcn2
landuse.lum	tile, sep, vfs, grww, bmp, urban	lum%tiledrain, %septic, %fstrip, %grasww, %bmpuser, %urb_lu

Purpose

SWAT+ simulates the pesticides listed in constituents.cs. It follows each one on plant foliage, in the soil layers, in surface runoff, lateral flow, tile flow and percolation, in channel water and bed sediment, in reservoirs and in aquifers. Four files describe them:

File	Named in	Content
pesticide.pes	hru_parm_db line of file.cio (default pesticide.pes)	Properties of each pesticide.
pest_hru.ini	init line of file.cio	Initial pesticide in soils and on plants.
pest_water.ini	init line of file.cio	Initial pesticide in channels, reservoirs and aquifers.
pest_metabolite.pes	fixed name, not in file.cio	Degradation products (daughters) of each parent pesticide.

Every pesticide named in constituents.cs must have a row in pesticide.pes with the same name. The database can contain many more pesticides than the run simulates. Pesticide applications in management.sch and decision tables name rows of pesticide.pes, and the application method comes from chem_app.ops.

pesticide.pes

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one pesticide with 17 values and a description, in the order below. The reader does not match columns by header name, so every value must be present and in this order.

#	Column	Units	Typical range	Description
1	name	-	-	Name of the pesticide, up to 16 characters. Must be unique.
2	soil_ads	(mg/kg)/(mg/L)	1 to 10^9	Soil adsorption coefficient normalized for soil organic carbon (K_{oc}).
3	frac_wash	fraction	0 to 1	Fraction of the pesticide on foliage washed off by a rainfall event.
4	hl_foliage	days	0 to 10,000	Half-life on foliage. 0 means no decay.
5	hl_soil	days	0 to 100,000	Half-life in the soil. 0 means no decay.
6	solub	mg/L	-	Solubility in water. Upper limit of the concentration in water leaving a soil layer.
7	aq_hlife	days	-	Half-life in channel, reservoir and aquifer water. 0 means no decay. Earlier documentation had aq_reac here, a reaction rate in 1/day; the column is now a half-life in days.
8	aq_volat	m/day	-	Volatilization coefficient in channel and reservoir water.
9	mol_wt	g/mol	-	Molecular weight. Used for the water and sediment mixing velocity and for the mass of metabolites.
10	aq_resus	m/day	-	Resuspension velocity of pesticide sorbed to bed sediment.
11	aq_settle	m/day	-	Settling velocity of pesticide sorbed to suspended sediment.
12	ben_act_dep	m	-	Depth of the active bed sediment layer.
13	ben_bury	m/day	-	Burial velocity in bed sediment.

#	Column	Units	Typical range	Description
14	`ben_hlife`	days	-	Half-life in bed sediment. 0 means no decay. Earlier documentation had `ben_reac` here, a reaction rate in 1/day; the column is now a half-life in days.
15	`pl_uptake`	-	0 and up	Transpiration stream concentration factor (TSCF): ratio of the concentration in the water taken up by roots to the dissolved concentration in the soil. 0 turns plant uptake off.
16	`harvest_sink`	0 or 1	0, 1	1: pesticide taken up by the plant leaves the system. 0: it stays in the plant and returns to the soil with the residue when the plant is harvested or killed.
17	`resistance_n`	fraction	0 to 1	Soil water used for the dissolved concentration in plant uptake. 0 uses the actual soil water, 1 uses field capacity, values in between blend the two.
18	`description`	-	-	Free text, one word.

Columns 15 to 17 are new in 62.0.1. Files written for earlier revisions have 14 values (name and 13 numbers) and then a description. With such a file the first 14 values of each row are read correctly, and the read of the row stops with an error when it meets the description where `pl_uptake` is expected. No message is written and the next row is read normally. The three new columns keep their built-in values (`pl_uptake` = 0, `harvest_sink` = 1, `resistance_n` = 1), so there is no plant uptake, which is the behaviour of earlier revisions. To use plant uptake, add the three columns before the description.

Half-lives and decay

Each day the pesticide in a compartment decays by first-order kinetics. The fraction left after one day is

$$f = \exp\left(-\frac{0.693}{t_{1/2}}\right)$$

where $t_{1/2}$ is the half-life of that compartment (`hl_foliage`, `hl_soil`, `aq_hlife` or `ben_hlife`). The half-life entered is a lumped value covering volatilization, photolysis, hydrolysis, biological degradation and other reactions. In channels the decay is scaled by the travel time through the reach, up to one day.

For most pesticides the foliar half-life is much shorter than the soil half-life because of volatilization and photodecomposition. If no foliar value is available, a common estimate is 0.25 to 0.5 times the soil half-life, lower for pesticides with a vapor pressure below 10^{-5} mm Hg, and at most 30 days.

Temperature-dependent decay

Column 7 of `parameters.bsn` switches temperature-dependent decay on. The SWAT+ Editor writes this column under the header `exp_sed`. When its value is 1, each half-life is taken as the value at 25 °C and adjusted with a Q10 of 2.58:

$$t_{1/2}(T) = t_{1/2,25}\ 2.58^{(25-T)/10}$$

The temperature T is the soil layer temperature for soil, the top soil layer temperature for foliage, and the water temperature for channel water and bed sediment. For aquifers the temperature is 0 °C in 62.0.1, so aquifer half-lives are about 10.7 times the 25 °C value. Reservoirs do not use the adjustment.

Any other value in that column gives the constant decay above. Check this column when a model is moved to 62.0.1: editor files commonly have 1.0 there, which switches the adjustment on.

Sorption and transport

The soil adsorption coefficient of a layer is

$$K_d = K_{oc} \frac{OC}{100}$$

where OC is the organic carbon content of the layer in percent. The pesticide in a layer is split between the soil and the water flowing through it according to K_d. The concentration in water leaving a layer is limited to `solub`. The concentration in surface runoff is the concentration in percolate times `pest_perc` of `parameters.bsn`.

On a day with at least 2.54 mm of precipitation, `frac_wash` of the pesticide on each plant's foliage is washed off to the top soil layer.

Plant uptake

When `pl_uptake` is above 0, growing plants take up pesticide from each soil layer with the water they transpire:

$$U = \text{TSCF} \frac{W_{up}}{\theta_{eff} + W_{up} + K_d \rho_b h} M$$

where W_{up} is the water taken up from the layer (mm), θ_{eff} is the soil water blended toward field capacity by `resistance_n`, ρ_b the bulk density, h the layer thickness (mm) and M the pesticide in the layer (kg/ha). Uptake is reported as `pl_uptake` in the pesticide output. With `harvest_sink` = 0, the pesticide kept in the plant returns to the soil when the plant is harvested or killed.

Metabolites

When a parent pesticide decays, a daughter listed in `pest_metabolite.pes` receives

$$M_{daughter} = \Delta M_{parent} \, f_{frac} \frac{MW_{daughter}}{MW_{parent}}$$

where f_{frac} is the daughter fraction for that compartment and MW the `mol_wt` values. Foliage uses `foliar_fr`, soil uses `soil_fr`, channel water uses `aq_fr` and channel bed sediment uses `ben_fr`. In 62.0.1 reservoirs (water and bed) and aquifers use `soil_fr`.

Calibration

These parameters can be changed during calibration through `calibration.cal`: `pst_koc`, `pst_washoff`, `pst_foliar_hlife`, `pst_soil_hlife`, `pst_solub`, `pst_aq_hlife`, `pst_aq_volat`, `pst_aq_settle`, `pst_aq_resus`, `pst_ben_hlife`, `pst_ben_bury` and `pst_ben_act_dep`. The daily decay factors are computed from the half-lives when `pesticide.pes` is read, so a calibration change to a half-life only has an effect when temperature-dependent decay is on.

pesticide.pes example

The first rows of `refdata/Osu_1hru/pesticide.pes` in the SWAT+ repository, as shipped:

```
pesticide.pes: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                    soil_ads     frac_wash    hl_foliage       hl_soil          solub       aq_hlife
245-tp                2600.00000       0.40000       5.00000      20.00000        2.50000        0.00700
2plus2                  20.00000       0.95000      10.00000      21.00000   660000.00000        0.00700
```

The file was written for revision 60.5.4 and has no `pl_uptake`, `harvest_sink` or `resistance_n` columns. As described above, each row is read up to `ben_hlife`, the description is not stored, and the three new columns take their built-in values, so no pesticide is taken up by plants.

Silvex (`245-tp`) sorbs strongly (K_{oc} of 2600) and has a soil half-life of 20 days. Mecoprop (`2plus2`) is very soluble and hardly sorbs.

Every row has `aq_hlife` = 0.007 and `ben_hlife` = 0.05. These match the reaction coefficients in 1/day of the older database format, but 62.0.1 reads them as half-lives in days. A half-life of 0.007 days leaves a fraction $\exp(-0.693/0.007) \approx 10^{-43}$ after one day, and 0.05 days leaves about 10^{-6}. Osu_1hru has 1 in column 7 of `parameters.bsn`, so the half-lives are adjusted for temperature (except in reservoirs), which lengthens them at temperatures below 25 °C but not enough to change the result. Any of these pesticides that reached channel water, bed sediment, a reservoir or an aquifer would be almost entirely gone within a day. Osu_1hru has no `constituents.cs`, so in that model no pesticide is simulated and the file is read without effect.

pest_hru.ini

Initial pesticide in the soil and on plants of HRUs. An HRU selects a record by name in the `pest` column of `soil_plant.ini`.

The file is read in two passes that expect different layouts:

- The first pass counts records. It expects a title line, then for each record a header line, a name line and one line per pesticide.
- The second pass reads the values. It expects a title line and one header line, then for each record a name line and one line per pesticide.

A file with one record satisfies both. With more than one record the second pass reads the header line of record 2 as its name and the values that follow are shifted, so use one record.

Each pesticide line has three values, one line per pesticide in the order of `constituents.cs`:

Value	Units	Description
name	-	Pesticide name, for readability. Not checked.
soil	mg/kg	Concentration in the soil. The same concentration is given to every soil layer.
plant	kg/ha	Pesticide on the foliage. It is shared among the plants by leaf area; if there is no leaf area at the start, nothing is put on the foliage.

pest_water.ini

Initial pesticide in channels, reservoirs, wetlands and aquifers. Channels, reservoirs and wetlands select a record by name in their initial condition files (`initial.cha`, `initial.res`), and aquifers in `initial.aqu`.

Line 1 is a title and line 2 a header. Each record starts with a name line. The values follow as one list: a label, then the water value of each pesticide, then the bed sediment value of each pesticide, in the order of `constituents.cs`. Line breaks within the list are free, but the counting pass expects two lines per pesticide after the name line. For two pesticides:

```
pest_water.ini: initial pesticide in water
name
ch_init
atrazine  0.002
0.001
0.05
0.02
```

Here `atrazine` is the label, the water values are 0.002 and 0.001, and the bed sediment values 0.05 and 0.02.

Value	Units	Description
water	mg/L	Concentration in the water. Channels multiply it by the initial channel water volume. Aquifers read it as mg/kg of aquifer material over the aquifer depth with a bulk density of 2.0 t/m³.
benthic	mg/kg	Concentration in the active bed sediment layer. Channels multiply it by the volume of the active bed layer.

Reservoirs use both values as given, without a conversion by volume.

pest_metabolite.pes

The degradation products of each parent pesticide. SWAT+ reads it if a file of this name exists in the run directory.

Line 1 is a title and line 2 a header. For each parent there is one line with the parent name and the number of daughters, followed by one line per daughter:

Value	Units	Description
`name`	-	Daughter pesticide name. Must also be listed in `constituents.cs` and `pesticide.pes`.
`foliar_fr`	fraction	Fraction of the parent decayed on foliage that becomes this daughter.
`soil_fr`	fraction	Fraction of the parent decayed in the soil that becomes this daughter. Also used in reservoirs and aquifers.
`aq_fr`	fraction	Fraction of the parent decayed in channel water that becomes this daughter.
`ben_fr`	fraction	Fraction of the parent decayed in channel bed sediment that becomes this daughter.

The parent name must match a row of `pesticide.pes`. If it does not, its daughter lines are not skipped and the rest of the file is read out of step.

```
pest_metabolite.pes: metabolites
parent        num_metab
aatrex        1
deethyl_atr   0.2   0.3   0.1   0.1
```

Atrazine (`aatrex`) has one daughter. 30% of the atrazine decayed in the soil, adjusted by the ratio of molecular weights, becomes `deethyl_atr`.

Related files

- `constituents.cs`: the list of pesticides simulated.
- `soil_plant.ini`: links HRUs to `pest_hru.ini` records.
- `initial.cha`, `initial.res`, `initial.aqu`: link channels, reservoirs and aquifers to `pest_water.ini` records.
- `chem_app.ops` and `management.sch`: pesticide applications.
- `parameters.bsn`: `pest_perc` and the temperature-dependent decay switch in column 7.
- Pathogens and heavy metals use the same initial condition layout.

Sources

- Readers: `pest_parm_read.f90`, `pest_hru_aqu_read.f90`, `pest_cha_res_read.f90`, `pest_metabolite_read.f90`
- Types: `type pesticide_db`, `type pesticide_cp`, `type daughter_decay_fractions` in `pesticide_data_module.f90`; `type cs_soil_init_concentrations` and `type cs_water_init_concentrations` in `constituent_mass_module.f90`
- Pesticide list crosswalk: `constit_db_read.f90`
- Initial conditions: `pesticide_init.f90`, `sd_hydsed_init.f90`, `res_initial.f90`, `aqu_read_init_cs.f90`

- Landscape processes: pest_decay.f90, pest_washp.f90, pest_lch.f90, pest_pl_up.f90, mgt_harvbiomass.f90, mgt_harvtuber.f90
- Temperature adjustment: pest_tempdecay_module.f90
- Channels, reservoirs and aquifers: ch_rtpest.f90, res_pest.f90, aqu_1d_control.f90
- Switch in parameters.bsn: bsn_prm%temp_decay in basin_module.f90
- Calibration parameters: cal_parm_select.f90

Column	Fortran variable
name	pestdb%name
soil_ads	pestdb%koc
frac_wash	pestdb%washoff
hl_foliage	pestdb%foliar_hlife
hl_soil	pestdb%soil_hlife
aq_hlife, aq_volat, aq_resus, aq_settle	pestdb% same names
ben_act_dep, ben_bury, ben_hlife	pestdb% same names
pl_uptake, harvest_sink, resistance_n	pestdb% same names
description	pestdb%descrip
parameters.bsn column 7 (exp_sed)	bsn_prm%temp_decay
pest_hru.ini record	pest_soil_ini%name, %soil, %plt
pest_water.ini record	pest_init_name, pest_water_ini%water, %benthic
pest_metabolite.pes daughter	pestcp%daughter

Purpose

SWAT+ can follow pathogens (bacteria and other microorganisms) listed in constituents.cs. Three files describe them:

File	Named in	Content
pathogens.pth	hru_parm_db line of file.cio (default pathogens.pth)	Die-off, growth and transport properties of each pathogen.
path_hru.ini	init line of file.cio	Initial pathogens in soils and on plants.
path_water.ini	init line of file.cio	Initial pathogens in channels, reservoirs and aquifers.

Every pathogen named in constituents.cs should have a row in pathogens.pth with the same name.

The pathogen code in 62.0.1 does not yet work as a complete model. Several columns are read but not used, and the landscape and channel routines have the limitations listed in Behaviour in 62.0.1. Read that section before using pathogen results.

pathogens.pth

Free format, whitespace-separated. Line 1 is a title and line 2 is the column header; both are skipped. Each later line is one pathogen with the 18 values below in this order. The reader does not match columns by header name.

#	Column	Units	Used in 62.0.1	Description
1	name	-	yes	Pathogen name, up to 16 characters.
2	do_soln	1/day	yes	Die-off factor in soil solution at 20 °C.
3	gr_soln	1/day	yes	Growth factor in soil solution at 20 °C.
4	do_sorb	1/day	no	Die-off factor for pathogens sorbed to soil particles at 20 °C.

#	Column	Units	Used in 62.0.1	Description
5	`gr_sorb`	1/day	no	Growth factor for pathogens sorbed to soil particles at 20 °C.
6	`kd`	-	yes	Partition coefficient between solution and sorbed phases.
7	`t_adj`	-	yes	Temperature adjustment factor for die-off and growth.
8	`washoff`	fraction	yes	Fraction of the pathogens on foliage washed off by a rainfall event.
9	`do_plnt`	1/day	yes	Die-off factor on foliage at 20 °C.
10	`gr_plnt`	1/day	yes	Growth factor on foliage at 20 °C.
11	`fr_manure`	fraction	no	Fraction of manure containing active colony forming units.
12	`perco`	-	yes	Percolation coefficient: ratio of the concentration in percolate to the concentration in the surface layer solution.
13	`det_thrshd`	cfu/m²	no	Detection threshold below which soil pathogens would be set to zero.
14	`do_stream`	1/day	yes	Die-off factor in streams at 20 °C.
15	`gr_stream`	1/day	no	Growth factor in streams at 20 °C.
16	`do_res`	1/day	no	Die-off factor in reservoirs at 20 °C.
17	`gr_res`	1/day	no	Growth factor in reservoirs at 20 °C.
18	`conc_min`	cfu/m²	yes	Minimum pathogen level on foliage and in the soil surface layer.

Die-off and growth

On foliage and in the top soil layer the pathogen count N changes each day by the net die-off rate, adjusted for the mean air temperature T:

$$N_d = N_{d-1}\,\exp\!\left(-(k_{die} - k_{grow})\,\theta^{\,T-20}\right) - c_{min}$$

where k_{die} and k_{grow} are `do_plnt` and `gr_plnt` on foliage, or `do_soln` and `gr_soln` in the soil, θ is `t_adj` and c_{min} is `conc_min`. The result is not allowed to fall below `conc_min`.

On a day with at least 2.54 mm of precipitation, `washoff` of the pathogens on each plant's foliage moves to the top soil layer.

path_hru.ini

Initial pathogens in the soil and on plants of HRUs. An HRU selects a record by name in the `path` column of `soil_plant.ini`.

Line 1 is a title. Each record has a header line, a name line, and then the values as one list: a label, the soil value of each pathogen, then the plant value of each pathogen, in the order of `constituents.cs`. Line breaks within the list are free, but the list must take one line per pathogen so that the records are counted correctly. For two pathogens:

```
path_hru.ini: initial pathogens
name
hru_init
ecoli  1000.  500.
50.  10.
```

Here `ecoli` is the label, the soil values are 1000 and 500 and the plant values 50 and 10.

Value	Units	Description
soil	cfu/m²	Pathogens in the top soil layer.
plant	cfu/m²	Pathogens on the foliage.

path_water.ini

Initial pathogens in channels, reservoirs and aquifers. Reservoirs and channels select a record by name in their initial condition files (initial.res, initial.cha), and aquifers in initial.aqu.

Line 1 is a title and line 2 a header. The record has a name line and then the values as one list: a label, the water value of each pathogen, then the bed sediment value of each pathogen. The list must take two lines per pathogen. Use one record (see below).

Value	Units	Description
water	-	Pathogens in the water. The value is used as given, without conversion, so give it in the units of the pathogen loads in the model.
benthic	-	Pathogens in the bed sediment, used as given.

Behaviour in 62.0.1

- Columns `do_sorb`, `gr_sorb`, `fr_manure`, `det_thrshd`, `gr_stream`, `do_res` and `gr_res` are not used.
- The landscape routines (die-off, wash-off, runoff and percolation) do not look up each pathogen's row by name. They use the row of `pathogens.pth` whose position equals the position of the HRU's record in `path_hru.ini`, and use that one row for every pathogen.
- Each day the runoff step removes all pathogens left in the top soil layer after percolation and reports them as lost in surface runoff, whether or not there is runoff that day.
- Initial soil pathogens from `path_hru.ini` are set to zero again for any soil with more than one layer. Initial plant pathogens are written to the balance output but not put on the foliage.
- `path_water.ini` is read with a header line before each record in the reading pass and without one in the counting pass, so only a file with one record is read correctly.
- Channels take their initial pathogens from the `path_water.ini` record in the position of the channel's pesticide initial record, not its pathogen record. Aquifers take the name match from `path_water.ini` but the values from the `path_hru.ini` soil column of the record in the same position. Reservoirs use their `path_water.ini` record.
- The channel die-off step does not identify the channel being routed, so `do_stream` does not act on the routed water.

Related files

- constituents.cs: the list of pathogens simulated.
- soil_plant.ini: links HRUs to `path_hru.ini` records.
- initial.cha, initial.res, initial.aqu: link channels, reservoirs and aquifers to `path_water.ini` records.
- Pesticides and heavy metals use a similar initial condition layout.

Sources

- Readers: path_parm_read.f90, path_hru_aqu_read.f90, path_cha_res_read.f90
- Types: `type pathogen_db` in pathogen_data_module.f90; `type cs_soil_init_concentrations` and `type cs_water_init_concentrations` in constituent_mass_module.f90
- Pathogen list crosswalk: constit_db_read.f90; `soil_plant.ini` link: hru_read.f90
- Initial conditions: pathogen_init.f90, sd_hydsed_init.f90, res_initial.f90, aqu_read_init_cs.f90
- Landscape processes: path_ls_swrouting.f90, path_ls_runoff.f90, path_ls_process.f90, called from hru_control.f90
- Channel: ch_rtpath.f90; temperature function: theta.f90

Column	Fortran variable
name	path_db%pathnm
other pathogens.pth columns	path_db% same names
path_hru.ini record	path_soil_ini%name, %soil, %plt
path_water.ini record	path_init_name, path_water_ini%water, %benthic

Purpose

Heavy metals listed in constituents.cs can be routed through the network as loads. In 62.0.1 they enter the model only through export coefficients (exco_hmet.exc) and delivery ratios (dr_hmet.del) and are routed with the water. There is no metal property database, and soils, plants, channels and reservoirs have no metal processes.

file.cio has slots for four metal files:

File	Named in	Status in 62.0.1
metals.mtl	hru_parm_db line of file.cio	No reader. The file is not opened.
hmet_hru.ini	init line of file.cio	Read, but the values are not used.
hmet_water.ini	init line of file.cio	No reader. The file is not opened.
exco_hmet.exc, dr_hmet.del	export coefficient and delivery ratio lines of file.cio	Read. Metal loads from routing objects.

Set metals.mtl, hmet_hru.ini and hmet_water.ini to null in file.cio unless a later revision uses them. The hmet columns of initial.cha and initial.aqu name records of these files and have no effect.

hmet_hru.ini

Initial heavy metals in the soil and on plants of HRUs. The reader expects this layout:

- Line 1: title.
- For each record: a header line, a name line, then for each metal in the order of constituents.cs two lines, label soil and then label plant.

The counting pass expects one line per metal instead of two, so files with more than one record are miscounted. The values (soil and plant concentrations in mg/kg) are stored but no part of the model uses them.

Related files

- constituents.cs: the list of metals.
- file.cio: the file name slots.
- Pesticides and pathogens: the constituents with full process models.

Sources

- Reader: hmet_hru_aqu_read.f90, called from proc_read.f90
- Type: type cs_soil_init_concentrations (hmet_soil_ini) in constituent_mass_module.f90
- File name slots: in_parmdb%hmetcom_db, in_init%hmet_soil, in_init%hmet_water in input_file_module.f90
- Metal list: constit_db_read.f90
- Metal loads: exco_read_hmet.f90, dr_read_hmet.f90

Item	Fortran variable
hmet_hru.ini record name	hmet_soil_ini%name
soil value	hmet_soil_ini%soil
plant value	hmet_soil_ini%plt
metals.mtl slot	in_parmdb%hmetcom_db

Purpose

gwflow is a cell-based groundwater model. When it is active it replaces the lumped aquifers, and aquifer.aqu, initial.aqu and the aquifer connect file are not used. The aquifer is divided into cells, and each day SWAT+ solves a water balance for every cell with Darcy flow between neighbouring cells. Cells exchange water with channels and receive recharge from HRUs or landscape units. Optional processes add groundwater ET, groundwater flow into the soil profile, saturation excess flow, tile drainage, pumping, exchange with reservoirs, wetlands, floodplains and canals, recharge ponds, phreatophyte uptake, solute transport and heat transport.

The inputs are spread over a set of small files, most of them with the .gw extension, one file for each part of the model. The files are not listed in file.cio; SWAT+ looks for them by name in the run directory. Units are metres, days and square metres unless a column says otherwise.

Activation

Set gwflow to 1 in codes.bsn. At start-up SWAT+ then looks for chancell.gw:

- If chancell.gw exists, gwflow is active. Unless object.cnt already gives a gwflow object count, SWAT+ counts the channel-cell rows in chancell.gw, uses that count as the number of gwflow objects, removes the aquifer objects from the object total, and reads the channel-cell connections from gwflow.con.
- If chancell.gw is missing or empty, gwflow is reset to 0 and the run uses the lumped aquifers.

When gwflow is active SWAT+ writes a log of what it read and which options it switched on or off to gwflow_record. Check this file first when a gwflow run does not behave as expected.

When the count comes from chancell.gw, SWAT+ skips three lines before it starts counting, while the file has two lines before the first data row. The first channel-cell row is therefore not counted and the count is one less than the number of rows. chancell.gw is then read for that many rows, so the last row of the file is not used. Giving the gwflow count in object.cnt avoids this.

File overview

File	Required	Read when	Purpose
chancell.gw	yes	always	Channel-cell links, streambed zone, depth zone and observation flag
gwflow.con	yes	always	Connect file for the channel-cell objects
codes.gw	yes	always	Grid size, process switches, time step and output switches
zones.gw	yes	always	Aquifer and streambed parameters by zone
cells.gw	yes	always	Properties and initial state of every cell
cellcon.gw	yes	always	Neighbours of every cell
outputs.gw	yes	always	Observation wells and output times
hrucell.gw	one of the two	conn_type 1, or 2 without lsucell.gw	HRU-cell overlaps for recharge
lsucell.gw	one of the two	conn_type 2, or 1 without hrucell.gw	Landscape unit-cell overlaps for recharge
chan_depth.gw	no	file exists	Daily channel water depth per depth zone
pumpex.gw	no	pumpex is 1 and file exists	Specified pumping from cells
hru_pump.gw	no	file exists	HRUs to report irrigation pumping for
tile.gw	no	tile is 1 and file exists	Tile drain parameters and tile cells
rescell.gw	no	res is 1 and file exists	Reservoir-cell links
gwflow.wetland	no	wet is 1 and file exists	Wetland bed thickness
floodplain.gw	no	fp is 1 and file exists	Floodplain-cell links

File	Required	Read when	Purpose
gwflow_canal.con	no	canal is 1 and file exists	Canal-cell links
phreato.gw, phreato_cell.gw	no	phreato.gw exists	Phreatophyte ET curve and cells
tvheads.gw	no	file exists	Time-varying heads for boundary cells
solute.gw	no	solute is 1 and file exists	Solute transport settings and solute parameters
cell_sol.gw	no	solute transport on and file exists	Initial solute concentration per cell
minerals.gw	no	solute transport on, salts simulated, file exists	Salt mineral fractions per cell
ponds.gw, pond_cell.gw	no	ponds.gw exists	Recharge ponds and their cells
pond_div.gw	no	ponds active and file exists	Daily diversion into each pond
sw_group.gw	no	file exists	Groups of channel cells for exchange output
transit.gw	no	structured grid and file exists	Cells for transit-time output

When a switch in codes.gw is on and its file is missing, SWAT+ notes it in gwflow_record and the process is not simulated. For tile drainage, floodplain exchange and solute transport the switch is also turned off for the rest of the run.

Format conventions

- Values are separated by spaces. Column widths do not matter.
- Line 1 of every file is a title line and is skipped. Most tables have a column header on line 2, also skipped, and data from line 3. Files with a different layout are described in the section for that file.
- Cell numbers run from 1 to ncell, in the order of cells.gw, and every file that names a cell uses these numbers.
- Channel numbers are the numbers of the channels in the channel connect file (chandeg.con). Reservoir numbers follow the reservoir connect file.
- In cells.gw and zones.gw, an optional value may be given as null, which means it is not set.

codes.gw

Grid size, process switches and output switches. Line 1 is a title. Line 2 is a row of key names and line 3 is a row of values, one value under each key. SWAT+ pairs keys and values by position and recognises the keys by name, so the order of the columns does not matter and a missing key keeps its default (0, or empty for grid_type).

Key	Units	Description
grid_type	-	structured (regular grid) or unstructured. Both use the same cell numbering.
ncell	-	Number of cells. cells.gw and cellcon.gw must have this many rows.
cell_size	m	Cell size of a structured grid. Read but not used; cell areas come from cells.gw.
n_rows, n_cols	-	Rows and columns of a structured grid. Used only by the structured-grid sections of tile.gw and minerals.gw.
bc_type	-	Default boundary condition for boundary cells: 1 = specified head, 2 = no flow. A cell can override it in cells.gw.
conn_type	-	How recharge reaches the cells: 1 = from HRUs (hrucell.gw), 2 = from landscape units (lsucell.gw). Any other value stops the run.
gw_soil	0/1	Groundwater transfer into the soil profile when the water table rises into it. Only with HRU connection.
satx	0/1	Saturation excess flow: groundwater above the ground surface goes to the channel nearest to the cell.
pumpex	0/1	Specified pumping from pumpex.gw.

Key	Units	Description
tile	0/1	Tile drainage from tile.gw.
res	0/1	Exchange with reservoirs from rescell.gw.
wet	0/1	Exchange with wetlands. Only with HRU connection.
fp	0/1	Exchange with floodplains from floodplain.gw.
canal	0/1	Seepage from canals in gwflow_canal.con.
solute	0/1	Solute transport from solute.gw.
heat	0/1	Heat transport.
time_step	day	Flow time step. The day is split into int(1/time_step) steps, so use 1 or a fraction of a day such as 0.5.
write_day, write_mon, write_yr, write_aa	0/1	Write the gwflow water balance files for each interval.
river_thresh	m	Distance subtracted from every channel-bed elevation in chancell.gw before the channel exchange is calculated. 0 uses the elevations as given.

With LSU connection, SWAT+ turns gw_soil and wet off and says so in gwflow_record.

Output switches in print.prt

The write_* keys are defaults. print.prt can list gwflow rows, and a row that is present replaces the corresponding setting before any gwflow output file is opened:

print.prt row	Controls	Without the row
gwflow_wb	Water balance files, by interval (replaces write_day to write_aa)	write_* keys of codes.gw
gwflow_obs	Observation well files, by interval	all four intervals written
gwflow_pump	HRU irrigation pumping output (on if any interval is y)	on
gwflow_flux	Canal, pond, tile, channel-group and channel-observation output (on if the daily or yearly column is y)	on
gwflow_heat	Heat balance files: daily, yearly and average annual columns	same intervals as the water balance
gwflow_solute	Solute balance files, by interval	same intervals as the water balance

The daily channel hydrograph separation file gwflow_chan_hydsep_day.txt is written when daily water balance output is on. The files are described in the gwflow output reference.

zones.gw

Aquifer and streambed parameters, one row per zone. Line 1 is a title and line 2 a column header. The zone number of a row is its position in the file (the first data row is zone 1); the zone_id column is not read.

#	Column	Units	Description
1	zone_id	-	Zone number. Not read; the row position decides.
2	aquifer_K	m/day	Horizontal hydraulic conductivity of the aquifer.
3	aquifer_Sy	fraction	Specific yield of the aquifer.
4	streambed_K	m/day	Hydraulic conductivity of the streambed, for channel cells that use this zone.
5	streambed_thick	m	Streambed thickness, for channel cells that use this zone.
6	thermal_K	J/day/m/°C	Thermal conductivity of the aquifer, used by heat transport. Optional; null or missing means 0.

Cells name a zone for conductivity and a zone for specific yield in `cells.gw`. Channel cells name a zone for streambed properties in `chancell.gw`.

cells.gw

One row per cell, in cell order. Line 1 is a title and line 2 a column header. Row i must have `id` equal to i, or the run stops with a message in `gwflow_record`. The first 14 columns are required. Columns 15 to 23 are optional: a row can end after column 14, and `null` in any of them means the value is not set.

#	Column	Units	Description
1	`id`	-	Cell number, 1 to `ncell`, equal to the row number.
2	`name`	-	Cell name, used in output files.
3	`gis_id`	-	GIS identifier of the cell, written to output files.
4	`status`	-	0 = inactive, 1 = active, 2 = boundary. Boundary cells keep their head and are treated by `bc_type`.
5	`elev`	m	Ground surface elevation.
6	`thck`	m	Aquifer thickness. The aquifer bottom is `elev - thck`.
7	`K_zone`	-	Zone (row of `zones.gw`) for hydraulic conductivity. Also used for `thermal_K`.
8	`Sy_zone`	-	Zone (row of `zones.gw`) for specific yield.
9	`delay`	day	Recharge delay through the unsaturated zone.
10	`exdp`	m	Extinction depth for groundwater ET, measured down from the ground surface. 0 turns groundwater ET off for the cell.
11	`init`	m	Initial groundwater head (elevation). A value below the aquifer bottom is raised to the bottom.
12	`x`	m	x coordinate of the cell centre.
13	`y`	m	y coordinate of the cell centre.
14	`area`	m²	Cell area.
15	`strK`	m/day	Streambed conductivity for this cell, in place of the zone value.
16	`strthick`	m	Streambed thickness for this cell, in place of the zone value.
17	`bc_type`	-	Boundary condition for this cell, in place of the default in `codes.gw`.
18	`tile_depth`	m	Tile depth for this cell, in place of the value in `tile.gw`.
19	`tile_area`	m²	Tile drainage area for this cell, in place of the value in `tile.gw`.
20	`tile_K`	1/day	Tile conductance for this cell, in place of the value in `tile.gw`.
21	`row`	-	Grid row of a structured cell (0 for unstructured). Written to output only.
22	`col`	-	Grid column of a structured cell (0 for unstructured). Written to output only.
23	`init_temp`	°C	Initial groundwater temperature, used by heat transport.

The x and y coordinates are used to find, for each cell, the nearest channel cell (for saturation excess flow and tile drainage) and, for each boundary cell, the nearest active cell, which receives the recharge that falls on the boundary cell.

cellcon.gw

The neighbours of every cell, which define where lateral flow can happen. Line 1 is a title and line 2 a column header. One row per cell, in cell order; the first value must equal the row number.

#	Column	Description
1	`cell_id`	Cell number, equal to the row number.
2	`neighbor_tot`	Number of neighbours.
3+	`neighbor_ids`	`neighbor_tot` cell numbers.

Flow between a cell and a neighbour uses Darcy's law with the harmonic mean of the two conductivities. The face width is the square root of the smaller of the two cell areas. No flow passes to an inactive neighbour or to a boundary neighbour with `bc_type` 2.

outputs.gw

Output settings, one per row. Line 1 is a title and line 2 a column header. Each row starts with a category and a value; anything after the value (SWAT+ Editor writes a note) is ignored. Rows can be in any order.

`category`	`value`	Effect
`observation_cell`	cell number	Adds an observation well. Head, water table depth, temperature, nitrate and phosphorus at each well are written to `gwflow_obs_*.txt`. One row per well.
`head_output_time`	date as `YYYYDDD`	A date for a full head output. In 62.0.1 these dates are read but produce no output.
`detail_debug_cell`	cell number or 0	Read but not used.

chancell.gw

The cells crossed by channels, one row per channel-cell pair. Line 1 is a title and line 2 a column header. A cell crossed by two channels appears on two rows.

#	Column	Units	Description
1	`cell_id`	-	Cell crossed by the channel.
2	`elev_m`	m	Channel bed elevation in the cell.
3	`channel`	-	Channel number in `chandeg.con`.
4	`riv_length_m`	m	Length of the channel within the cell.
5	`zone`	-	Zone (row of `zones.gw`) for streambed conductivity and thickness.
6	`dep_zone`	-	Depth zone: column of `chan_depth.gw` that gives the daily water depth. Optional, default 0.
7	`obs`	0/1	1 writes daily channel exchange for this cell to `gwflow_chan_obs_flow_day.txt`. Optional, default 0.

Channel exchange

Exchange is calculated each day for every active channel cell with Darcy's law across the streambed. With bed elevation z_b (`elev_m` minus `river_thresh`), water depth d, stage $z_s = z_b + d$, groundwater head h, streambed conductivity K_b and thickness m_b, and flow area $A = w\,L$ (channel width times `riv_length_m`):

$$Q = \frac{K_b}{m_b}\,A \times \begin{cases} d & h < z_b \\ z_s - h & z_b < h < z_s \\ -(h - z_s) & h > z_s \end{cases}$$

A positive Q (m^3/day) is seepage from the channel into the aquifer and is limited to the water in the channel. A negative Q is groundwater discharge to the channel and is limited to the water stored in the cell. The width

w is the channel width from the channel data. The depth d is the channel depth from the channel data, unless `chan_depth.gw` is present, in which case it is the day's depth for the cell's depth zone.

gwflow.con

The connect file for the channel-cell objects. It has the usual connect-file layout (title line, header line, then one row per object); see connect files. Each row is one channel cell, with the cell number in the `CELL` column and one outflow to the channel (`sdc`, the channel number, `tot`, 1.0).

hrucell.gw

Overlaps between HRUs and cells, used to pass recharge from HRUs to cells and, with `gw_soil`, groundwater back into the soil. Line 1 is a title and line 2 a column header.

#	Column	Units	Description
1	`hru`	-	HRU number.
2	`area_m2`	m²	Total area of the HRU.
3	`cell_id`	-	Cell that overlaps the HRU.
4	`overlap_m2`	m²	Area of the overlap.

Rows must be grouped by HRU and the HRUs must be in increasing order, since SWAT+ reads the rows for HRU 1, then HRU 2, and so on. A row with an HRU number outside 1 to the number of HRUs is skipped. Each HRU's recharge is split between its cells by `overlap_m2 / area_m2`.

The HRU's recharge delay is taken from the `delay` of its cells in `cells.gw`. When an HRU overlaps several cells, the delay of the last cell listed for that HRU is used.

lsucell.gw

Overlaps between landscape units and cells, used in place of `hrucell.gw` with LSU connection. The layout differs from the other tables:

Line	Content
1	Title
2	Number of landscape units in the model
3	Number n of landscape units connected to cells
next n lines	One landscape unit number per line
next 2 lines	Headers, skipped
rest	One row per overlap: landscape unit number, landscape unit area (m²), cell number, overlap area (m²)

The overlap rows must be grouped by landscape unit in increasing order. With LSU connection every HRU uses a recharge delay based on the mean `delay` of all cells.

SWAT+ Editor writes `lsucell.gw` as a title followed by rows of cell, landscape unit and area, which is a different layout from the one the model reads.

chan_depth.gw

Daily channel water depth for each depth zone. Line 1 is a title and line 2 a column header. Then one row per simulated day, in order: day of year, year, and one depth (m) for each depth zone 1 to N, where N is the largest `dep_zone` in `chancell.gw`. One row is read per day in sequence; the day and year values are not checked.

pumpex.gw

Specified pumping from cells, read when `pumpex` is 1. Line 1 is a title and line 2 a column header. One row per pumping period.

#	Column	Units	Description
1	`name`	-	Label, not used.
2	`cell_id`	-	Pumped cell.
3	`rate`	m^3/day	Pumping rate during the period.
4	`yr_start`	year	First year of the period.
5	`dy_start`	day of year	First day of the period.
6	`yr_end`	year	Last year of the period.
7	`dy_end`	day of year	Last day of the period.

Consecutive rows with the same `cell_id` form one pump with several periods (up to 1000). Rows for one cell that are not next to each other create separate pumps. Each date is converted to a day count from the start of the simulation as (year − first year) × 365 + day, without leap days. A year before the first simulated year is counted as day 1.

hru_pump.gw

HRUs whose groundwater irrigation pumping is written each day to `gwflow_cell_wb_ppag_obs_day.txt`. Line 1 is a title, line 2 is the number of HRUs, and then one HRU number per line.

tile.gw

Tile drainage, read when `tile` is 1. Each value is on a separate line:

Line	Content
1	Title
2	Tile depth below the ground surface (m), for all cells
3	Tile drainage area (m^2), for all cells
4	Tile conductance (1/day), for all cells
5	Group flag: 1 if tile cell groups follow, 0 if not
groups	If the flag is 1: the number of groups, then for each group a name line, the number of cells, and one cell number per line
next	Header line, skipped
rest	Tile flag per cell (1 = tile drained, 0 = not). Unstructured grid: one value per line for all `ncell` cells. Structured grid: `n_rows` lines of `n_cols` values.

Columns 18 to 20 of `cells.gw` replace the depth, area and conductance for single cells. Each day, an active tile cell whose head h is above the tile elevation z_t (`elev - tile_depth`) drains

$$Q_{tile} = A_{tile}\, K_{tile}\, (h - z_t)$$

in m^3/day to the channel nearest to the cell. With groups, the daily tile flow of each group is written to `gwflow_tile_group_day.txt`.

The structured-grid form of the tile flags depends on grid arrays that 62.0.1 does not set up, so give the tile flags in the unstructured form. SWAT+ Editor writes the three tile values on one line followed by a list of tile cells, which is a different layout from the one the model reads.

rescell.gw

Reservoir-cell links, read when `res` is 1.

Line	Content
1, 2	Title and a second line, skipped
3	Reservoir bed thickness (m), for all reservoirs
4	Reservoir bed conductivity (m/day), for all reservoirs
5	Number of rows that follow
6	Header, skipped
rest	`cell_id`, `res_id`, `res_stage`

`res_id` is the reservoir number; a row with 0 is skipped. Exchange uses the ground elevation of the cell from `cells.gw` as the reservoir bed elevation. `res_stage` is read but not used.

gwflow.wetland

Wetland bed thickness, read when `wet` is 1 and HRUs are connected to cells. Lines 1 and 2 are skipped. Each later row has a wetland name and a bed thickness (m). The digits in the name give the HRU number, so `wet019` sets the bed thickness of the wetland in HRU 19. Wetlands not listed use 0.25 m.

SWAT+ Editor writes this information to a file named `wetland.gw`, which the model does not read, so wetlands then use 0.25 m.

floodplain.gw

Floodplain-cell links, read when `fp` is 1. Line 1 is a title, line 2 the number of rows, line 3 a header, then one row per cell: cell number, channel number, floodplain bed conductivity (m/day) and area of the cell inside the floodplain (m^2). The area is limited to the cell area.

gwflow_canal.con

Canal-cell links, read when `canal` is 1. The canal dimensions, bed properties and active days come from the canal records in `water_canal.wal`. Line 1 is a title. Each later row describes one canal:

```
canal_id  n  cell_1 length_1 elev_1  cell_2 length_2 elev_2  ...
```

with `n` triplets of cell number, canal length in the cell (m) and canal bed elevation in the cell (m). A canal with a diversion source is treated as a diversion canal. A canal with no diversion and a start day above 0 is treated as a canal fed from outside the model. Any other canal is linked to a channel through water allocation.

phreato.gw and phreato_cell.gw

Groundwater uptake by phreatophytes. Both files are needed when `phreato.gw` exists; there is no switch in `codes.gw`.

`phreato.gw` is the curve of uptake rate against water table depth. Line 1 is a title and line 2 a header, then one point per row: depth to the water table (m) and ET rate (m/day), in order of increasing depth.

`phreato_cell.gw` lists the cells with phreatophytes. Line 1 is a title and line 2 a header, then one row per cell: cell number and area covered by phreatophytes (m^2).

tvheads.gw

Line 1 is a title and line 2 a header. Each later row gives a cell number and one head per simulated year. In 62.0.1 the file is read but the heads are not applied, so boundary cells keep their initial heads.

solute.gw

Solute transport settings, read when `solute` is 1.

Line	Content
1, 2	Title and a second line, skipped
3	Number of transport time steps per flow time step
4	Longitudinal dispersivity (m)
5	Skipped (SWAT+ Editor writes the number of solutes here)
rest	One row per solute: name, retardation factor, reaction rate constant (1/day), concentration in canal water from outside the model (g/m^3)

The solutes and their order are set by the model: nitrate and phosphorus first, then the salt ions and then the other constituents listed in `constituents.cs`. The file must have one row per solute in that order; the name column is not checked. Further values on a row are ignored. SWAT+ Editor writes an initial concentration as a fifth value, which the model does not read; initial values come from `cell_sol.gw`.

The retardation factor divides the transport time step, so 1 means no sorption. The reaction term adds concentration × groundwater volume × rate constant (g/day) to the cell, so a negative rate constant gives first-order decay.

When constituents are simulated, SWAT+ also tries to read per-cell reaction groups and shale maps after `cell_sol.gw`. In 62.0.1 no file is open at that point, so gwflow solute transport cannot run together with the constituent module.

cell_sol.gw

Initial solute concentration in every cell. Line 1 is a title and line 2 a header, then one row per cell in cell order: cell number and one concentration (g/m^3) per solute, in the order of `solute.gw`. Without this file all concentrations start at 0.

minerals.gw

Salt mineral fractions per cell, read when solute transport is on, salt ions are simulated and the file exists. Line 1 is a title, line 2 the number of minerals, line 3 a header, then for an unstructured grid one row per cell with one fraction per mineral. The structured-grid form is read but the values are not stored. SWAT+ Editor does not write this file.

ponds.gw, pond_cell.gw and pond_div.gw

Recharge ponds. They are active when `ponds.gw` exists, and `pond_cell.gw` must then exist too.

`ponds.gw` has a title line, a header line and one row per pond:

#	Column	Units	Description
1	`pond_id`	-	Pond number, matched in `pond_cell.gw`.
2	`area`	m^2	Pond area.
3	`chan`	-	Channel that supplies the diverted water. 0 if none.
4	`canal`	-	Canal that supplies the diverted water, used when `chan` is 0. 0 if none.
5	`unl`	0/1	1 = the diverted water comes from outside the model, used when `chan` and `canal` are 0.
6	`bed_k`	m/day	Conductivity of the pond bed.
7	`wsta`	-	Weather station for rainfall and evaporation.
8	`evap_co`	-	Evaporation coefficient. Daily evaporation is `evap_co` × potential ET of the station × pond area.
9	`start_yr`	year	Year the pond starts operating.

#	Column	Units	Description
10	`start_mo`	month	Month the pond starts operating.
11	`start_day`	day	Day of the month the pond starts operating.
12+	`unl_conc_1 …`	g/m³	Concentration of each solute in water from outside the model (none when solute transport is off).

The start date is converted to a day count with 365-day years and fixed month lengths; a start year before the first simulated year means day 1.

`pond_cell.gw` has a title line, a header line and one row per pond-cell connection: pond number, cell number and connection area (m²). The rows for a pond give its cells.

`pond_div.gw` has a title line, a header line and one row per simulated day: year, month, day and the volume diverted into each pond that day (m³), in the order of `ponds.gw`. One row is read per day in sequence, so the file needs a row for every simulated day; the dates are not checked. Without the file, no water is diverted into the ponds.

sw_group.gw

Groups of channel cells whose daily channel exchange is summed and written to `gwflow_gwsw_group_day.txt`. Line 1 is a title and line 2 a header. Each later row gives a group number, the number of cells n in the group and n cell numbers.

transit.gw

Cells for groundwater transit-time output. Read only for a structured grid. Line 1 is a title, line 2 a header, line 3 the number of cells, then one cell number per line. SWAT+ Editor does not write this file.

Example

Start of a set of inputs for a structured grid of 1454 cells, with illustrative values.

`codes.gw` (abbreviated):

```
codes.gw: written by SWAT+ editor
     grid_type         ncell     cell_size        n_rows        n_cols   ...      time_step     write_aa  i
    structured          1454       5000.00            52            53   ...           1.00            1
```

`zones.gw`:

```
zones.gw: written by SWAT+ editor
   zone_id     aquifer_K    aquifer_Sy   streambed_K streambed_thick     thermal_K
         1       0.50000       0.20000       0.00500         0.50000       0.00000
         2       2.00000       0.20000       0.00500         0.50000       0.00000
```

`cells.gw` (first row, abbreviated):

```
 id  name      gis_id  status   elev   thck K_zone Sy_zone  delay  exdp   init ...  row  col init_temp
  1  cell0001       1       2 395.00  34.99      1       1   0.00  1.00 390.00 ...   52    1      null
```

Cell 1 is a boundary cell. Its aquifer bottom is at 395.00 - 34.99 = 360.01 m and it starts with a head of 390.00 m.

`chancell.gw`:

```
chancell.gw: written by SWAT+ editor
 cell_id   elev_m  channel  riv_length_m  zone  dep_zone  obs
      42   326.00       66       1853.55     1         0    1
      43   324.50       66        912.10     1         0    0
```

Channel 66 crosses cells 42 and 43, both with streambed properties from zone 1. With `river_thresh` at 5.00, the exchange at cell 42 uses a bed elevation of 321.00 m. Daily exchange at cell 42 is written to the channel observation output.

Related files

- `codes.bsn`: `gwflow` switches the module on.
- `object.cnt`: can give the number of gwflow objects directly.
- `print.prt`: gwflow output rows.
- `hru-data.hru` and connect files: the HRUs, landscape units, channels and reservoirs that cells connect to.
- `water_canal.wal`: canal data used by canal seepage.
- `constituents.cs` and salt inputs: the salts and constituents added to the gwflow solutes.
- gwflow outputs.

Sources

- Activation and object count: `basin_read_objs.f90`
- Readers: `gwflow_chan_read.f90` (`chancell.gw`, `chan_depth.gw` header) and `gwflow_read.f90` (all other `.gw` files and `gwflow_canal.con`), both called from `hyd_connect.f90`
- Wetland bed thickness: `wet_read_hyd.f90`
- print.prt rows: `basin_print_codes_read.f90`; override logic in `gwflow_read`; output files opened in `gwflow_output_init` in `gwflow_output.f90`
- Variables: `gwflow_module.f90`
- Daily solution: `gwflow_simulate.f90`, `gwflow_lateral.f90`, `gwflow_rech.f90`, `gwflow_gwet.f90`, `gwflow_channel_exch.f90`, `gwflow_tile.f90`, `gwflow_reservoir.f90`, `gwflow_wetland.f90`, `gwflow_floodplain.f90`, `gwflow_canal.f90`, `gwflow_phreatophyte.f90`, `gwflow_pond.f90`, `gwflow_solute.f90`, `gwflow_chem.f90`, `gwflow_heat.f90`

File and column	Fortran variable
`codes.gw` `n_rows`, `n_cols`	`grid_nrow`, `grid_ncol`
`codes.gw` `bc_type`	`bc_type_int`, copied to `bc_type_array`
`codes.gw` `gw_soil`, `satx`, `pumpex`, `tile`, `res`, `wet`, `fp`, `canal`, `solute`, `heat`	`gw_soil_flag`, `gw_satx_flag`, `gw_pumpex_flag`, `gw_tile_flag`, `gw_res_flag`, `gw_wet_flag`, `gw_fp_flag`, `gw_canal_flag`, `gw_solute_flag`, `gw_heat_flag`
`codes.gw` `time_step`	`gw_time_step`
`codes.gw` `write_day` to `write_aa`	`gwflag_day`, `gwflag_mon`, `gwflag_yr`, `gwflag_aa`
`codes.gw` `river_thresh`	`gw_bed_change`
`zones.gw` columns 2 to 6	`zones_aquK`, `zones_aquSy`, `zones_strK`, `zones_strbed`, `zones_Kt`
`cells.gw` `status`, `elev`, `thck`, `exdp`, `init`, `x`, `y`, `area`	`gw_state%stat`, `%elev`, `%thck`, `%exdp`, `%init`, `%xcrd`, `%ycrd`, `%area`
`cells.gw` `delay`	`delay`, converted to `gw_delay = exp(-1/delay)` per HRU
`cells.gw` columns 15 to 20	`cell_strK_over`, `cell_strthick_over`, `bc_type_array`, `cell_tile_depth_over`, `cell_tile_area_over`, `cell_tile_K_over`
`cellcon.gw`	`gw_state%ncon`, `cell_con%cell_id`
`outputs.gw`	`gw_obs_cells`, `gw_output_yr` and `gw_output_day`, `gw_cell_obs_ss`
`chancell.gw`	`gw_chan_id`, `gw_chan_elev`, `gw_chan_chan`, `gw_chan_len`, `gw_chan_zone`, `gw_chan_dpzn`, `gw_chan_obs`
`tile.gw` lines 2 to 5	`gw_tile_depth`, `gw_tile_drain_area`, `gw_tile_K`, `gw_tile_group_flag`
`rescell.gw` lines 3 and 4	`res_thick`, `res_K`
`solute.gw`	`num_ts_transport`, `gw_long_disp`, `gwsol_sorb`, `gwsol_rctn`, `canal_out_conc`
`ponds.gw`	`gw_pond_info`

Purpose

The salt module tracks eight salt ions in soil water, groundwater, channels, reservoirs and wetlands, together with five salt minerals in soils and aquifers that dissolve and precipitate. Salt enters from initial conditions, fertilizer

and manure, road salt, atmospheric deposition and point sources, and leaves with plant uptake, runoff, percolation and flow. High soil salinity can reduce plant growth.

The salt files are not listed in `file.cio`; SWAT+ looks for them by name in the run directory. The constituent module, which uses the same master file, is on the constituent inputs page.

Activation

Salt simulation is switched on by listing salt ions in the salt block of `constituents.cs`. With no salt ions listed, or without `constituents.cs`, the salt files are ignored.

The ions must be listed in this order, because several routines use the position of an ion and not its name:

Position	Ion
1	`so4` (sulfate)
2	`ca` (calcium)
3	`mg` (magnesium)
4	`na` (sodium)
5	`k` (potassium)
6	`cl` (chloride)
7	`co3` (carbonate)
8	`hco3` (bicarbonate)

Every file below that has one value per ion gives the values in this order. Files with mineral fractions give five values in this order: $CaCO_3$, $MgCO_3$, $CaSO_4$, $MgSO_4$, NaCl.

File overview

File	Content	Linked to objects by
`salt_hru.ini`	Initial ion concentrations and mineral fractions in HRU soils	`salt` column of `soil_plant.ini`
`salt_aqu.ini`	Initial ion concentrations and mineral fractions in aquifers	`salt` column of `initial.aqu_cs`
`salt_channel.ini`	Initial ion concentrations in channels	`salt` column of `initial.cha_cs`
`salt_res`	Initial ion concentrations in reservoirs and wetlands	`salt` column of `reservoir.res_cs` and `wetland.wet_cs`
`salt_irrigation`	Ion concentrations in irrigation water from outside the model	position, see below
`salt_fertilizer.frt`	Ion content of fertilizers and manures	position in the fertilizer list
`salt_road`	Road salt applied to the land surface	atmospheric deposition stations
`salt_atmo.cli`	Ion concentration in rainfall and dry deposition	atmospheric deposition stations, see climate inputs
`salt_recall.rec`	Ion loads from point sources	see recall inputs
`salt_plants`	Salt tolerance of each plant	position in `plants.plt`
`salt_uptake`	Daily ion uptake by each plant	position in `plants.plt`
`salt_urban`	Ion content of sediment from impervious areas	urban type name in `urban.urb`

Missing files

`salt_hru.ini`, `salt_aqu.ini` and `salt_channel.ini` are opened whether or not they exist. If one is missing, SWAT+ creates an empty file of that name in the run directory and reads no records from it. An empty file, whether created this way or supplied, is read the same way: SWAT+ finds no title line and no records.

The test projects `refdata/Ames_sub1` and `refdata/Osu_1hru` in the SWAT+ repository contain empty `salt_hru.ini`, `salt_aqu.ini` and `salt_channel.ini` files created by earlier runs. Both projects give `null` for the constituent file in `file.cio`, so no salt ions are simulated and the empty files have no effect on the results. If salt ions are listed in `constituents.cs`, an empty file behaves as follows:

- Empty `salt_hru.ini`: there is no record for the HRUs, but HRU setup still reads record 1, which does not exist. Give at least one record.
- Empty `salt_aqu.ini`: no aquifer finds a record, so aquifers start with no salt ions and no salt minerals.
- Empty `salt_channel.ini`: there is no record for the channels, but channel setup still reads record 1, which does not exist. Give at least one record.

Some files are needed as soon as salt ions are listed, because the setup reads from them for every object without checking that they were found:

- `salt_irrigation` is read for every HRU during setup.
- `salt_road` is read every day for every HRU when `atmodep.cli` is in use.

Provide these files whenever salts are simulated, with zero values if there is nothing to add.

salt_hru.ini

Initial salt in HRU soils. Lines 1 to 5 are skipped. Each record then takes three lines:

Line of record	Content
1	Record name
2	Soil values: 8 ion concentrations in soil water (mg/L), then 5 mineral fractions (percent)
3	Plant values: 13 numbers, read but not used

An HRU uses the record named in the `salt` column of its `soil_plant.ini` record. If the name is not found, the first record is used. Every soil layer starts with the same values. The ion mass in a layer is the concentration times the water stored in the layer at the start of the run.

salt_aqu.ini

Initial salt in aquifers. Lines 1 to 4 are skipped. Each later line is one record: name, 8 ion concentrations in groundwater (mg/L), and 5 mineral fractions (percent).

Aquifers find their record through `initial.aqu_cs`, which has a title line, a header line and one row per aquifer with the columns `name`, `pest`, `path`, `hmet`, `salt` and `cs`. The `salt` column names a record in `salt_aqu.ini`. For salts, aquifer n uses row n of `initial.aqu_cs`, whatever the row's name, so the rows must be in aquifer order. The ion mass is the concentration times the groundwater stored at the start of the run.

salt_channel.ini

Initial salt in channel water. Lines 1 and 2 are skipped. Each later line is one record: name and 8 ion concentrations (mg/L).

Channels find their record through `initial.cha_cs` (title, header, then rows with `name`, `pest`, `path`, `hmet`, `salt`, `cs`). A channel uses the `initial.cha_cs` row at the same position as the `initial.cha` row it selects by name; the `salt` column of that row names a record in `salt_channel.ini`. If the name is not found, the first record is used.

salt_res

Initial salt in reservoirs and wetlands. Lines 1 to 11 are skipped (two title lines, eight comment lines and a header). Each later line is one record: name and 8 ion concentrations (mg/L). The file is ignored when the reservoir nutrient file in `file.cio` is `null`.

Reservoirs find their record through `reservoir.res_cs` and wetlands through `wetland.wet_cs`. Both have two header lines and one row per reservoir (or wetland) data record with the columns `id`, `pest`, `weir`, `salt` and `cs`. `id` is the position of the record in `reservoir.res` or `wetland.wet`, and `salt` names a record in `salt_res`. A wetland without a matching record starts with no salt.

salt_irrigation

Ion concentrations in irrigation water taken from outside the model. Lines 1 and 2 are skipped. Each later line is one record: name and 8 ion concentrations (mg/L).

An HRU uses the record at the same position as the `salt_hru.ini` record it uses (record 3 of `salt_hru.ini` goes with record 3 of `salt_irrigation`); the name is not used. In 62.0.1 the transfer of salt with irrigation water is switched off in water allocation, so these concentrations do not change the salt balance.

salt_fertilizer.frt

Ion content of each fertilizer. Line 1 is a title and line 2 a header. Then one row per record of `fertilizer.frt`, in the same order and with the same number of rows:

#	Column	Units	Description
1	`name`	-	Fertilizer name. Not checked against `fertilizer.frt`.
2	`so4`	kg/kg	Sulfate per kg of material applied.
3	`ca`	kg/kg	Calcium per kg of material applied.
4	`mg`	kg/kg	Magnesium per kg of material applied.
5	`na`	kg/kg	Sodium per kg of material applied.
6	`k`	kg/kg	Potassium per kg of material applied.
7	`cl`	kg/kg	Chloride per kg of material applied.
8	`co3`	kg/kg	Carbonate per kg of material applied.
9	`hco3`	kg/kg	Bicarbonate per kg of material applied.

The salt added (kg/ha) is the amount applied (kg/ha) times the value in the table.

Where salt is added in 62.0.1:

- A manure operation in a management schedule or a `manure` action in a decision table adds salt, split between the top two soil layers by the surface fraction of the application method in `chem_app.ops`. The row used is the row at the position of the manure in `manure_db.frt`, and salt is added only when the manure's `org_min` entry matches a record in `manure_om.frt` other than the first.
- A fertilizer operation in a management schedule adds salt only when the HRU has standing water, and the salt goes into the ponded water.
- A fertilizer operation on dry soil adds no salt.

Salt from a fertilizer whose name in `fertilizer.frt` starts with `a` is reported as a soil amendment in the salt balance output; other fertilizers are reported as fertilizer.

salt_road

Road salt applied to the soil surface. The file is organised by the atmospheric deposition stations of `atmodep.cli` and uses the same time step as that file. Lines 1 to 4 are skipped. Then, for each station in the order of `atmodep.cli`, one line with the station name followed by one line per ion (8 lines), each starting with the ion name:

`atmodep.cli` time step	Values on each ion line	Daily application
`aa`	1 value, kg/ha per year	the value / 365, every day
`mo`	1 value per month of the deposition record, kg/ha per month	the month's value / days in the month

atmodep.cli time step	Values on each ion line	Daily application
yr	1 value per year of the deposition record, kg/ha per year	spread over the days with precipitation and a mean temperature below 0 °C, in proportion to the day's precipitation

Each HRU receives the road salt of the deposition station of its weather station, on its whole area, into the top soil layer. For the yr time step the precipitation and temperature used to spread the salt come from the weather station at the same position in the station list as the deposition station.

salt_plants

Salt tolerance of plants and switches for salt stress.

Line	Content
1, 2	Title and header, skipped
3	Conversion factor from total dissolved solids (mg/L) to electrical conductivity (dS/m)
4	Skipped
5	Salt stress switch: 0 = off, 1 = on
6	Soil type: 1 = gypsiferous ($CaSO_4$) soils, 2 = NaCl soils
7	Stress method: 1 = salt stress applied after the other stresses, 2 = salt stress included with the other stresses
8 to 11	Skipped
12	Header, skipped
13+	One row per plant in plants.plt, in the same order: name, threshold a (dS/m), slope b (% per dS/m)

The plant name is not checked; the row position decides.

Salt stress

When the switch is on, SWAT+ computes for each soil layer within the root zone the total dissolved solids of the soil water at saturation, after mineral dissolution, and converts it to the electrical conductivity of the saturation extract EC_e with the factor on line 3. The relative yield for that layer is

$$Y_r = \frac{100 - b\,(EC_e - a)}{100} \qquad \text{for } EC_e > a$$

limited to 0 to 1, and 1 when $EC_e \le a$. For gypsiferous soils 2 dS/m is added to a. A plant with $a = 0$ has no salt stress. The salt stress of the plant is the smallest Y_r over its root zone layers.

With stress method 1, plant growth is multiplied by the salt stress after the smallest of the water, temperature, nitrogen, phosphorus and aeration stresses has been applied. With method 2, the salt stress is one of the stresses and the smallest of them is used.

salt_uptake

Specified daily salt uptake by plants. Lines 1 to 3 are skipped. Then one row per plant in plants.plt, in the same order: name, and 8 uptake values (kg/ha per day). The plant name is not checked.

While a plant has roots, its daily uptake of each ion is split between the soil layers in proportion to the root mass in each layer and is limited to the ion mass in the layer. Without the file, plants take up no salt.

salt_urban

Ion content of sediment washed from impervious areas. Lines 1 and 2 are skipped. Each later line is an urban type name from urban.urb and 8 concentrations (mg of ion per kg of sediment). In 62.0.1 the values are read but not used.

Example

salt_plants in the layout SWAT+ Editor writes, with salt stress on and illustrative values:

```
salt_plants: written by SWAT+ editor
conversion factor from TDS --> EC (EC=TDS/factor)
640.0
plant salinity tolerance
1            flag (0 = off; 1 = on)
2            flag (1=caso4 soils; 2=nacl soils)
1            flag (1=salt stress applied after other stresses applied; 2=included with other stresses)
NOTE: a and b values are for nacl soils
NOTE: if a and b values = 0, no salt impact
a = EC_sat threshold for impact on crop yield
b = slope of salinity impact on crop yield
            name           a           b
            agrl        1.70       12.00
            corn        1.70       12.00
```

TDS is divided by 640 to give EC_e. For corn, with $EC_e = 4$ dS/m in a layer, the relative yield is $(100 - 12 \times (4 - 1.7))/100 = 0.72$. The rows after the header continue for every plant in plants.plt.

Related files

- constituents.cs: lists the salt ions and switches the salt module on.
- soil_plant.ini: names the salt_hru.ini record for each HRU.
- fertilizer.frt: sets the row order of salt_fertilizer.frt.
- atmodep.cli, salt_atmo.cli: deposition stations and salt deposition.
- salt_recall.rec: salt loads from point sources.
- gwflow inputs: salt ions become groundwater solutes when gwflow solute transport is on.
- constituent inputs: the selenium and boron module.

Sources

- Master list: constit_db_read.f90
- Readers: salt_hru_read.f90, salt_aqu_read.f90, salt_cha_read.f90, res_read_saltdb.f90, salt_irr_read.f90, salt_fert_read.f90, salt_roadsalt_read.f90, salt_plant_read.f90, salt_uptake_read.f90, salt_urban_read.f90
- Link files: aqu_read_init_cs.f90 (initial.aqu_cs), ch_read_init_cs.f90 (initial.cha_cs) with sd_channel_read.f90, res_read_salt_cs.f90, wet_read_salt_cs.f90, soil_plant_init.f90 and hru_read.f90
- Initialisation: salt_hru_init.f90, sd_hydsed_init.f90, res_initial.f90, wet_initial.f90
- Processes: salt_fert.f90, salt_fert_wet.f90 (called from mgt_sched.f90 and actions.f90), salt_irrig.f90, salt_roadsalt.f90, salt_uptake.f90, salt_chem_soil_single.f90, salt_chem_aqu.f90, salt stress in pl_waterup.f90 and pl_biomass_gro.f90
- Types and variables: constituent_mass_module.f90, salt_module.f90, salt_data_module.f90, res_salt_module.f90

File and column	Fortran variable
salt_hru.ini soil and plant lines	salt_soil_ini%soil, salt_soil_ini%plt
salt_aqu.ini	salt_aqu_ini%conc, salt_aqu_ini%frac

File and column	Fortran variable
salt_channel.ini	salt_cha_ini%conc
salt_res	res_salt_data%c_init
salt_irrigation	salt_water_irr%water, copied to cs_irr%saltc
salt_fertilizer.frt	fert_salt (type fert_db_salt)
salt_road	rdapp_salt%salt%road, %roadmo, %roadyr, %roadday
salt_plants lines 3, 5, 6, 7	salt_tds_ec, salt_tol_sim, salt_soil_type, salt_effect
salt_plants columns a, b	salt_stress_a, salt_stress_b
salt_uptake	salt_uptake_kg
salt_urban	salt_urban_conc
initial.aqu_cs	aqu_init_dat_c_cs
initial.cha_cs	ch_init_cs
reservoir.res_cs, wetland.wet_cs	res_dat_c_cs, wet_dat_c_cs

Purpose

constituents.cs lists the pesticides, pathogens, heavy metals, salt ions and other constituents that a run simulates. The last group, the other constituents, is the constituent module: selenate (SeO_4), selenite (SeO_3) and boron in soils, aquifers, channels, reservoirs and wetlands. The module simulates sorption, selenium reduction and selenium release from shale, and the constituents move with water like the other dissolved substances.

The salt ion files are described on the salt inputs page. Apart from constituents.cs, which is named in file.cio, the files are not listed in file.cio; SWAT+ looks for them by name in the run directory.

constituents.cs

The master list, named on the simulation line of file.cio (default constituents.cs). If the file is missing or named null, no pesticides, pathogens, salts or other constituents are simulated.

Line 1 is a title. Then come five groups, each as a count followed by that many names:

Group	Count	Names
1	number of pesticides	pesticide names, matched by name to pesticide.pes
2	number of pathogens	pathogen names, matched by name to pathogens.pth
3	number of heavy metals	metal names, not matched to any database (see metals)
4	number of salt ions	salt ion names, in the order given on the salt inputs page
5	number of other constituents	constituent names

Each count is read from a separate line and the names that follow are read as one list, starting on the next line, so they can be on one line or several. A count of 0 still needs its names line: SWAT+ skips one line after the count, so leave a blank line (or any text) there. A pesticide or pathogen name that is not in its database is kept in the list but gets no parameters.

The constituent module uses the position of each constituent and not its name. Fertilizer input, selenium reactions and the reservoir parameters assume this order:

Position	Constituent
1	seo4 (selenate)
2	seo3 (selenite)
3	boron

File overview

File	Content	Linked to objects by
cs_hru.ini	Initial dissolved and sorbed constituents in HRU soils	cs column of soil_plant.ini
cs_aqu.ini	Initial dissolved and sorbed constituents in aquifers	cs column of initial.aqu_cs
cs_channel.ini	Initial concentrations in channels	cs column of initial.cha_cs
cs_res	Reservoir and wetland parameters and initial concentrations	cs column of reservoir.res_cs and wetland.wet_cs
cs_irrigation	Concentrations in irrigation water from outside the model	position, see below
fertilizer.frt_cs	Constituent content of fertilizers and manures	position in the fertilizer list
cs_reactions	Reaction parameters for soils and aquifers	HRU and aquifer order
cs_uptake	Daily constituent uptake by each plant	position in plants.plt
cs_atmo.cli	Concentration in rainfall and dry deposition	atmospheric deposition stations, see climate inputs
cs_recall.rec	Loads from point sources	see recall inputs
cs_plants_boron	Boron tolerance of plants	read but not used
cs_urban	Constituent content of sediment from impervious areas	read but not used
cs_streamobs	Channels for daily concentration output	not read

Missing files

cs_hru.ini, cs_aqu.ini and cs_channel.ini are opened whether or not they exist. If one is missing, SWAT+ creates an empty file of that name in the run directory and reads no records from it. An empty file, whether created this way or supplied, is read the same way.

The test project refdata/Ames_sub1 in the SWAT+ repository contains empty cs_hru.ini, cs_aqu.ini and cs_channel.ini files created by earlier runs. It gives null for the constituent file in file.cio, so no constituents are simulated and the empty files have no effect on the results. If constituents are listed in constituents.cs, an empty file behaves as follows:

- Empty cs_hru.ini: there is no record for the HRUs, but HRU setup still reads record 1, which does not exist. Give at least one record.
- Empty cs_aqu.ini: when initial.aqu_cs is present, the aquifer search runs over the number of records in cs_irrigation and reads cs_aqu.ini records that do not exist. Give at least as many records as cs_irrigation has.
- Empty cs_channel.ini: there is no record for the channels, but channel setup still reads record 1, which does not exist. Give at least one record.

Two files are needed as soon as other constituents are listed in constituents.cs, because they are read for every HRU without a check that they were found:

- cs_irrigation is read for every HRU during setup.
- cs_uptake is read every day for every HRU with a growing plant.

Provide both whenever constituents are simulated, with zero values if there is nothing to add.

cs_hru.ini

Initial constituents in HRU soils. Lines 1 to 5 are skipped. Each record then takes three lines:

Line of record	Content
1	Record name
2	Soil values: one dissolved concentration (mg/L) per constituent, then one sorbed concentration (mg/kg of soil) per constituent
3	Plant values: the same number of values, read but not used

An HRU uses the record named in the `cs` column of its `soil_plant.ini` record; if the name is not found, the first record is used. `soil_plant.ini` has the `cs` column only when `event` in `codes.bsn` is not 0. With `event` at 0, the column is not read and every HRU uses the first record of `cs_hru.ini`. Every soil layer starts with the same values.

cs_aqu.ini

Initial constituents in aquifers. Lines 1 to 3 are skipped. Each later line is one record: name, one dissolved concentration (mg/L) per constituent, then one sorbed concentration (mg/kg) per constituent. The sorbed mass is calculated with an assumed aquifer bulk density of 2000 kg/m³.

Aquifers find their record through `initial.aqu_cs` (title, header, then rows with `name`, `pest`, `path`, `hmet`, `salt`, `cs`). Aquifer n uses row n of `initial.aqu_cs`, whatever the row's name, and the `cs` column of that row names a record in `cs_aqu.ini`. See salt inputs for the same link used by salts.

cs_channel.ini

Initial concentrations in channel water. Lines 1 and 2 are skipped. Each later line is one record: name and one concentration (mg/L) per constituent. Channels find their record through `initial.cha_cs` as described for `salt_channel.ini`, using its `cs` column.

cs_res

Constituent parameters and initial concentrations for reservoirs and wetlands. Lines 1 to 15 are skipped (two title lines, twelve comment lines and a header). The file is ignored when the reservoir nutrient file in `file.cio` is `null`. Each later line is one record:

#	Column	Units	Default	Description
1	`name`	-	-	Record name, named in the `cs` column of `reservoir.res_cs` or `wetland.wet_cs`.
2	`v_seo4`	m/day	0.001	Settling velocity of selenate.
3	`v_seo3`	m/day	0.001	Settling velocity of selenite.
4	`v_born`	m/day	0.001	Settling velocity of boron.
5	`k_seo4`	1/day	0.05	First-order loss rate of selenate.
6	`k_seo3`	1/day	0.03	First-order loss rate of selenite.
7	`k_born`	1/day	0.0	First-order loss rate of boron.
8	`theta_seo4`	-	1.08	Temperature factor for the selenate rate.
9	`theta_seo3`	-	1.08	Temperature factor for the selenite rate.
10	`theta_born`	-	1.08	Temperature factor for the boron rate.
11	`c_seo4`	mg/L	0.0	Initial selenate concentration.
12	`c_seo3`	mg/L	0.0	Initial selenite concentration.
13	`c_born`	mg/L	0.0	Initial boron concentration.

All 13 values must be given; a short line takes its remaining values from the next line. A reservoir without a `cs_res` record starts with no constituents and uses the default rates in the table. `reservoir.res_cs` and `wetland.wet_cs` are described under `salt_res`.

cs_irrigation

Concentrations in irrigation water from outside the model. Lines 1 and 2 are skipped. Each later line is one record: name and one concentration (mg/L) per constituent.

An HRU uses the record at the same position as the `cs_hru.ini` record it uses; the name is not used. In 62.0.1 the transfer of constituents with irrigation water is switched off in water allocation, so these concentrations do not change the constituent balance.

fertilizer.frt_cs

Constituent content of each fertilizer. Line 1 is a title and line 2 a header. Then one row per record of `fertilizer.frt`, in the same order and with the same number of rows:

#	Column	Units	Description
1	`name`	-	Fertilizer name. Not checked against `fertilizer.frt`.
2	`seo4`	kg/kg	Selenate per kg of material applied.
3	`seo3`	kg/kg	Selenite per kg of material applied.
4	`boron`	kg/kg	Boron per kg of material applied.

The mass added (kg/ha) is the amount applied (kg/ha) times the value in the table. Constituents are added in the same cases as salts (see `salt_fertilizer.frt`): with manure operations and decision-table `manure` actions whose manure has an `org_min` entry other than the first record of `manure_om.frt`, and with fertilizer operations on HRUs with standing water. For manure, the whole amount goes into the top soil layer.

cs_reactions

Reaction parameters for soils and aquifers, organised in reaction groups and shale formations. Every HRU and every aquifer must have a row.

Section	Content
Line 1, 2	Title and header, skipped
Line 3	Number of parameters n_p and number of reaction groups n_g
next n_p lines	One parameter per line, with one value per reaction group
next line	Header, skipped
next line	Number of shale formations n_s
next n_s lines	Per shale formation: S/Se ratio of the shale, rate constant for autotrophic reduction of oxygen (1/day), rate constant for autotrophic reduction of nitrate (1/day)
next line	Header, skipped
next lines	One line per HRU, in HRU order: HRU number, reaction group, n_s shale fractions (fraction of the HRU area over each shale formation)
next line	Header, skipped (only when the model has aquifers)
next lines	One line per aquifer, in aquifer order: aquifer number, reaction group, n_s shale fractions

The parameter lines are used in this order:

Line	Parameter	Units	Used for
1	Selenium reduction inhibition factor for nitrate	-	soils, aquifers
2	Oxygen concentration in soil water	mg/L	soils
3	Oxygen concentration in groundwater	mg/L	aquifers
4	Sorption coefficient K_d of selenate	L/kg	soils, aquifers

Line	Parameter	Units	Used for
5	Sorption coefficient K_d of selenite	L/kg	soils, aquifers
6	Sorption coefficient K_d of boron	L/kg	soils, aquifers
7	Rate constant for selenate reduction in soil	1/day	soils
8	Rate constant for selenate reduction in groundwater	1/day	aquifers

The first value of the HRU and aquifer lines is not checked; the line position decides. Further parameter lines are read but not used by soils and aquifers. The rate constant for selenite reduction is not read for soils or aquifers and stays 0, so selenite is not reduced there.

cs_uptake

Specified daily constituent uptake by plants. Lines 1 to 3 are skipped. Then one row per plant in `plants.plt`, in the same order: name and one uptake value (kg/ha per day) per constituent. The plant name is not checked. While a plant has roots, its daily uptake of each constituent is split between the soil layers in proportion to the root mass in each layer and is limited to the mass in the layer.

cs_plants_boron

Boron tolerance of plants. Line 1 is a title and line 2 is skipped, line 3 is a switch (0 = off, 1 = on), lines 4 to 7 are skipped, and then one row per plant in `plants.plt`: name, threshold a and slope b. In 62.0.1 the values are read but no routine uses them, so boron has no effect on plant growth.

cs_urban

Constituent content of sediment washed from impervious areas. Lines 1 and 2 are skipped. Each later line is an urban type name from `urban.urb` and one concentration (mg per kg of sediment) per constituent. In 62.0.1 the values are read but not used.

cs_streamobs

A list of channels for daily concentration and load output to `cs_streamobs_output`: a title line, the number of channels, then one channel number per line. In 62.0.1 the switch that would read it is never set, so the file is not read and the output is not written.

Example

`constituents.cs` for a run with one pesticide, the eight salt ions and the three constituents:

```
constituents.cs: constituent list
1
atrazine
0

0

8
so4 ca mg na k cl co3 hco3
3
seo4 seo3 boron
```

There are no pathogens or heavy metals, so their counts are 0 and each is followed by an empty names line.

A matching `cs_aqu.ini` record with 0.005 mg/L selenate in groundwater and 0.2 mg/kg sorbed selenate, values illustrative:

```
cs_aqu.ini: initial constituents in aquifers
seo4 seo3 boron: dissolved (mg/L), then sorbed (mg/kg)
name          seo4     seo3    boron   seo4_s   seo3_s  boron_s
aqu_cs1      0.005    0.001    0.100    0.200    0.050    1.000
```

Related files

- salt inputs: the salt ions listed in `constituents.cs`, and the link files `initial.aqu_cs`, `initial.cha_cs`, `reservoir.res_cs` and `wetland.wet_cs`.
- `file.cio`: names `constituents.cs`.
- `soil_plant.ini`: names the `cs_hru.ini` record for each HRU.
- `codes.bsn`: `event` decides whether `soil_plant.ini` has the `cs` column.
- `fertilizer.frt`: sets the row order of `fertilizer.frt_cs`.
- `pesticide.pes`, `pathogens.pth`, metals: the other groups in `constituents.cs`.
- `cs_atmo.cli` and `cs_recall.rec`: deposition and point-source loads.
- gwflow inputs: constituents become groundwater solutes when gwflow solute transport is on.

Sources

- Master list: `constit_db_read.f90`; type `cs_db_type` in `constituent_mass_module.f90`
- Readers: `cs_hru_read.f90`, `cs_aqu_read.f90`, `cs_cha_read.f90`, `res_read_csdb.f90`, `cs_irr_read.f90`, `cs_fert_read.f90`, `cs_reactions_read.f90`, `cs_uptake_read.f90`, `cs_plant_read.f90`, `cs_urban_read.f90`
- Link files and initialisation: `soil_plant_init.f90`, `cs_hru_init.f90`, `aqu_read_init_cs.f90`, `sd_channel_read.f90`, `sd_hydsed_init.f90`, `res_read_salt_cs.f90`
- Processes: `cs_fert.f90`, `cs_fert_wet.f90`, `cs_irrig.f90`, `cs_uptake.f90` (called from `pl_biomass_gro.f90`), `cs_sorb_hru.f90`, `cs_sorb_aqu.f90`, `se_reactions_soil.f90`, `se_reactions_aquifer.f90`, `res_cs.f90`
- Types and variables: `cs_module.f90`, `cs_data_module.f90`, `res_cs_module.f90`

File and column	Fortran variable
`constituents.cs` counts	`cs_db%num_pests`, `%num_paths`, `%num_metals`, `%num_salts`, `%num_cs`
`constituents.cs` names	`cs_db%pests`, `%paths`, `%metals`, `%salts`, `%cs`
`cs_hru.ini` soil and plant lines	`cs_soil_ini%soil`, `cs_soil_ini%plt`
`cs_aqu.ini`	`cs_aqu_ini%aqu`
`cs_channel.ini`	`cs_cha_ini%conc`
`cs_res`	`res_cs_data` (type `reservoir_cs_data`)
`cs_irrigation`	`cs_water_irr%water`, copied to `cs_irr%csc`
`fertilizer.frt_cs`	`fert_cs` (type `fert_db_cs`)
`cs_reactions` parameter lines 1 to 8	`rct(1:8, group)`, copied to `se_ino3`, `oxy_soil`, `oxy_aqu`, `kd_seo4`, `kd_seo3`, `kd_born`, `kseo4` of `cs_rct_soil` and `cs_rct_aqu`
`cs_reactions` shale lines	`rct_shale(:,1:3)`, copied to `sseratio`, `ko2a`, `kno3a`
`cs_uptake`	`cs_uptake_kg`
`cs_plants_boron`	`bor_tol_sim`, `bor_stress_a`, `bor_stress_b`
`cs_urban`	`cs_urban_conc`
`cs_streamobs`	`cs_str_obs`

Note

In revision 62.0.1 the water allocation calculation follows these inputs only for a network with a single place of use, other than an irrigation district, whose POD lines are listed in POD-number order. Other layouts can take the wrong amounts or stop the run, and several columns are read but have no effect. See Limitations in 62.0.1 before setting up a network.

Purpose

Water allocation moves water from where it is taken to where it is used: irrigation districts, towns, industry, treatment plants, reservoirs, canals and places outside the basin. Revision 62.0.1 describes the network with three kinds of object:

- A **place of use** (POU) is where water goes: an irrigation district, a water treatment plant, a water use, a water tower, a canal, a channel, a reservoir, an aquifer or a receiver outside the basin. Each POU has a daily demand (its duty).
- A **point of diversion** (POD) is where water is taken: a channel, reservoir, aquifer, canal, water tower, outside-basin source or an unlimited source. A POD can supply several POUs, and a POU can draw on several PODs.
- A **point of return** (POR) is where a POU sends water after it has been used or treated: back to a channel, reservoir or aquifer, on to another treatment plant or use, or out of the basin.

Two files define the network: `place_of_use.wro` lists every POU with its PODs and PORs, and `point_of_diver.wro` lists every POD with the POUs it serves. The `.wal` files describe the objects between them: treatment plants, water uses, water towers, canals, pipes, irrigation districts and outside-basin sources.

This structure replaces the `water_allocation.wro` transfer file of earlier revisions, whose columns in earlier documentation (`rule_typ`, `srcs`, `ob_typ`, `withdr`, `limit_mon`, `cha_ob`, `rcv_ob`, `rcv_num`) have no counterpart in 62.0.1. `water_allocation.wro`, `element.wro` and `water_rights.wro` are no longer read.

Files and activation

File	Name set by	Contents
`place_of_use.wro`	`water_rights` line of `file.cio`, first name	Places of use, with their PODs and PORs
`point_of_diver.wro`	`water_rights` line of `file.cio`, second name	Points of diversion and the POUs each serves
`water_hru_irr.wal`	fixed name	HRUs in each irrigation district
`wtps_wuses.wal`	fixed name	Numbers of treatment plants and water uses
`water_treat.wal`	fixed name	Water treatment plants
`water_use.wal`	fixed name	Water uses (domestic, industrial, commercial)
`outside_src.wal`	fixed name	Sources outside the basin
`om_treat.wal`, `om_use.wal`, `om_osrc.wal`	fixed names	Water quality sets for treated water, used water and outside sources
`water_tower.wal`	fixed name	Water towers (storage)
`water_canal.wal`	fixed name	Canals
`water_pipe.wal`	fixed name	Pipes
`outside_rcv.wal`	fixed name	Receivers outside the basin (not read)

The `water_rights` line of `file.cio` now takes two file names, the POU file and the POD file, with defaults `place_of_use.wro` and `point_of_diver.wro`. A `file.cio` written for an earlier revision has `water_allocation.wro element.wro water_rights.wro` on that line, and SWAT+ 62.0.1 then reads `water_allocation.wro` as the POU file and `element.wro` as the POD file. Update the line before running an older project; older file.cio files explains how the old line is read.

Water allocation runs when the POU file has at least one place of use. If a `.wro` file or `wtps_wuses.wal` is missing, or the line in `file.cio` says `null`, SWAT+ creates an empty file of that name in the run folder (a file called `null` in the second case) and reads nothing from it. The empty file does no harm.

All files have a title on line 1. The `.wro` files and most `.wal` files then give the number of records on line 2 and a column header on line 3. Records are read as free-format values separated by spaces, by position, so the header text is not checked.

Daily sequence

Each day, before any object is simulated:

1. The day's POD and POR totals are set to zero.
2. Outside-basin source PODs, water tower PODs and unlimited PODs give water to their POUs.
3. POUs with no PODs (a treatment plant or use fed only by returns from other POUs) treat or use the water they have stored and pass it to their PORs.
4. The demand of every POU is set, from `rate_max` or from its decision table, and split among its PODs by `frac` (or by a decision table).

During the day, natural PODs give water when their object is simulated: a channel POD after the channel is routed (the water comes out of the channel's outflow for the day), a reservoir POD after the day's inflow is added, an aquifer POD after recharge is added. A canal POD gives water when the canal receives water.

Each time a POD gives water, every POU whose PODs have all given water for the day receives it. Water delivered to a POU that is a treatment plant, use or canal is processed at once (treated, used, or routed through the canal), and the POU's outflow is passed to its PORs.

Because step 2 runs before step 4, outside-source, water tower and unlimited PODs work with the previous day's demand, and with zero demand on the first day.

place_of_use.wro

Line 1 is a title, line 2 the number of POUs, line 3 a header. Each POU then takes one line, followed by one line for each of its PODs and one line for each of its PORs.

POU line

#	Column	Units	Description
1	`num`	-	POU number, 1 to the number on line 2. The record is stored under this number.
2	`name`	-	POU name.
3	`typ`	-	POU type, from the table below.
4	`typ_num`	-	Number of the object of that type that receives the water (for example the channel number, or the record number in `water_treat.wal`). For `irr`, the record number in `water_hru_irr.wal`.
5	`pods`	-	Number of POD lines that follow. 0 for a POU fed only by returns from other POUs.
6	`pors`	-	Number of POR lines that follow.
7	`dtbl_mx`	-	Name of a flow control decision table (`flo_con.dtl`) that sets the daily demand, or `null`.
8	`rate_max`	m^3/s	Maximum demand rate. Without a decision table the POU demands rate_max × 86400 m^3 every day. With a decision table, its demand is capped at that amount.
9	`dtbl_pod_fr`	-	Name of a flow control decision table that sets the fraction taken from each POD, or `null` to use the `frac` column of the POD lines.
10	`dtbl_por_fr`	-	Name of a flow control decision table that sets the fraction sent to each POR, or `null` to use the `frac` column of the POR lines.

POU type	Water goes to
`irr`	The HRUs of an irrigation district in `water_hru_irr.wal`
`cha`	A channel (added to the channel's inflow)
`res`	A reservoir
`aqu`	An aquifer
`wtp`	A water treatment plant in `water_treat.wal`
`use`	A water use in `water_use.wal`
`wtow`	A water tower in `water_tower.wal`
`can`	A canal in `water_canal.wal`

POU type	Water goes to
`orcv`	A receiver outside the basin in `outside_rcv.wal`

POD lines

One line per POD, starting with the word `POD`.

#	Column	Units	Description
1	`kind`	-	The word `POD`.
2	`num`	-	Not used (overwritten by column 5).
3	`name`	-	POD name, for reference.
4	`typ`	-	POD type, for reference. The type used is the one in `point_of_diver.wro`.
5	`typ_num`	-	Object number, for reference.
6	`conv_typ`	-	Conveyance type, `pipe` or `pump`. Read but not used.
7	`conv_num`	-	Conveyance number. Read but not used.
8	`dtbl_min`	-	Decision table for the minimum level. Read but not used.
9	`const_min`	depends on POD type	Minimum level below which the POD gives no water. See the POD type table under point_of_diver.wro.
10	`dtbl_wdraw`	-	Name of a flow control decision table that manages the annual withdrawal limit, or `null`. When a name is given, the limit is not reset at the start of each year; the `duty_reset` decision table action resets it instead.
11	`ann_max`	m^3/s	Annual withdrawal limit for this POD and POU. Once the volume taken during the year reaches ann_max $\times$ 86400 m^3, the POD gives this POU no more water until the limit is reset.
12	`frac`	fraction	Fraction of the POU's daily demand taken from this POD. The fractions of a POU's PODs normally add up to 1.
13	`comp`	-	`y` to let this POD make up a shortfall when the POU's PODs together delivered less than its demand, `n` otherwise. Channel PODs cannot make up a shortfall.

POR lines

One line per POR, starting with the word `POR`.

#	Column	Units	Description
1	`kind`	-	The word `POR`.
2	`num`	-	Not used (overwritten by column 5).
3	`name`	-	POR name, for reference.
4	`typ`	-	Receiving object type: `cha`, `res`, `aqu`, `wtp`, `use`, `wtow`, `can` or `orcv`. `hru` is accepted but receives nothing.
5	`typ_num`	-	Number of the receiving object of that type.
6	`conv_typ`	-	Conveyance type. Read but not used.
7	`conv_num`	-	Conveyance number. Read but not used.
8	`dtbl_max`	-	Read but not used.
9	`const_max`	-	Read but not used.
10	`ann_max`	-	Read but not used.
11	`frac`	fraction	Fraction of the POU's outflow sent to this POR.

A water treatment plant POU sends each POR its `frac` of the treated water, and a water use POU sends each POR its `frac` of its effluent. For the other POU types, canals included, each POR receives its `frac` of the water withdrawn for the POU, in addition to the delivery to the POU itself.

point_of_diver.wro

Line 1 is a title, line 2 the number of PODs, line 3 a header. Each POD takes one line: five fixed columns, then six columns for each POU it serves.

#	Column	Units	Description
1	`num`	-	POD number.
2	`name`	-	POD name.
3	`typ`	-	POD type, from the table below.
4	`typ_num`	-	Number of the object of that type (channel number, reservoir number, record number in `outside_src.wal`, and so on).
5	`pous`	-	Number of POUs served.
6 to 11	`pou_num`, `pou_name`, `pod_num`, `pou_typ`, `pou_typ_num`, `right`	-	Repeated for each POU: the POU number in `place_of_use.wro`, its name, the position of this POD among that POU's POD lines, the POU type and object number, and the water right (`sr` senior, `jr` junior).

The `right` column is read but does not change the order of withdrawals in 62.0.1.

POD type	Water taken from	Meaning of `const_min`
`cha`	Channel outflow for the day	Minimum daily flow volume (m^3). No water is taken on days when the channel outflow is at or below it.
`res`	Reservoir storage	Fraction of the principal spillway volume. No water is taken when storage is at or below it.
`aqu`	Aquifer storage	Water table depth (m). Water is taken only while the water table is shallower than this depth.
`can`	Canal storage	Fraction of the current canal storage, so values up to 1 set no limit.
`wtow`	Water tower storage	Fraction of the current tower storage, so values below 1 set no limit.
`osrc`	Outside-basin source	Not used.
`unl`	Unlimited source with no water quality	Not used.
`hru`	Accepted, gives no water	-
`gwf`	Accepted, gives no water	-

The amount taken is the POD's duty (its `frac` of the POU demand), limited by the water the object has above the minimum. Withdrawals from channels, reservoirs, canals and towers take sediment and nutrients with the water in proportion. Aquifer withdrawals take water, nitrate and mineral P.

Two types do not follow this rule:

- An `osrc` POD gives its `frac` of the daily volume available from the outside source, whatever the POU demand. The available volume is the `flo` value of the `om_osrc.wal` record with the same number as the source. The water has no sediment or nutrients.
- A `unl` POD gives frac times its duty, which is $frac^2$ times the POU demand.

water_hru_irr.wal

Irrigation districts for `irr` POUs. Line 1 is a title, line 2 the number of districts, line 3 a header. Each district takes one line with its name and number of HRUs, followed by one line per HRU.

Line	Columns	Description
District	`name`, `num_hrus`	District name and the number of HRU lines that follow.
HRU	`hru`, `dtbl_lum`	HRU number and the name of the decision table in `lum.dtl` that sets its irrigation demand.

The demand of an irrigation POU is meant to be the sum of the demands of its HRUs, set by their decision tables. See Limitations in 62.0.1 for how 62.0.1 handles it.

wtps_wuses.wal

Sets up storage for treatment plants and water uses. It is needed whenever `water_treat.wal` or `water_use.wal` is used.

Line	Contents
1	Title
2	Number of treatment plants, number of water uses
3	Header (skipped)
4	Names of the treatment plants
5	Names of the water uses

The two numbers must match the number of records in `water_treat.wal` and `water_use.wal`. Lines 4 and 5 must both be present, even when one of the numbers is 0. The names are read but not used.

water_treat.wal

Water treatment plants. Line 1 is a title, line 2 the number of plants, line 3 a header. Each plant takes one line, followed by one line per treatment level.

Plant line:

#	Column	Units	Description
1	`name`	-	Plant name.
2	`stor_mx`	m^3	Maximum storage. Read but not used.
3	`lag_days`	days	Treatment time. Read but not used: treated water leaves the same day.
4	`loss_fr`	fraction	Water loss during treatment. Read but not used; the loss comes from the `flo` value of the quality set.
5	`num_treats`	-	Number of treatment level lines that follow.

Treatment level line:

#	Column	Description
1	`org_min_typ`	How the quality of the treated water is set: `const` (a record in `om_treat.wal`), `dtbl` (a decision table) or `recall` (a recall record).
2	`org_min_name`	Name of the `om_treat.wal` record, decision table or recall record.
3, 4	`pests_typ`, `pests_name`	Same pair for pesticides. Read but not used.
5, 6	`paths_typ`, `paths_name`	Same pair for pathogens. Read but not used.
7, 8	`salts_typ`, `salts_name`	Same pair for salt ions. Read but not used.
9, 10	`constit_typ`, `constit_name`	Same pair for other constituents. Read but not used.
11	`description`	Text.

If the project has pesticides, each treatment level line is followed by a header line and a line of pesticide concentrations in the treated water, one value per pesticide. If it has pathogens, a header line and a line of pathogen concentrations follow in the same way.

When a plant has more than one treatment level, the last level sets the quality of the outflow.

Treated outflow to each POR is

$$Q_{out} = V_{stor}\, f_{\text{om}}\, \text{frac}_{POR}$$

where V_{stor} is the water delivered to the plant that day (m^3), f_{om} is the `flo` value of the quality set (the fraction of water that leaves the plant after treatment losses) and $frac_{POR}$ is the POR's fraction. Sediment and nutrient loads are the quality set's concentrations times Q_{out}, limited to the loads that came into the plant. The difference between incoming and outgoing loads is reported as the amount removed by treatment.

water_use.wal

Water uses (domestic, industrial, commercial). Line 1 is a title, line 2 the number of uses, line 3 a header. Each use takes one line, followed by the optional pesticide and pathogen lines described for `water_treat.wal`.

#	Column	Units	Description
1	`name`	-	Use name.
2	`stor_mx`	m^3	Maximum storage. Read but not used.
3	`lag_days`	days	Read but not used.
4	`loss_fr`	fraction	Read but not used.
5 to 14	`org_min_typ`, `org_min_name`, … `constit_typ`, `constit_name`	-	Quality pairs as in a `water_treat.wal` treatment level. `org_min_name` names a record in `om_use.wal` when the type is `const`.
15	`description`	-	Text.

The outflow of a use is the water delivered that day times the `flo` value of its `om_use.wal` record (the fraction returned as effluent), with the record's concentrations. Each POR receives its `frac` of the outflow.

outside_src.wal

Sources outside the basin, used by `osrc` PODs. Line 1 is a title, line 2 the number of sources, line 3 a header. Each source takes one line, followed by the optional pesticide and pathogen lines described for `water_treat.wal`.

#	Column	Description
1	`name`	Source name.
2 to 11	`org_min_typ`, `org_min_name`, … `constit_typ`, `constit_name`	Quality pairs as in a `water_treat.wal` treatment level. `org_min_name` names a record in `om_osrc.wal` when the type is `const`.
12	`description`	Text.

om_treat.wal, om_use.wal and om_osrc.wal

Water quality sets for treated water, used water (effluent) and outside sources. Line 1 is a title, line 2 the number of records, line 3 a header. Each record is a name followed by 18 values.

#	Column	Units	Description
1	`name`	-	Record name.
2	`flo`	see text	In `om_treat.wal` and `om_use.wal`, the fraction of the incoming water that leaves the plant or use. In `om_osrc.wal`, the volume available from the source each day (m^3).
3	`sed`	mg/L	Sediment.
4	`orgn`	mg N/L	Organic nitrogen.
5	`sedp`	mg P/L	Organic phosphorus.
6	`no3`	mg N/L	Nitrate nitrogen.
7	`solp`	mg P/L	Soluble (mineral) phosphorus.
8	`chla`	mg/L	Chlorophyll a.
9	`nh3`	mg N/L	Ammonia nitrogen.
10	`no2`	mg N/L	Nitrite nitrogen.
11	`cbod`	mg/L	Carbonaceous biological oxygen demand.
12	`dox`	mg/L	Dissolved oxygen.

#	Column	Units	Description
13 to 18	`san`, `sil`, `cla`, `sag`, `lag`, `grv`	mg/L	Sand, silt, clay, small aggregates, large aggregates and gravel.
19	`temp`	°C	Water temperature.

water_tower.wal

Water towers: storage between a source and its users. Line 1 is a title, line 2 the number of towers, line 3 a header, then one line per tower.

#	Column	Units	Description
1	`id`	-	Tower number.
2	`name`	-	Tower name.
3	`stor_mx`	m³	Storage capacity. Used by the `wtow_vol` decision table condition and the `fill_wtow` action.
4	`ddown_days`	days	Read but not used.
5	`loss_fr`	fraction	Read but not used.

A tower is filled by a `wtow` POU and emptied by a `wtow` POD. Its water quality does not change in storage.

water_canal.wal

Canals. Line 1 is a title, line 2 the number of canals, line 3 a header, then one line per canal.

#	Column	Units	Description
1	`id`	-	Canal number.
2	`name`	-	Canal name.
3	`w_sta`	-	Nearby weather station. Read but not used.
4	`init`	-	Initial concentrations. Read but not used.
5	`dtbl`	-	Decision table for canal outflow. Must be `null` in 62.0.1: with any other name the canal releases no water.
6	`ddown_days`	days	Days to draw the storage down. Each day the canal releases its storage divided by `ddown_days`.
7	`w`	m	Bottom width.
8	`d`	m	Depth.
9	`l`	km	Length.
10	`s`	m/m	Slope. Read but not used.
11	`ss`	m/m	Side slope (horizontal to vertical) of the trapezoidal section.
12	`sat_con`	mm/day	Seepage rate from the canal bed to groundwater.
13	`evap_co`	fraction	Fraction of the canal storage kept after evaporation each day. 1 means no evaporation loss; 0 empties the canal.
14	`num_aqu`	-	Number of aquifers that receive seepage.
15 onward	`aqu_fr`	fraction	One value per aquifer: the fraction of the seepage that goes to aquifer 1, aquifer 2, and so on. The aquifers are always the first `num_aqu` aquifers of the model, in order.

The canal capacity is computed from its shape as $V_{max} = (w + ss\,d)\,d\,l \times 1000$ m³ (with l in km), and is used by the `fill_can` decision table action.

When a canal receives water, it first supplies any POUs for which it is a POD. Evaporation is then applied, and seepage of sat_con $\times\ d \times l$ m³ is divided among the aquifers by their fractions, with nitrate and soluble P in proportion. Finally the canal releases its storage divided by `ddown_days`. In 62.0.1 this release is removed from

the canal but is not passed on; the PORs of a canal POU receive their share of the water withdrawn for the canal instead.

When gwflow canal seepage is switched on, gwflow takes canal width, depth and `sat_con` from this file. It also expects a bed thickness, a diversion ID and start and end days for each canal, which `water_canal.wal` does not supply in 62.0.1, so gwflow treats every canal as connected to a channel.

water_pipe.wal

Pipes. In 62.0.1 pipes are read but have no effect: water moves from PODs to POUs and from POUs to PORs without conveyance losses.

Line 1 is a title, line 2 the number of pipes, line 3 a header. For each pipe the reader then takes a header line and two record lines: the first gives the number of aquifers, and the second is read as the full record. Both record lines have the same layout:

#	Column	Units	Description
1	`id`	-	Pipe number.
2	`name`	-	Pipe name.
3	`stor_mx`	m^3	Maximum storage.
4	`ddown_days`	days	Drawdown time.
5	`loss_fr`	fraction	Water lost along the pipe.
6	`num_aqu`	-	Number of aquifers that receive the loss.
7 onward	`aqu_fr`	fraction	One value per aquifer: fraction of the loss to that aquifer.

outside_rcv.wal

Receivers outside the basin, with columns `id`, `name` and `filename` (an output file for the receiver). SWAT+ 62.0.1 does not read this file, so `orcv` POUs and PORs have no receiving object.

Decision tables

Water allocation uses flow control decision tables (`flo_con.dtl`) and land use decision tables (`lum.dtl`). See decision tables for the file format.

Item	Kind	Effect
`wallo_dmd` with option `set_dmd`	action	Sets the POU demand to the action's constant (m^3).
`wallo_dmd` with option `irr_dmd`	action	Adds the current demand of the POU named by the action's object number.
`wallo_dmd` with option `fill_res`, `fill_wtow` or `fill_can`	action	Sets the demand to the volume needed to fill a reservoir (to a fraction of its principal or emergency volume), a water tower or a canal (to a fraction of capacity), times a second constant.
`duty_reset`	action	Sets a new annual withdrawal limit for one POD of one POU and resets the volume taken so far.
`irr_pou_dmd`	condition	Tests the current demand of a POU.
`wtow_vol`	condition	Tests water tower storage as a fraction of `stor_mx`.

Limitations in 62.0.1

Several parts of the daily calculation do not follow the input structure above:

- The duty used for a withdrawal is looked up with the POD's number and the position of the POU in that POD's list, in place of the POU's number and the POD's position in that POU. Results follow the input only when these coincide: POD number n is the n-th POD line of every POU it serves, and the k-th POU listed

for a POD in `point_of_diver.wro` is POU number k. A network with one POU whose POD lines are listed in POD-number order meets this. Other layouts take the wrong duty or fail.

- For an irrigation district (`irr` POU), the HRU demands from the district's decision tables are summed and then replaced by rate_max × 86400, so the POU always demands its maximum rate. Delivery to the district does not step through the listed HRUs.
- No decision table action sets the POD and POR fractions, so naming a table in `dtbl_pod_fr` or `dtbl_por_fr` sets all fractions to zero.
- For an aquifer POD, the water delivered is computed after the aquifer storage has been reduced, so it is smaller than the water removed from the aquifer by the factor $(1 - r)$, where r is the fraction of storage withdrawn.
- Projects with pesticides or pathogens fail when `water_treat.wal`, `water_use.wal` or `outside_src.wal` is read, because the storage for the treated-water concentrations is not set up.
- Water taken by start-of-day PODs (outside sources, towers, unlimited sources) is not counted in the day's delivered totals, which are reset when the day's demand is set.

Example

A town water use that draws on a channel and an aquifer and returns its effluent to another channel. The single POU lists its PODs in POD-number order, which keeps POD numbers and POU positions aligned as 62.0.1 requires.

`place_of_use.wro`:

```
place_of_use.wro: town supply
1
num  name   typ  typ_num  pods  pors  dtbl_mx  rate_max  dtbl_pod_fr  dtbl_por_fr
1    town   use  1        2     1     null     0.05      null         null
POD  1  river   cha  12  pipe  0  null  20000.0  null  20.0  0.7  n
POD  2  wells   aqu  3   pump  0  null  15.0     null  20.0  0.3  y
POR  1  outfall cha  14  pipe  0  null  0.0      0.0   1.0
```

`point_of_diver.wro`:

```
point_of_diver.wro: town supply
2
num  name   typ  typ_num  pous  pou_num  pou_name  pod_num  pou_typ  pou_typ_num  right
1    river  cha  12       1     1        town      1        use      1            sr
2    wells  aqu  3        1     1        town      2        use      1            sr
```

`wtps_wuses.wal`, `water_use.wal` and `om_use.wal`:

```
wtps_wuses.wal: town supply
0  1
treatment plant names on line 4, water use names on line 5
none
town
```

```
water_use.wal: town supply
1
name  stor_mx  lag_days  loss_fr  org_min_typ  org_min_name  pests_typ  pests_name  paths_typ  paths_name
town  0.0      0.0       0.0      const        town_eff      null       null        null       null
```

```
om_use.wal: town supply
1
name      flo  sed  orgn  sedp  no3  solp  chla  nh3  no2  cbod  dox  san  sil  cla  sag  lag  grv  temp
town_eff  0.8  20   2     0.5   10   1     0     5    0.1  20    4    0    0    0    0    0    0    18
```

The town demands $0.05 \times 86400 = 4320$ m³ a day: 70% (3024 m³) from channel 12 and 30% (1296 m³) from aquifer 3. The channel gives water only on days its outflow exceeds 20 000 m³, and the aquifer only while its water table is shallower than 15 m. The aquifer may make up a shortfall from the channel. Each POD may supply at most $20 \times 86400 = 1\,728\,000$ m³ a year to the town. 80% of the water delivered returns to channel 14 as effluent with the concentrations in `town_eff`.

Outputs

Set the `water_allo` row of print.prt to write `water_allo_day.txt` (and the monthly, yearly and average annual files) with each POU's demand and delivery, the withdrawal from each POD and the water sent to each POR. Treatment plants and water uses are reported in `wallo_treat_*.txt` and `wallo_use_*.txt`.

Related files

- file.cio: the `water_rights` line names the POU and POD files.
- Decision tables: flow control and land use tables used for demand, limits and irrigation.
- Channels, reservoirs and aquifers: PODs and receiving objects.
- print.prt: output switches.
- gwflow: canal seepage to the groundwater grid.

Sources

- File names and `file.cio` line: `type input_water_allocation` in input_file_module.f90, readcio_read.f90
- Read order: main.f90.in
- Readers: water_allocation_read.f90 (POU and POD files), water_hru_irr_read.f90, wallo_wtps_wuses.f90, water_treatment_read.f90, water_use_read.f90, water_osrc_read.f90, om_treat_read.f90, om_use_read.f90, om_osrc_read.f90, water_tower_read.f90, water_canal_read.f90, water_pipe_read.f90, water_orcv_read.f90 (not called)
- Types: water_allocation_module.f90; `hyd_output` in hydrograph_module.f90
- Daily calculation: wallo_start_day.f90 (called from time_control.f90), wallo_control.f90, wallo_withdraw.f90, wallo_pou_deliv.f90, wallo_return.f90, wallo_treatment.f90, wallo_use.f90, wallo_canal.f90, wallo_transfer.f90 (not called)
- POD hooks: sd_channel_control3.f90, res_control.f90, aqu_1d_control.f90, hru_control.f90
- Decision tables: actions.f90, conditions.f90
- Outputs: header_water_allocation.f90, wallo_pou_output.f90
- gwflow canals: gwflow_read.f90

File	Column	Fortran variable
place_of_use.wro POU line	num, name, typ, typ_num, pods, pors, dtbl_mx, rate_max, dtbl_pod_fr, dtbl_por_fr	ipou, pou%name, %typ, %typ_num, %pods, %pors, %dtbl_mx, %rate_max, %dtbl_pod_fr, %dtbl_por_fr
POD line	columns 2 to 13	pou%pod%num, %name, %typ, %num, %conv_typ, %conv_num, %dtbl_min, %const_min, %dtbl_wdraw, %ann_max, %frac, %comp
POR line	columns 2 to 11	pou%por%num, %name, %typ, %num, %conv_typ, %conv_num, %dtbl_max, %const_max, %ann_max, %frac
point_of_diver.wro	num, name, typ, typ_num, pous	pod%num, %name, %typ, %typ_num, %pous
point_of_diver.wro per POU	pou_num, pou_name, pod_num, pou_typ, pou_typ_num, right	pod%pou%num, %name, %pod_num, %typ, %typ_num, %right
water_hru_irr.wal	name, num_hrus, hru, dtbl_lum	hruirr_db%name, %hrus, %hru_num, %dtbl_lum
water_treat.wal	name, stor_mx, lag_days, loss_fr, num_treats	wtp%name, %stor_mx, %lag_days, %loss_fr, %num_treats
treatment level, water_use.wal, outside_src.wal	quality pairs	%conc%org_min_typ, %org_min_name, %pests_typ, %pests_name, %paths_typ, %paths_name, %salts_typ, %salts_name, %constit_typ, %constit_name
om_*.wal	18 values	wtp_om_treat, wuse_om_efflu, osrc_om (type hyd_output)
water_tower.wal	stor_mx, ddown_days, loss_fr	wtow%stor_mx, %ddown_days, %loss_fr
water_canal.wal	w_sta … num_aqu, aqu_fr	canal%w_sta, %init, %dtbl, %ddown_days, %w, %d, %l, %s, %ss, %sat_con, %evap_co, %num_aqu, %aqu_loss_fr
water_pipe.wal	stor_mx … aqu_fr	pipe%stor_mx, %ddown_days, %loss_fr, %num_aqu, %aqu_loss_fr